

Brandenburgische Technische Universität Cottbus-Senftenberg
Institut für Informatik
Fachgebiet Programmiersprachen und Compilerbau

Masterarbeit



Brandenburgische
Technische Universität
Cottbus - Senftenberg

Modellierung und Untersuchung von Constraint-Problemen für Quanten-Annealer

Tristan Nuck

Matrikelnummer: 3909798

Studiengang: Informatik

Betreuer :

Prof. Dr. rer. nat. habil. Petra Hofstedt

Dr. rer. nat. Sven Löffler

Inhaltsverzeichnis

1	Einleitung	7
1.1	Motivation	7
1.2	Aufgabenstellung	8
1.3	Aufbau der Arbeit	9
2	Grundlagen CP und Quantenmechanik	11
2.1	Constraint Satisfaction und Optimization	11
2.1.1	Constraint Satisfaction-Probleme	12
2.1.2	Constraint Optimization-Probleme	13
2.1.3	Solver	14
2.2	Grundlagen der Quantenmechanik	15
2.2.1	Motivation	15
2.2.2	Wellenfunktionen	19
2.2.3	Bra-Kets und Qubits	21
2.2.4	Eigenwerte und Eigenzustände	23
2.2.5	Operatoren	24
2.3	Quantum Adiabatic Computation	26
2.4	Verwandte Arbeiten	31
3	Quantum Annealing	33
3.1	Der Quanten-Annealer	33
3.1.1	Quantentechnologien	33
3.1.2	Qubits	35
3.1.3	Bias und Coupling von Qubits	37
3.1.4	Vom Ising-Modell zur Qubo	39
3.1.5	Quantum Adiabatic Theorem in der Praxis	41
3.2	Lösen von CSPs auf einem Quanten-Annealer	46
3.2.1	Graph-Färbung als Qubo-Modell	46
3.2.2	Rotating Rostering-Problem als Qubo-Modell	48

3.2.3	Embedding und Anneal-Parameter	55
3.2.4	Auswertung der Anneal-Ergebnisse	63
4	Modellierung	69
4.1	Variablenkodierung	69
4.1.1	One Hot-Kodierung	70
4.1.2	Binärkodierung	71
4.1.3	Domain Wall-Kodierung	72
4.1.4	Vergleich	73
4.2	Umwandlung von Constraintproblemen	76
4.3	Aufstellen von Qubo-Ausdrücken	78
4.3.1	Allgemeingültige Umformungen	78
4.3.2	Qubo aus Wahrheitswertetabellen erstellen	79
4.3.3	Qubo-Ausdrücke erstellen mithilfe von CSPs	86
4.4	Überführung zum Ising-Modell	91
4.4.1	Ising-Ausdrücke erzeugen	91
4.5	Optimierung von Ising-Ausdrücken	92
4.5.1	Energielücke	93
4.5.2	Koeffizienten-Domäne	95
4.5.3	Anzahl der Variablen und Kanten	98
5	Evaluation	101
5.1	Vergleich Qubo/Ising-Finder	101
5.1.1	Beispielproblem	101
5.1.2	Auswertung	102
5.2	Graphfärbung	104
5.2.1	Grapherzeugung	105
5.2.2	Auswertung Kodierung	105
5.2.3	Auswertung Anneal-Zeit	106
5.2.4	Auswertung Anneal Schedule	111
5.2.5	Fazit	114
5.3	Rotating Rostering	115
5.3.1	Problem	115
5.3.2	Auswertung Kodierung	116
5.3.3	Auswertung Anneal-Zeit und Schedule	118
5.3.4	Fazit	120
5.4	COPs	121
5.4.1	Sonet-Problem	121
5.4.2	Auswertung	122
5.4.3	Fazit	126

6	Zusammenfassung, Fazit und Ausblick	127
6.1	Zusammenfassung	127
6.2	Fazit	129
6.3	Ausblick	130
7	Anhang	133
7.1	One Hot-Kodierung	133
7.2	Qubo-Ausdrücke für den Vergleich der Transformationstechniken . . .	134
7.3	Kodierungen des Graphfärbungsproblems	135
7.4	Anneal-Ergebnisse des Anneal-Zeit Tests	137
7.5	Anneal-Ergebnisse des Anneal Schedule-Tests	139
7.6	Domain Wall-Kodierung des Rotating Rostering	139
7.7	Anneal-Ergebnisse mit vier Angestellten	144
7.8	Sonet-Problem Modellierung	147

Kapitel 1

Einleitung

In diesem Kapitel wird zunächst in Abschnitt 1.1 eine kleine Motivation rund um das Thema der Quanten-Computer und deren potentiellen Nutzen präsentiert. Anschließend ist in Abschnitt 1.2 die Aufgabenstellung dieser Arbeit beschrieben, gefolgt von dem Aufbau und einer Übersicht der einzelnen Kapitel in Abschnitt 1.3.

1.1 Motivation

Durch die immer weiter voranschreitende Globalisierung, neue technische Innovationen und eine ständig expandierende Wirtschaft ergeben sich immer komplexer werdende Probleme, die gelöst werden müssen. Dazu zählen unter anderem logistische Aufgaben, Raumplanung oder auch Netzwerkprobleme. Natürlich ist jedes Unternehmen daran interessiert, für diese Probleme optimale Lösungen zu finden, die zusätzlich zu gegebenen Randbedingungen noch eine hohe Wirtschaftlichkeit garantieren.

Viele dieser Probleme können mithilfe der Constraint-Programmierung gelöst werden. Dabei liegt der Fokus auf der Problembeschreibung und weniger auf dem Finden eines konkreten Lösungsverfahrens. Somit können Probleme wie das Erstellen eines Schichtplans in deklarativer Form beschrieben werden und ein Solver übernimmt danach das Lösen dieses Problems. So lassen sich viele verschiedene kombinatorische Probleme, für die kein effizienter Lösungsalgorithmus existiert, lösen. Die verschiedenen Solver können sich dabei auf bestimmte Constraint-Probleme spezialisieren und versuchen, durch verschiedene Heuristiken und andere Optimierungstechniken eine optimale Lösung in einer akzeptablen Zeit zu finden.

Dabei arbeiten die Solver aber im Allgemeinen nicht effizient und für besonders große Probleminstanzen kann dies zu langen Rechenzeiten führen, die in der Praxis nicht

akzeptabel sind. Natürlich entwickeln sich sowohl die Solver, Modellierungstechniken als auch die Hardware, auf der diese Programme ausgeführt werden, stetig weiter. Gerade bei der Hardware sieht es aber aktuell so aus, dass eine physikalische Grenze der Fertigungstechnologie erreicht wird. Auch die Software und Algorithmen können sich nur begrenzt weiterentwickeln.

Hier bieten die Quantencomputer einen neuen Ansatz zum Lösen solcher Probleme. Sie sind nicht an die Gesetze der klassischen Physik gebunden und nutzen die besonderen Eigenschaften der Quantenmechanik aus. Sowohl in der Hardware als auch in der Software ergeben sich so neue Möglichkeiten, um kombinatorische Probleme effizienter lösen zu können. Der Fokus soll dabei auf dem Quanten-Annealer liegen, der auf dem Quantum Adiabatic Theorem basiert und in der Lage ist, verschiedenste Constraint-Probleme zu lösen.

Quantencomputer stehen noch am Anfang ihrer Entwicklung und haben mit vielen Limitationen zu kämpfen. In der Zukunft könnten sie sich aber zu wertvollen Werkzeugen zum Lösen von Constraint-Problemen oder auch im Einsatz anderer praxisrelevanter Felder erweisen. Um mit dieser neuen Technologie korrekt und effizient umgehen zu können, ist es wichtig, ihren Aufbau und ihre interne Funktionsweise zu verstehen. Dazu werden in dieser Arbeit der Aufbau und die Funktionsweise eines Quanten-Annealers genauer beleuchtet. Dabei ist das Ziel, die nötigen Schritte und Hürden, die beim Lösen von realen Constraint-Problemen auf dem Quanten-Annealer auftreten, zu veranschaulichen.

1.2 Aufgabenstellung

Eine Vielzahl von Problemen, die im Alltag auftreten, lässt sich als Constraint-Problem formulieren, z.B. die Schichtplanung in einem Krankenhaus oder die Routenplanung bei der Post. In der Regel sind derartige Probleme lediglich mit einem hohen Aufwand exakt lösbar. Aufgrund der hohen praktischen und wirtschaftlichen Relevanz existiert eine Vielzahl von Solvern, die versuchen, diese Probleme mithilfe von verschiedenen Heuristiken und optimierten Methoden effizient zu lösen. All diese Solver arbeiten auf klassischer Hardware und sind somit an die Gesetze der klassischen Physik gebunden.

Inzwischen steht mit Quantencomputern eine neue Form der Hardware zur Verfügung, die auf den Besonderheiten der Quantenmechanik wie Superposition und Verschränkung (Entanglement) basiert und somit möglicherweise eine alternative Lösungsmethode für die genannten Probleme bietet. Dabei erfährt der Quantum-Annealer, der auf dem Quantum Adiabatic-Theorem basiert, besondere Aufmerksamkeit, da er in der Lage ist, Constraint-Probleme zu lösen.

Ziel der Arbeit ist es verschiedene Constraint-Probleme (CSPs und COPs) so zu modellieren, dass sie auf dem Quanten-Annealer lösbar sind. Hierbei sollen verschiedene Modellierungsansätze untersucht und evaluiert werden. Dabei soll insbesondere die Entwicklung der benötigten Lösungszeiten beim Hochskalieren der Probleme untersucht werden. Neben der Modellierung der Probleme selbst ist auch die korrekte Anpassung der Annealing-Parameter, wie Annealing-Zeit und Schedule, von entscheidender Bedeutung. Auch diese Parameter sollen untersucht und verschiedene Konfigurationen verglichen und evaluiert werden.

1.3 Aufbau der Arbeit

Die nachfolgende Arbeit gliedert sich in fünf Kapitel zuzüglich eines Anhangs.

Das Kapitel 2 teilt sich in zwei thematische Abschnitte auf. Im ersten Abschnitt werden die Grundlagen der Constraint-Programmierung eingeführt. Dabei werden vor allem Begriffe wie Constraint Satisfaction-Problem (CSP), Constraint Optimization-Problem (COP) und Solver definiert. Der zweite Abschnitt thematisiert die Quantenmechanik. Zunächst werden zwei motivierende Beispiele zur Quantenmechanik präsentiert. Anschließend werden grundlegende Begriffe wie Wellenfunktionen, Dirac-Notation, Eigenwerte und Operatoren eingeführt. Abschließend wird auf das zugrunde liegende Quantum Adiabatic Theorem und verwandte Arbeiten eingegangen.

Ein Überblick über das Quanten-Annealing wird im Kapitel 3 präsentiert. Dazu werden zunächst der Aufbau und die Funktionsweise des Quanten-Annealers erklärt. Danach werden zwei motivierende Beispiele für den Annealer modelliert, ausgeführt und ausgewertet. Dabei wird die Auswertungsmethodik für den Quanten-Annealer genauer beleuchtet.

In Kapitel 4 wird ausführlich auf die Modellierung von Problemen für einen Quanten-Annealer eingegangen. Dazu wird auf drei verschiedene Variablenkodierungen und anschließend auf die Umwandlung von Constraints in Qubo- und Ising-Ausdrücke eingegangen. Weiterhin werden verschiedene Techniken zum Erzeugen von Qubo-Ausdrücken präsentiert und erklärt. Abschließend wird die Relevanz von geeigneten Ising-Ausdrücken aufgezeigt und wie diese optimiert werden können.

Im letzten Kapitel 5 wird das in der Arbeit entwickelte Werkzeug zum Finden von Qubo- und Ising-Ausdrücken evaluiert und mit bisherigen Techniken verglichen. Außerdem werden zwei Constraint-Probleme, das Rotating Rostering-Problem und das Graphfärbungsproblem, untersucht und die Möglichkeiten des Quanten-Annealers, diese Probleme zu lösen, ausgewertet. Zuletzt wird mit dem Sonet-Problem noch ein Constraint-Optimierungsproblem evaluiert.

Im letzten Kapitel 6 wird die Arbeit zunächst zusammengefasst. Anschließend wird ein Fazit bezüglich des Quanten-Annealers gezogen und ein Ausblick auf weitere Arbeiten und offene Fragen präsentiert.

Kapitel 2

Grundlagen Constraint-Probleme und Quantenmechanik

In diesem Kapitel werden die Grundlagen im Umgang mit Constraint-Problemen und der Quantenmechanik eingeführt. Dabei wird im Abschnitt 2.1 eine kleine Motivation zur Nutzung der Constraintprogrammierung geben und grundlegenden Begriffe im Umgang mit Constraint Satisfaction-Problemen (CSP) und Constraint Optimization-Problemen (COP) und deren Solver eingeführt. Die zugrunde liegende Prädikatenlogik wird nicht eingeführt und ein Wissen über diese wird vorausgesetzt. Mehr Informationen zur Prädikatenlogik können aus [HW07] entnommen werden. Im Abschnitt 2.2 wird das Thema der Quantenmechanik motiviert und wichtige Begriffe wie Wellenfunktionen, Dirak-Notation, Eigenwerte und Operatoren eingeführt. Anschließend wird in Abschnitt 2.3 das Thema der Quantum Adiabatic Computation und das Quantum Adiabatic Theorem eingeführt und hergeleitet. Abschließend werden in Abschnitt 2.4 verwandte Arbeiten erwähnt.

2.1 Constraint Satisfaction und Optimization

Viele Probleme, die im Alltag auftreten, lassen sich als Constraint-Problem formulieren. Constraints sind dabei Einschränkungen bzw. Bedingungen die an Variablen gestellt werden. Dadurch lassen sich auf elegante und einheitlicher Weise verschiedenste Probleme modellieren, die danach von schon existierenden Solvern gelöst werden können. Dazu gehören zum Beispiel Schichtpläne, Raumverwaltungen oder die Routenplanung für Logistikunternehmen. Alle diese Probleme können mithilfe von Constraintprogrammierung modelliert und gelöst werden. Das Vorgehen ist dabei deklarativer Natur. Der Fokus liegt auf der Problembeschreibung mithilfe von Va-

riablen, Domänen und Constraints. Der Nutzer muss sein Problem "nur" korrekt modellieren und ein Solver übernimmt dann, das automatische Finden einer Lösung. Dieses Vorgehen bietet viele Vorteile. Zum einen wird dem Nutzer das Problem der Lösungsfindung abgenommen. Außerdem sind Modellierung und Solver voneinander getrennt. Dadurch können beide Komponenten individuell entwickelt und verbessert werden. Mittlerweile gibt es eine Vielzahl von Constraint-Bibliotheken und Anwendungen, die die Modellierung vereinfachen. Diese unterstützen einen großen Katalog an verschiedenen Constraints, z. B. arithmetische, Zähl- oder aussagenlogische Constraints. In den nächsten Abschnitten 2.1.1 und 2.1.2 werden die Begriffe rund um CSP und COP genau definiert.

2.1.1 Constraint Satisfaction-Probleme

Die nachfolgenden Definitionen orientieren sich an [RN20]. Ein CSP wird als Tripel $P = (V, D, C)$ aus Variablen $V = \{v_1, v_2, \dots, v_n\}$, Domänen $D = \{D_1, D_2, \dots, D_n\}$ und Constraints $C = \{c_1, c_2, \dots, c_m\}$ dargestellt.

Definition 2.1.1 (Constraint Satisfaction-Problem). Ein Constraint Satisfaction-Problem kurz CSP ist ein Tripel $P = (V, D, C)$ wobei:

- $V = \{v_1, v_2, \dots, v_n\}$ die Menge der Variablen,
- $D = \{D_1, D_2, \dots, D_n\}$ die Menge der Domänen,
- $C = \{c_1, c_2, \dots, c_m\}$ die Menge der Constraints ist. Δ

Zu jeder Variable v_i gehört die Domäne D_i , welche den Wertebereich vorgibt. Die Constraints c_j sind Relationen über den Variablen aus V und geben an, welche Belegungen der Variablen erlaubt sind.

Definition 2.1.2 (Constraint). Ein Constraint $c_i = R_i(v_j, v_k, \dots, v_l)$ ist eine Relation über dem Kreuzprodukt der Domänen der beteiligten Variablen $R_i \subseteq D_j \times D_k \times \dots \times D_l$. Ein Tupel in R_i stellt eine erfüllende Belegung des Constraints dar. Δ

Constraints können explizit durch das Angeben aller Tupel der Relation oder implizit durch eine Formel beschrieben werden.

Beispiel 2.1.1 (XOR-CSP). Gegeben sind zwei boolesche Variablen $V = \{v_1, v_2\}$ mit den Domänen $D_1 = D_2 = \{0, 1\}$. Als Constraint wählen wir die XOR-Relation $c_1 = \{(0, 1), (1, 0)\}$. Um unser CSP zu lösen, müssen die Variablen v_i so mit Werten aus ihren Domänen D_i belegt werden, dass kein Constraint aus C verletzt wird. In diesem Fall ist z.B. $v_1 = 1$ und $v_2 = 0$ eine erfüllende Belegung. $\Diamond$

Definition 2.1.3 (Variablenbelegung). Eine Variablenbelegung eines CSPs ist eine Funktion $\phi : V \rightarrow D$, die jeder Variable x_i einen Wert d_j aus ihrer Domäne D_i zuweist. $\triangle$

Besonders relevant sind Variablenbelegungen die alle Constraints eines CSPs gleichzeitig erfüllen. Solche Belegungen werden auch Lösungen bzw. erfüllende Belegungen genannt.

Definition 2.1.4 (Lösung). Gegeben sei ein CSP $P = (V, D, C)$. Eine Lösung l von P ist eine Variablenbelegung ϕ , sodass alle Constraints C gleichzeitig erfüllt sind. $\triangle$

Die Domänen für CSPs können sich aus den unterschiedlichsten Mengen zusammensetzen. Klassische Beispiele sind die boolesche Domäne $D = \{0, 1\}$, wobei 0 für *False* und 1 für *True* steht oder endliche Domänen, die entweder eine endliche Teilmenge der natürlichen Zahlen oder eine Ansammlung von Objekten umfasst. Natürlich können auch die reellen Zahlen als Domäne dienen. In dieser Arbeit werden jedoch ausschließlich boolesche oder endliche Domänen betrachtet.

Oftmals wird nicht nur eine beliebige erfüllende Belegung gesucht, sondern eine besonders gute. Die Güte einer Lösung hängt dann von einer Funktion f über den Variablen des CSPs ab, die es zu minimieren oder maximieren gilt. Solche Constraint Optimization Probleme werden im folgenden Abschnitt 2.1.2 definiert.

2.1.2 Constraint Optimization-Probleme

Die Constraint Optimization-Probleme kurz COP verhalten sich ähnlich wie die CSPs. Gegeben sei ein CSP $P = (V, D, C)$, analog zur Definition 2.1.1. Unser COP P' lässt sich leicht aus P erweitern. Wir fügen unserem Problem eine Zielvariable v_z und dazugehörige Zielfunktion f hinzu.

Definition 2.1.5 (Constraint Optimization-Problem [HW07]).

Ein Constraint Optimization-Problem ist ein Vier-Tupel $P = (V, D, C, f)$ wobei:

- $V = \{v_1, v_2, \dots, v_n\} \cup \{v_z\}$ die Menge der Variablen,
- $D = \{D_1, D_2, \dots, D_n\} \cup \{D_z\}$ die Menge der Domänen,
- $C = \{c_1, c_2, \dots, c_m\}$ die Menge der Constraints,
- $f : D_1 \times D_2 \times \dots \times D_n \rightarrow D_z$ die Zielfunktion ist. $\triangle$

Ziel ist es, eine Belegung der Variablen v_1 bis v_n so zu wählen, dass alle Constraints C erfüllt sind und die Zielfunktion f minimiert bzw. maximiert wird. Die Variable v_z erhält hier den Wert der Zielfunktion und ihre Domäne $D_z = \{0, -1, -2, -3\}$

ist meist eine Teilmenge der ganzen oder reellen Zahlen. Das vorherige *XOR*-CSP-Beispiel lässt sich durch das Einführen der Zielfunktion $f(v_1, v_2) = -2 \cdot v_1 - 1 \cdot v_2$ in ein COP überführen. Soll die Zielfunktion minimiert werden, dann ist nur noch die Belegung $(1, 0)$ eine optimale Lösung mit Zielfunktionswert -2 . Die Belegung $(0, 1)$ erfüllt zwar ebenfalls alle Constraints, liefert aber mit -1 einen schlechteren Zielfunktionswert. Solver können solche COPs und auch CSPs automatisch lösen. Ein kleiner Überblick über diese folgt im nächsten Abschnitt 2.1.3.

2.1.3 Solver

Constraint-Probleme können je nach Anwendungsfall stark unterschiedlich aussehen. Viele Probleme lassen sich mit endlichen Domänen beschreiben, manchmal sind jedoch auch unendliche Domänen erforderlich. Auch die Art der Constraints kann stark variieren. Aus diesem Grund existieren spezielle Solver, die besonders gut mit bestimmten Constraint-Problemen umgehen können. Zum Lösen von linearen arithmetischen Constraints kann z. B. ein Simplex-Solver genutzt werden. Das Simplex-Verfahren wird in [Hur09] genauer beschrieben. Eine Implementierung des Verfahrens ist z.B. in CPLEX [cpl] zu finden.

Um mit unendlichen Domänen umgehen zu können, bedarf es anderer Techniken als für endliche Domänen. Es existieren aber auch Solver, die mit einer großen Menge von Constraints auf verschiedenen Domänen umgehen können. Das Vorgehen für endliche Domänen ist dabei meist ähnlich und orientiert sich am folgenden Verfahren. Mithilfe von Tiefensuche werden verschiedene Variablenbelegungen getestet und nach jeder neuen festgelegten Variable ein Konsistenz-Schritt durchgeführt. Dabei werden offensichtlich nicht mehr erfüllbare Belegungen sofort ausgeschlossen (Propagation), um diese nicht später in der Suche berücksichtigen zu müssen. Wird eine Sackgasse erreicht, wird mithilfe von Backtracking zurückgesprungen und eine neue Belegung ausprobiert. Eine ausführliche Beschreibung von Konsistenztechniken und Backtracking ist in [HW07] zu finden. Dieses Verfahren wird z.B. in Solvern wie CHOCO [cho] implementiert.

All diese Verfahren lösen zwar ein Constraint-Problem, sind aber im Allgemeinen nicht effizient. Gerade für größere Probleminstanzen kann es hier zu langen Lösungszeiten kommen. Einen alternativen Ansatz liefert hier das Quanten-Annealing. Mithilfe von quantenmechanischen Prozessen könnte es vielleicht schneller Constraint-Probleme lösen als klassische Computer und Verfahren.

2.2 Grundlagen der Quantenmechanik

Die theoretischen Grundlagen der Quantenmechanik sind vor allem zum Verständnis des Aufbaus und der Funktionsweise des Quantencomputers relevant. Dabei kann ein besseres Verständnis über die Funktionsweise zu einer effizienteren Nutzung führen.

2.2.1 Motivation

Die Physik versucht mithilfe der Mathematik die Regeln und Gesetze unserer Welt zu verstehen und zu formalisieren. Dabei ist die klassische Physik für uns im Alltag wohl am wichtigsten. Sie beschreibt z.B. wie sich makroskopische Objekte bewegen und welche Kräfte auf welche Weise wirken. Dazu zählen unter anderem die Flugbahn eines geworfenen Balls oder die Beschleunigung eines Fahrrads, das den Berg hinunter fährt. All diese Prozesse lassen sich mit klassischer Physik beschreiben und erklären. Hier stellt sich die Frage, ob diese Beschreibung schon ausreichend ist. Diese Frage lässt sich aber schnell mit 'Nein' beantworten. Bei den Geschwindigkeiten und makroskopischen Energiegrößen, die uns im Alltag begegnen, funktioniert unsere klassische Physik gut. Bewegen sich Objekte allerdings mit nahezu Lichtgeschwindigkeit, treten relativistische Phänomene auf, die sich klassisch nicht mehr erklären lassen. Etwas Ähnliches lässt sich bei mikroskopisch kleinen Systemen, wie einzelnen Elektronen beobachten. Hier treten quantenmechanische Phänomene auf, um die es in diesem Abschnitt gehen soll.

Beispiel 2.2.1 (Doppelspalt). Der Doppelspalt-Versuch ist ein weitverbreitetes Experiment, das gerne im Physik-Unterricht gezeigt wird. Der Aufbau, zu sehen in Abbildung 2.1, ist denkbar einfach. Eine Lichtquelle (meist einen Laser) wird auf eine Wand mit zwei dünnen Schlitzen gerichtet. Hinter dieser Wand befindet sich ein Schirm, auf dem das Licht nach dem Passieren der Schlitze zu sehen ist. Fällt das Licht nun durch beide Spalte, sehen wir ein Interferenzmuster, anstatt, wie naiv zu erwarten, zwei Streifen an den Stellen der Spalten. Dies lässt sich klassisch mithilfe von Welleneigenschaften erklären.

Licht ist eine Welle. Wenn diese auf beide Spalte trifft, entstehen an diesen Stellen neue Wellen, die sich gleichförmig im Raum ausbreiten. Da die Spalte einen gewissen Abstand voneinander haben, müssen die Wellen unterschiedliche Distanzen zurücklegen, um die gleichen Punkte auf dem Anzeigeschirm zu erreichen. Dadurch treffen in bestimmten Abständen Wellenberge auf Wellentäler, was zu destruktiver Interferenz führt. Analog treffen an anderen Punkten zwei Wellenberge oder zwei Wellentäler aufeinander, was zu konstruktiver Interferenz führt. Somit lässt sich das regelmäßige Interferenzmuster auf dem Schirm erklären.

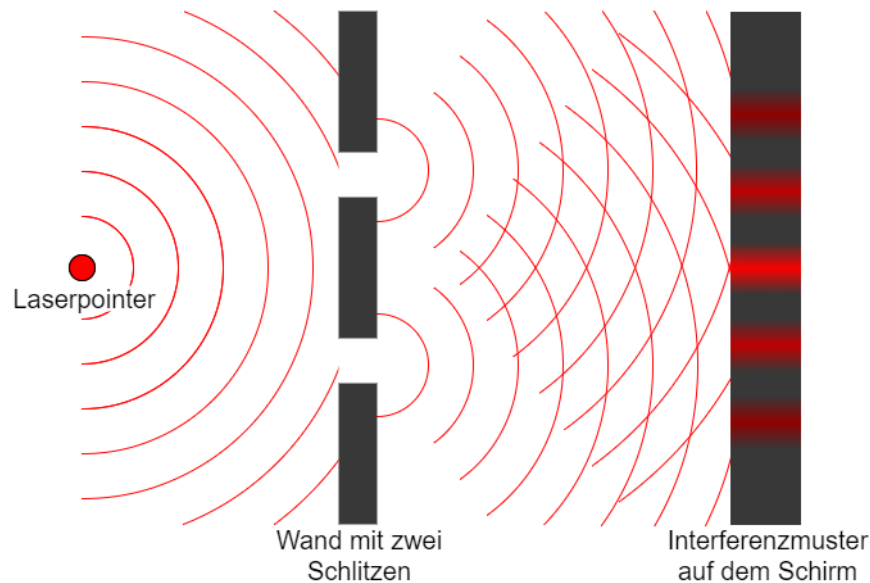


Abbildung 2.1: Aufbau und Ergebnis des Doppelspaltexperiments

Dieses Experiment lässt sich auch auf eine andere Weise durchführen. Anstelle eines kontinuierlichen Lichtstrahls, schicken wir einzelne Elektronen durch die Spalte. Dabei wird jedes Elektron nacheinander durch den Doppelspalt geschickt. Auf dem Schirm werden dann die Landepunkte der Elektronen dargestellt (Abbildung 2.2). Zu erwarten wären zwei Streifen hinter unserem Doppelspalt. Tatsächlich beobachten wir das gleiche Interferenzmuster wie bei dem Lichtstrahl. Dieses Phänomen lässt sich nicht so einfach mit der klassischen Physik erklären. Das Elektron sollte sich entweder durch den linken oder rechten Spalt hindurch bewegen. Hier kann die Quantenmechanik helfen. Anstatt nur den Zustand "linker Spalt" und "rechter Spalt" zu betrachten, kann sich das Elektron auch in einer Überlagerung (Superposition) dieser Zustände befinden. Es durchquert sowohl den linken als auch den rechten Spalt "gleichzeitig". Natürlich benötigt eine exakte Erklärung des Interferenzmusters noch eine ausführlichere mathematisch korrekte Beschreibung (siehe z.B. [Bea12]). Dies soll aber erst einmal nur einen kleinen Einblick in die Quantenmechanik liefern und zeigen, dass Teilchen auch Welleneigenschaften aufweisen. $\diamond$

Beispiel 2.2.2 (Polarisationsfilter). Ein weiteres Experiment zur Demonstration der Quantenmechanik lässt sich mit Polarisationsfiltern durchführen. Licht, also elektromagnetische Wellen, können klassisch als zwei senkrecht zueinander stehende, oszillierende elektrische und magnetische Welle (siehe Abbildung 2.3) beschrieben werden. Die Ausrichtung der elektrischen Welle gibt die Polarisationsrichtung an. Ein

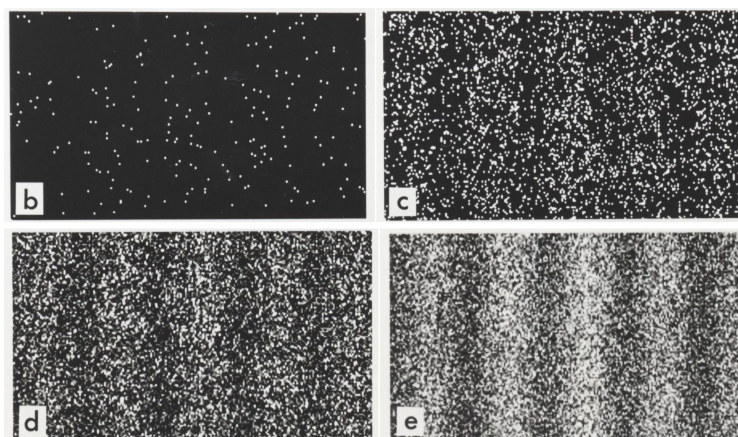


Abbildung 2.2: Ergebnis des Doppelspaltexperiments mit Elektronen. Weiße Punkte stellen die Einschlagsorte der Elektronen auf dem Schirm dar. Die Teilbilder stellen den zeitlichen Verlauf des Experimentes dar. Bei b wurden bisher nur wenige Elektronen durch den Doppelspalt geschickt und bei Teilbild e die meisten. [Bel12]

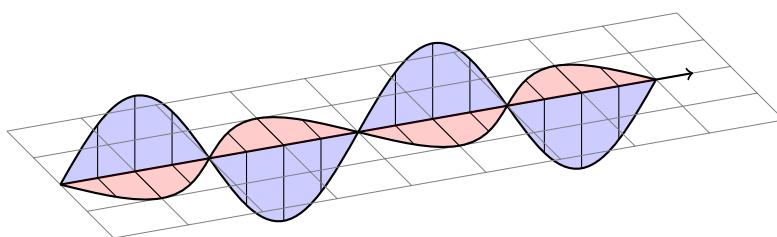


Abbildung 2.3: Vertikal polarisierte elektromagnetische Welle die sich im Raum bewegt. Der schwarze Pfeil gibt die Ausbreitungsrichtung an. Rot ist die Stärke des magnetischen Feldes und blau die Stärke des elektrischen Feldes.

horizontaler Polarisationsfilter absorbiert vertikal polarisiertes Licht und lässt somit nur horizontales Licht passieren. Elektromagnetische Wellen können nicht nur vertikal oder horizontal polarisiert sein, sondern auch in beliebigen Ausrichtungen dazwischen. Dies kann als Überlagerung von einer vertikal und einer horizontal polarisierten Welle interpretiert werden. Der horizontale Polarisationsfilter absorbiert in diesem Fall nur den vertikalen Anteil der Welle.

Wenn ein Polarisationsfilter mit horizontaler Ausrichtung vor eine Lichtquelle gehalten wird, werden 50% der Lichtwellen absorbiert (Abbildung 2.4a). Legen wir dahinter einen Polarisationsfilter mit vertikaler Ausrichtung, werden auch horizontale elektrische Wellen absorbiert und es kommt kein Licht mehr durch den Filter (Abbildung 2.4b). Soweit stimmt alles mit unserer klassischen Betrachtungsweise überein. Le-

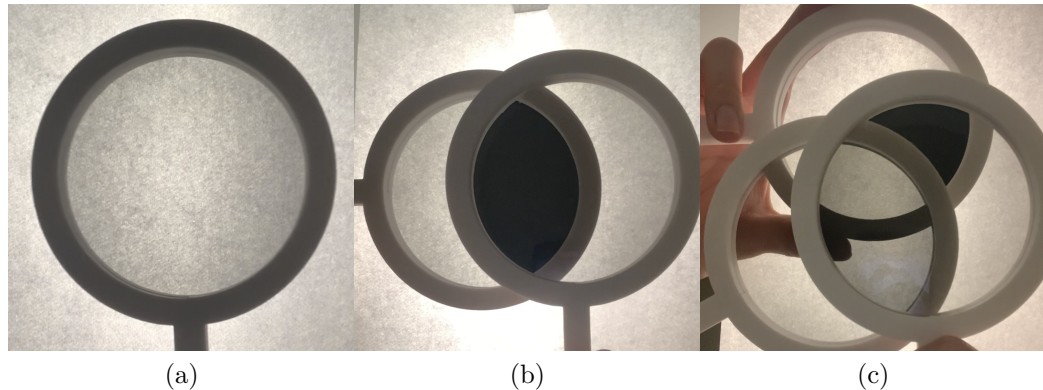


Abbildung 2.4: (a) Einzelner Polarisationsfilter, der 50% des Lichts absorbiert. (b) Zwei orthogonale Polarisationsfilter, die Licht vollständig absorbieren. (c) Drei Polarisationsfilter, die nun mehr Licht durchlassen als die zwei orthogonalen Filter.

gen wir jetzt aber einen weiteren Filter, der diagonal polarisiertes Licht filtert (45° zu beiden Filtern gedreht), zwischen die beiden anderen Filter, so können wir etwas Merkwürdiges beobachten: Die Lichtquelle erscheint heller, es kommt also mehr Licht durch alle drei Filter als nur durch zwei (Abbildung 2.4c). Klassisch ist dieses Phänomen mit dem eben erwähnten Wellenbild nicht zu erklären und es bedarf der Quantenmechanik.

Photonen, die Teilchen des Lichts, existieren nur in diskreten Energiepaketen. Sie sind somit quantisiert und es können keine halben Photonen existieren. Ein Polarisationsfilter kann entweder das gesamte Photon absorbieren oder es durchlassen. Außerdem können Photonen sich nicht nur im Zustand "vertikal" oder "horizontal" polarisiert befinden, sondern auch in Überlagerungen dieser. Solche Superpositionszustände beschreiben beispielsweise diagonal polarisiertes Licht. Trifft ein Photon, das sich gleichermaßen in einem vertikalen sowie in einem horizontalen Zustand befindet, auf einen horizontalen Polarisationsfilter, besteht eine 50%-Wahrscheinlichkeit, dass das Photon diesen Filter passieren wird. Nach dem Durchschreiten des Filters befindet sich das Photon im vertikal polarisierten Zustand, wäre dies nicht der Fall, hätte es den Filter nicht passieren können. Trifft das Photon nun auf einen vertikalen Polarisationsfilter, besteht keine Chance mehr für das Photon diesen zu passieren. Legen wir allerdings unseren diagonalen Polarisationsfilter dazwischen (Abbildung 2.4c), existiert eine Chance, dass das Photon im horizontalen Zustand diesen durchdringt. Danach befindet sich das Photon wieder in einer Überlagerung aus der horizontalen und vertikalen Polarisation. Dadurch hat es wiederum eine bestimmte Wahrscheinlichkeit, auch den letzten vertikalen Filter zu passieren. Dieses Experiment zeigt,

dass Wellen auch Teilcheneigenschaften aufweisen und diese nötig sind, um bestimmte Phänomene zu beschreiben. Im nachfolgenden Abschnitt 2.2.2 wird ein besonderer Fokus auf die Darstellung von Teilchen als Wellen gelegt. $\diamond$

2.2.2 Wellenfunktionen

Das Doppelspaltexperiment zeigt, dass Teilchen auch Welleneigenschaften besitzen und sie nicht fest definiert nur an einem Ort existieren. Aus diesem Grund wird in der Quantenmechanik jedes Teilchen durch eine Wellenfunktion ψ beschrieben. Dabei ist ψ von einem Ort x und Zeit t abhängig. Ein typisches Beispiel ist die Wellenfunktion eines Elektrons im Potenzialtopf.

Beispiel 2.2.3 (Potenzialtopf). Der Potenzialtopf wird durch eine unendlich hohe Potenzialbarriere (z.B. einem unendlich starken elektrischen Feld) links von der Koordinate 0 und einer unendlich hohen Potenzialbarriere rechts von der Koordinate L beschrieben. Innerhalb dieses Topfes liegt das Potenzial bei 0. Die allgemeine zeitabhängige Wellengleichung eines Elektrons, gefangen in diesem Topf sieht wie folgt aus [Zim05]:

$$\psi(x, t) = a \cdot e^{-i \cdot E_n \cdot t / \hbar} \cdot \sin\left(\frac{n \cdot \pi \cdot x}{L}\right) \quad \text{wobei} \quad E_n = \frac{n^2 \cdot \pi^2 \cdot \hbar^2}{2 \cdot m \cdot L^2} \quad (2.1)$$

Der Faktor n ist eine Natürliche Zahl und gibt an, in welchem diskreten Energiezustand sich das Teilchen mit der Masse m befindet. $\hbar$ ist die reduzierte Planck-Konstante. Das a stellt einen Normierungsfaktor da und muss so gewählt werden, dass das folgende Integral 1 ergibt.

$$\int_{-\infty}^{+\infty} |\psi(x, t)|^2 dx = 1 \quad (2.2)$$

Dieses Integral gibt an, mit welcher Wahrscheinlichkeit sich das Teilchen im gesamten Raum aufhält. Da das Teilchen nicht verschwinden kann, muss dieses Integral 1 ergeben. Wenn nur der Zeitpunkt $t = 0$ betrachtet wird, die Zeitabhängigkeit also vernachlässigt wird, vereinfacht sich die Funktion zu:

$$\psi(x) = \sqrt{\frac{2}{L}} \cdot \sin\left(\frac{n \cdot \pi \cdot x}{L}\right) \quad (2.3)$$

Das a lässt sich in der vereinfachten Zeitunabhängig Form leicht durch die Gleichung 2.2 bestimmen und entspricht $\sqrt{\frac{2}{L}}$. In Abbildung 2.5 sind die Energieniveaus für $n = 1$ bis $n = 3$ dargestellt.

Diese Funktion $\psi(x)$ beschreibt zwar das Elektron, liefert aber noch keine in der Realität messbare Größe. Erst durch das Bilden des Betragsquadrats der Wellenfunktion

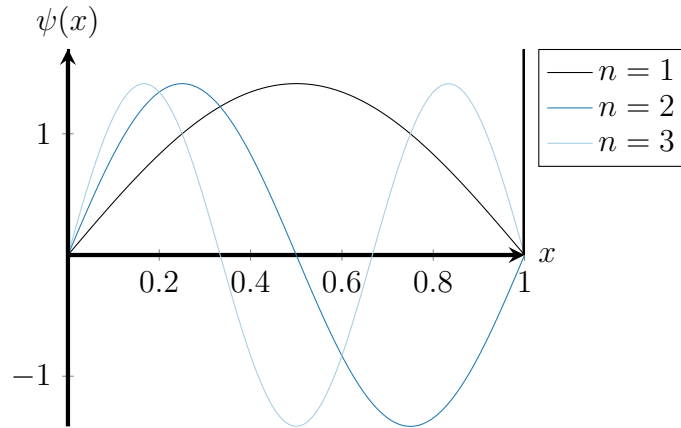


Abbildung 2.5: Die drei niedrigsten Energiezustände eines Elektrons im unendlichen Potenzialtopf der Länge $L = 1\text{m}$.

$|\psi|^2$ erhalten wir eine messbare Größe, die Aufenthaltswahrscheinlichkeit des Elektrons. Dies ist in Abbildung 2.6 wieder für $n = 1$ bis $n = 3$ dargestellt.

Es ist allerdings zu beachten das $|\psi|^2$ nicht direkt die Aufenthaltswahrscheinlichkeit an der Stelle x angibt. Die Wahrscheinlichkeit das Teilchen in einem bestimmten Raumbereich anzutreffen, entspricht der Fläche unter der Funktion $|\psi|^2$. Da ein Teilchen sich irgendwo im Raum befinden muss, ergibt sich die Fläche unter der kompletten Funktion zu 1. Um die Wahrscheinlichkeit in einem vorgegebenen Bereich des Potenzialtopfes zu bestimmen, muss $|\psi|^2$ somit in diesem Bereich integriert werden. Die Wahrscheinlichkeit, dass sich das Elektron in der Mitte des Topfes, also im Bereich von $x = 0.4$ bis $x = 0.6$, befindet, kann für $n = 1, L = 1$ wie folgt berechnet werden:

$$\int_{0.4}^{0.6} |\psi(x)|^2 dx = \int_{0.4}^{0.6} \left| \sqrt{2} \cdot \sin(\pi \cdot x) \right|^2 dx \approx 0.387 \quad (2.4)$$

Das Integral von 0 bis 1 muss 1 ergeben. Solch eine Annahme ist sinnvoll, da das Elektron irgendwo im Potenzialtopf sein muss. Elektronen können sich aber auch in Überlagerungen (Superposition) von Zuständen, wie beim Doppelspaltexperiment beobachtet, befinden. Durch das einfache Addieren zweier Wellenfunktionen lässt sich so eine Superposition mathematisch beschreiben. Die einzige Bedingung an solche Zustände ist, dass das Integral über den gesamten Raum ebenfalls 1 ergeben muss. Die Überlagerung der beiden niedrigsten Zustände $n = 1$ und $n = 2$ könnte wie folgt realisiert werden:

$$\psi' = \frac{1}{\sqrt{2}} \cdot \psi_1 + \frac{1}{\sqrt{2}} \cdot \psi_2 = \sin(\pi \cdot x) + \sin(\pi \cdot x \cdot 2) \quad (2.5)$$

◇

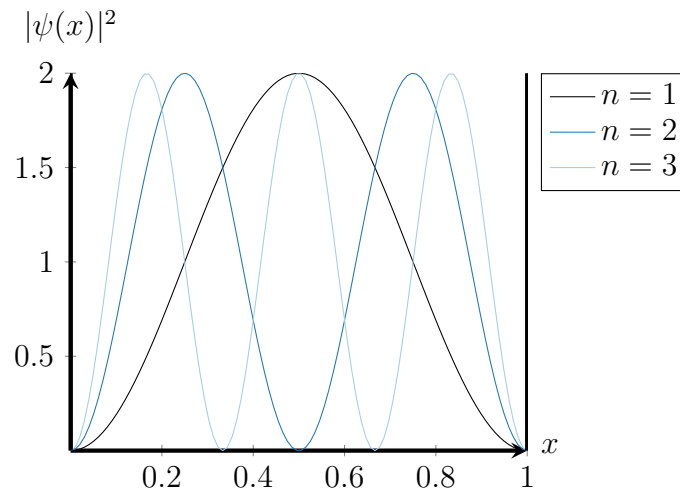


Abbildung 2.6: Aufenthaltswahrscheinlichkeit eines Elektrons in den drei niedrigsten energetischen Zuständen innerhalb eines unendlichen Potenzialtopfes der Länge $L = 1\text{m}$.

Das Quadrat des Vorfaktors $\frac{1}{\sqrt{2}}$ gibt dabei an, wie hoch die Wahrscheinlichkeit ist, dass sich das Teilchen im Zustand $n = 1$ oder $n = 2$ befindet. In diesem Fall existiert eine 50% Wahrscheinlichkeit das Teilchen in Zustand 1 oder in Zustand 2 zu finden. Die in diesem Abschnitt verwendete Funktionsnotation für die Wellenfunktionen von Teilchen ist die intuitive Darstellung von Wellenfunktionen. Es existiert allerdings auch eine alternative, die Bra-Ket-Notation. Um diese soll es im nachfolgenden Abschnitt 2.2.3 gehen.

2.2.3 Bra-Kets und Qubits

Eine heutzutage weitverbreitete Notation in der Quantenmechanik ist die Bra-Ket- bzw. Dirac-Notation. Wellenfunktionen werden hier als sogenannte Kets beschrieben, welche analog zu einem Spaltenvektor fungieren. Diese Vektornotation ist zunächst nicht so intuitiv greifbar wie die Funktionsnotation bietet aber bei quantenmechanischen Rechnungen Vorteile in der Handhabung, wodurch es einfacher wird, quantenmechanische Resultate herzuleiten. Eine allgemeine Wellenfunktion ψ lässt sich als Linearkombination der Basisvektoren dieses Vektorraums ausdrücken. Das Betragsquadrat der Vorfaktoren a_i gibt die Wahrscheinlichkeit an, das Teilchen in diesem Zustand zu messen. Für diskrete Größen wie z.B. n Energieniveaus lässt sich das Ket

$|\psi\rangle$ als einfache Summe ausdrücken.

$$|\psi\rangle = a_1 \cdot |\psi_1\rangle + a_2 \cdot |\psi_2\rangle + \dots + a_n \cdot |\psi_n\rangle \quad \text{mit} \quad \sum_{i=1}^n |a_i|^2 = 1 \quad (2.6)$$

Ein Bra $\langle\psi|$ ist der analoge Zeilenvektor, in dem die einzelnen Elemente komplex konjugiert sind. Zusammengesetzt $\langle\psi|\psi\rangle$ beschreibt dieser Ausdruck das innere Produkt der beiden Vektoren. Dieses kann ähnlich dem Skalarprodukt klassischer Vektorräume wie dem $\mathbb{R}^n$ interpretiert werden.

$$\langle\psi|\psi\rangle = \sum_{i=1}^n a_i^* \cdot a_i \quad (2.7)$$

Um beliebige Wellenfunktionen als Vektor zu repräsentieren, bedarf es unendlich vieler Basisvektoren. Eine Wellenfunktion $\psi(x)$ benötigt für jeden x (1-D Position) Wert einen entsprechenden Eintrag im Vektor. Da x eine kontinuierliche Größe ist, müssen unendlich viele Punkte im Vektor angegeben werden, um die Funktion darzustellen. Eine weitere Folge ist, dass anstatt einer Summe für kontinuierliche Größen ein Integral genutzt werden muss, um den Wellenvektor anzugeben.

$$|\psi\rangle = \int a(x) |x\rangle dx \quad (2.8)$$

Die Kets $|x\rangle$ beschreiben wieder Basisvektoren und die $a(x)$ sind die Vorfaktoren, die eine Wahrscheinlichkeit angeben. Rechnungen müssen meist in einem unendlichen komplexen Vektorraum, dem sogenannten Hilbertraum, durchgeführt werden.

Für Qubits wird diese Betrachtung einfacher. Ein Qubit kann sich nur in zwei Zuständen 0 und 1 befinden. In Ket-Notation durch $|0\rangle$ und $|1\rangle$ angegeben. Durch dieses einfache quantenmechanische System genügt es, zwei Basisvektoren für die beiden Grundzustände zu definieren. Eine beliebige Superposition der Grundzustände entspricht wieder einer Linearkombination der beiden Basiszustände.

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad |q\rangle = a \cdot |0\rangle + b \cdot |1\rangle = \begin{pmatrix} a \\ b \end{pmatrix} \quad \text{mit} \quad |a|^2 + |b|^2 = 1 \quad (2.9)$$

Diese Notation eignet sich somit besonders gut, um Qubits zu betrachten und mit ihnen zu rechnen. Auch Operatoren über Qubits können dann, wie später zu sehen sein wird, als endliche Matrizen betrachtet werden. Aus diesem Grund wird in den nachfolgenden Kapiteln ausschließlich diese Notation verwendet. Die Beschreibung von Qubits in Bra-Ket-Notation ist zunächst rein mathematisch. Wie genau ein echtes Quantensystem (physische Objekte die untereinander Wechselwirken und sich

quantenmechanisch Verhalten) realisiert wird, das sich diesen mathematischen Beschreibungen beugt, ist vom konkreten Quantencomputer abhängig. Eine Übersicht zu verschiedenen technisch möglichen Realisierungen lässt sich in [JBTS19] nachlesen.

In der Bra-Ket-Notation werden Zustände als Vektoren und Operatoren als Matrizen dargestellt. Für bestimmte Matrizen lassen sich Eigenwerte und dazu gehörige Eigenvektoren finden. Diese haben auch eine physikalische Interpretation welche in Abschnitt 2.2.5 erläutert wird. Im folgenden Abschnitt 2.2.4 wird zunächst das Konzept der Eigenwerte formal eingeführt.

2.2.4 Eigenwerte und Eigenzustände

Eigenwerte und Eigenvektoren sind ein Konzept aus der lineare Algebra. Obwohl es oft für Matrizen und Vektoren untersucht wird, gelten analoge Resultate für Funktionen mit entsprechenden Eigenfunktionen und Eigenwerten. Der Fokus in diesem Kapitel liegt auf Eigenwerten in Vektorräumen, da die Dirac-Notation quantenmechanische Systeme in Vektorräumen beschreibt. Die Definitionen in diesem Kapitel orientieren sich an [Lan13], dort findet sich auch eine ausführliche Darstellung.

Definition 2.2.1 (Eigenwert und Eigenvektor). Gegeben sei eine $n \times n$ Matrix A . Ein Vektor v der Länge n heißt Eigenvektor, wenn er folgende Gleichung erfüllt:

$$A \cdot v = \lambda_v \cdot v \quad (2.10)$$

λ_v ist ein Skalar und der Eigenwert zum Vektor v . △

Anstelle von einfachen Matrizen wird in der Quantenmechanik von Operatoren gesprochen. Die Vektoren entsprechen den Zuständen des Systems und werden deswegen auch Eigenzustände genannt. Gerade für kleine Matrizen lassen sich diese leicht per Hand berechnen. Eigenwerte einer Matrix A müssen folgende Gleichung erfüllen.

$$\det(A - \lambda_v I) = 0 \quad (2.11)$$

Für den Pauli-x-Operator σ_x sieht diese Rechnung wie folgt aus.

$$\det \left(\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right) = \det \begin{pmatrix} -\lambda & 1 \\ 1 & -\lambda \end{pmatrix} = \lambda^2 - 1 = 0 \quad (2.12)$$

Die beiden Eigenwerte sind dann $\lambda_1 = 1$ und $\lambda_2 = -1$. Die dazugehörigen Eigenvektoren lassen sich durch Lösen der Gleichung $(\sigma_x - \lambda_i I)v = 0$ berechnen.

$$\left(\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} - \lambda_i I \right) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} -\lambda_i & 1 \\ 1 & -\lambda_i \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 0 \quad (2.13)$$

Das Ergebnis ist ein Gleichungssystem mit zwei Unbekannten und zwei Variablen, wobei für jeden Eigenwert ein eigenes Gleichungssystem entsteht.

$$-\lambda_i x_1 + x_2 = 0 \quad (2.14)$$

$$x_1 + -\lambda_i x_2 = 0 \quad (2.15)$$

Die beiden Lösungen sind alle Vielfache der Vektoren $v_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ und $v_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$. Die beiden Eigenwerte sind reell. Was die Eigenwerte physikalisch Repräsentieren und in welcher Verbindung diese zu Operatoren stehen wird im folgenden Abschnitt 2.2.5 erläutert.

2.2.5 Operatoren

Ein Operator in der Quantenmechanik ist ein mathematisches Objekt, welches auf den Zustand eines quantenmechanischen Systems angewendet werden kann und als Ergebnis einen neuen Zustand liefert. Er beschreibt somit einen physikalischen Prozess, der zu einer Änderung eines Zustands führt. Dies kann beispielsweise ein Operator sein, der die Entwicklung des Systems über einen bestimmten Zeitraum angibt, oder die Energie eines Systems wiedergibt. Ein Zustand entspricht dabei einem Vektor im Hilbertraum. Das Wissen über diese wird vorausgesetzt. Eine genaue Definition und eine ausführliche Betrachtung von Hilberträumen lässt sich in [Von32] nachlesen. Dieser Abschnitt fokussiert sich auf die Nutzung von Operatoren im Rahmen der Dirac-Notation. Das Verständnis dieser ist notwendig, um die theoretische Funktionsweise des Quanten-Annealer besser verstehen zu können. Die Matrixdarstellung solcher Operatoren steht hier im Vordergrund. Im Allgemeinen können Operatoren analog auch auf Funktionen angewendet werden, z.B. Ableitungsoperatoren. Viele Eigenschaften und Rechenregeln, die in diesem Kapitel gezeigt werden, lassen sich analog auf Funktionen übertragen. Die folgenden Ausführungen folgen denen aus [BG00].

Definition 2.2.2 (Operator). Ein Operator ist ein mathematisches Objekt A , das auf einen Zustandsvektor $|\psi\rangle$ in einem Hilbertraum H wirkt und einen neuen Zustandsvektor $|\phi\rangle$ liefert.

$$A : H \rightarrow H \quad A|\psi\rangle = |\phi\rangle \quad (2.16)$$

Der Zustandsvektor $|\phi\rangle$ bleibt dabei im gleichen Hilbertraum wie $|\psi\rangle$. $\triangle$

Eigenschaften von Operatoren. [Von32] Die folgenden Eigenschaften sind nur ein Ausschnitt und zeigen die relevanten Eigenschaften, die im Beweis vom Quantum

Adiabatic Theorem [BF28] und für spätere Betrachtungen von Hamiltonoperatoren benötigt werden.

- Hier untersuchte Operatoren sind linear.

$$A(a_1 |\psi_1\rangle + a_2 |\psi_2\rangle) = a_1 A |\psi_1\rangle + a_2 A |\psi_2\rangle \quad (2.17)$$

- Es existiert ein Einheitsoperator I .

$$I |\psi\rangle = |\psi\rangle \quad (2.18)$$

- Operatoren sind im allgemeinen nicht kommutativ.

$$AB |\psi\rangle \neq BA |\psi\rangle \quad (2.19)$$

- Ein Operator angewendet auf das Ket und Bra des gleichen Zustandes liefert im allgemeinen nicht das Ket und Bra des gleichen neuen Zustandes.

$$A |\psi\rangle = |\phi\rangle \not\Rightarrow \langle\psi| A = \langle\phi| \quad (2.20)$$

Operatoren können aber nicht nur physikalische Prozesse, die ein System verändern, repräsentieren. Ihnen können auch reale Messgrößen wie Position, Energie und Impuls zugewiesen werden. Solche Größen werden oft Observablen eines Systems genannt. Ein wichtiger Vertreter dieser Operatoren ist der Hamiltonoperator. Er wird der Energie zugeordnet und seine Eigenwerte entsprechen den verschiedenen Energieniveaus, die ein System annehmen kann.

Definition 2.2.3 (Hamiltonoperator [BG00]). Ein Hamiltonoperator H ist ein Operator, der der Gesamtenergie eines Systems zugeordnet wird. Seine Eigenwerte entsprechen den verschiedenen Energieniveaus, die ein System annehmen kann.

$$H |\psi_i\rangle = E_i |\psi_i\rangle \quad (2.21)$$

Δ

Zu jedem Operator A existiert ein adjungierter Operator $A^\dagger$. Er bildet sich aus den komplex konjugierten Einträgen der transponierten Matrix A und besitzt folgende Eigenschaft.

$$A |\psi\rangle = |\phi\rangle \Rightarrow \langle\psi| A^\dagger = \langle\phi| \quad (2.22)$$

Operatoren, denen Observablen zugeordnet sind und somit auch der Hamiltonoperator, haben die Eigenschaft, hermitesch zu sein. Das bedeutet, sie sind identisch

mit ihrem Adjungierten $H = H^\dagger$. Diese Eigenschaft ermöglicht es, einen hermiteschen Operator, der zwischen einem Bra und einem Ket steht, auf einen der beiden Vektoren beliebig anzuwenden.

$$\langle \psi | H | \phi \rangle = \langle \psi | H \phi \rangle = \langle H \psi | \phi \rangle \quad (2.23)$$

Für Eigenzustände $|\psi_i\rangle$ und deren entsprechenden Eigenwerten E_i lässt sich die Eigenschaft analog formulieren. Diese Eigenschaft wird besonders wichtig und notwendig in der Herleitung des Quantum Adiabatic Theorems.

$$\langle \psi_i | H | \psi_j \rangle = \langle \psi_i | E_j | \psi_j \rangle = E_j \langle \psi_i | \psi_j \rangle \quad (2.24)$$

Um die Energie des Quanten-Annealers zu beschreiben, sind vor allem zwei Operatoren von Bedeutung. Der Pauli-x-Operator σ_x und der Pauli-z-Operator σ_z .

$$\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (2.25)$$

Diese liefern vorteilhafte Eigenzustände und Eigenwerte, um mit dem Quanten - Annealer Optimierungsprobleme lösen zu können. Das Vorgehen dazu wird in Kapitel 3 behandelt. Mit dem Wissen über Eigenwerte und Eigenvektoren kann jetzt die theoretische Grundlage des Quanten-Annealers, das Quantum Adiabatic Theorem, in Abschnitt 2.3 hergeleitet werden.

2.3 Quantum Adiabatic Computation

Idee. Das Quantum Adiabatic Theorem besagt, dass ein zeitlich veränderliches Quantensystem, das sich initial in einem Zustand $|\psi\rangle$ befindet, in diesem bleibt, solange der Hamiltonoperator nur langsam genug verändert wird.

Satz 2.3.1 (Quantum Adiabatic Theorem [BF28]). Gegeben sei ein zeitlich veränderliches Quantensystem mit den Eigenzuständen $|\psi_0(t)\rangle, \dots, |\psi_n(t)\rangle$, welches sich im folgenden initialen Zustand befindet:

$$|\psi(0)\rangle = \sum_{i=0}^n c_i(0) \cdot |\psi_i(0)\rangle \quad (2.26)$$

Solange die Änderung des zu dem Quantensystem gehörigen Hamiltonoperators langsam genug ist, entwickelt sich das System vom Zeitpunkt $t = 0$ bis $t = T$ in den folgenden Zustand.

$$|\psi(T)\rangle = \sum_{i=0}^n c_i(T) \cdot |\psi_i(T)\rangle \quad (2.27)$$

Wobei die Koeffizienten $c_i(t)$ im Betragsquadrat für beide Zeitpunkte gleich bleiben.

$$\forall i \in \{0, \dots, n\} : |c_i(0)|^2 = |c_i(T)|^2 \quad (2.28)$$

▽

Dieser Vorgang muss adiabatisch erfolgen, das heißt, es darf kein Energieaustausch mit der Umgebung stattfinden. Was genau "langsam" bedeutet wird sich während der Herleitung klären. Um ein intuitives Verständnis für dieses Theorem zu vermitteln, kann ein Pendel betrachtet werden. Verändern wir die Länge des Fadens an dem das Pendel schwingt, abrupt, ist der Zustand, in dem sich das Pendel danach befindet, nur schwer vorherzusehen. Ändern wir dagegen die Länge des Fadens langsam, schwingt das Pendel mit der gleichen Periode weiter. Der folgende Abschnitt zeigt die Herleitung dieses Verhaltens aus der Schrödinger-Gleichung für quantenmechanische Systeme. Das Vorgehen orientiert sich an [SN20].

Quantum Adiabatic Theorem. Wir betrachten ein quantenmechanisches System mit einem zeitlich veränderlichen Hamiltonoperator $H(t)$. Wellenfunktionen bzw. Zustände, die unser System beschreiben, müssen die zeitabhängige Schrödinger-Gleichung 2.29 erfüllen. Das Ziel ist es, eine Zustandsbeschreibung zu finden, die die Schrödinger-Gleichung erfüllt.

$$i\hbar \frac{\partial}{\partial t} |\psi(t)\rangle = H(t) |\psi(t)\rangle \quad (2.29)$$

Der Ausdruck $\frac{\partial}{\partial t}$ steht für die partielle Zeitliche Ableitung und $\hbar$ für die reduzierte Planck-Konstante. Diese Differentialgleichung stellt eine Bedingung dar, welche die Funktionen der Zustände eines Quantensystems erfüllen müssen. Es sind somit nur Zustandsbeschreibungen erlaubt, die nach dem Einsetzen in die Schrödinger-Gleichung eine wahre Aussage ergeben. Jeder Zustand $|\psi(t)\rangle$ lässt sich als Kombination aus Basiszuständen formulieren. Wir benötigen Basiszustände $|\psi_n(t)\rangle$, die die Schrödinger-Gleichung zu jedem Zeitpunkt t lösen. Wir definieren $|\psi_n(t)\rangle$ als die Eigenzustände von $H(t)$ und $E_n(t)$ die dazu gehörigen Eigenwerte.

$$H(t) |\psi_n(t)\rangle = E_n(t) |\psi_n(t)\rangle \quad (2.30)$$

Diese $|\psi_n(t)\rangle$ lösen noch nicht die zeitabhängige Schrödinger-Gleichung. Sie müssen aber für ein festes t eine Basis im Vektorraum von $|\psi(t)\rangle$ bilden. $|\psi(t)\rangle$ lässt sich somit für ein festes t durch die $|\psi_n(t)\rangle$ beschreiben.

$$|\psi(t)\rangle = \sum_n c_n(t) \cdot e^{i\theta_n(t)} |\psi_n(t)\rangle \quad \text{mit} \quad \theta_n(t) = -\frac{1}{\hbar} \int_0^t E_n(t') dt' \quad (2.31)$$

Der Term $(e^{i\theta_n(t)})$ ist dabei nur eine Verallgemeinerung der allgemeinen Lösung $e^{-\frac{i}{\hbar} \cdot E_n \cdot t}$ für den Fall, dass der Hamiltonoperator zeitunabhängig ist. Dieser Ausdruck wird jetzt in die Schrödinger-Gleichung eingesetzt. In den folgenden Rechnungen sind die c_i, ψ_i, θ_i, H und E_i immer noch zeitabhängig. Für eine bessere Lesbarkeit wird aber auf das (t) verzichtet.

$$i\hbar \frac{\partial}{\partial t} \sum_n c_n \cdot e^{i\theta_n} |\psi_n\rangle = H(t) \sum_n c_n \cdot e^{i\theta_n} |\psi_n\rangle \quad (2.32)$$

Das Anwenden des Ableitungsoperators auf der linken Seite der Gleichung liefert folgenden Ausdruck.

$$\begin{aligned} & i\hbar \sum_n \dot{c}_n \cdot e^{i\theta_n} |\psi_n\rangle + c_n \cdot e^{i\theta_n} |\dot{\psi}_n\rangle + c_n(t) \cdot i\dot{\theta}_n e^{i\theta_n} |\psi_n\rangle \\ &= i\hbar \sum_n \left(\dot{c}_n |\psi_n\rangle + c_n |\dot{\psi}_n\rangle + c_n \cdot i\dot{\theta}_n |\psi_n\rangle \right) e^{i\theta_n} \end{aligned} \quad (2.33)$$

Die Ableitung von θ lässt sich einfach auswerten $i\dot{\theta}_n = i \frac{\partial}{\partial t'} \frac{-1}{\hbar} \int_0^t E_n(t') dt' = -\frac{i}{\hbar} E_n$. Die $|\psi_n\rangle$ sind als Eigenzustände des Hamiltonoperators definiert. Damit lässt sich die rechte Seite der Gleichung 2.32 vereinfachen.

$$\sum_n c_n \cdot e^{i\theta_n} \cdot E_n |\psi_n\rangle \quad (2.34)$$

Der Term entspricht genau dem dritten Summanden aus Gleichung 2.33. Wir streichen beide und erhalten eine neue Gleichung.

$$\begin{aligned} & i\hbar \sum_n \left(\dot{c}_n |\psi_n\rangle + c_n |\dot{\psi}_n\rangle \right) e^{i\theta_n} = 0 \\ \Leftrightarrow & \sum_n \dot{c}_n \cdot e^{i\theta_n} |\psi_n\rangle = - \sum_n c_n \cdot e^{i\theta_n} |\dot{\psi}_n\rangle \end{aligned}$$

Sei $|\psi_m\rangle$ der Eigenzustand des Eigenwertes E_m des Hamiltonoperators H . Wir bilden das innere Produkt mit $\langle\psi_m|$ und erhalten folgende Formel.

$$\sum_n \dot{c}_n \cdot e^{i\theta_n} \langle\psi_m|\psi_n\rangle = - \sum_n c_n \cdot e^{i\theta_n} \langle\psi_m|\dot{\psi}_n\rangle \quad (2.35)$$

Da sowohl $|\psi_m\rangle$ als auch $|\psi_n\rangle$ Basisvektoren sind, müssen diese, sofern sie voneinander verschieden sind (für $m \neq n$), senkrecht zueinander stehen. Das innere Produkt zweier Basisvektoren ist somit 1, wenn diese gleich sind, und 0, wenn diese voneinander verschieden sind. Die Summe vereinfacht sich wie folgt.

$$\begin{aligned} \dot{c}_m \cdot e^{i\theta_m} &= - \sum_n c_n \cdot e^{i\theta_n} \cdot \langle\psi_m|\dot{\psi}_n\rangle \\ \dot{c}_m &= - \sum_n c_n \cdot e^{i(\theta_n - \theta_m)} \langle\psi_m|\dot{\psi}_n\rangle \end{aligned}$$

Um einen Ausdruck für $\langle \psi_m | \dot{\psi}_n \rangle$ zu finden, wird die stationäre Schrödinger-Gleichung nach der Zeit abgeleitet.

$$\begin{aligned} \frac{\partial}{\partial t} H |\psi_n\rangle &= \frac{\partial}{\partial t} E_n |\psi_n\rangle \\ \dot{H} |\psi_n\rangle + H |\dot{\psi}_n\rangle &= \dot{E}_n |\psi_n\rangle + E_n |\dot{\psi}_n\rangle \end{aligned}$$

Wieder wird das innere Produkt mit $\langle \psi_m |$ auf beide Seiten angewendet.

$$\langle \psi_m | \dot{H} |\psi_n\rangle + \langle \psi_m | H |\dot{\psi}_n\rangle = \langle \psi_m | \dot{E}_n |\psi_n\rangle + \langle \psi_m | E_n |\dot{\psi}_n\rangle$$

Der Hamiltonoperator ist hermitesch, dadurch lässt er sich beliebig auf das rechte Ket oder linke Bra anwenden. Für den Fall $m \neq n$ ergibt sich der folgende Ausdruck.

$$\langle \psi_m | \dot{H} |\psi_n\rangle + E_m \langle \psi_m | \dot{\psi}_n\rangle = \dot{E}_n \langle \psi_m | \psi_n\rangle + E_n \langle \psi_m | \dot{\psi}_n\rangle \quad (2.36)$$

$$\langle \psi_m | \dot{\psi}_n\rangle = \frac{\langle \psi_m | \dot{H} |\psi_n\rangle}{E_n - E_m} \quad (2.37)$$

Die zeitliche Änderung der c_m , die den Wert angibt, wie wahrscheinlich es ist, dass sich das System im Zustand ψ_m befindet, wird von der obigen Gleichung 2.37 beschrieben. Befinden wir uns im Zustand ψ_k , ist der Wert für $c_k = 1$ und für alle anderen c_m mit $m \neq k$ gleich 0. Die Änderung dieser c_m ist zunächst nur durch den Term $c_n \cdot \frac{\langle \psi_k | \dot{H} |\psi_n\rangle}{E_n - E_k}$ beeinflusst. Dieser Term kann vernachlässigt werden, wenn der Energieunterschied der Zustände $|\psi_n\rangle$ und $|\psi_k\rangle$ groß genug ist. Außerdem muss die zeitliche Ableitung des Hamiltonoperators klein sein, das System darf sich nur langsam verändern. Die Änderung des c_k kann durch direktes Lösen der Differentialgleichung der $\dot{c}_k$ angegeben werden.

$$c_k(t) = c_k(0) \cdot e^{i\gamma(t)} \text{ mit } \gamma(t) = i \int_0^t \langle \psi_k(t') | \dot{\psi}_k(t') \rangle dt' \quad (2.38)$$

Bis auf einen Phasenfaktor $e^{i\gamma(t)}$ bleibt das System in dem Zustand $|\psi_k(t)\rangle$. Das bedeutet, wenn sich ein System in seinem energetisch niedrigsten Eigenzustand befindet, wird es, solange die Energielücke groß genug und das System sich langsam genug ändert, auch weiterhin im energetisch niedrigsten Eigenzustand bleiben. Genau dieses Verhalten macht sich der Quanten-Annealer zu nutze, um Optimierungsprobleme zu lösen.

Um ein greifbares Resultat zu erhalten, müssen ein paar weitere Rechenschritte und Abschätzungen durchgeführt werden. Der Ausdruck für $\langle \psi_m | \dot{\psi}_n \rangle$ kann in die Differentialgleichung für $\dot{c}_n$ eingesetzt werden.

$$\dot{c}_m = - \sum_n c_n \frac{\langle \psi_m | \dot{H} |\psi_n\rangle}{E_n - E_m} \cdot e^{-\frac{i}{\hbar} \int_0^t E_n(t') - E_m(t') dt'}$$

Ein Ergebnis für die oben besprochene Belegung der c_i mit $c_k = 1$ und $c_{m \neq k} = 0$ ist von Interesse. Daher werden diese c_i eingesetzt und die Ergebnisse für $\dot{c}_{m \neq k}$ untersucht.

$$\dot{c}_{m \neq k} = \frac{\langle \psi_m | \dot{H} | \psi_k \rangle}{E_k - E_m} \cdot e^{-\frac{i}{\hbar} \int_0^t E_k(t') - E_m(t') dt'} \quad (2.39)$$

Dieser Ausdruck kann integriert werden, um ein Ergebnis für $c_{m \neq k}$ zu erhalten.

$$c_{m \neq k}(T) = \int_0^T \frac{\langle \psi_m | \dot{H} | \psi_k \rangle}{E_k - E_m} \cdot e^{-\frac{i}{\hbar} \int_0^t E_k(t') - E_m(t') dt'} dt \quad (2.40)$$

Um dieses Integral zu lösen, werden zwei Abschätzungen vorgenommen. Für die Energielücke kann das Minimum $\overline{E_k - E_m}$ über den gesamten Zeitraum t eingesetzt werden. Analog kann für den Zähler das Maximum $\langle \psi_m | \dot{H} | \psi_k \rangle$ gewählt werden.

$$c_{n \neq m}(T) \approx \int_0^T \frac{\overline{\langle \psi_m | \dot{H} | \psi_k \rangle}}{\overline{E_k - E_m}} \cdot e^{-\frac{i}{\hbar} \int_0^t \overline{E_k - E_m} dt'} dt \quad (2.41)$$

$$= \frac{\overline{\langle \psi_m | \dot{H} | \psi_k \rangle}}{\overline{E_k - E_m}} \int_0^T e^{-\frac{i}{\hbar} \overline{E_k - E_m} t} dt \quad (2.42)$$

$$= \frac{\overline{\langle \psi_m | \dot{H} | \psi_k \rangle}}{\overline{E_k - E_m}} \left[\frac{-i\hbar}{\overline{E_k - E_m}} \cdot e^{-\frac{i}{\hbar} \overline{E_k - E_m} t} \right]_0^T \quad (2.43)$$

$$= \frac{\overline{\langle \psi_m | \dot{H} | \psi_k \rangle}}{\overline{E_k - E_m}} \cdot \frac{-i\hbar}{\overline{E_k - E_m}} \left(e^{\frac{i}{\hbar} \overline{E_k - E_m} T} - 1 \right) \quad (2.44)$$

$$\approx \frac{\overline{\langle \psi_m | \dot{H} | \psi_k \rangle}}{\overline{E_k - E_m}} \cdot \frac{-i\hbar}{\overline{E_k - E_m}} \quad (2.45)$$

Der Term $\left(e^{\frac{i}{\hbar} \overline{E_k - E_m} T} - 1 \right)$ oszilliert und kann mit 1 nach oben abgeschätzt werden. Der Wert für $|c_{m \neq k}(T)|^2$ sollte 0 sein. Wir müssen somit untersuchen, wann der Ausdruck 2.45 klein wird. Die imaginäre Einheit i kann weggelassen werden, da nur das Betragsquadrat des Ausdrucks minimiert werden muss. Insgesamt soll der folgende Ausdruck viel kleiner als 1 sein.

$$\frac{\hbar \overline{\langle \psi_m | \dot{H} | \psi_k \rangle}}{\overline{E_k - E_m}^2} \ll 1 \quad (2.46)$$

Damit das Quantum Adiabatic Theorem anwendbar ist, muss der Quotient aus der maximalen zeitlichen Änderung des Hamiltonoperators und dem Quadrat der Energielücke viel kleiner als 1 sein. Praktisch bedeutet das, damit der Quanten-Annealer ein korrektes Ergebnis erhalten kann, muss die Ungleichung 2.46 erfüllt sein. Der Term

$(\langle \psi_m | \dot{H} | \psi_k \rangle)$ lässt sich durch modifizieren der Modellierung des Problems oder anpassen der verwendeten Anneal-Zeit manipulieren. Der Term $(\overline{E_k - E_m})^2$ lässt sich auch durch die Problemmodellierung beeinflussen. Dies zeigt wie wichtig die Modellierung, für das Lösen von Problemen mithilfe des Quanten Adiabatic Theorems ist.

2.4 Verwandte Arbeiten

Das Forschungsgebiet der Quantencomputer und zu einem gewissen Grad auch die Quantenmechanik selber, stecken noch in den Kinderschuhen. Trotzdem existiert eine Vielzahl an theoretischen und praktischen Arbeiten, die sich sowohl der Problemmodellierung als auch dem Benchmarking widmen.

Die konkrete Modellierung von sehr klassischen Constraint-Problemen wie dem Magic Square oder n-Queens wurde schon im Paper [Cod21] durchgeführt. Allerdings können auch allgemeinere Techniken entwickelt und angewendet werden, um Probleme in ein Qubo- oder Ising-Modell zu überführen. Ein Ansatz dazu wurde in dem Papern [WG23] und [Wol24] vorgestellt, in denen MiniZinc-Programme automatisch in Qubo-Modelle transformiert wurden. Auf diese automatischen Transformationen wird in den Abschnitten 4.2 und 4.3 genauer eingegangen.

Ein alternativer Ansatz zum Transformieren von Constraint-Problemen ist das direkte Aufstellen von Qubo-Ausdrücken aus Wahrheitswerttabellen. Dazu nutzt D-Wave einen LP-Ansatz, welcher in [pen] zu finden ist. Um dieses Verfahren zu beschleunigen, wurde in dem Paper [Pak21] eine Variablenheuristik eingeführt. Das allgemeine LP-Verfahren zum Finden von Qubo-Ausdrücken und die LP-Heuristik werden im Abschnitt 4.3.2 gesondert erklärt und untersucht.

Auch die Constraint-Programmierung kann genutzt werden, um Qubo- und Ising-Ausdrücke aus Wahrheitswerttabellen zu erstellen. Die Idee dafür stammt aus dem Paper [PVVL22], in dem Constraintprogrammierung zum Finden linearer Gleichungen genutzt wurde. Solche linearen Gleichungen können durch einfaches Quadrieren in einen Qubo-Ausdruck umgewandelt werden. Auf das originale Verfahren und die in dieser Arbeit entwickelte Erweiterung wird in Abschnitt 4.3.3 gesondert eingegangen.

Zum Modellieren von Problemen ist nicht nur das Finden von Qubo-Ausdrücken relevant. Auch die Kodierung der Variablen spielt eine wichtige Rolle. Zu diesem Zweck wurde in dem Paper [Cha19] die Domain Wall-Kodierung mit besonderem Fokus auf das Quanten-Annealing entwickelt. Eine genaue Erklärung zu dieser Kodierungstechnik ist in Abschnitt 4.1 gegeben.

Zuletzt sind auch das Benchmarking und das Auswerten von Anneal-Ergebnissen

relevant. Zu diesem Zweck existieren bestimmte Metriken, die sich besonders gut zum Vergleichen mit vor allem klassischen Verfahren eignen. Im Paper [KYN⁺15] wurden Probleme mit Hilfe der Time To Target-Metrik getestet und ausgewertet. Die Metrik wird in Abschnitt 3.2.4 eingeführt. Eigene Auswertungen von Problemen mithilfe der Time To Target-Metrik sind in den Abschnitten 3.2.4 und 5.2 zu finden.

Kapitel 3

Quantum Annealing

Dieses Kapitel beschäftigt sich mit der praktischen Anwendung des Quantum Adiabatic Theorems. Dazu wird in Abschnitt 3.1 der Quanten-Annealer von D-Wave eingeführt und sein Aufbau und die Funktionsweise erklärt. Im Abschnitt 3.2 werden zwei Constraint-Probleme für den Quanten-Annealer modelliert. Danach wird in Abschnitt 3.2.3 und 3.2.4 ein genauerer Blick auf die einzelnen Arbeitsschritte des Annealers geworfen und wie die Ergebnisse des Annealings auszuwerten sind.

3.1 Der Quanten-Annealer

Das Quanten-Annealing, welches mithilfe des Quanten Adiabatic Theorems Ising-Modelle lösen kann, ist nur eine von verschiedenen Quantentechnologien, die genutzt werden können, um CSPs zu lösen. Der folgende Abschnitt 3.1.1 soll einen kleinen Überblick über diese geben.

3.1.1 Quantentechnologien

Die bisher am weitesten verbreiteten und technologisch fortschrittlichsten Technologien unterscheiden sich in Gate- und Anneal-basierte Ansätze. Die Gate-Technologie ist dabei vergleichbar mit klassischen Computern, wohingegen die Anneal-Technologien eher der Funktionsweise von Analogcomputern entsprechen. Im folgenden Abschnitt werden die verschiedenen Technologien näher beleuchtet sowie deren Vor- und Nachteile aufgezeigt.

Gate-Quantencomputer. Ein Gate-Quantencomputer besteht aus Qubits, auf denen Gate-Operationen ausgeführt werden können. Solche Gates sind ähnlich zu den

klassischen Logik-Gates innerhalb eines digitalen Computers zu verstehen. Qubits können sich jedoch auch in Überlagerungen von Zuständen befinden, wodurch eine viel größere Anzahl an möglichen Quanten-Gates existiert als klassischer Logik-Gates. Durch diese Vielzahl an möglichen Operationen sind Gate-Technologien flexibler einsetzbar. Mithilfe des Shor-Algorithmus [Sho97] lassen sich die Primfaktoren einer Zahl wesentlich schneller als mit einem klassischen Computer finden. Dies kann genutzt werden, um Verschlüsselungstechniken wie RSA [RSA78] zu knacken.

Die potentielle Flexibilität erhöht allerdings auch die Komplexität. Solche Quantencomputer sind extrem schwer zu bauen und bisher nur begrenzt skalierbar. Ein Überblick über die verschiedenen Ansätze und bisherigen Ergebnisse von IBM ist hier [Abu25] zu finden. Außerdem ist es keine leichte Aufgabe ein Programm bzw. einen Algorithmus für solche Computer zu schreiben. Aktuell muss eine Art Assembler-Code für Quantencomputer geschrieben werden, was sehr spezifisches Fachwissen zu Quanten-Gates voraussetzt. Aus diesen Gründen wurde sich gegen die Nutzung von Gate-Technologien entschieden.

Annealer. Bei den Annealern werden keine Programme geschrieben, die auf der jeweiligen Hardware ausgeführt werden. Stattdessen wird deklarativ beschrieben, welche Eigenschaften eine Lösung besitzen soll. Die Hardware kümmert sich danach mit Hilfe quantenmechanischer Effekte um das Finden einer solchen Lösung. Der Fokus dieser Arbeit liegt auf dem Quantum Annealing, wobei die Quanten-Annealer von D-Wave verwendet werden. Es existiert auch ein digitaler Annealer [dig] von Fujitsu. Dieser versucht mit klassischer Hardware die Funktionsweise eines Quanten-Annealers nachzuahmen. Aufgrund der Kosten für die Nutzung dieser Hardware wurde dieser Annealer in der Arbeit nicht weiter betrachtet.

Der Quanten-Annealer von D-Wave ist eine Hardware, die das Quanten Adiabatic Theorem nutzt, um Ising-Modelle zu lösen. Er findet, gegeben ein Ising-Modell, eine Variablenbelegung mit minimaler Energie. Der Annealer besteht aus Qubits, die untereinander gekoppelt sind. Der Vorteil eines Annealers liegt darin, dass lediglich ein Ising-Modell übergeben werden muss, welches das ursprüngliche Problem kodiert. Dabei ist zu beachten, dass alle Finite Domänen CSPs in ein Ising-Modell überführt werden können. Außerdem stellt D-Wave einen kostenlosen Zugriff über die Cloud auf verschiedenste Annealer zur Verfügung. Aus diesen Gründen fokussiert sich die Arbeit auf den Quanten-Annealer von D-Wave.

Architekturen. Mittlerweile stellt D-Wave verschiedene Quanten-Annealer mit unterschiedlichen Architekturen zur Verfügung. Die Architektur gibt dabei an, welche Qubits untereinander gekoppelt sind. In der einfachsten, der Chimera-Architektur, ist

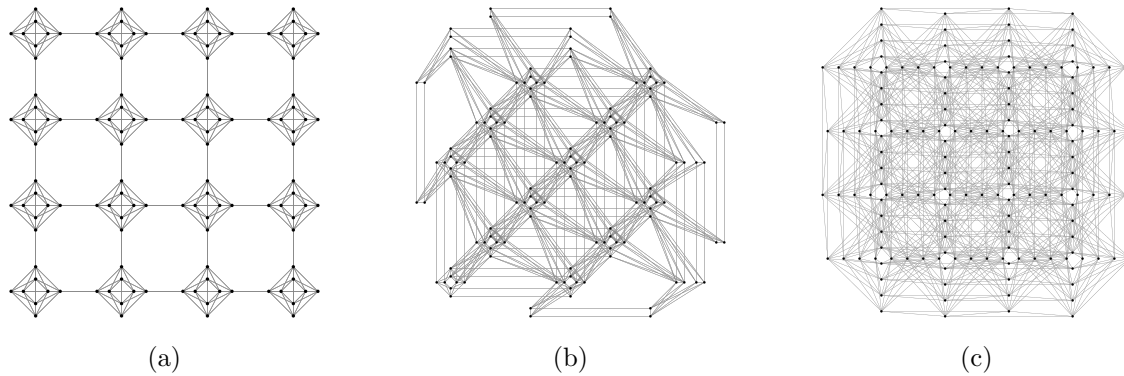


Abbildung 3.1: Topologien der verschiedenen Quanten-Annealer-Architekturen. Ein Knoten entspricht einem Qubit, und Kanten sind zwischen zwei gekoppelten Qubits eingezeichnet. (a) Chimera-Architektur, (b) Pegasus-Architektur, (c) Zephyr-Architektur. Diese Abbildungen stellen nur einen kleinen exemplarischen Ausschnitt der echten Quanten-Annealer dar.

ein einzelnes Qubit mit nur sechs weiteren Qubits verbunden. Spätere Architekturen wie Pegasus oder Zephyr bieten mehr Verbindungen zwischen den Qubits. In der Pegasus Architektur ist ein Qubit mit 15 und in der Zephyr Architektur ist ein Qubit mit 20 weiteren Qubits verbunden. Die Abbildung 3.1 zeigt die drei bisherigen Architekturen. In dieser Arbeit werden sowohl der Advantage 7.1 (Pegasus-Architektur) sowie der Advantage2 2.6 (Zephyr-Architektur) verwendet. Innerhalb des Advantage 7.1 sind zwar Qubits weniger dicht miteinander verknüpft, insgesamt bietet er aber mit seinen rund 5500 Qubits einen Größenvorteil gegenüber dem Advantage2 2.6 mit nur rund 1200 Qubits. Für kleine Probleme wird jedoch aufgrund der aktuelleren und verbesserten Hardware der Advantage2 2.6 verwendet. Wie genau solche System aufgebaut sind, wird in den zwei folgenden Abschnitten vorgestellt.

3.1.2 Qubits

Dieser Abschnitt gibt nur einen groben Einblick in den Aufbau eines Qubits. Eine umfangreichere Untersuchung kann in [HJB⁺10] nachgelesen werden. Alle Quanten-Annealer besteht aus Qubits, die untereinander gekoppelt sind und sich gegenseitig beeinflussen. Ein Qubit ist im Allgemeinen ein quantenmechanisches System (physikalische Objekte die untereinander Wechselwirken und sich quantenmechanisch Verhalten), welches sich in nur zwei klassischen Zuständen befinden kann. Dies ist analog zum Bit zu verstehen, welches sich nur in Zustand 0 oder Zustand 1 befinden kann. Ein Beispiel für echte Zweizustandssysteme ist der Spin eines Elektrons. Dieser kann

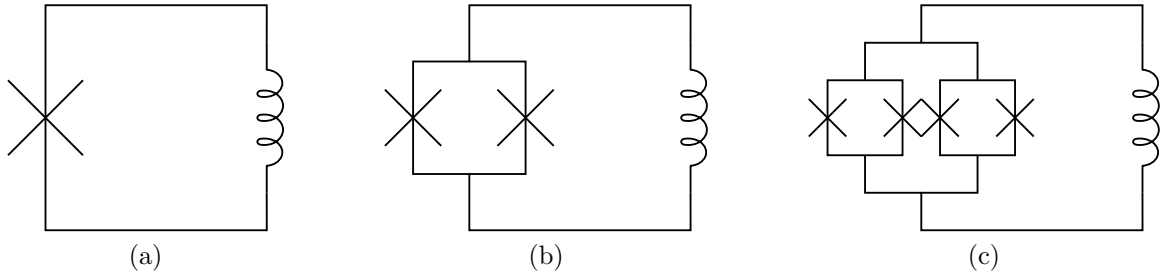


Abbildung 3.2: (a) Einfaches Qubit, bestehend aus einer Supraleiterschleife und einem Josephson-Kontakt. (b) Komplexeres Qubit, bestehend aus einer Supraleiterschleife und zwei parallel geschalteten Josephson-Kontakten. (c) Qubit, bestehend aus einer Supraleiterschleife und zwei parallel geschalteten Blöcken, die jeweils aus zwei parallel geschalteten Josephson-Kontakten bestehen. Die Spulen symbolisieren die natürliche Induktivität der Supraleiterschleife der Qubits.[HJB⁺10]

nur zwei Werte annehmen und somit als Qubit genutzt werden. Die Nutzung eines einzelnen Elektrons ist in der Praxis aber nur schwer umzusetzen. Es ist nicht einfach, einzelne Teilchen in einer kontrollierten Umgebung zu halten und diese zu manipulieren. Aus diesem Grund werden für den Quanten-Annealer Supraleiter-Qubits verwendet.

Der Aufbau eines Supraleiter-Qubits, zu sehen in Abbildung 3.2, ähnelt dem eines LC-Schwingkreises (Spule Kondensator). Im einfachsten Fall besteht ein Qubit aus einer Supraleiterschleife. Diese besitzt, wie jede Leiterschleife, eine bestimmte Induktivität und übernimmt die Rolle der Spule. Anstelle eines Kondensators wird ein Josephson-Kontakt genutzt. Dieser besteht aus einer dünnen isolierenden Schicht, die die Supraleiterschleife unterbricht. Da die Eigenschaft von Supraleitern, Strom verlustfrei, also ohne Widerstand zu transportieren, erst bei sehr geringen Temperaturen auftritt, müssen die Qubits auf ungefähr 20 mK gekühlt werden (ideal wäre 0 K). Bei diesen Temperaturen schließen sich im Leiter zwei Elektronen zu sogenannten Cooper-Paaren zusammen und können verlustfrei durch den Leiter fließen (Supraleitung [BCS57]). Treffen solche Cooper-Paare auf einen Josephson-Kontakt, besteht eine Wahrscheinlichkeit, dass sie durch die schmale isolierende Schicht durchtunneln können (Tunneleffekt [Raz03]). Beide dieser Phänomene, also Supraleitung und Tunneleffekt, sind quantenmechanische Effekte.

Ein Josephson-Kontakt ist im Vergleich zum Kondensator ein nichtlineares Bauelement. Das bedeutet, dass die Energieniveaus, die sich für diese Schaltung ergeben, unterschiedliche Abstände besitzen. Die Differenz von Energieniveau E_1 zu Energieniveau E_2 ist kleiner als die Differenz von Energieniveau E_2 zu Energieniveau E_3 .

Werden für externe Anregungen der Qubits nur Energien genutzt, die sich im Bereich der Differenz von E_1 und E_2 befinden, wird das Qubit nie in einen höheren Energiezustand gebracht. Es wird somit künstlich ein Zweizustandssystem erzeugt, welches sich gut kontrollieren lässt. Die beiden Zustände, die genutzt werden, entsprechen jeweils unterschiedlichen Stromrichtungen in der Supraleiterschleife. In einem Zustand zeigt das Magnetfeld nach oben und im anderen Zustand nach unten. Daher werden die Zustände meist mit $|\uparrow\rangle$ und $|\downarrow\rangle$ gekennzeichnet. Die Bezeichnung ist allerdings rein symbolisch und kann auch mit $|0\rangle$ und $|1\rangle$ erfolgen. Die zweite Notation wird im folgenden Teil vermehrt verwendet.

Wie in Abschnitt 2.2.3 beschrieben, stellen die Kets Vektoren dar. Für ein einzelnes Qubit erhalten wir unsere zwei Basisvektoren in zwei Dimensionen.

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (3.1)$$

Für zwei gekoppelte Qubits erhalten wir andere Vektoren. Zwei gekoppelte Qubit können sich analog zu zwei Bits in vier verschiedenen Zuständen befinden. Anstelle von zwei Dimensionen ergeben sich somit vier.

$$|00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad |01\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \quad |10\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} \quad |11\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

Natürlich sind auch wieder Superpositionen dieser Zustände möglich. Erfolgt allerdings eine Messung, wird immer nur einer dieser Basiszustände gemessen. Dadurch wird nur eine klassische Größe wie z.B. die Richtung des Magnetfeldes messbar. Die Überlagerung der Zustände gibt dann die Wahrscheinlichkeit an, mit der bestimmte klassische Zustände gemessen werden. Wie Qubits im Quanten-Annealer manipuliert werden können, wird im nächsten Abschnitt 3.1.3 erklärt.

3.1.3 Bias und Coupling von Qubits

Auch hier erfolgt nur eine grobe Einführung in das Thema der Qubit-Manipulation. Eine ausführlichere Betrachtung des Themas lässt sich in [HLB⁺09] nachlesen. Um konkrete Probleme mithilfe des Quanten-Annealers lösen zu können, müssen die gekoppelten Qubits von außen programmierbar sein. Dafür werden Qubit-Bias und Coupler eingesetzt. Ein Qubit-Bias, zu sehen in Abbildung 3.3, wird realisiert, indem ein Magnetfeld, gesteuert durch eine externe Stromquelle, mit dem Magnetfeld des Qubits in Wechselwirkung gebracht wird. Je nach Stärke des Feldes, wird es wahrscheinlicher, dass sich das Qubit passend zum externen Magnetfeld ausrichtet und die

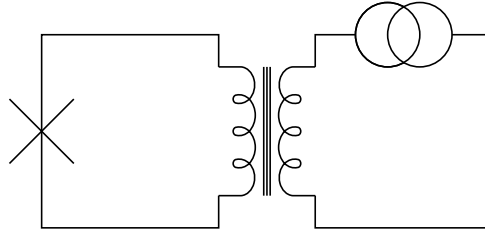


Abbildung 3.3: Externer Bias (rechts) zum Manipulieren eines einzelnen Qubits (links).[HLB⁺09]

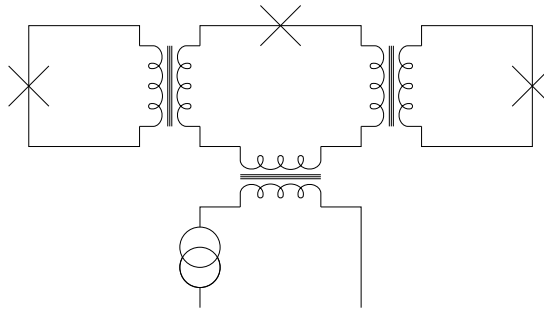


Abbildung 3.4: Coupler zwischen zwei Qubits (links und rechts), der durch eine externe Stromquelle gesteuert werden kann.[HLB⁺09]

entgegengesetzte Richtung annimmt. Die Vorstellung ist hier ähnlich zu zwei Stabmagneten die, wenn sie nebeneinander gelegt werden, sich entgegengesetzt zueinander ausrichten.

Um zwei Qubits untereinander zu koppeln, wird eine ähnliche Technik verwendet. Der Aufbau ist in Abbildung 3.4 zu sehen. Die Magnetfelder beider Qubits werden über das Magnetfeld einer dritten Supraleiter-Schleife gegenseitig gekoppelt. Das Magnetfeld der dritten Schleife wird wiederum durch ein externes Magnetfeld, gesteuert durch eine Stromquelle, manipuliert. So entsteht eine steuerbare Wechselwirkung zwischen zwei Qubits.

Alle Qubits untereinander zu koppeln stellt eine technische Herausforderung dar. Aus diesem Grund ist der D-Wave Quanten-Annealer so aufgebaut, dass nur bestimmte Qubits untereinander gekoppelt werden. Welche Auswirkung dies auf die Modellierung von Problemen hat, wird in einem späteren Abschnitt über konkrete Problemmodellierung behandelt. Es existieren verschiedene Architekturen, die jeweils verschiedene Qubit-Topologien liefern. Die einfachste Architektur von D-Wave ist die Chimera-Topology. Hier werden Qubits in 4x4-Blöcken angeordnet und untereinander gekoppelt. Die Qubits entsprechen dabei den langen Schleifen, zu sehen in Abbildung 3.5.

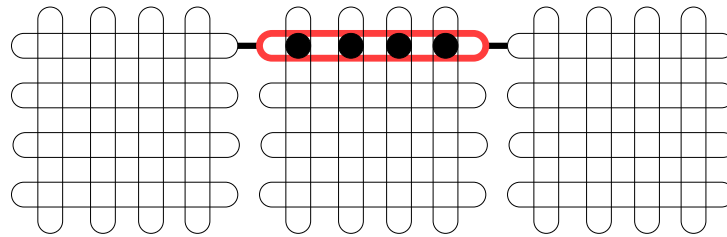


Abbildung 3.5: Drei benachbarte Einheitszellen der Chimera-Architektur. Qubits sind in 4x4-Blöcken angeordnet und durch lange Schleifen dargestellt. In der Abbildung sind die Coupler für ein horizontales, mit rot markiertes, Qubit eingezeichnet. Die schwarzen Punkte symbolisieren die Kopplungen des ersten horizontalen Qubit mit den vier vertikalen Qubits der Einheitszelle. Die dicken schwarzen Linien kennzeichnen die Coupler zu den benachbarten Einheitszellen.

Ein solcher Block bildet eine Einheitszelle. Innerhalb dieser Zelle werden jeweils alle horizontalen mit allen vertikalen Qubits gekoppelt, sodass jedes Qubit mit vier weiteren Qubits gekoppelt ist.

Der Quanten-Annealer setzt sich aus mehreren solcher Einheitszellen zusammen. Bei benachbarten Einheitszellen werden zusätzlich noch die jeweils benachbarten vertikalen und horizontalen Qubits gekoppelt (siehe Abbildung 3.5). Dadurch ergibt sich ein großes System aus Qubits, die mit bis zu sechs weiteren Qubits gekoppelt sind. D-Wave ist kontinuierlich an der Weiterentwicklung der Architekturen und hat mittlerweile die Zephyr-Architektur umgesetzt. In dieser ist ein Qubit mit bis zu zwanzig weiteren Qubits gekoppelt. Unabhängig von der Architektur ergeben sich programmierbare Coupler und Bias für verschiedene Qubits mit einem "Up"- und "Down"-Zustand, welche untereinander wechselwirken. Dieser Aufbau ähnelt sehr einem klassischen Modell aus der Physik, dem Ising-Modell [SJ21]. Um dieses soll es im folgenden Abschnitt gehen.

3.1.4 Vom Ising-Modell zur Quadratischen unrestringierten binären Optimierung (Qubo)

Das Ising-Modell beschreibt in der ursprünglichen Form die Wechselwirkung zwischen benachbarten atomaren Spins, welche z.B. die Werte -1 und 1 annehmen können, in einem Feststoff (meist in einer Gitterstruktur angeordnet). Ein reales physikalisches System, wie z.B. Atome die in einer Gitterstruktur gebunden sind, strebt dabei immer den energetisch niedrigsten Zustand an. Durch externe Einflüsse wie Temperatur, elektrische oder magnetische Felder, kann dieser sehr unterschiedlich aussehen. Untereinander gekoppelten Spins von z.B. Elektronen, die extern beeinflusst werden

können, ähnelt sehr dem Aufbau von gekoppelten Qubits. Tatsächlich beschreibt der Quanten-Annealer genau ein solches Ising-Modell.

Definition 3.1.1 (Ising Modell [Cod21]). Ein Ising-Modell besteht aus Spin-Variablen $s_i \in \{-1, 1\}$. Diese Variablen sind durch lokale Bias h_i und die gegenseitige Wechselwirkung zwischen zwei Variablen J_{ij} beeinflusst. Die Energie des Modells ergibt sich wie folgt:

$$E(s) = \sum_i \sum_j J_{ij} \cdot s_i \cdot s_j + \sum_i h_i \cdot s_i \quad (3.2)$$

Das heißt, die Energie berechnet sich aus der Summe von allen Variablen multipliziert mit ihrem Bias und der Summe aus dem Produkt aller gekoppelten Variablen multipliziert mit ihrem Kopplungsfaktor. Ziel ist es, eine Belegung der Spin-Variablen so zu wählen, dass die Energie minimal wird. Δ

CSPs oder auch COPs mit endlichen Domänen können direkt als Ising-Modell modelliert werden. Für CSPs entsprechen dann die niedrigsten Energielösungen des Ising-Modells erfüllenden Belegungen. Die Wahl der Variablenbelegung mit Werten von -1 und 1 (im Ising-Modell) ist meist, vor allem im Bereich der Informatik, nicht intuitiv. Allerdings können Ising-Modelle in Qubos und Qubos analog in Ising Modelle überführt werden. Eine Qubo ist dabei ein analoges Optimierungsproblem zum Ising-Modell, mit dem Unterschied, dass die Variablen nur Werte von 0 und 1 annehmen können. Dieses Modell nutzt somit boolesche Variablen, was für eine intuitive Modellierung von Problemen bzw. die Nutzung von bereits existierenden Modellierungen mit booleschen Variablen erlaubt. Aus diesem Grund wird meist eine Modellierung als Qubo bevorzugt.

Definition 3.1.2 (Quadratische unrestringierte binäre Optimierung (Qubo) [Cod21]). Eine Qubo besteht aus booleschen Variablen $x_i \in \{0, 1\}$. Die Variablen sind durch Parameter Q_{ij} untereinander verknüpft. Die Energie des Systems ergibt sich wie folgt:

$$E(x) = \sum_i \sum_j Q_{ij} \cdot x_i \cdot x_j \quad (3.3)$$

Ziel ist es, eine Belegung der booleschen Variablen so zu wählen, dass die Energie minimal wird. Δ

Ein Unterschied zum Ising-Modell wird schnell deutlich. Die lokalen Bias der Variablen, die im Ising-Modell mit h_i (vgl. Gleichung 3.2) gekennzeichnet sind, kommen im Qubo Modell nicht direkt vor. Da das Quadrat einer booleschen Variablen wieder der booleschen Variable selbst entspricht, können die lokalen Bias einfach in die Parameter Q_{ii} aufgenommen werden. Ein Beispiel für diese Vereinfachung sieht wie folgt aus:

Beispiel 3.1.1. Gegeben seien lineare und quadratische Terme für eine Qubo:

$$2 \cdot x_1 + 3 \cdot x_2 + 4 \cdot x_1 \cdot x_2 \quad (3.4)$$

Die Qubo-Darstellung aus Gleichung 3.3 ist dann wie folgt gegeben:

$$E(x_1, x_2) = 2x_1 + 4x_1x_2 + 0x_2x_1 + 3x_2 \quad (3.5)$$

$$= 2x_1^2 + 4x_1x_2 + 0x_2x_1 + 3x_2^2 \quad (3.6)$$

$$= \begin{pmatrix} x_1 & x_2 \end{pmatrix} \cdot \begin{pmatrix} 2 & 4 \\ 0 & 3 \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (3.7)$$

◇

Um ein Ising-Modell in eine Qubo zu überführen, kann die Variablentransformation $s_i = 2 \cdot x_i - 1$ gewählt werden. Für $x_i = 1$ ergibt sich $s_i = 1$ und für $x_i = 0$ ergibt sich für $s_i = -1$. Analog kann eine Qubo mit der Transformation $x_i = \frac{s_i+1}{2}$ in ein Ising-Modell transformiert werden. Beim Modellieren von Problemen können also beide Modelle genutzt werden. Es können beliebige CSPs mit endlichen Domänen in Äquivalente Qubos durch Hilfsvariablen überführt werden. Ein Verfahren zur Automatischen Konvertierung von MiniZinc Programmen in Qubos lässt sich hier finden [Wol24, WG23]. Ein solches automatisiertes Verfahren führt aber häufig zur Einführung von vielen Hilfsvariablen. Die Nutzung von Hilfsvariablen kann aufgrund der stark beschränkten Anzahl an Qubits schnell zu Problemen führen. Aus diesem Grund kann es trotzdem sinnvoll sein, händisch CSPs in Qubos umzuwandeln.

Nachdem das gegebene Problem erfolgreich in ein Ising-Modell überführt wurde, kann das Minimum dieses Modells bestimmt werden, um eine Lösung für das Problem zu erhalten. Eine unbeantwortete Frage ist, wie genau das Minimum eines Ising-Modells berechnet wird. Dabei kann das Quanten Adiabatic Theorem helfen. Um die konkrete Umsetzung dieses Theorems im Quanten-Annealer geht es im folgenden Abschnitt.

3.1.5 Quantum Adiabatic Theorem in der Praxis

Das Quantum Adiabatic Theorem [SN20], welches in Abschnitt 2.3 hergeleitet wurde, besagt, dass zeitlich veränderliche quantenmechanische Systeme in demselben Eigenzustand bleiben, solange sie sich nur langsam genug verändert. Dieses theoretische Resultat kann genutzt werden, um mithilfe von Quanten-Annealern das Minimum eines Ising-Modells zu bestimmen. Hierfür wird der Annealer in einen einfachen Startzustand gebracht, dessen energetisch minimaler Eigenzustand bekannt ist und leicht präpariert werden kann. Danach wird das System langsam zum Ziel-Ising-Modell

überführt und bleibt dabei theoretisch im energetisch minimalen Eigenzustand. Der Starthamiltonien des Systems hat dabei folgende Form.

$$\sum_{i=1}^n \sigma_x^{(i)} \text{ wobei } \sigma_x^{(i)} = I_1 \otimes \dots \otimes I_{i-1} \otimes \sigma_x \otimes I_{i+1} \otimes \dots \otimes I_n \quad (3.8)$$

$\sigma_x^{(i)}$ ist der Pauli-x Operator aus Abschnitt 2.2.5, der auf das i -te Qubit wirkt. Dieser Hamiltonien, auch transversaler Feldhamiltonien bezeichnet, entspricht dem Anlegen eines magnetischen Feldes, das senkrecht zu dem Magnetfeld der Qubits ausgerichtet ist. Warum dieser Hamiltonien sich besonders gut als Starthamiltonien eignet wird bei der Betrachtung der Eigenwerte deutlich. Die Berechnung der Eigenwerte für ein Zwei-Qubit-System wird nachfolgend verdeutlicht.

$$\begin{aligned} \sigma_x^{(1)} + \sigma_x^{(2)} &= \sigma_x \otimes I + I \otimes \sigma_x \\ &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \otimes \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix} = H_s \end{aligned}$$

Die Eigenwerte und Eigenvektoren des Starthamiltoniens H_s lauten wie folgt:

$$\text{Eigenwerte: } -2, 2, 0, 0 \quad \text{Eigenvektoren: } \begin{pmatrix} 1 \\ -1 \\ -1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}, \begin{pmatrix} -1 \\ 0 \\ 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \\ -1 \\ 0 \end{pmatrix} \quad (3.9)$$

Der energetisch niedrigste Eigenzustand ist eine Superposition aller vier klassischen Zustände, die die Qubits annehmen können. Analog ist Für n -Qubits der energetisch niedrigste Zustand die Superposition aller möglichen 2^n klassischen Zustände. Dieser Zustand kann mithilfe kontrollierter Mikrowellenpulsen erzeugt werden. Es ist also möglich, den Annealer in einen initialen energetisch minimalen Eigenzustand zu versetzen. Von diesem energetisch niedrigsten Eigenzustand aus startet der Annealing-Prozess. Danach wird der Zielhamiltonien über eine bestimmte Zeit zugeschaltet. Der Zielhamiltonien lässt sich aus dem problemspezifischen Ising-Modell ableiten.

Dazu wird der negative Pauli-z Operator verwendet. Dieser liefert, angewendet auf ein einzelnes Qubit, genau die Eigenwerte, die den Werten der Spin-Variablen entsprechen.

$$-\sigma_z |0\rangle = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = -1 \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = -1 \cdot |0\rangle \quad (3.10)$$

$$-\sigma_z |1\rangle = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 1 \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 1 \cdot |1\rangle \quad (3.11)$$

Die Spin-Variablen im Ising-Modell können somit durch den negativen Pauli-z Operator ersetzt werden. Einsen, die durch die Substitution aus booleschen Variablen entstehen, werden durch Einheitsmatrizen ersetzt. Die vollständige Transformation eines CSPs bis hin zu einem Zielhamiltonien wird am XOR-Beispiel mit zwei Variablen verdeutlicht.

Beispiel 3.1.2. Gegeben seien das CSP $P = (\{x, y\}, \{D_x, D_y\}, \{c_{x,y}\})$ mit $D_x = \{0, 1\}$, $D_y = \{0, 1\}$ und $c_{x,y} = x \text{ XOR } y$. Zunächst muss eine Qubo gefunden werden, die minimal ist, wenn das XOR-Constraint erfüllt ist. Ein möglicher Ausdruck sieht wie folgt aus:

$$2 \cdot xy - x - y \quad (3.12)$$

Dieser Ausdruck liefert -1 , wenn das Constraint erfüllt ist und den Wert 0 , wenn es nicht erfüllt ist. Jetzt muss der Term für die Qubo in einen Term für ein Ising-Modell überführt werden. Dabei wird für jede boolesche Variable x der Ausdruck $\frac{x_s+1}{2}$ eingesetzt. Dadurch werden alle booleschen Variablen in Spin-Variablen mit der Domäne $\{-1, 1\}$ überführt.

$$2 \cdot \frac{x_s+1}{2} \cdot \frac{y_s+1}{2} - \frac{x_s+1}{2} - \frac{y_s+1}{2} \quad (3.13)$$

Dieser Ausdruck kann jetzt in Operator-Form gebracht werden. Dazu werden die

Spin-Variablen durch $-\sigma_z$ und die Einsen durch I , der Einheitsmatrix, ersetzt.

$$\begin{aligned}
& 2 \cdot \frac{I - \sigma_z^{x_s}}{2} \cdot \frac{I - \sigma_z^{y_s}}{2} - \frac{I - \sigma_z^{x_s}}{2} - \frac{I - \sigma_z^{y_s}}{2} \\
&= 2 \cdot \frac{I - \sigma_z}{2} \otimes \frac{I - \sigma_z}{2} - \frac{I - \sigma_z}{2} \otimes I - I \otimes \frac{I - \sigma_z}{2} \\
&= 2 \left(\begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \otimes \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \otimes \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \right) \\
&= 2 \cdot \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} - \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} - \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \\
&= \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} = H_t
\end{aligned}$$

Die Matrix H_t ist der Zielhamiltonien für das XOR-Problem. Die Berechnung der Eigenwerte zeigt, dass die energetisch niedrigsten Eigenzustände ($\lambda_1 = \lambda_2 = -1$) mit unseren erwarteten Lösungen übereinstimmen.

$$\begin{aligned}
\lambda_1 = -1, v_1 &= \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = |10\rangle & \lambda_2 = -1, v_2 &= \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} = |01\rangle \\
\lambda_3 = +0, v_3 &= \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = |11\rangle & \lambda_4 = +0, v_4 &= \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = |00\rangle
\end{aligned}$$

◇

Für die klassischen Zustände $|10\rangle$ und $|01\rangle$ befindet sich das System im energetisch niedrigsten Zustand. Das Quantum Adiabatic Theorem kann jetzt genutzt werden, um das XOR-Problem zu lösen. Das System wird in den Startzustand, beschrieben durch den Starthamiltonien H_s , gebracht. Der energetisch niedrigste Zustand ist dabei durch Mikrowellenpulse leicht zu präparieren. Danach wird der initiale Hamiltonian abgeschwächt und über eine bestimmte Zeit der Zielhamiltonian H_t zugeschaltet. Die Rate, mit der die einzelnen Hamiltonian manipuliert werden, hängt vom sogenannten Annealing Schedule ab. Das Standard-Anneal Schedule ist in Abbildung 3.6 zu sehen.

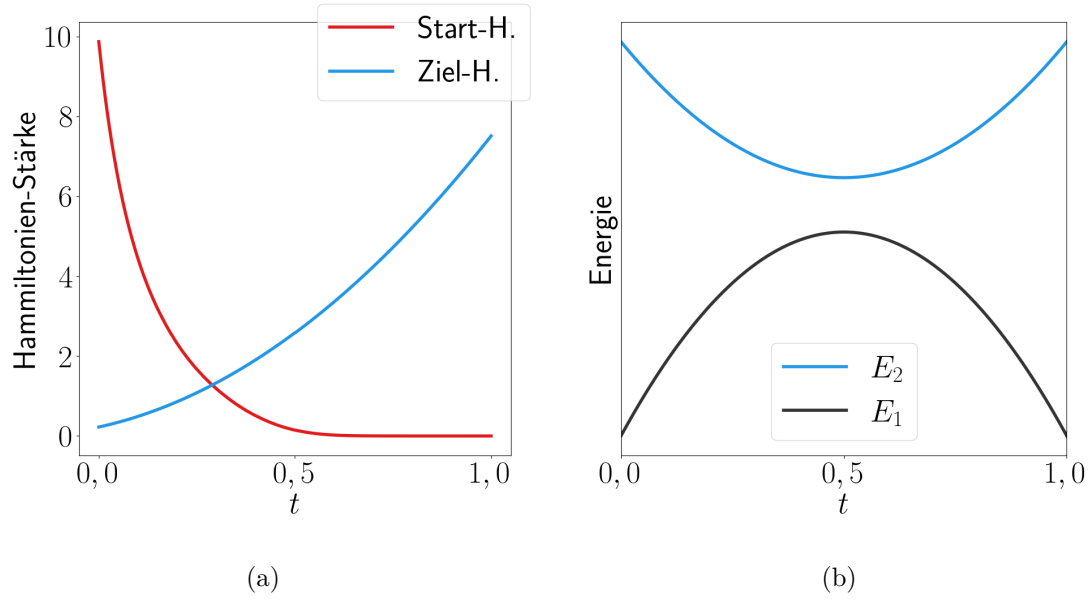


Abbildung 3.6: (a) Kurven des Standard-Anneal Schedules. Der Parameter $A(t)$ ist durch die rote und der Parameter $B(t)$ ist durch die blaue Kurve dargestellt. (b) Qualitative Darstellung der zwei kleinsten Eigenwerte E_1 und E_2 während des Annealings.

Da sowohl der Start- als auch der Zielhamiltonian kontinuierlich verändert werden, verändern sich auch die Eigenwerte des zusammengesetzten Hamiltonian H . Wobei H wie folgt definiert ist:

$$H = A(t) \cdot H_s + B(t) \cdot H_t \quad (3.14)$$

Die Koeffizienten $A(t)$ und $B(t)$ sind jeweils durch das Annealing-Schedule gegeben, welches von $t = 0$ bis $t = 1$ während des Annealings abgefahren wird. Die konkrete Veränderung der Eigenwerte von H ist dabei problemspezifisch. In Abbildung 3.6 ist der Verlauf der Eigenwerte qualitativ dargestellt. Dabei sei angemerkt, dass die minimale Energielücke zwischen den zwei energetisch minimalen Eigenzuständen während des Annealings abnimmt, was sich negativ auf das Anneal-Ergebnis auswirkt. Wird der Prozess des Annealings langsam genug vollzogen, befindet sich das System am Ende im energetisch niedrigsten Zustand des Zielhamiltonian. Durch Messen der Qubits kann dann eine Lösung für das XOR-Problem abgelesen werden.

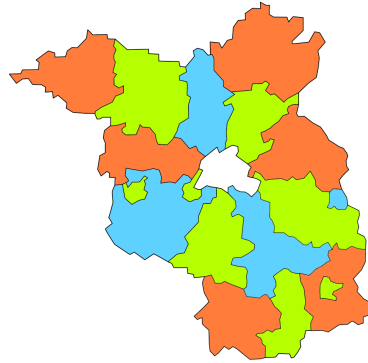


Abbildung 3.7: Färbung der Landkreise in Brandenburg. Ohne Berlin werden drei Farben benötigt, mit Berlin vier Farben.

3.2 Lösen von CSPs auf einem Quanten-Annealer

Um ein CSP auf dem Quanten-Annealer lösen zu können, muss dieses in eine für den Quanten-Annealer nutzbare Form transformiert werden. Der Ablauf dazu soll am Problem der Graphfärbung und des Rotating Rosterings demonstriert werden.

3.2.1 Graph-Färbung als Qubo-Modell

Eine Färbung eines Graphen weist jedem Knoten eine Farbe zu. Ziel ist es, jeden Knoten so einzufärben, dass keine adjazenten Knoten dieselbe Farbe besitzen. Bei solchen Problemen ist meist eine begrenzte Anzahl an verschiedenen Farben vorgegeben. Dieses zunächst rein mathematische Problem kann, auf das Färben von z.B. Landkarten übertragen werden. Hierbei müssen alle Länder (Knoten) eine andere Farbe als ihre Nachbarländer (adjazente Knoten) erhalten. Das Problem wird beispielhaft für die Landkreise in Brandenburg gelöst. Jeder der 18 Landkreis soll eine andere Farbe, als seine benachbarten Landkreise erhalten. Eine mögliche Lösung für dieses Problem ist in Abbildung 3.7 zu sehen.

Qubo-Modellierung. Der Quanten-Annealer ist in der Lage Variablenbelegungen zu finden, die ein gegebenes Ising-Modell minimieren. Um eine Lösung für das Graphfärbungsproblem zu finden, muss dieses in ein Ising-Modell überführt werden, welches für erfüllende Variablenbelegungen minimal ist. Qubo und Ising-Modelle sind äquivalent ineinander überführbar. Das CSP wird hier als Qubo modelliert, weil die Integer-Variablen des ursprünglichen Problems mithilfe der One Hot-Kodierung intuitiv durch boolesche Variablen dargestellt werden können.

Variablenkodierung. Im ursprünglichen Problem wird jedem Landkreise eine Variable x_i mit der Domäne $D_i = \{0, 1, 2\}$ bzw. $D_i = \{0, 1, 2, 3\}$ zugewiesen. Die einzelnen Werte stehen dabei jeweils für die Farben 0 =rot, 1 =grün, 2 =blau und 3 =weiß. Diese werden mithilfe der One Hot-Kodierung durch drei bzw. vier boolesche Variablen ersetzt. Eine Variable $x_{i,j}$ ist genau dann wahr, wenn der Landkreis i mit der Farbe j eingefärbt wird. Daraus folgt auch, dass von allen $x_{i,j}$ für ein festes i immer nur exakt eine Variable den Wert wahr annehmen darf. Dies muss als zusätzliches Constraint festgehalten werden. Eine arithmetische Darstellung, bei der „wahr“ dem Wert 1 und „falsch“ dem Wert 0 entspricht, lässt sich wie folgt aufstellen:

$$\forall i \in \{1, \dots, 18\} : \sum_{j=0}^3 x_{i,j} = 1 \quad (3.15)$$

Solche arithmetischen Ausdrücke lassen sich durch äquivalente Umformungen leicht in eine Qubo-Form überführen. Dabei wird die Gleichung so umgestellt, dass auf einer Seite eine 0 steht und beide Seiten quadriert.

$$\sum_{j=0}^3 x_{i,j} = 1 \quad (3.16)$$

$$\sum_{j=0}^3 x_{i,j} - 1 = 0 \quad (3.17)$$

$$\left(\sum_{j=0}^3 x_{i,j} - 1 \right)^2 = 0^2 \quad (3.18)$$

$$\sum_{j=0}^3 x_{i,j}^2 + \sum_{j < k} 2x_{i,j}x_{i,k} - \sum_{j=0}^3 2 \cdot x_{i,j} + 1 = 0 \quad (3.19)$$

$$\sum_{j < k} 2x_{i,j}x_{i,k} - \sum_{j=0}^3 x_{i,j} = -1 \quad (3.20)$$

Die Konstante kann in dem Qubo-Ausdruck weggelassen werden. Die übrigen Terme entsprechen alle dem Qubo-Format. Eine ausführlichere Beschreibung der einzelnen Rechenschritte lässt sich im Anhang 7.1 finden. Variablenbelegungen, die der One Hot-Kodierung entsprechen, ergeben eingesetzt in den Ausdruck 3.20, den Wert -1 und erfüllen das Constraint. Alle anderen Belegungen, die nicht der One Hot-Kodierung entsprechen, ergeben Werte die größer als -1 sind. Das One Hot-Kodierungs-Constraint wurde durch diese Umwandlung in eine äquivalente Qubo-Darstellung gebracht. Als nächstes wird die Nachbarschaft modelliert.

Nachbarschaft. Zwei benachbarte Landkreise sollen in zwei verschiedenen Farben eingefärbt werden. Diese Bedingung lässt sich sehr einfach modellieren. Für jedes

	Woche 1	Woche 2	Woche 3	Woche 4
Angestellter 1	Sp. Woche 1	Sp. Woche 2	Sp. Woche 3	Sp. Woche 4
Angestellter 2	Sp. Woche 2	Sp. Woche 3	Sp. Woche 4	Sp. Woche 1
Angestellter 3	Sp. Woche 3	Sp. Woche 4	Sp. Woche 1	Sp. Woche 2
Angestellter 4	Sp. Woche 4	Sp. Woche 1	Sp. Woche 2	Sp. Woche 3

Tabelle 3.1: Visualisierung des rotierenden Schichtplans. Die Abkürzung Sp. steht hier für Schichtplan.

Paar benachbarter Landkreise x_i und x_j werden jeweils für alle Farben die folgenden Qubo-Terme eingeführt:

$$\forall (i, j) \in \text{Nachbarn} \forall k \in \{0, 1, 2, 3\} : x_{i,k} \cdot x_{j,k} \quad (3.21)$$

Der finale Qubo-Ausdruck ist die Summe aller One Hot (siehe Gleichung 3.20) und aller Nachbarschaftsterme (siehe Term 3.21). Durch das One Hot-Constraint wird garantiert, dass jedem Landkreis eine Farbe zugewiesen wird. Zudem verhindern die Terme 3.21, dass zwei benachbarte Landkreise dieselbe Farbe erhalten. Die einzelnen Terme erreichen nur dann ihr Minimum, wenn je zwei benachbarten Landkreisen verschiedene Farben zugewiesen werden.

Das Problem der Graphfärbung lässt sich mithilfe der One Hot-Kodierung sehr direkt in ein Qubo-Format überführen. Allerdings existieren CSPs, bei denen diese Umwandlung nicht so leicht möglich ist und viele verschiedene Modellierungstechniken angewendet werden müssen. Ein Beispiel dafür ist das Rotating Rostering-Problem, welches im folgenden Abschnitt eingeführt und in eine Qubo-Darstellung transformiert wird.

3.2.2 Rotating Rostering-Problem als Qubo-Modell

Das Rotating Rostering-Problem modelliert das Problem der Schichtplanerstellung. Für eine gegebene Anzahl von Angestellten soll ein Schichtplan mit den Schichten Früh, Spät, Nacht und Frei entworfen werden, der bestimmte Constraints erfüllt. Hierbei handelt es sich um einen rotierenden Schichtplan. Es wird für n Angestellte ein n -wöchiger Schichtplan erstellt. In der ersten Woche wird dem ersten Arbeitnehmer der Schichtplan der ersten Woche und dem zweiten Arbeitnehmer der Schichtplan der zweiten Woche zugeordnet. In der zweiten Woche erhält der erste Mitarbeiter den Schichtplan für die zweite Woche und der zweite Mitarbeiter den Schichtplan für die dritte Woche. So rotiert der Schichtplan, bis wieder von vorne begonnen wird. Der Ablauf ist nochmals in Tabelle 3.1 verdeutlicht.

Der erstellte Schichtplan muss verschiedene Constraints erfüllen, die entweder vom Arbeitsbetrieb abhängen, als gute Arbeitspraxis gelten, oder gesetzlich vorgeschrieben sind. Die Constraints aus der Modellierung in [LBH] sind im folgenden Teil aufgelistet:

C1 Schichtbedarf:

Für jeden Wochentag und jede Schicht ist vorgegeben, wie viele Angestellte der jeweiligen Schicht zugeteilt werden müssen. Eine beispielhafte Zuteilung ist in Tabelle 3.2 zu sehen. In diesem Beispiel wird am Wochenende weniger Personal benötigt, als unter der Woche.

	Mo	Di	Mi	Do	Fr	Sa	So
Frei	2	2	2	2	2	4	4
Früh	2	2	2	2	2	2	2
Spät	2	2	2	2	2	1	1
Nacht	2	2	2	2	2	1	1

Tabelle 3.2: Schichtbedarf der einzelnen Wochentage für acht Angestellte.

C2 Wochenendschichten:

Für einen Angestellten müssen am Wochenende (Samstag und Sonntag) immer die gleichen Schichten vergeben werden.

C3 Minimale Schichtwiederholung:

Jede Schicht muss mindestens s_{min} mal hintereinander im Schichtplan auftreten, bevor eine andere Schicht verteilt werden kann.

C4 Vorwärtsrotation:

Zugeteilte Schichten dürfen nur in einer bestimmten Reihenfolge vergeben werden. Auf eine beliebige Schicht, darf nur eine spätere oder eine Freischicht folgen. Zum Beispiel kann nach einer Spätschicht nur eine Nachtschicht oder eine Freischicht folgen.

C5 Maximale Schichtwiederholung:

Jede Schicht darf maximal s_{max} oft wiederholt vergeben werden, bis eine andere Schicht gewählt werden muss.

C6 Erholungstage:

Für jeden Angestellten müssen innerhalb von zwei Wochen mindestens zwei freie Tage liegen.

Qubo-Modellierung. Die Modellierung ähnelt der für das Graphfärbungsproblem. Die Integer Variablen werden wieder One Hot-kodiert und danach werden die einzel-

nen Constraints in ein Qubo-Format gebracht. Die Umwandlung orientiert sich stark an der existierenden Modellierung aus der CSPLib [csp99]. Es gibt verschiedene Möglichkeiten das Rotating Rostering-Problem als CSP zu modellieren, die alle jeweils andere Qubo-Transformationen zur Folge haben. Hierbei wird jedoch die in [LBH] vorgeschlagene Variante verwendet.

Um einzelne Constraints in äquivalente Qubo-Ausdrücke zu überführen, müssen diese Qubo-Ausdrücke eine bestimmte Bedingung erfüllen. Diese Bedingung wurde bei der Graphfärbung nicht aktiv betrachtet und durch die Einfachheit der Umwandlung automatisch erfüllt. Diese Bedingung wird in den folgenden Abschnitten als Qubo-Bedingung bezeichnet:

Definition 3.2.1 (Qubo-Bedingung). Wird ein Constraint in eine Qubo-Form umgewandelt, müssen alle erfüllenden Belegungen dieses Constraints, eingesetzt in den Qubo-Term, denselben minimalen Wert ergeben. Alle nicht erfüllenden Belegungen müssen beliebige Werte ergeben, die größer als das Minimum sind. Δ

Der Grund für diese Einschränkung ist folgender. Das Vorgehen bei der Umwandlung arbeitet jedes Constraint einzeln ab. Das bedeutet, für jedes Constraint wird ein Qubo-Term gefunden, der genau dann minimal ist, wenn die Variablen-Belegung das Constraint erfüllt. Würde es erfüllende Belegungen geben, die unterschiedliche Werte annehmen können, entsteht folgendes Problem. Es kann vorkommen, dass es für den finalen Qubo-Ausdruck (der Summe über alle einzelnen Qubo-Constraint-Ausdrücke) besser ist, ein Constraint zu verletzen, dafür aber zwei andere Constraints mit einem besonders niedrigen Wert zu erfüllen. Um dieses Problem zu verhindern, müssen alle erfüllenden Belegungen den selben minimalen Qubo-Wert erhalten. Ein weiterer Vorteil ist, dass alle Lösungen den selben minimalen Energiewert besitzen. Dadurch ist es sehr einfach zu überprüfen, ob eine Variablenbelegung eine Lösung ist.

Variablenkodierung. Die Variablenkodierung erfolgt analog zur Graphfärbung mithilfe der One Hot-Kodierung. Beim Rotating Rostering-Problem werden Variablen x_1 bis $x_{w \cdot 7}$, wobei w der Anzahl der Wochen bzw. Angestellten entspricht, verwendet. Diese besitzen folgende Domänen: $D_1 = D_2 = \dots = D_{w \cdot 7} = \{0, 1, 2, 3\}$. Die Werte stehen dabei jeweils für die Schichten 0 = Frei, 1 = Früh, 2 = Spät und 3 = Nacht. Eine Variable x_i wird in vier neue boolesche Variablen $x_{i,0}, x_{i,1}, x_{i,2}, x_{i,3}$ umgewandelt. Eine Variable $x_{i,j}$ ist genau dann wahr, wenn die Variable x_i den Wert $j \in \{0, 1, 2, 3\}$ annimmt. Der Term auf der linken Seite der Gleichung 3.20 muss wieder für jede One Hot-kodierte Variable eingefügt werden:

$$\forall i \in \{1, \dots, w \cdot 7\} : \sum_{j < k}^3 2x_{i,j}x_{i,k} - \sum_{j=0}^3 x_{i,j} \quad (3.22)$$

Schichtbedarf. C1 Sei der Schichtbedarf durch eine Tabelle (vgl. Tabelle 3.2) mit Einträgen $b_{i,j}$, wobei i der Wochentag und j der Schicht entspricht, gegeben. Dann sieht der dazugehörige Qubo-Term wie folgt aus:

$$\forall i \in \{1, \dots, w\} \forall j \in \{0, 1, 2, 3\} : \left(\sum_{k=0}^{n-1} x_{i+k \cdot 7, j} - b_{i,j} \right)^2 \quad (3.23)$$

Wochenendschichten. C2 Dass am Wochenende immer dieselbe Schicht zugewiesen werden soll, kann auch mit einfacher Gleichungsumformung realisiert werden. Der entsprechende Ausdruck lautet wie folgt:

$$\forall i \in \{1, \dots, w\} \forall j \in \{0, 1, 2, 3\} : (x_{i+5, j} - x_{i+6, j})^2 \quad (3.24)$$

Ein anderes Vorgehen ist bei der minimalen Schichtwiederholung nötig. Eine einfache Repräsentation als Summengleichung ist hier nicht mehr möglich.

Minimale Schichtwiederholung. C3 Die Umwandlung dieses Constraints wird beispielsweise für den Wert $s_{min} = 2$ gezeigt. Es sollen immer mindestens zwei gleiche Schichten aufeinander folgen. Im Fall von Integer-Variablen sieht das Constraint wie folgt aus:

$$\forall i \in \{1, \dots, w \cdot 7\} : (x_i \neq x_{(i+1)_{\text{mod}}}) \Rightarrow (x_i = x_{(i-1)_{\text{mod}}}) \quad (3.25)$$

Der Ausdruck i_{mod} steht dabei für $(i \bmod (w \cdot 7)) + 1$. Im folgenden Teil werden aus Übersichtsgründen die mod-Notationen an Indizes weggelassen. Diese sind bei Additionen oder Subtraktionen aber weiterhin notwendig. Die Variablen sind One Hot-Kodiert, wodurch das Constraint eine andere Form annimmt. Anstelle von drei aufeinanderfolgenden Integer-Variablen x_{i-1}, x_i, x_{i+1} werden für jede Schicht individuell drei aufeinanderfolgende One Hot-Variablen $x_{i-1, j}, x_{i, j}, x_{i+1, j}$ verglichen. Die erfüllenden Belegungen sind durch die Tabelle 3.3 gegeben.

Hierbei ist es wichtig, dass auch wirklich nur Belegungen als falsch gewertet werden, die das Constraint aktiv verletzen. Die Belegung 1, 0, 0 wirkt zunächst falsch. Allerdings kann es sein, dass eine andere Schicht an den Tagen i und $i + 1$ gewählt wurde. Dies lässt sich nur durch das Betrachten der Schichtvariablen für j nicht erfassen. Falls an den Tagen i und $i + 1$ auch nicht dieselbe Schicht gewählt werden würde, wird dies von dem Constraint über die Variablen $x_{i-1, k}, x_{i, k}, x_{i+1, k}$ erfasst. k bezeichnet dabei die Schicht, die für Tag i gewählt wurde. Für diese Wahrheitstabelle lässt sich händisch ein passender Qubo-Term finden. Dieser muss für alle möglichen Tage und alle möglichen Schichten hinzugefügt werden.

$$\forall i \in \{1, \dots, w \cdot 7\} \forall j \in \{0, 1, 2, 3\} : x_{i, j} - x_{i-1, j} \cdot x_{i, j} + x_{i-1, j} \cdot x_{i+1, j} - x_{i, j} \cdot x_{i+1, j} \quad (3.26)$$

$x_{i-1,j}$	$x_{i,j}$	$x_{i+1,j}$	CSP	Qubo
0	0	0	Wahr	0
0	0	1	Wahr	0
0	1	0	Falsch	> 0
0	1	1	Wahr	0
1	0	0	Wahr	0
1	0	1	Falsch	> 0
1	1	0	Wahr	0
1	1	1	Wahr	0

Tabelle 3.3: Wahrheitswertetabelle für das Constraint, dass mindestens zweimal dieselbe Schicht aufeinanderfolgen muss.

Dieser Ausdruck ist für alle erfüllenden Belegungen 0 und für die beiden nicht erfüllenden Belegungen 1. Der Qubo-Term erfüllt die notwendige Qubo-Bedingung 3.2.1 und kann für die Modellierung genutzt werden.

Vorwärtsrotation. C4 Das Vorgehen, um das Constraint der Schichtreihenfolge in einen Qubo-Term zu transformieren, kann ähnlich gestalten werden. Wieder kann eine Wahrheitswertetabelle angegeben werden und daraus Qubo-Ausdrücke ohne großen Aufwand händisch bestimmt werden. Die beiden benötigten Qubo-Ausdrücke sehen wie folgt aus:

$$\forall i \in \{1, \dots, w \cdot 7\} : x_{i,3} - x_{i,3} \cdot x_{i+1,0} - x_{i,3} \cdot x_{i+1,3} + 2 \cdot x_{i+1,0} \cdot x_{i+1,3} \quad (3.27)$$

$$\begin{aligned} \forall i \in \{1, \dots, w \cdot 7\} : & x_{i,2} - x_{i,2} \cdot x_{i+1,0} - x_{i,2} \cdot x_{i+1,2} - x_{i,2} \cdot x_{i+1,3} \\ & + 2 \cdot x_{i+1,0} \cdot x_{i+1,2} + 2 \cdot x_{i+1,2} \cdot x_{i+1,3} + 2 \cdot x_{i+1,0} \cdot x_{i+1,3} \end{aligned} \quad (3.28)$$

Der erste Term stellt sicher, dass nach einer Nachtschicht nur eine weitere Nachtschicht oder ein freier Tag folgen kann. Der zweite Term garantiert, dass nach einer Spätschicht nur eine Spät- oder Nachtschicht oder ein freier Tag folgen kann. Für freie Tage und Frühschichten sind solche Constraints nicht notwendig, da nach diesen jede beliebige Schicht folgen kann.

Maximale Schichtwiederholung. C5 Für das Constraint, dass nur eine maximale Anzahl an selben Schichten in Folge zugeteilt werden darf, muss ein anderes Vorgehen verwendet werden. Dieses wird beispielhaft an dem Wert $s_{max} = 4$ verdeutlicht. In diesem Constraint werden jeweils fünf aufeinanderfolgende One Hot-Variablen der selben Schicht ($x_{i,j}$ wobei $i \in \{1, \dots, 5\}$) betrachtet. Alle Belegungen mit der Ausnahme, dass alle Variablen 1 sind, sollen akzeptiert werden. Hier ist zu beachten, dass zwar Belegungen wie 0, 1, 0, 1, 0 nicht das maximale Schichten-Constraint verletzen,

aber das Constraint der minimalen Schichtlänge. Da alle Qubo-Ausdrücke für erfüllende Belegungen einen eindeutigen Minimalwert besitzen, ist es egal, ob in diesem Constraint diese Belegungen als erfüllend oder nicht erfüllend modelliert werden. Das vollständige Problem erreicht nur dann sein eindeutiges Minimum, wenn alle Qubo-Terme zusammen ihr eindeutiges Minimum erreichen. Aus diesem Grund wird nur die Belegung 1, 1, 1, 1, 1 als nicht erfüllend modelliert.

Es existiert kein direkter Qubo-Ausdruck für dieses Problem. Das eindeutige Minimum des Qubo-Ausdrucks muss 0 sein, da die Belegung 0, 0, 0, 0, 0 eine erfüllende Belegung ist und immer den Qubo-Wert 0 liefert. Das bedeutet, dass auch die Qubo-Ausdrücke für eine einzelne 1 in der Belegung 0 liefern müssen. Daraus folgt, dass die Parameter a_i der Ausdrücke $a_i \cdot x_i$ auch gleich 0 sind. Weiterhin müssen die Qubo-Werte für genau zwei Einsen in der Belegung auch 0 ergeben. Daraus folgt wiederum, dass die Parameter $a_{i,j}$ für Ausdrücke $a_{i,j} \cdot x_i \cdot x_j$ auch 0 sind. Dadurch sind zwangsläufig alle Parameter $a_i = 0$ und die Belegung 1, 1, 1, 1, 1 liefert den Wert 0, was kein gültiger Qubo-Ausdruck ist. Um dieses Problem zu umgehen, müssen Hilfsvariablen eingeführt werden.

Hilfsvariablen. Eine Hilfsvariable a_i ist eine weitere boolesche Variable, die dem CSP hinzugefügt wird, um das Finden eines Qubo-Ausdruckes zu ermöglichen. Eine wichtige Einschränkung ist dabei, dass mindestens eine Belegung der Hilfsvariablen existiert, für die eine erfüllende Belegung des ursprünglichen Problems sein eindeutiges Minimum erreicht. Es stellt dabei kein Problem dar, wenn für andere Belegungen der Hilfsvariablen für erfüllende Belegungen nicht das eindeutige Minimum, sondern ein größerer Wert erreicht wird. Es darf sich jedoch kein Wert ergeben, der kleiner als das eindeutige Minimum ist, da sonst das gleiche Problem auftreten kann, dass am Anfang dieses Kapitels besprochen wurde. Durch das Hinzufügen einer Hilfsvariable lässt sich leicht ein Qubo-Ausdruck für folgende Gleichung finden: $z = (x_i \wedge x_j)$. Die Variable z kann als Hilfsvariable genutzt werden, um eine Verkettung von $\wedge$ -Ausdrücken zu reduzieren und das maximale Schichten-Constraint umzuwandeln. Durch Einfügen einer Hilfsvariable a ergibt sich der folgender Ausdruck:

$$(x_i + x_j - 2 \cdot z - a)^2 \quad (3.29)$$

Der Ausdruck kann genau dann 0 werden, wenn z dem Ergebnis der Operation $x_i \wedge x_j$ entspricht. In allen anderen Fällen lässt sich auch keine Belegung der Hilfsvariable finden, die es ermöglichen würde, den Ausdruck auf 0 zu bringen. Ein maschinelles Vorgehen zum Finden solcher Ausdrücke wird in [PVVL22] beschrieben. Mithilfe dieses neuen Ausdrucks ist es möglich, das Constraint in einen Qubo-Ausdruck umzuwandeln. Unser ursprüngliches Problem ist es, einen Ausdruck zu finden, der > 0 wird, wenn alle Variablen 1 sind und sonst 0 liefert. Aussagenlogisch sieht dies wie

folgt aus:

$$\forall i \in \{1, \dots, w \cdot 7\} \forall j \in \{0, 1, 2, 3\} : x_{i,j} \wedge x_{i+1,j} \wedge x_{i+2,j} \wedge x_{i+3,j} \wedge x_{i+4,j} \quad (3.30)$$

Die Verkettung aus $\wedge$ lässt sich aufbrechen und vereinfachen, bis nur noch ein einzelner $\wedge$ -Ausdruck übrig ist.

$$x_{i,j} \wedge x_{i+1,j} = z_1, x_{i+2,j} \wedge x_{i+3,j} = z_2, z_1 \wedge z_2 = z_3, z_3 \wedge x_{i+4,j} \quad (3.31)$$

Für einen einzelnen $\wedge$ -Ausdruck lässt sich leicht ein Qubo-Term finden. Eine Möglichkeit ist der Ausdruck $z_3 \cdot x_{i+4,j}$. Dieser liefert eine 1, wenn beide Variablen 1 sind und sonst 0. Jetzt können alle Qubo Terme aufaddiert werden, um den finalen Qubo-Ausdruck zu erzeugen.

$$\begin{aligned} & (x_{i,j} + x_{i+1,j} - 2 \cdot z_1 - a_1)^2 \\ & + (x_{i+2,j} + x_{i+3,j} - 2 \cdot z_2 - a_2)^2 \\ & + (z_1 + z_2 - 2 \cdot z_3 - a_3)^2 \\ & + (z_3 \cdot x_{i+4,j}) \end{aligned}$$

Durch das Einfügen von sechs Hilfsvariablen kann das maximale Schichten-Constraint in eine Qubo-Form gebracht werden. Welche Auswirkungen die Nutzung von Hilfsvariablen für Auswirkungen im Bezug auf den Quanten-Annealer hat, wird in einem späteren Kapitel 3.2.3 behandelt.

Erholungstage. C6 Das einzige, noch nicht betrachtete Constraint ist, dass innerhalb von zwei Wochen mindestens zwei freie Tage liegen müssen. Dieses Constraint stellt wieder eine neue Herausforderung dar. Es müssen je 14 aufeinander folgende Tage betrachtet werden. Das Aufstellen einer Wahrheitswertetabelle und Ablesen eines Qubo-Ausdrucks ist hier nicht mehr realistisch möglich. Es muss eine alternative Technik angewendet werden. Formal bedeutet das Constraint, dass von den 14 Variablen, mindestens zwei einen Wert von 1 haben müssen. Als Formel ausgedrückt sieht dies wie folgt aus:

$$\forall i \in \{1, \dots, w \cdot 7\} : \sum_{k=0}^{13} x_{i+k,0} \geq 2 \quad (3.32)$$

Eine Summe ist leicht in einen Qubo-Term zu transformieren. Bei einer Ungleichung funktioniert die Technik nicht. Allerdings kann durch das Einfügen einer Slack-Variable s jede Ungleichung in eine Gleichung überführt werden. Damit die Slack-Variable nur positive Werte annehmen muss, wird die Ungleichung vorher durch Multiplikation mit -1 in eine kleiner-gleich-Ungleichung umgewandelt.

$$\forall i \in \{1, \dots, w \cdot 7\} : \sum_{k=0}^{13} -x_{i+k,0} + s_i = -2 \text{ wobei } s_i \in \{0, \dots, 12\} \quad (3.33)$$

Die Variable s_i muss mithilfe von booleschen Variablen kodiert werden. Dazu kann wieder die One Hot-Kodierung genutzt werden. Dadurch würden jedoch 13 neue Hilfsvariablen hinzukommen. Da die Slack-Variable nur in diesem Constraint vorkommt, kann sie auch binär kodiert werden. Demnach werden statt 13 nur vier Hilfsvariablen benötigt. Der komplett umgewandelte Qubo-Ausdruck sieht dann wie folgt aus:

$$\forall i \in \{1, \dots, w \cdot 7\} : \left(\sum_{k=0}^{13} -x_{i+k,0} + s_1 + 2 \cdot s_2 + 4 \cdot s_3 + 8 \cdot s_4 + 2 \right)^2 \quad (3.34)$$

Falls nur eine oder keine ursprüngliche Variable mit 1 belegt ist, wird der Ausdruck immer größer 0 sein. Für exakt zwei Variablen mit 1 wird der Ausdruck 0. Wenn mehr als zwei Variablen mit 1 belegt sind, können die Slack-Variablen $s_1, \dots, s_4$ so gewählt werden, dass der Ausdruck wieder exakt 0 ergibt. Dieses Vorgehen kann auch genutzt werden, um das maximale Schichten-Constraint umzuwandeln. Das Rotating Rostering-Problem ist somit vollständig in Qubo-Form umgewandelt und kann auf dem Quanten-Annealer ausgeführt werden. Welche Schritte dafür notwendig sind, wird im folgenden Abschnitt 3.2.3 behandelt.

3.2.3 Embedding und Anneal-Parameter

Der Quanten-Annealer von D-Wave ist in der Lage, den energetisch niedrigsten Zustand eines Ising-Modells zu finden. Das Graphfärbungs und Rotating Rostering-Problem befindet sich aktuell in einer Qubo-Form. Die Python Library PyQubo [ZTT21, TTMT19] kann bei der Modellierung und Transformation zwischen Ising und Qubo-Modellen helfen. Ein beliebiger Qubo-Ausdruck, wie beispielsweise $x_1 + 2 \cdot x_2 - x_1 \cdot x_2$, kann mithilfe von PyQubo wie folgt realisiert werden:

```
x1, x2 = Binary("x1"), Binary("x2")
H = x1 + 2*x2 - x1*x2
```

Außerdem ist es möglich, das erstellte Model zwischen Qubo- und Ising-Darstellung zu transformieren. Das theoretische Vorgehen dazu wurde in 3.1.4 beschrieben. Ein Modell kann kompiliert werden und durch einfache Methodenaufrufe zwischen den einzelnen Darstellungen umgewandelt werden.

```
model = H.compile()
quboModel, qOffset = model.to_qubo()
isingModell isingModellQ, iOffset = model.to_ising()
```

Es werden nicht nur die Koeffizienten der jeweiligen Modelle berechnet und zurückgegeben, sondern auch das Energie-Offset. Konstanten können innerhalb der Qubo- oder auch der Ising-Modelle weggelassen werden. Dadurch verschiebt sich allerdings

das eindeutige Minimum um den Wert dieser Konstante. Der Wert Offset liefert genau diese Verschiebung. Jetzt ist nur noch ein weiterer Schritt nötig, bevor der Quanten-Annealer das CSP lösen kann. Es muss ein passendes Embedding gefunden werden. Wie in Abschnitt 3.1.3 beschrieben, entspricht ein Qubit einer Spin-Variable des Ising-Modells. Die einzelnen Variablen des erstellten Ising-Modells müssen somit auf die physischen Qubits abgebildet werden.

Definition 3.2.2 (Graph Embedding[LL24]). Gegeben seien zwei Graphen G und H . Ein Embedding von G in H ist eine Abbildung $f : V(G) \rightarrow \mathcal{P}(V(H))$ mit folgenden Eigenschaften.

- $\forall v \in V(G) : f(v)$ induziert einen disjunkten verbundenen Teilgraphen in H
- $\forall vw \in E(G) : \text{Es existiert eine Kante in } H, \text{ die die Mengen } f(v) \text{ und } f(w) \text{ verbindet.}$ Δ

Das Embedding f muss keine eins zu eins Abbildung von Knoten aus G auf Knoten von H sein. Es ist auch möglich, einen einzelnen Knoten aus G auf mehrere Knoten aus H abzubilden. Diese Menge an Knoten aus H entspricht einer Kette an physischen Qubits, die zusammen ein logisches Qubit von G repräsentieren. Die erste Bedingung stellt sicher, dass für diese Kette an Qubits auch Kanten in H existieren, die diese verbinden können. Die zweite Bedingung sorgt dafür, dass alle Kanten, die im Originalgraph G vorhanden sind, auch in H existieren. Hier reicht es, wenn eine einzige Kante zwischen zwei Qubit-Ketten (die zwei logische Qubits repräsentieren) im Graph H liegt.

Ein solches Embedding zu finden, ist ein NP-schweres Problem. Mithilfe von Heuristiken kann jedoch versucht werden, schnell eine Lösung zu finden, die so wenig physische Qubits wie möglich verwendet. Im allgemeinen gilt, je mehr Qubits verwendet werden, desto größer ist die Anfälligkeit für äußere Störfaktoren. Dies schadet der Lösungsqualität und es kann vorkommen, dass weniger bis gar keine Variablenbelegungen mehr gefunden werden, die alle Constraints erfüllen. Zu den äußeren Störfaktoren zählen unter anderem thermische Prozesse oder auch die ungewollte magnetische Wechselwirkung von nicht gekoppelten Qubits untereinander. Aus diesem Grund sollte versucht werden, jedes logische Qubit auch nur einem physischen Qubit zuzuweisen.

D-Wave stellt zu diesem Zweck die Python Library *minorminer*[min17] zur Verfügung. Diese implementiert ein heuristisches Verfahren, beschrieben in [CMR14], zur Berechnung eines solchen Embeddings. Zunächst wird ein Embedding für das Graphfärbungsproblem gesucht. Ein solches Embedding lässt sich auf dem AdvantageSystem2 2.6 leicht finden. Die kleine Problemgröße mit nur $3 \cdot 18 = 54$ bzw. $4 \cdot 19 = 76$ Variablen und die zugrunde liegende Graphenstruktur, dargestellt in Abbildung 3.8,

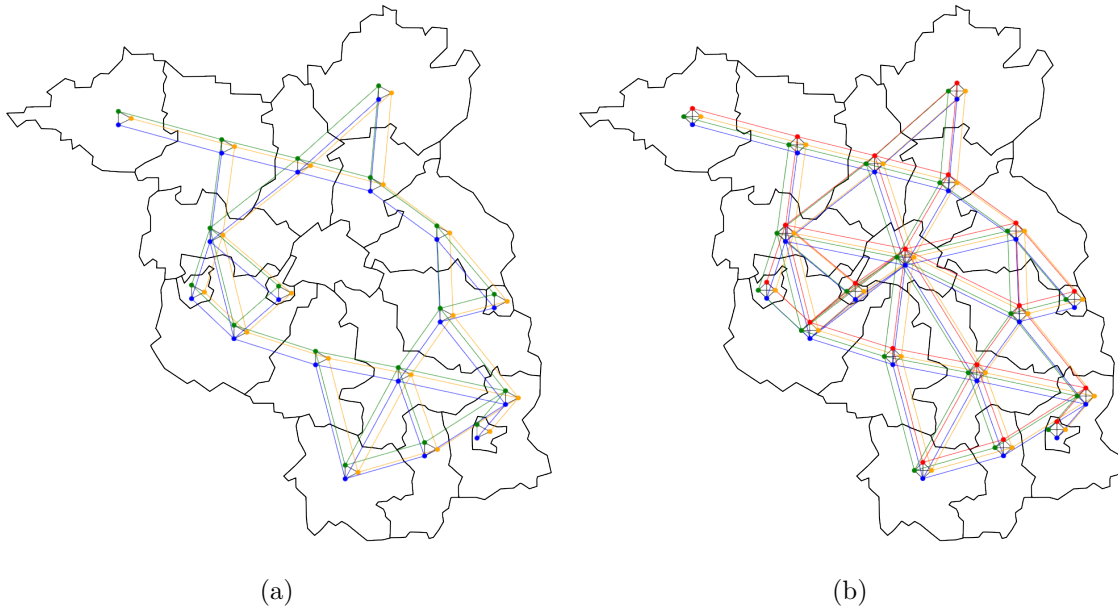


Abbildung 3.8: Graph des Ising-Modells für das Graphfärbungsproblems ohne Berlin mit drei Farben (a) und mit Berlin und vier Farben (b). Jeder Knoten entspricht einer Variable im Ising-Modell und jede Kante einer Variablenverknüpfung.

lassen sich gut auf dem Annealer embedden. Im allgemeinen eignen sich Probleme, in denen Variablen nur sehr lokal und nicht zu dicht untereinander verknüpft sind. Probleme, in denen beispielsweise 18 Variablen alle untereinander verknüpft sind (siehe Term 3.34), lassen sich nur schwer bis gar nicht embedden. Für das Färbungsproblem mit drei verschiedenen Farben müssen maximal zwei Qubits zu einem logischen Qubit kombiniert werden, während für das Problem mit vier Farben maximal drei Qubits kombiniert werden müssen.

Beim Versuch, ein Embedding für das Rotating Rostering-Problem auf dem Advantage System 7.1 zu finden, wird dieses Verfahren aber scheitern. Der Grund liegt vor allem in der Menge der untereinander gekoppelten Variablen, denn die reine Betrachtung der benötigten Variablen liefert zunächst keinen Widerspruch. Der Advantage System 7.1 besitzt 5.554 nutzbare Qubits, wobei ein Qubit mit 15 weiteren gekoppelt ist. Das ursprüngliche Rotating Rostering CSP besteht aus $8 \cdot 7 = 56$ Integer-Variablen. Diese wurden durch die One Hot-Kodierung in $4 \cdot 56 = 224$ boolesche Variablen umgewandelt. Für die Umwandlung einiger Constraints wurden Hilfsvariablen genutzt. Allein durch das maximale Schichten-Constraint werden $56 \cdot 6 \cdot 4 = 1.344$ neue Variablen eingefügt. Das Constraint zur Sicherstellung von genügend freien Ta-

gen innerhalb von zwei Wochen fügt nochmals $56 \cdot 4 = 224$ neue Variablen hinzu. Insgesamt ergeben sich 1.792 Variablen.

Modellierungsoptimierung. Allerdings ist nicht nur die Anzahl der Variablen, sondern auch die Anzahl von untereinander gekoppelten Variablen relevant. Insgesamt ergeben sich 9.344 gekoppelte Variablen. Je nachdem, wie die zugrundeliegende Graphstruktur unseres Problems aussieht, kann es schwerer sein, ein Embedding auf dem Quanten-Annealer zu finden. Selbst wenn es ein Embedding gibt, kann es vorkommen, dass das heuristische Verfahren dieses nicht findet. Allerdings können noch Optimierungen an der Modellierung vorgenommen werden, um die Anzahl der Variablen stark zu reduzieren. Das maximale Schichten-Constraint, welches die meisten neuen Variablen einfügt, kann mithilfe von Ungleichungen und Slack-Variablen in folgenden Qubo-Ausdruck überführt werden.

$$\begin{aligned} & \forall i \in \{1, \dots, w \cdot 7\} \forall j \in \{0, 1, 2, 3\} : \\ & (x_{i,j} + x_{i+1,j} + x_{i+2,j} + x_{i+3,j} + x_{i+4,j} + s_1 + 2 \cdot s_2 - 4)^2 \end{aligned} \quad (3.35)$$

Dadurch werden statt 1.344 Variablen nur 448 Variablen hinzugefügt. Dies reicht jedoch nicht um ein Embedding zu finden. Um das Problem demonstrativ auf dem Annealer ausführen zu können, wird das Constraint für mindestens zwei freie Tage innerhalb von zwei Wochen entfernt. Dadurch werden weitere Variablen und Kopplungen reduziert. Außerdem wird der Kopplungsgrad der Variablen (die Anzahl der Verbindungen zu anderen Variablen), die einer Freischicht entsprechen, stark reduziert. Mit diesem Constraint besitzen die Variablen einen Grad von 102, welcher ohne Verkettungen von mindestens sieben physischen Qubits auf dem Annealer ohnehin nicht umgesetzt werden kann. Das Entfernen dieses Constraints senkt den maximalen Grad auf 30.

Leider ist noch eine weitere Vereinfachung bei der Umsetzung des Rotating Rostering-Problems notwendig. Die Anzahl an maximalen Schichtwiederholungen muss von vier auf drei reduziert werden. Außerdem wird ein besserer Qubo Ausdruck für dieses Constraint benutzt. Wie solche optimierten Qubo-Ausdrücke gefunden werden können, wird in Kapitel 4 behandelt. Der optimierte Qubo-Ausdruck sieht wie folgt aus:

$$\begin{aligned} & \forall i \in \{1, \dots, w \cdot 7\} \forall j \in \{0, 1, 2, 3\} : \\ & 3s_1 + x_{i,j}x_{i+1,j} + x_{i,j}x_{i+2,j} + x_{i,j}x_{i+3,j} - 2x_{i,j}s_1 + x_{i+1,j}x_{i+2,j} \\ & + x_{i+1,j}x_{i+3,j} - 2x_{i+1,j}s_1 + x_{i+2,j}x_{i+3,j} - 2x_{i+2,j}s_1 - 2x_{i+3,j}s_1 \end{aligned} \quad (3.36)$$

Nach diesen Optimierungen werden insgesamt nur 448 Variablen und 2.760 Coupler benötigt. Eine Übersicht über die verschiedenen Modellierungen und deren Einfluss

	<i>Normal</i>	<i>Optimiert</i>	<i>Ohne Frei</i>	<i>Opt. + Ohne Frei</i>
<i>Variablen</i>	1792	672	1568	448
<i>Kanten</i>	9344	6736	5304	2760
<i>max. Grad</i>	100	96	24	22

Tabelle 3.4: Übersicht der Variablen, Kanten und des maximalen Grades verschiedener Qubo-Modellierungen für das Rotating Rostering-Problem. *Normal* entspricht der Modellierung aus 3.2.2. In *Optimiert* wurde das maximale Schichten Constraint durch den Ausdruck 3.36 ersetzt. Für die dritte Spalte wurde das Constraint für zwei freie Tage innerhalb von zwei Wochen weggelassen.

auf die Variablen und Kantenanzahl lässt sich in Tabelle 3.4 finden. Für diese Konfiguration des Problems lässt sich ein Embedding finden, das auf dem Quanten-Annealer ausgeführt werden kann. Der Graph des Ising-Modells und ein Ausschnitt von vier Schichtvariablen sind in Abbildung 3.9 zu sehen. Schichtvariablen werden durch schwarze Knoten dargestellt und sind in Blöcke sortiert. Jeder einzelne Block gehört zu einem Angestellten. Eine Spalte in so einem Block entspricht dem Wochentag und eine Zeile der Schicht von Frei bis Nacht. Durch die Abbildung wird deutlich, wie stark vernetzt das Rotating Rostering-Problem ist.

Obwohl das Problem nur aus 448 Variablen besteht, ist die zugrundeliegende Graphstruktur des Problems sehr stark verknüpft und nicht leicht für das heuristische Verfahren auf die Graphstruktur des Annealers zu übertragen. Weder die Anzahl der Variablen noch der Grad dieser, lässt dies offensichtlich deutlich werden. Es existieren Variablen mit einem Grad von 4, 20, 21 und 22. Daraus könnte man vermuten, dass nur maximale Ketten der Länge zwei benötigt werden, da ein Qubit mit 15 weiteren verbunden ist. Dies ist aber nicht der Fall. Die Topologie eines Quanten-Annealers folgt einem bestimmtes Muster, welches bereits in Abschnitt 3.1.3 beschrieben wurde. Es existiert eine Einheitszelle aus Qubits, in der bestimmte aber nicht alle, Qubits untereinander verbunden sind. Qubits innerhalb einer Einheitszelle sind mit ausgewählten Qubits benachbarten Einheitszellen verbunden. Daraus ergibt sich eine gut verzweigte Struktur, die jedoch keine vollständigen Graphen für vier oder mehr Knoten realisiert (für die Pegasus-Topologie).

Das Constraint der One Hot-Kodierung besteht aus vier Variablen, die alle untereinander verbunden sind (zu sehen in Abbildung 3.9 (b)). Um einen K_4 (vollständig verbundener Graph mit vier Knoten) zu embedden, wird mindestens ein zusätzliches Qubit benötigt. Dies gilt für alle Constraints, in denen vier oder mehr Variablen untereinander gekoppelt sind. Diese sind zudem teilweise mit zusätzlichen weiteren Variablen verbunden. Diese eng vernetzte Struktur erschwert das Embedding auf

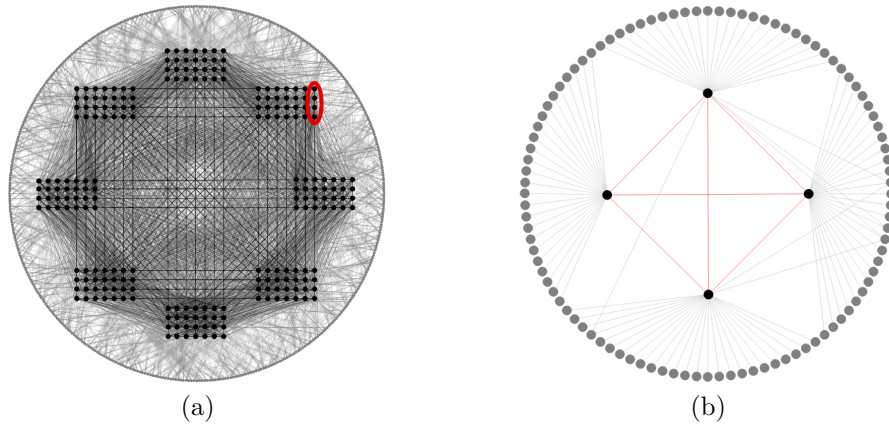


Abbildung 3.9: (a) Graph des Ising-Modells für Rotating Rostering-Problem. Jeder Knoten entspricht einer Variable im Ising-Modell. Schwarze Knoten repräsentieren Schichtvariablen des originalen Problems und graue Knoten (kreisförmig am Rand angeordnet) repräsentieren Hilfsvariablen. Eine Kante zwischen zwei Knoten ist eingezeichnet, wenn im Ising-Modell zwei Variablen über einen Koeffizienten a verknüpft sind. (b) Vergrößerte und neu sortierte Graphdarstellung der in (a) Rot markierten Variablen. Graue Knoten sind Variablen die mit den markierten vier Schichtvariablen verbunden sind.

dem Quanten-Annealer erheblich. Aus diesem Grund müssen sehr viele physikalische Qubits zu einem logischen Qubit verkettet werden. Dadurch entstehen Ketten von bis zu 36 Qubits. Insgesamt werden 4.099 physische Qubits benötigt, um den vollständigen Graph des Ising-Modells zu embedden. Das Embedding von den vier Schichtvariablen aus Abbildung 3.9 (b) ist in Abbildung 3.10 zu sehen. Die langen Qubit-Ketten, die erzeugt werden müssen, sollten wenn möglich verhindert werden. Diese können sich negativ auf das Ergebnis des Annealers auswirken, was in Abschnitt 3.2.4 sichtbar wird.

Anneal-Parameter. Nun kann mithilfe des Quanten-Annealers eine minimale Lösung für das Ising-Modell gefunden werden. Durch Aufrufen der Sample-Methode in der D-Wave Ocean SDK [oce] können Ising-Modelle mit einem Embedding an den Quanten-Annealer übergeben werden. Danach werden die Biase und Coupler entsprechend dem Ising-Modell programmiert und der Annealing-Prozess kann gestartet werden. Für den Lösungsprozess existieren einige Parameter, die angepasst werden können. Die wichtigsten werden im folgenden Abschnitt besprochen.

In einer idealen Umgebung und unter der Bedingung, dass das System langsam genug

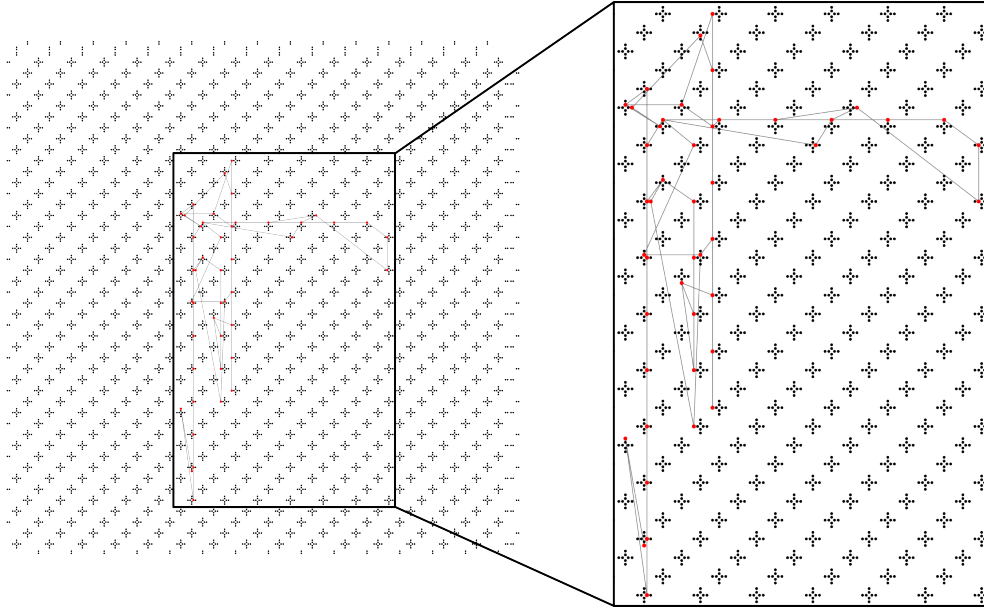


Abbildung 3.10: Embedding der in Abbildung 3.9 (b) dargestellten vier Schichtvariablen. Alle rot eingezeichneten Qubits werden genutzt, um die vier logischen Qubits der Schichtvariablen darzustellen. Eingezeichnete Kanten werden entweder zur Kettenbildung oder zur Kopplung der Variablen untereinander benötigt.

in den Zielzustand überführt wird, sollte das Quantum Adiabatic Theorem sicherstellen, dass der Annealer in seinem energetisch niedrigsten Eigenzustand bleibt. Allerdings handelt es sich hier um reale Bedingungen, die immer mit äußeren Störungen und einem nicht adiabatischen Vorgang verknüpft sind. Aus diesem Grund muss der Annealing-Vorgang mehrmals wiederholt werden, um brauchbare Ergebnisse zu erhalten. Die Anzahl der Wiederholungen wird über den Parameter *num_reads* gesteuert. Wie lange ein einzelner Anneal-Lauf andauert, also wie schnell der Annealer vom Start in den Ziel-Hamiltonian überführt wird, kann durch den Parameter *annealing_time* gesteuert werden. Die minimale Annealing-Zeit ist technisch limitiert und hängt vom konkreten Annealer ab. Im Falle des Advantage System 7.1 liegt diese bei $0,5\mu s$. Die maximale Anneal-Zeit ist künstlich von D-Wave festgelegt und liegt hier beispielsweise bei $2.000\mu s$. Es sei jedoch angemerkt, dass längere Anneal-Zeiten nicht unbedingt bessere Ergebnisse liefern müssen. Die Qubits sind auch länger äußeren Einflüssen ausgesetzt, welche die gewünschten quantenmechanischen Eigenschaften zerstören können. Zusätzlich zu der Anneal-Zeit kann auch noch das Anneal Schedule angepasst werden. Hier kann beispielsweise festgelegt werden, dass in bestimmten Zeitintervallen, wie in den ersten $5\mu s$ von $20\mu s$, das Annealing besonders schnell vonstatten gehen soll und in den restlichen $15\mu s$ langsamer. Je nach Problem kann dies

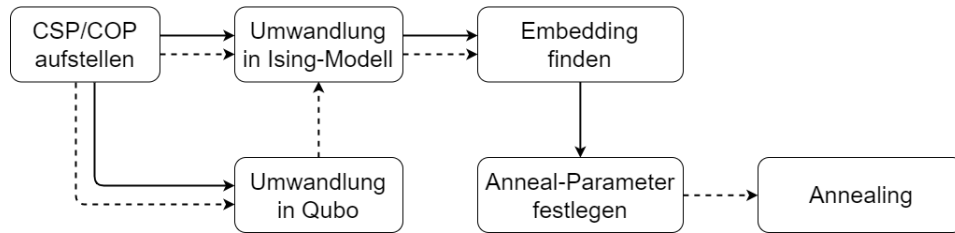


Abbildung 3.11: Übersicht über die einzelnen Arbeitsschritte zum Lösen eines CSP/COP auf einem Quanten-Annealer. Die gestrichelten Pfeile führen zu Prozessen die automatisiert ausgeführt werden können. Normale Pfeile führen zu Prozessen die händisch ausgeführt werden können bzw. sollten.

zu besseren Ergebnissen führen. Es existieren noch viele weitere Parameter, wie z.B. die Kettenstärke oder das automatische Skalieren von Bias und Coupler-Werten. Hier sollen zunächst nur die Parameter erwähnt werden, die in späteren Teilen der Arbeit genutzt werden.

Übersicht. Bis ein CSP oder auch ein COP Problem auf einem Quanten-Annealer gelöst werden kann, sind wie in diesem und dem vorherigen Abschnitt beschrieben verschiedenste Arbeitsschritte notwendig. Eine Übersicht über diese verschiedenen Schritte ist in Abbildung 3.11 zu sehen. Initial muss das Problem als CSP oder COP formuliert werden. Danach müssen die einzelnen Constraints in ein Qubo oder ein Ising-Modell überführt werden. Falls das Problem in ein Qubo-Modell überführt wurde, kann es automatisch in ein äquivalentes Ising-Modell transformiert werden. Danach kann automatisch ein Embedding gefunden werden, dieses kann aber durch händische Optimierung (wie dem Vorgeben von einem initialen Embedding) verbessert werden. Danach müssen nur noch die Anneal-Parameter festgelegt werden und der Anneal-Vorgang kann gestartet werden.

Das Graphfärbungsproblem wurde auf dem Advantage2 2.6 ausgeführt. Dabei wurde eine Anneal-Zeit von $0.5\mu s$ und eine Anzahl von 500 Läufen gewählt. Das Rotating Rostering-Problem wurde in seiner wie oben beschriebenen modifizierten Variante auf dem Advantage System 7.1 ausgeführt. Hier wurde eine Anneal-Zeit von $20\mu s$ und 2.000 Läufen gewählt. Wie die Ergebnisse des Quanten-Annealers interpretiert und mit klassischen Verfahren verglichen werden können, wird im nächsten Abschnitt 3.2.4 behandelt.

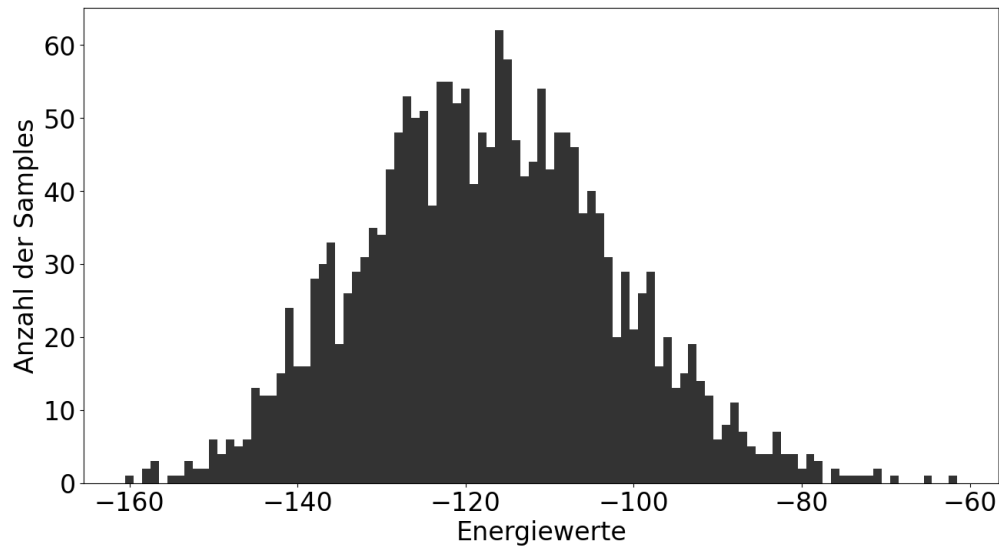


Abbildung 3.12: Ergebnisse des Rotating Rostering-Problem, ausgeführt auf dem Advantage System 7.1. Es wurden 2000 Durchläufe mit jeweils 20 *us* Anneal-Zeit durchgeführt.

3.2.4 Auswertung der Anneal-Ergebnisse

Der Quanten-Annealer liefert für jeden durchgeführten Lauf eine Variablenbelegung und den dazugehörigen Energiewert dieser. Für die modifizierte Version des Rotating Rostering-Problems ist die Verteilung von Ergebnissen in der Abbildung 3.12 dargestellt. Das eindeutige Minimum dieser modifizierten Version liegt bei -348 . Der Zielwert wurde mit einer Energie von 188 verfehlt. Da die meisten Constraints so modelliert wurden, dass eine Abweichung vom eindeutigen Minimum einen Wert von $+1$ oder $+2$ zur Folge hat, liegt die Vermutung nahe, dass viele Constraints verletzt wurden. Durch das Betrachten des Schichtplans für die Lösung mit Energie -160 (zu sehen in Tabelle 3.5) bestätigt sich diese Vermutung.

Das minimale Schichten Constraint und die One Hot-Kodierung wurde an verschiedensten Stellen verletzt. Dieses CSP mit dieser Modellierung ist aufgrund seiner Struktur und Größe ungeeignet für den Quanten-Annealer. Aus diesem Grund wird für die Auswertung von Ergebnissen in diesem Abschnitt nur das Graphfärbungsproblem betrachtet. Für die beiden Varianten des Problems sind die Ergebnisse in Abbildung 3.13 zu sehen. Das Optimum für die beiden Färbungsprobleme liegt jeweils bei -18 und -19 . Beide Werte wurden sehr häufig erreicht, was vor allem an

	Mo	Di	Mi	Do	Fr	Sa	So
Angestellter 1	Frei	Frei	Früh	E2	E1	Frei	Frei
Angestellter 2	Früh	Früh	Nacht	Früh	Früh	Frei	Frei
Angestellter 3	Spät	Spät	Frei	Frei	Nacht	Früh	Früh
Angestellter 4	Frei	Spät	Spät	Frei	Früh	Nacht	Nacht
Angestellter 5	Frei	Spät	Nacht	Nacht	Spät	Frei	E1
Angestellter 6	Spät	Nacht	Früh	Spät	Spät	Frei	Frei
Angestellter 7	Früh	Früh	Früh	Nacht	E1	Spät	Spät
Angestellter 8	Nacht	Nacht	Nacht	Spät	Frei	Früh	Früh

Tabelle 3.5: Bester Rotating Rostering Schichtplan aus 2000 Anneal-Durchläufen. Die Schicht E1 wird eingetragen, wenn alle Schichtvariablen für diesen Tag gleich 0 sind. Die Schicht E2 wurde eingetragen, da mehrere Schichten gleichzeitig ausgewählt wurden. Konkret wurden Früh und Spätschicht gewählt.

der kleinen Problemistanz liegt. Für das Problem mit vier Farben wurden häufiger schlechtere Werte als das Optimum gefunden. Die Ursache dafür kann vielseitig sein. Die gewählte Anneal-Zeit kann zu kurz gewesen sein, die längeren Qubit-Ketten können Probleme verursacht haben oder die größere Problemistanz ist im allgemeinen anfälliger gegenüber äußeren Störfaktoren.

Arbeitsschritte des Annealers. Um den Quanten-Annealer mit einem klassischen Verfahren vergleichen zu können, muss eine genaue Betrachtung der benötigten Lösungszeit unternommen werden. Wenn ein Ising-Modell mit einem Embedding an einen Quanten-Annealer geschickt wird, sind verschiedenste Arbeitsschritte erforderlich, bis alle Samples gesammelt sind. Ein grober Überblick über die einzelnen Arbeitsschritte des Annealers liefert die Abbildung 3.14. Der Ablauf eines Anneal-Prozesses erfolgt in mehreren Phasen. Zunächst werden die Bias und Coupler des Annealers so programmiert, dass ein Übergang vom Startzustand in den Zielzustand (definiert durch das Ising-Modell) möglich ist. Danach wird ein erster Anneal-Durchlauf gestartet. Innerhalb der angegebenen Anneal-Zeit wird das System vom Startzustand in seinen Zielzustand überführt. Anschließend werden die Werte der einzelnen Qubits ausgelesen und als erstes Sample gespeichert. Die Delay-Phase sorgt dafür, dass das System nach dem Auslesen der Qubits, welches Wärme erzeugt, wieder auf seine Betriebstemperatur heruntergekühlt wird. Die Anneal, Readout und Delay-Phasen werden abhängig vom angegebenen Anneal-Parameter wiederholt. Die Overhead-Zeit ergibt sich beispielsweise aus Verzögerungen beim Auslesen der Qubits oder generellem Kommunikations-Overhead zwischen den einzelnen Schnittstellen.

Nachdem das Annealing abgeschlossen wurde, entstehen noch Postprocessing-Zeiten.

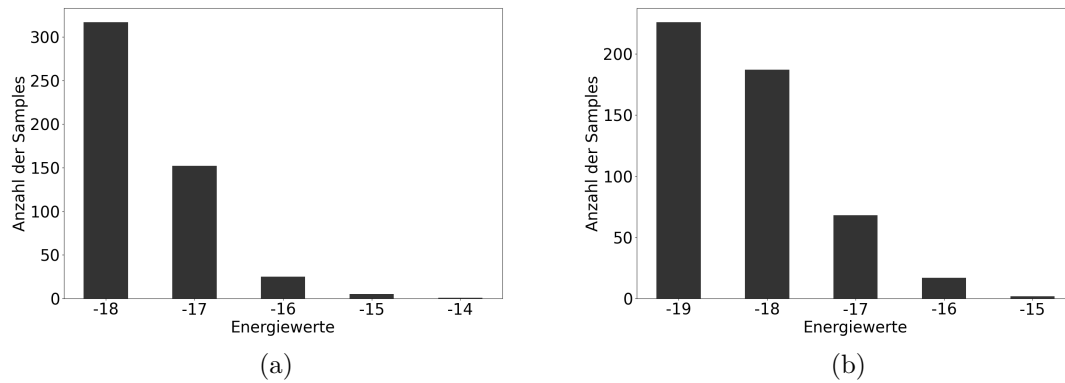


Abbildung 3.13: Anneal-Ergebnisse für das Färbungsproblem mit drei (a) und vier (b) Farben.

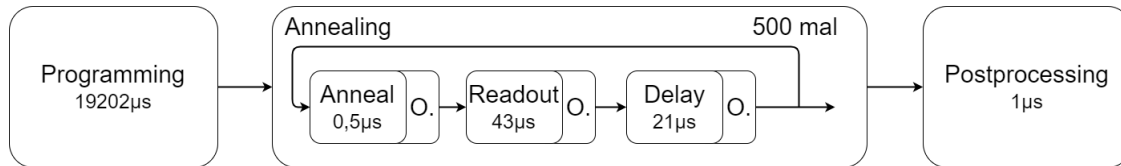


Abbildung 3.14: Übersicht über die verschiedenen Arbeitsprozesse des Quanten-Annealers. Die Abkürzung O. steht für Overhead. In jedem einzelnen Arbeitsprozess sind beispielhaft die Zeiten für das Färbungsproblem mit drei Farben angegeben.

Postprocessing wird beispielsweise genutzt, um Unstimmigkeiten in Qubit-Ketten zu lösen. Im optimalen Fall befinden sich alle Qubits innerhalb einer Kette im selben Zustand. Durch thermische Prozesse oder andere Störfaktoren kann es jedoch vorkommen, dass innerhalb von einer Kette einzelne Qubits in unterschiedlichen Zuständen liegen. In solchen Fällen entscheidet das Postprocessing darüber, wie die Zustände solcher Qubits gewertet werden sollen. Standardmäßig wird das Majority Vote-Prinzip verwendet, bei dem der Zustand ausgewählt wird, der innerhalb der Kette am häufigsten vorkommt. Ein Überblick über die verschiedenen Zeiten für die beiden Graphfärbungsprobleme sind in der Tabelle 3.6 dargestellt. Die Zeiten Anneal, Readout und Delay fallen pro durchgeführten Lauf an, wohingegen Overhead und Postprocessing über alle Läufe aufsummiert werden. Das Programmieren des Annealers erfolgt einmalig zu Beginn der Anneal-Phase.

¹Gesamtzeit des Anneal-Vorgangs inklusive der durchgeführten Wiederholungen.

Farben	Progam.	Anneal	Readout	Delay	Overhead	Postproc.	Gesamt ¹
3	19202	0,5	43	21	787	1	52.240
4	19201	0,5	78	21	805	39	69.795

Tabelle 3.6: Übersicht der verschiedenen Zeiten eines Anneal-Vorgangs für die beiden Färbungsprobleme. Alle Zeiten sind in μs angegeben.

Time To Target Metrik. Da Quantum-Annealing ein stochastischer Vorgang ist, lassen sich Zeiten zum Erzielen einer einzelnen Lösung nicht direkt mit der Lösungszeit eines klassischen Solvers vergleichen. Deshalb wird die Metrik Time To Target (TTT) [KYN⁺15] eingeführt. Die Samples des Quanten-Annealers liefern eine Wahrscheinlichkeitsverteilung für die jeweiligen Energiewerte. Die TTT-Metrik berechnet die durchschnittliche Zeit, die benötigt wird, um ein gewünschtes Ergebnis zu erhalten. Das Target kann dabei entweder einem konkreten Wert, wie beim Rotating Rostering und Färbungsproblem, oder einer bestimmte Schwelle bei z.B. Optimierungsproblemen entsprechen. Die Time To Target berechnet sich wie folgt:

$$p_t = \frac{|T|}{|S|}, \quad m = \left\lceil \frac{1}{p_t} \right\rceil, \quad TTT = t_s \cdot m \quad (3.37)$$

Hierbei ist p_t die Wahrscheinlichkeit, ein Sample zu erhalten, das dem Target Wert entspricht. Sie berechnet sich aus der Anzahl der Samples, die den Target Wert erreicht haben ($|T|$), geteilt durch die Anzahl aller Samples ($|S|$). Die TTT ergibt sich dann als Produkt der Zeit pro Sample t_s und der durchschnittliche Anzahl an benötigten Samples m . Um einen fairen Vergleich mit klassischen Solvern zu ermöglichen, die immer eine korrekte Antwort liefern, kann auch eine beliebige Wahrscheinlichkeitsschwelle für die TTT gesetzt werden. Diese modifizierte TTT_p gibt die Zeit an, die benötigt wird, um mit einer Wahrscheinlichkeit p das Target zu erreichen. Die Berechnung wird entsprechend angepasst:

$$p_t = \frac{|T|}{|S|}, \quad m_p = \left\lceil \frac{\log(1-p)}{\log(1-p_t)} \right\rceil, \quad TTT_p = t_s \cdot m_p \quad (3.38)$$

Die Anzahl der Samples, die gezogen werden müssen, um mit einer Wahrscheinlichkeit von p einen Target Wert zu erreichen m_p , wurde angepasst. Ansonsten ist die Berechnung identisch geblieben. Bei einem Vergleich zwischen klassischen Solvern und dem Quanten-Annealer sollten Zeiten wie Overhead, Postprocessing und Programmierzeiten mit berücksichtigt werden, um einen fairen Vergleich zu ermöglichen. Um verschiedene Annealer-Parameter oder Modellierungen zu testen, kann die Programming-Zeit vernachlässigt werden, da sie konstant ist. Die beiden TTT-Metriken werden im folgenden anhand der Färbbarkeit für drei Farben exemplarisch berechnet.

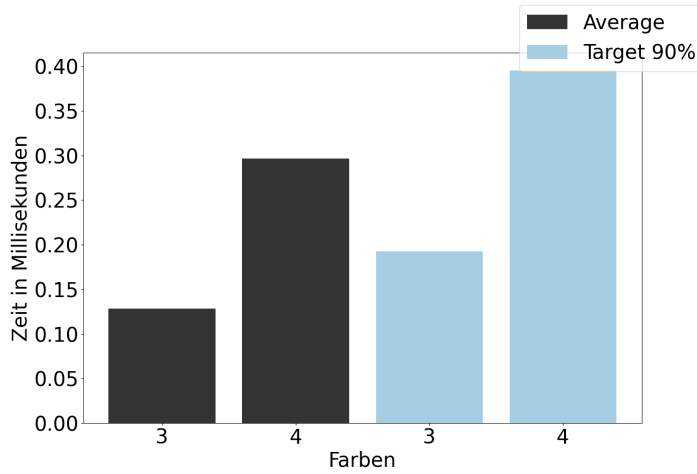


Abbildung 3.15: TTT-Metriken für Graph-Färbung mit drei und vier Farben. Die blauen Balken repräsentieren die normalen TTT-Metriken und die orangen Balken der TTT Metrik mit einer Zielwahrscheinlichkeit von 90%.

Für drei Farben wurden 500 Anneal-Durchläufe mit einer Anneal-Zeit von $0,5 \mu s$ durchgeführt. Die Readout-Zeit beträgt $43 \mu s$ und die Delay-Zeit $21 \mu s$. Insgesamt wird pro Lauf eine Zeit von $64,5 \mu s$ benötigt. Von den 500 Samples haben 317 den Wert von -18 erreicht und damit eine optimale Lösung erzielt. Daraus ergibt sich eine Wahrscheinlichkeit von $\frac{317}{500} = 0,634$. Im Durchschnitt werden also $\left\lceil \frac{1}{0,634} \right\rceil = 2$ Läufe benötigt, um ein optimales Ergebnis zu erhalten. Um mit einer Wahrscheinlichkeit von 90% eine Lösung zu erhalten, sind $\left\lceil \frac{\log(1-0,9)}{\log(1-0,634)} \right\rceil = 3$ Läufe erforderlich. Die geforderte Wahrscheinlichkeit ist theoretisch beliebig wählbar. Es ist aber zu beachten, dass die Anzahl der benötigten Läufe exponentiell mit der geforderten Wahrscheinlichkeit wächst. Aus diesem Grund sollte die Wahrscheinlichkeit nicht zu hoch gewählt werden. Multiplizieren wir diese beiden Werte mit der benötigten Zeit pro Lauf, ergeben sich die beiden TTT-Metriken:

$$TTT = 64,5 \mu s \cdot 2 = 129 \mu s = 0,129 ms \quad (3.39)$$

$$TTT_p = 64,5 \mu s \cdot 3 = 193,5 \mu s = 0,194 ms \quad (3.40)$$

Ein Plot der TTT-Metriken für drei und vier Farben ist in Abbildung 3.15 zu sehen. Zum einen lässt sich ein Anstieg in der benötigten Lösungszeit zwischen der normalen TTT und der Target TTT, welcher zu erwarten war beobachten. Außerdem ist auch ein Anstieg in der Lösungszeit für das Färbungsproblem mit vier Farben zu erkennen. Gegeben der höheren Komplexität des Problems ist dies nicht ungewöhnlich.

Kapitel 4

Modellierung

In diesem Kapitel werden verschiedene Techniken und Strategien vorgestellt, um CSPs und COPs in Qubo- bzw. Ising-Modelle umzuwandeln. Die konkrete Modellierung der Probleme spielt eine große Rolle, da diese direkten Einfluss darauf haben kann, ob und wie gut ein CSP bzw. COP auf dem Quanten-Annealer gelöst werden kann. Im Abschnitt 4.1 werden verschiedene Möglichkeiten der Variablenkodierung vorgestellt. Danach werden in Abschnitt 4.2 mögliche Techniken für die Umwandlung von Constraints in Qubo bzw. Ising-Ausdrücke präsentiert. Der Abschnitt 4.3 fokussiert sich dabei auf Qubo-Ausdrücke, während in Abschnitt 4.4 gesondert auf Ising-Modelle eingegangen wird. Zuletzt werden Mögliche Optimierungen solcher Qubo bzw. Ising-Modelle in Abschnitt 4.5 besprochen.

4.1 Variablenkodierung

Innerhalb von Qubo- oder Ising-Modellen sind nur binäre Variablen erlaubt. Aus diesem Grund müssen Variablen mit einer größeren Domäne mithilfe von binären Variablen kodiert werden. In dieser Arbeit werden nur Probleme mit endlichen Domänen untersucht, aus diesem Grund wird auch hier nur die Kodierung von endlichen Domänen gezeigt. In diesem Abschnitt werden drei verschiedene Kodierungstechniken vorgestellt, One Hot-Kodierung [Pet15], Binärkodierung [Pet15] und die Domain Wall-Kodierung [Cha19]. Jedes Verfahren wird anhand eines kleinen Beispiels vorgestellt und es werden jeweils Vor- und Nachteile der Kodierungen aufgezeigt. Das durchgehende Beispiel dieses Abschnitts besteht aus zwei Variablen x und y mit folgenden Domänen:

$$x = \{0, 1, 2, 3\}, y = \{0, 1, 2\} \quad (4.1)$$

Die Zahlen 0 bis 3 können dabei für beliebige Werte wie Farben, Schichten oder Räume stehen. Diese beiden Variablen werden innerhalb des Beispiels mit einer Gleichheitsbedingung verknüpft $x = y$. Diese muss in der jeweiligen Variablenkodierung umgesetzt werden.

4.1.1 One Hot-Kodierung

Die One Hot-Kodierung wurde schon in Kapitel 3.2 verwendet, um die Variablen des Graph-Färbung und des Rotating Rostering-Problems zu kodieren. Zur Vollständigkeit wird die Kodierung hier noch einmal formal eingeführt.

Um eine Variable x mit der Domäne D_x zu kodieren, werden $|D_x| = n$ viele neue boolesche Variablen $x_1, \dots, x_n$ eingeführt. Dabei nimmt eine Variable x_i genau dann den Wert 1 an, wenn die Variable x den i -ten Wert in der Domäne D_x annimmt. Somit nimmt immer nur exakt eine Variable der x_i den Wert 1 an.

Beispiel 4.1.1 (One Hot-Kodierung). Die beiden Variablen x und y werden durch jeweils vier und drei boolesche Variablen, x_0, x_1, x_2, x_3 und y_0, y_1, y_2 , kodiert. Eine Variable x_i nimmt genau dann den Wert 1 an, wenn x dem Wert i entspricht. Daraus folgt, dass immer nur exakt eine Variable x_i den Wert 1 annehmen darf. Dieses Constraint kann wie schon in Kapitel 3.2 zu sehen, mit folgendem Qubo-Ausdruck umgesetzt werden.

$$\sum_{i=0}^3 x_i = 1 \quad (4.2)$$

$$\left(\sum_{i=0}^3 x_i - 1 \right)^2 = 0 \quad (4.3)$$

$$-x_1 - x_2 - x_3 - x_3 + 2x_0x_1 + 2x_0x_2 + 2x_0x_3 + 2x_1x_2 + 2x_1x_3 + 2x_2x_3 = -1 \quad (4.4)$$

Wir erhalten einen Qubo-Ausdruck der genau dann -1 wird, wenn das One Hot-Constraint erfüllt ist. Für jede andere Variablenbelegung ergibt sich ein Wert größer als -1 . Um das Constraint $x = y$ zu kodieren, muss ein Qubo-Ausdruck gefunden werden, der für alle gültigen Belegungen sein eindeutiges Minimum erreicht und für alle nicht erfüllenden Belegungen einen größeren Wert als dieses Minimum annimmt. Im Abschnitt 4.2 wird sich genauer mit der Umwandlung von Constraints beschäftigt. Die erfüllenden Belegungen sind in der Tabelle 4.1 aufgelistet. Ein möglicher Qubo-Ausdruck für dieses Constraint sieht wie folgt aus:

$$x_0x_1 + x_0x_2 + x_1y_2 + x_2y_1 + x_3y_0 + x_3y_1 + x_3y_2 + y_0y_1 + y_1y_2 - x_0y_0 - x_1y_1 - x_2y_2 \quad (4.5)$$

Wird eine erfüllende Belegung in diesen Ausdruck eingesetzt, erhalten wir einen Wert von -1 . Für alle anderen Belegungen werden wieder größere Werte angenommen. $\diamond$

x	y	x_0	x_1	x_2	x_3	y_0	y_1	y_2
0	0	1	0	0	0	1	0	0
1	1	0	1	0	0	0	1	0
2	2	0	0	1	0	0	0	1

Tabelle 4.1: Erfüllende Belegungen für das Constraint $x = y$.

x	y	x_2	x_1	y_2	y_1
0	0	0	0	0	0
1	1	0	1	0	1
2	2	1	0	1	0

Tabelle 4.2: Erfüllende Belegungen für $x = y$ in der Binärcodierung.

4.1.2 Binärcodierung

Bei der Binärcodierung werden die Variablen binär durch boolesche Variablen dargestellt. Um eine Variable mit n verschiedenen Werten zu repräsentieren werden somit $\lceil \log(n) \rceil$ boolesche Variablen benötigt. Falls die Größe der Domäne der zu kodierenden Variable einer Zweierpotenz entspricht, sind auch keine weiteren Kodierungs-Constraints nötig. Für den Fall, dass die Domänengröße keiner Zweierpotenz entspricht, müssen zusätzliche Beschränkungen an die booleschen Variablen gestellt werden. Für das konkrete Beispiel sieht dies wie folgt aus:

Beispiel 4.1.2 (Binärcodierung). Die Variable x kann durch zwei boolesche Variablen x_2 und x_1 dargestellt werden. Die Übersetzung ist etwas komplizierter. Die Variable x wird wie folgt codiert:

$$x = 2 \cdot x_2 + 1 \cdot x_1 \quad (4.6)$$

Dadurch lassen sich alle Werte von 0 bis 3 ausdrücken. Analog kann y durch die booleschen Variablen y_2 und y_1 dargestellt werden. Allerdings umfasst die Domäne von y nicht den Wert 3. Dieser muss mit einem zusätzlichen Constraint ausgeschlossen werden. Die Variablenbelegung $y_2 = y_1 = 1$ muss durch einen passenden Qubo-Ausdruck ausgeschlossen werden. Ein solcher Qubo-Ausdruck lässt sich leicht finden:

$$y_2 \cdot y_1 \quad (4.7)$$

Die beiden Variablen x und y sind jetzt binär kodiert. Um das Constraint $x = y$ umzusetzen, muss wieder ein passender Qubo-Ausdruck gefunden werden. Die erfüllenden Belegungen für dieses Problem sind in Tabelle 4.2 dargestellt. Ein möglicher

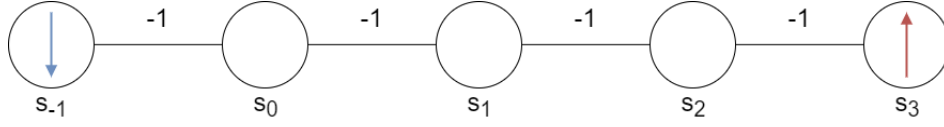


Abbildung 4.1: Darstellung Domain Wall-Kodierung

Qubo-Ausdruck für dieses Problem sieht wie folgt aus:

$$x_2 + x_1 + y_2 + y_1 - 2x_2y_2 + x_1y_2 - 2x_1y_1 \quad (4.8)$$

◇

4.1.3 Domain Wall-Kodierung

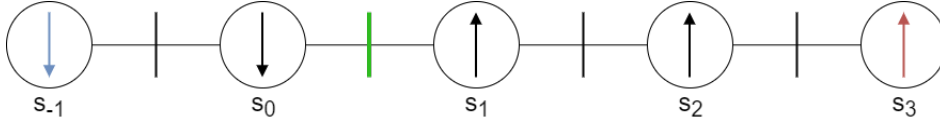
Bei der Domain Wall-Kodierung [Cha19] handelt es sich um eine Kodierungstechnik, die sich augenscheinlich besonders gut für den Quanten-Annealer eignet. Da diese Kodierung speziell auf den Quanten-Annealer ausgelegt ist, werden zunächst Ising-Variablen mit den Werten -1 und 1 betrachtet. Eine Variable s mit der Domänengröße n wird dann durch eine Kette von mehreren Ising-Variablen kodiert. Dabei befindet sich an dem linken Ende der Kette eine Variable die konstant auf -1 gesetzt und am rechten Ende eine Variable die Konstant auf $+1$ gesetzt ist. Eine schematische Darstellung ist in Abbildung 4.1 zu sehen. Die Variablen innerhalb der Kette werden über den folgenden Ausdruck verknüpft:

$$\sum_{i=-1}^{n-2} -s_i \cdot s_{i+1} \quad (4.9)$$

Die einzelnen Terme in der Summe werden minimal, wenn benachbarte Variablen denselben Wert annehmen, also entweder $1, 1$ oder $-1, -1$. Da die beiden äußeren Variablen allerdings auf jeweils unterschiedliche Werte gesetzt sind, muss eine Stelle in der Kette existieren, an der sich zwei benachbarte Variablen unterscheiden. Diese Stelle wird Domain Wall genannt und kodiert einen Wert der ursprünglichen Variable s . Befindet sich also eine Domain Wall zwischen den Variablen s_i und s_{i+1} , kodiert diese Konfiguration den Wert $i + 1$. Für das Beispiel sieht die Kodierung wie folgt aus:

Beispiel 4.1.3 (Domain Wall-Kodierung). Die Variable x wird durch eine Kette aus fünf Variablen s_{-1}, s_0, s_1, s_2 und s_3 kodiert. Die beiden äußeren Variablen werden wieder als Grenzen verwendet. Innerhalb der Summe 4.9 können die beiden Konstanten $s_{-1} = -1$ und $s_3 = 1$ direkt eingesetzt werden. Daraus ergibt sich der folgende Ausdruck:

$$s_0 - s_0s_1 - s_1s_2 - s_2 \quad (4.10)$$

Abbildung 4.2: Domain Wall-Kodierung des Wertes 2 für die Variable x .

x	y	x_0	x_1	x_2	y_0	y_1
0	0	0	0	0	0	0
1	1	0	0	1	0	1
2	2	0	1	1	1	1

Tabelle 4.3: Erfüllende Belegungen des Constraints $x = y$ für die Domain Wall-Kodierung.

Es werden insgesamt nur drei Spin-Variablen s_0, s_1 und s_2 benötigt. Dieser Ausdruck kann in einen äquivalenten Qubo-Ausdruck überführt werden. Für jede Spin-Variable s_i wird der Term $(2 \cdot x_i - 1)$ eingesetzt. Als Ergebnis erhalten wir einen äquivalenten Qubo-Ausdruck für die Verkettung der Variablen:

$$4x_0 - 4x_0x_1 - 4x_1x_2 + 4x_1 \quad (4.11)$$

In der Abbildung 4.2 ist beispielhaft die Kodierung des Wertes zwei für die Variable x eingezeichnet. Analog kann die Variable y mithilfe von zwei Variablen y_0 und y_1 kodiert werden. Der Qubo-Ausdruck ist gegeben durch:

$$4y_0 - 4y_0y_1 \quad (4.12)$$

Für die Umwandlung des Constraints muss wieder ein passender Qubo-Term gefunden werden. Die erfüllenden Belegungen für das Constraint $x = y$ in der Domain Wall-Kodierung sind in der Tabelle 4.3 aufgelistet. Ein möglicher Qubo-Term sieht wie folgt aus:

$$x_0 + x_1 + x_2 + 2y_0 + 2y_1 - 2x_1y_0 - 2x_2y_0 - 3x_2y_1 \quad (4.13)$$

◇

4.1.4 Vergleich

Jede der drei zuvor genannten Kodierungen ist in der Lage, Variablen mit endlichen Domänen binär zu kodieren. Bei der binären Kodierung werden am wenigsten Variablen benötigt. Um eine Variable mit n möglichen Werten zu kodieren, benötigt die binäre Kodierung nur $\lceil \log(n) \rceil$ viele Variablen. Gerade für große Domänen liefert dies

einen deutlichen Vorteil gegenüber der One Hot-Kodierung mit n Variablen oder der Domain Wall-Kodierung mit $n - 1$ Variablen.

Handelt es sich bei der Domänengröße um ein Vielfaches von zwei, so besitzt die Binärkodierung einen weiteren Vorteil. Es müssen keine weiteren Kodierungsconstraints eingefügt werden. Für Domänen, wo dies nicht der Fall ist, können unter anderem relativ komplizierte Domänenconstraints auftreten. Es kann sogar Fälle geben, in denen ohne Einführen einer Hilfsvariable kein Qubo-Ausdruck für die Domänenbeschränkung gefunden werden kann, wie z.B. für drei Variablen und Domänengröße siehe. Dieses Beispiel ist äquivalent zu dem Beispiel $\neg(x_1 \wedge x_2 \wedge x_3)$ 4.3.2 welches in Abschnitt 4.3 genauer untersucht wird. Für die One Hot-Kodierung lässt sich immer ein passender Qubo-Ausdruck finden. Dieser erzeugt allerdings einen vollständig verbundenen Graphen über allen Variablen, was beim Embedding zu Problemen führen kann. Bei der Domain Wall-Kodierung lässt sich auch immer ein Qubo-Ausdruck finden. Außerdem müssen die Variablen nur in einer Kette verknüpft werden, was beim Embedding sehr leicht umzusetzen ist.

Bei der konkreten Repräsentation der Werte ist die One Hot-Kodierung natürlich am einfachsten. Hier entspricht jeder Wert exakt einer Variable. Zum einen lassen sich Constraints damit intuitiver auf die neue Repräsentation umformulieren und Ergebnisse händisch leichter evaluieren. Diese Punkte sind natürlich nur relevant, falls Constraints von Hand umgeformt und Ergebnisse nicht automatisch verarbeitet werden. Außerdem lassen sich einzelne Werte durch einen Bias beeinflussen, ohne dass Coupler zwischen Qubits benötigt werden. Bei der Domain Wall-Kodierung ist die Darstellung komplizierter. Ein einzelner Wert wird durch eine Domain Wall kodiert, welche von zwei Variablen abhängt. Dadurch müssen zwei Bias genutzt werden, um einzelne Werte zu bevorzugen. Für die Binärdarstellung ist die Repräsentation am kompliziertesten. Hier hängt ein einzelner Wert von allen Variablen gleichzeitig ab. Dadurch müssen im Zweifel sogar Coupler genutzt werden, um einzelne Werte in der Kodierung zu bevorzugen.

Die Darstellung einzelner Constraints ist stark von der jeweiligen Bedingung, die an die Variablen gestellt wird, abhängig. Für das obige Beispiel einer Gleichheitsbedingung kommt die Binärkodierung mit den wenigsten Knoten und Kanten aus. Ein Vergleich der einzelnen entstandenen Graphen ist in Abbildung 4.3 zu sehen. Im Allgemeinen kann aber nicht festgelegt werden, welche Kodierung besser ist. Es wird immer bestimmte Constraints geben, die in einer Kodierung besser als in der anderen umgesetzt werden können. Das Problem der Graph-Färbung aus Abschnitt 3.2.1 lässt sich beispielsweise ohne viele zusätzlichen Kanten mit der One-Hot Kodierung darstellen.

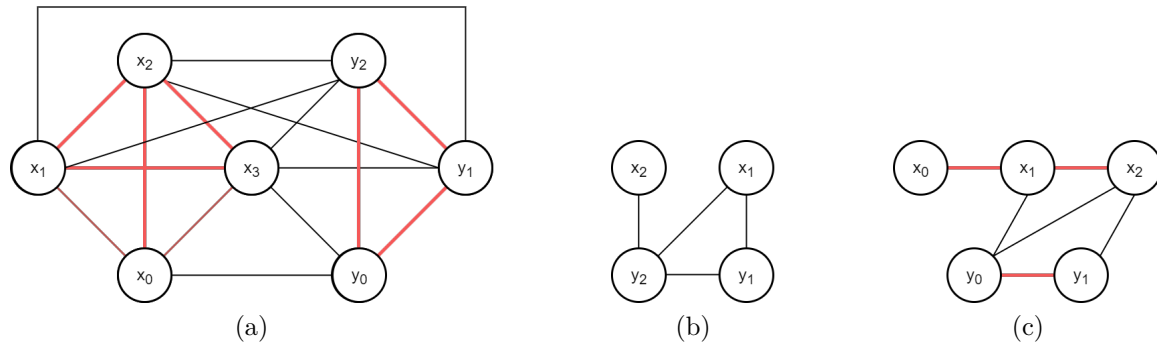


Abbildung 4.3: (a) Graphdarstellungen der Beispiele 4.1.1 bis 4.1.3. (a) One Hot-Kodierung, (b) Binärkodierung, (c) Domain Wall-Kodierung.

	One Hot	Binär	Domain Wall
Variablen	n	$\lceil \log_2(n) \rceil (+h)$	$n - 1$
Coupler	$n \cdot (n - 1)$	0 bis $\binom{n+h}{2}$	$n - 2$

Tabelle 4.4: Benötigte Variablen und Coupler um eine Variable mit der Domänengröße n zu kodieren. Die Variable h bezeichnet die Anzahl je nach Domänengröße nötigen Hilfsvariablen.

Zusammenfassung. Abschließend kann festgehalten werden, dass zum Kodieren von Variablen mit einer Domänengröße, die einer Zweier-Potenz entsprechen, die Binärkodierung sehr gut funktioniert. Für Domänengrößen, die davon abweichen, kann die Domain Wall-Kodierung genutzt werden. Eine Übersicht über die Anzahl an benötigten Variablen und Couplern ist nochmal in Tabelle 4.4 gegeben. Allerdings ist es problemspezifisch, welche Kodierung die wenigsten Hilfsvariablen und Kanten benötigt, um die Constraints umzuwandeln. Hierbei sollte sich auch die Option offen gelassen werden, Variablen, die nur in bestimmten Constraints vorkommen, so zu kodieren, dass die Kodierung besonders kleine Graphen erzeugt, die sich leicht embedden lassen.

4.2 Umwandlung von Constraintproblemen

Um CSPs in eine Qubo-Form zu überführen, wurden für das Rotating Rostering-Problem einzelne Constraints umgewandelt. Für das Graphfärbungsproblem konnte ein direkter Ansatz gewählt werden. Dieses Problem ist gerade in der One Hot-Kodierung sehr nahe dem Qubo-Modell. Dadurch konnte aus der Adjazenzmatrix des Graphen direkt die Qubo erstellt werden. Es existieren noch einige andere Graphenprobleme, die sich direkt in eine Qubo umwandeln lassen. Eins dieser Probleme ist das Problem der maximalen unabhängigen Menge (muM-Problem). Gegeben ist ein Graph G mit Knoten $v_i \in V$ und Kanten $(v_i, v_j) \in E$, es soll die größte Menge an unabhängigen Knoten gefunden werden. Dieses Problem lässt sich direkt als Qubo darstellen. Ein Knoten v_i entspricht in der Qubo-Modellierung einer booleschen Variable x_i . Diese Variable nimmt den Wert 1 an, wenn der Knoten für die muM gewählt wurde, und ansonsten 0. Der benötigte Qubo-Term für die Modellierung sieht dann wie folgt aus:

$$\sum_{v_i \in V} -x_i + \sum_{(v_i, v_j) \in E} 2 \cdot x_i \cdot x_j \quad (4.14)$$

Das Minimum wird genau dann erreicht, wenn die maximale Anzahl an Knoten v_i gewählt wurde, die nicht über eine Kante verbunden sind, also die muM gefunden wurde. Beliebige Finite Domänen-CSPs können mithilfe der Konflikt-Graph-Technik in das Problem der muM überführt werden.

Konflikt-Graph-Technik [SS05]. Ein CSP $P = \{V, D, C\}$ wird wie folgt in einen sog. Konflikt-Graphen umgewandelt. Für jedes Constraint $c_i \in C$ wird eine duale Variable y_i eingeführt. Die Domäne D_i dieser Variable besteht aus allen erfüllenden Belegungen des Constraints c_i . Jede Belegung der Variable y_i wird als eigener Knoten in den Konflikt-Graphen eingezeichnet. Zwei Knoten sind genau dann verbunden, wenn für mindestens eine Variable zwei verschiedene Belegungen durch die beiden Knoten gegeben sind. Somit sind für ein Constraint c_i alle daraus resultierenden Knoten untereinander verbunden, da es sich jeweils um verschiedene Belegungen der gleichen Variablen handelt. Zusätzlich können noch Knoten zwischen Constraints, die sich die gleichen Variablen teilen, verbunden sein.

Wird in einem solchen Konflikt-Graph eine Menge U von $|C|$ unabhängigen Knoten gefunden, wurde das CSP gelöst. Alle Knoten, die zu einem Constraint c_i gehören, sind untereinander verbunden, wodurch in U für jedes Constraint ein Knoten gewählt werden muss. Weiterhin existieren innerhalb dieser Menge keine Kanten, wodurch keine Konflikte in den Variablenbelegungen entstehen. Werden alle Variablenbelegungen der Knoten kombiniert, erhalten wir eine Belegung, die alle Constraints gleichzeitig erfüllt und das CSP löst. Das allgemeine Vorgehen zum Erstellen eines solchen

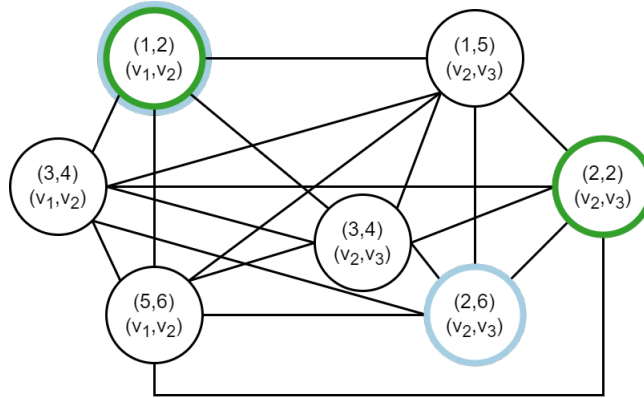


Abbildung 4.4: Konflikt-Graph für das Beispiel 4.2.1. Die zwei maximalen unabhängigen Mengen an Knoten sind mit grün und orange markiert.

Konflikt-Graphen wird an einem kleinen Beispiel noch einmal verdeutlicht.

Beispiel 4.2.1 (Konflikt-Graph). Gegeben sei das folgende CSP $P = \{V, D, C\}$ mit $V = \{v_1, v_2, v_3\}$, $D = \{D_{v_1}, D_{v_2}, D_{v_3}\}$, $C = \{c_1, c_2\}$ wobei die Domänen und Constraints wie folgt definiert sind:

$$D_1 = D_2 = D_3 = \{1, 2, 3, 4, 5\} \quad (4.15)$$

$$c_1(v_1, v_2) = \{(1, 2), (3, 4), (5, 6)\} \quad (4.16)$$

$$c_2(v_2, v_3) = \{(1, 5), (2, 6), (3, 4), (2, 2)\} \quad (4.17)$$

Die beiden Constraints c_1 und c_2 sind jeweils durch die Menge ihrer erfüllenden Belegungen der beteiligten Variablen gegeben. Jetzt werden für jedes Constraint duale Variablen y_1, y_2 mit den entsprechenden Domänen eingeführt. Die Domänen entsprechen den erfüllenden Belegungen der Constraints, wie sie oben gegeben sind. Jeder Wert der Domäne entspricht im Graphen einem eigenen Knoten.

Zwei Knoten besitzen eine Kante, wenn für mindestens eine geteilte Variable ein Widerspruch in der Belegung auftritt. Zum Beispiel existiert eine Kante zwischen $(3, 4)$ aus y_1 und $(2, 6)$ aus y_2 , da hier v_2 einmal mit 4 und einmal mit 2 belegt ist. Der vollständige Konflikt-Graph ist in Abbildung 4.4 dargestellt. Die Qubo für das CSP wird direkt aus dem Graphen mithilfe des Terms 4.14 abgelesen. Danach kann es dem Quanten-Annealer zum Lösen übergeben werden. Gerade für solche kleinen Probleme findet der Annealer leicht die beiden möglichen Lösungen mit den Knoten $\{(1, 2), (2, 6)\}$ und $\{(1, 2), (2, 2)\}$. Die Lösungen sind in Abbildung 4.4 farblich markiert. $\diamond$

Die Konflikt-Graph-Technik bietet den Vorteil, dass Variablen nicht vorher kodiert

und Constraints nicht in Qubo-Ausdrücke übersetzt werden müssen. Das Problem kann also direkt in einen solchen Graphen übersetzt werden. Allerdings müssen alle erfüllenden Belegungen aller Constraints aufgezählt werden und mit Belegungen anderer Constraints, die die gleichen Variablen teilen, verglichen werden. Aus diesem Grund eignet sich diese Technik besonders für Probleme mit Constraints, die nur wenig erfüllende Belegungen besitzen.

Das Rotating Rostering-Problem besitzt z.B. ein Constraint, welches ungeeignet für diese Transformation ist. Das Constraint des Schichtbedarfs hat sehr viele erfüllende Belegungen. Hier müssen vier Schichten von acht Mitarbeitern so zugeteilt werden, dass der Schichtbedarf gedeckt ist. Im Falle von vier Schichten und einem Schichtbedarf von zwei für jede Schicht an diesem Tag ergeben sich 2520 mögliche Schichtbelegungen. Addieren wir alle Möglichkeiten für alle einzelnen Wochentage zusammen, erhalten wir 14280 Kombinationen. Jede einzelne Belegung würde einem Knoten im Konfliktgraphen entsprechen. Selbst der größte Quanten-Annealer von D-Wave mit rund 5500 Qubits hat zu wenig Qubits, um dieses Problem zu lösen.

Auch wenn die Konflikt-Graph-Technik in vielen Fällen vermutlich ungeeignet ist, soll hier dennoch deutlich werden, dass es verschiedene Möglichkeiten gibt, CSPs in eine Qubo-Form zu überführen. Es kann sein, dass einige Probleme auf natürliche Weise als Qubo dargestellt werden können (siehe Graphfärbung) oder Probleme leicht auf ein ähnliches Graphproblem reduziert werden können, welches sich wieder natürlich als Qubo darstellen lässt (siehe Konflikt-Graph-Technik). Eine Technik, die ebenfalls immer funktioniert, ist das konkrete Umwandeln von gegebenen Constraints in Qubo-Terme (Abschnitt 4.3.1 und 4.3.2).

4.3 Aufstellen von Qubo-Ausdrücken

Beim direkten Umwandeln von Constraints wird für jedes Constraint ein Qubo-Ausdruck gefunden, der die Qubo-Bedingung 3.2.1, welche schon in Abschnitt 3.2 eingeführt wurde, erfüllt. In Abschnitt 3.2 wurden beispielhaft verschiedene Techniken gezeigt, um Qubo-Ausdrücke zu finden. Dieser Abschnitt 4.3 stellt drei verschiedene Techniken vor und zeigt deren Vor- und Nachteile auf.

4.3.1 Allgemeingültige Umformungen

Einfache linear-arithmetische Constraints auf booleschen Variablen x_i lassen sich durch rein mathematische Umformungsschritte in einen Qubo-Ausdruck überführen. Eine Gleichung wird so umgeformt, dass auf einer Seite eine 0 steht, und danach werden beide Seiten quadriert. Alle erfüllenden Belegungen liefern den Wert 0 und alle

anderen Belegungen einen Wert größer 0.

$$\sum_{i=1}^n a_i \cdot x_i = b \quad (4.18)$$

$$\sum_{i=1}^n a_i \cdot x_i - b = 0 \quad (4.19)$$

$$\left(\sum_{i=1}^n a_i \cdot x_i - b \right)^2 = 0^2 \quad (4.20)$$

Ungleichungen müssen zunächst mithilfe von Slack-Variablen zu Gleichungen umgeformt werden. Die Slack-Variable kann hier binär kodiert werden, um die Anzahl an zusätzlich benötigten Hilfsvariablen zu verringern.

$$\sum_{i=1}^n a_i \cdot x_i \leq b \quad (4.21)$$

$$\sum_{i=1}^n a_i \cdot x_i + \sum_{j=0}^{m-1} 2^j \cdot s_j = b \quad (4.22)$$

Bei Ungleichungen ist zu beachten, dass die Slack-Variable den Unterschied von der aktuellen Summe bis zum Wert b ausgleichen soll. Wenn s nur eine Binärkodierung der Zahlen von 0 bis $2^m - 1$ ist, müssen die Koeffizienten a_i ganze Zahlen sein. Andere Unterschiede zwischen b und der Summe lassen sich sonst in dieser Form nicht kodieren.

Es existieren noch eine Menge weiterer solcher möglicher allgemeingültiger Umformungen für bestimmte Constraint-Formen. Eine umfangreichere Ausführung lässt sich in [Wol24, WG23] nachlesen. Der Vorteil liegt hier in der Automatisierung des Verfahrens. Ist das CSP in einer korrekten Form gegeben, z.B. als MiniZinc-Programm, kann es durch diese sehr allgemeingültigen Umformungsschritte vollständig automatisch in eine Qubo-Form gebracht werden. Allerdings muss diese Qubo-Darstellung nicht optimal sein. Das bedeutet, dass sie entweder mehr Hilfsvariablen als nötig nutzt oder die Abstände der einzelnen Energielevel sehr klein sind.

4.3.2 Qubo aus Wahrheitswertetabellen erstellen

Ein alternativer Ansatz ist, direkt aus einer gegebenen Wahrheitswertetabelle einen Qubo-Ausdruck zu generieren. Wie schwer es ist, einen solchen Ausdruck zu finden, hängt dabei von der Anzahl der Variablen und den konkreten erfüllenden Belegungen ab. Für mehr als zwei Variablen ist meist ein händisches Finden eines solchen Ausdrucks nicht mehr realistisch möglich. Aus diesem Grund muss auf maschinelle

Verfahren zurückgegriffen werden. Auch das Brute-Forcing wird bei zu vielen Variablen zu lange dauern. Aus diesem Grund werden drei alternative Verfahren vorgestellt, um solche Qubo-Ausdrücke effizienter zu finden.

D-Wave Penalty-Modell [pen]. Das Finden eines Qubo-Ausdrucks für eine gegebene Wahrheitswertetabelle kann als lineares Programmierungsproblem (LP-Problem) formuliert werden. Dieses Vorgehen wird von D-Waves Penalty-Modell implementiert und im folgenden Abschnitt genauer beschrieben. Ein LP-Problem ist dabei wie folgt definiert:

Definition 4.3.1 (LP-Problem [Van20]). Gegeben seien Entscheidungsvariablen $x_1, \dots, x_n \in \mathbb{R}$. Ziel ist es, eine Zielfunktion der Form $c_1x_1 + c_2x_2 + \dots + c_nx_n$, wobei die $c_i \in \mathbb{R}$ liegen, zu minimieren bzw. zu maximieren. Zusätzlich müssen noch Nebenbedingungen der folgenden Form eingehalten werden.

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n &\leq b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n &\leq b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n &\leq b_m \\ x_1, x_2, \dots, x_n &\geq 0 \end{aligned}$$

Die a_{ij} sind auch Elemente der reellen Zahlen. Die Nicht-Negativitätsbedingungen der Entscheidungsvariablen können so modifiziert werden, dass auch negative Werte für die Entscheidungsvariablen zugelassen werden. Die oben dargestellte Variante entspricht der Standardform. $\triangle$

Solche LP-Probleme lassen sich z.B. mit dem Simplex-Verfahren lösen. Das Simplex-Verfahren arbeitet nicht auf allen Probleminstanzen effizient, findet aber meist schnell eine optimale Lösung. Wie eine Wahrheitswertetabelle für einen Qubo-Ausdruck in ein LP-Problem umgewandelt werden kann, soll durch ein kleines Beispiel verdeutlicht werden.

Beispiel 4.3.1 (Wahrheitswertetabelle als LP-Problem). Für den aussagenlogischen Ausdruck $(x_1 \vee x_2) \wedge x_3$ soll ein Qubo-Ausdruck gefunden werden, der für erfüllende Belegungen minimal ist. Die vollständige Wahrheitswertetabelle dieses Ausdrucks ist in Tabelle 4.5 zu sehen. Das Problem wird wie folgt in ein LP-Problem umgewandelt. Zunächst wird für jede erfüllende Belegung eine Gleichheitsbedingung der Form

$$a_1x_1 + a_2x_2 + a_3x_3 + a_{12}x_1x_2 + a_{13}x_1x_3 + a_{23}x_2x_3 = t \quad (4.23)$$

x_1	x_2	x_3	$(x_1 \vee x_2) \wedge x_3$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Tabelle 4.5: Wahrheitswertetabelle für $(x_1 \vee x_2) \wedge x_3$.

eingefügt. Dabei ist zu beachten, dass die a_i die zu suchenden Variablen sind und die x_i die konstanten Faktoren, die für jede erfüllende Belegung eine Gleichung erzeugen. Doppelt indizierte a_{ij} werden zur besseren Übersicht für die verknüpften Terme $x_i x_j$ verwendet. Weiterhin entspricht das t dem eindeutigen Minimum des Qubo-Ausdrucks, welches auch bestimmt werden muss. Für das Beispiel mit den drei erfüllenden Belegungen $(0, 1, 1)$, $(1, 0, 1)$ und $(1, 1, 1)$ ergeben sich konkret diese Gleichungen:

$$0a_1 + 1a_2 + 1a_3 + 0 \cdot 1a_{12} + 0 \cdot 1a_{13} + 1 \cdot 1a_{23} = t \quad (4.24)$$

$$1a_1 + 0a_2 + 1a_3 + 1 \cdot 0a_{12} + 1 \cdot 1a_{13} + 0 \cdot 1a_{23} = t \quad (4.25)$$

$$1a_1 + 1a_2 + 1a_3 + 1 \cdot 1a_{12} + 1 \cdot 1a_{13} + 1 \cdot 1a_{23} = t \quad (4.26)$$

Gleichungen entsprechen allerdings nicht der Standardform eines LP-Problems. Sie können aber äquivalent als zwei Ungleichungen dargestellt werden. Eine Umformung einer solchen Gleichung in die LP-Standardform ist im folgenden Beispiel zu sehen:

$$0a_1 + 1a_2 + 1a_3 + 0 \cdot 1a_{12} + 0 \cdot 1a_{13} + 1 \cdot 1a_{23} = t \quad (4.27)$$

$$\Leftrightarrow 0a_1 + 1a_2 + 1a_3 + 0 \cdot 1a_{12} + 0 \cdot 1a_{13} + 1 \cdot 1a_{23} - t = 0 \quad (4.28)$$

$$\Leftrightarrow 0a_1 + 1a_2 + 1a_3 + 0 \cdot 1a_{12} + 0 \cdot 1a_{13} + 1 \cdot 1a_{23} - t \leq 0 \quad (4.29)$$

$$0a_1 + 1a_2 + 1a_3 + 0 \cdot 1a_{12} + 0 \cdot 1a_{13} + 1 \cdot 1a_{23} - t \geq 0 \quad (4.30)$$

$$\Leftrightarrow 0a_1 + 1a_2 + 1a_3 + 0 \cdot 1a_{12} + 0 \cdot 1a_{13} + 1 \cdot 1a_{23} - t \leq 0 \quad (4.31)$$

$$0a_1 - 1a_2 - 1a_3 - 0 \cdot 1a_{12} - 0 \cdot 1a_{13} - 1 \cdot 1a_{23} + t \leq 0 \quad (4.32)$$

Für jede nicht erfüllende Belegung muss eine Bedingung eingeführt werden, die dafür sorgt, dass der Qubo-Ausdruck Werte liefert, die größer als t sind. Ein konkretes Beispiel für die Belegung $(0, 0, 1)$ sieht wie folgt aus:

$$0a_1 + 0a_2 + 1a_3 + 0 \cdot 0a_{12} + 0 \cdot 1a_{13} + 0 \cdot 1a_{23} > t \quad (4.33)$$

Um ein echt Größer-als in ein Größer-gleich umzuwandeln, kann die rechte Seite mit einer kleinen Zahl ϵ (z.B. 0,01) addiert werden. Dadurch wird der Lösungsbereich zwar eingeschränkt, allerdings sind die Coupler innerhalb eines Quanten-Annealers auch nicht beliebig präzise konfigurierbar. Lösungen wie 0,00001 oder ähnlich kleine Werte sind somit nicht erwünscht. Zusätzlich kann noch eine weitere Variable g auf die rechte Seite addiert werden. Diese repräsentiert den Energieunterschied zwischen dem Minimum des Qubo-Ausdrucks und den nicht erfüllenden Belegungen. Wünschenswert ist ein möglichst hoher Abstand, und somit soll diese Variable später maximiert werden. Insgesamt ergibt sich ein Ausdruck der Form:

$$0a_1 + 0a_2 + 1a_3 + 0 \cdot 0a_{12} + 0 \cdot 1a_{13} + 0 \cdot 1a_{23} > t \quad (4.34)$$

$$\Leftrightarrow 0a_1 + 0a_2 + 1a_3 + 0 \cdot 0a_{12} + 0 \cdot 1a_{13} + 0 \cdot 1a_{23} \geq t + \epsilon + g \quad (4.35)$$

$$\Leftrightarrow 0a_1 + 0a_2 + 1a_3 + 0 \cdot 0a_{12} + 0 \cdot 1a_{13} + 0 \cdot 1a_{23} - t - g \geq \epsilon \quad (4.36)$$

$$\Leftrightarrow 0a_1 - 0a_2 - 1a_3 - 0 \cdot 0a_{12} - 0 \cdot 1a_{13} - 0 \cdot 1a_{23} + t + g \leq -\epsilon \quad (4.37)$$

Die Variable g ist in diesem einfachen Beispiel die einzige zu maximierende Variable. Die Domänen der a_i sollen von -1 bis $+1$ beschränkt sein. Die Variable g muss größer gleich 0 sein und die Variable t ist unbeschränkt. Das vollständige LP-Problem ist somit wie folgt gegeben. Die Gleichungen wurden zur kompakteren Darstellung nicht umgeformt.

$$1a_1 + 0a_2 + 1a_3 + 0a_{12} + 1a_{13} + 0a_{23} - 1t + 0g = 0 \quad (4.38)$$

$$0a_1 + 1a_2 + 1a_3 + 0a_{12} + 0a_{13} + 1a_{23} - 1t + 0g = 0 \quad (4.39)$$

$$1a_1 + 1a_2 + 1a_3 + 1a_{12} + 1a_{13} + 1a_{23} - 1t + 1g = 0 \quad (4.40)$$

$$-0a_1 - 0a_2 - 0a_3 - 0a_{12} - 0a_{13} - 0a_{23} + 1t + 1g \leq -\epsilon \quad (4.41)$$

$$-0a_1 - 0a_2 - 1a_3 - 0a_{12} - 0a_{13} - 0a_{23} + 1t + 1g \leq -\epsilon \quad (4.42)$$

$$-0a_1 - 1a_2 - 0a_3 - 0a_{12} - 0a_{13} - 0a_{23} + 1t + 1g \leq -\epsilon \quad (4.43)$$

$$-1a_1 - 0a_2 - 0a_3 - 0a_{12} - 0a_{13} - 0a_{23} + 1t + 1g \leq -\epsilon \quad (4.44)$$

$$-1a_1 - 1a_2 - 0a_3 - 1a_{12} - 0a_{13} - 0a_{23} + 1t + 1g \leq -\epsilon \quad (4.45)$$

Ein LP-Solver liefert für dieses Problem die Lösung $a_2 = a_3 = a_{13} = -1$, $a_{12} = 1$ und $a_1 = a_{23} = 0$. Dies entspricht dem folgenden Qubo-Ausdruck $-x_2 - x_3 + x_1x_2 - x_1x_3$, welcher die benötigte Qubo-Bedingung erfüllt. $\diamond$

Durch dieses Verfahren lassen sich gerade für Constraints mit einer geringe Anzahl von Variablen schnell Qubo-Ausdrücke aus einer Wahrheitstabelle erzeugen. Allerdings existieren auch aussagenlogische Ausdrücke, die sich nicht direkt als Qubo-Ausdruck darstellen lassen. In diesem Fall existiert für das entstandene LP-Problem keine Lösung und das Verfahren muss modifiziert werden.

LP-Problem mit Hilfsvariablen. Existiert für eine gegebene Wahrheitstabelle kein gültiger Qubo-Ausdruck, müssen solange Hilfsvariablen eingeführt werden, bis eine Lösung gefunden werden kann. Das Aufstellen des LP-Problems wird dadurch aber ungemein schwieriger. Wird eine Hilfsvariable hinzugefügt, müssen die Bedingungen an erfüllende und nicht erfüllende Belegungen modifiziert werden. Für erfüllende Belegungen muss mindestens eine Belegung der Hilfsvariablen existieren, für das das eindeutige Minimum erreicht wird. Für alle anderen Belegungen der Hilfsvariablen reicht es, wenn diese einen größeren Wert als das eindeutige Minimum erhalten.

Für nicht erfüllende Belegungen muss für jede Belegung der Hilfsvariablen ein Wert erreicht werden, der größer als das Minimum ist. Für eine Hilfsvariable müssen somit zwei und für zwei Hilfsvariablen vier Nebenbedingungen eingeführt werden. Das Hauptproblem liegt aber bei den erfüllenden Belegungen. Da vorher nicht festgelegt werden kann, für welche Belegung der Hilfsvariablen das eindeutige Minimum erreicht werden kann, müssen alle Kombinationen ausprobiert werden. Genau dieses Verfahren implementiert D-Waves Penalty-Modell. Das modifizierte Verfahren soll an dem Beispiel $\neg(x_1 \wedge x_2 \wedge x_3)$, für das kein direkter Qubo-Ausdruck existiert, verdeutlicht werden.

Beispiel 4.3.2 (LP-Problem mit Hilfsvariablen). Für jede nicht erfüllende Belegung (in diesem Fall nur $(1, 1, 1)$) muss für jede mögliche Belegung an Hilfsvariablen eine Ungleichung analog zu z.B. Ungleichung 4.45 eingeführt werden. Zunächst wird nur eine Hilfsvariable h_1 eingeführt.

$$-a_1 - a_2 - a_3 - 0a_4 - a_{12} - a_{13} - 0a_{14} - a_{23} - 0a_{24} - 0a_{34} + t + g \leq \epsilon \quad (4.46)$$

$$-a_1 - a_2 - a_3 - a_4 - a_{12} - a_{13} - a_{14} - a_{23} - a_{24} - a_{34} + t + g \leq \epsilon \quad (4.47)$$

Die Koeffizienten a_4 und a_{i4} gehören zur Hilfsvariable h_1 bzw. zu kombinierten Termen $x_i \cdot h_1$. Somit ist für jede nicht erfüllende Belegung und jede mögliche Belegung der Hilfsvariablen sichergestellt, dass ein größerer Wert als das eindeutige Minimum t erreicht wird. Als Nächstes müssen die Nebenbedingungen für die erfüllenden Belegungen aufgestellt werden.

Hierbei tritt ein Problem auf. Zwar muss nicht für jede Belegung an Hilfsvariablen das Minimum t erreicht werden, es reicht eine, allerdings ist vorher nur schwer abzuschätzen, für welche konkrete Belegung der Hilfsvariablen welche Belegung der eigentlichen Entscheidungsvariablen ihr Minimum erreicht. Es müssen also alle möglichen Kombinationen iteriert werden und jedes Mal überprüft werden, ob das LP-Problem lösbar ist. Für die erfüllende Belegung $(0, 0, 1)$ sehen die beiden möglichen Nebenbedingun-

gen wie folgt aus:

$$0a_1 + 0a_2 + 1a_3 + 0a_4 + 0a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + 0a_{34} = t \quad (4.48)$$

$$0a_1 + 0a_2 + 1a_3 + 1a_4 + 0a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + 1a_{34} \geq t \quad (4.49)$$

$$0a_1 + 0a_2 + 1a_3 + 0a_4 + 0a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + 0a_{34} \geq t \quad (4.50)$$

$$0a_1 + 0a_2 + 1a_3 + 1a_4 + 0a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + 1a_{34} = t \quad (4.51)$$

Die Gleichungen bzw. Ungleichungen 4.48 und 4.49 repräsentieren den Fall, dass die Belegung $(0, 0, 1, 0)$ das eindeutige Minimum erreichen soll, während die Gleichungen bzw. Ungleichungen 4.50 und 4.51 den Fall das $(0, 0, 1, 1)$ das eindeutige Minimum erreicht abdecken. Für jede erfüllende Belegung müssen diese beiden Möglichkeiten kombiniert und versucht werden, das LP-Problem zu lösen.

Insgesamt existieren sieben erfüllende Belegungen der ursprünglichen drei Entscheidungsvariablen. Durch das Einfügen einer einzelnen Hilfsvariable müssen für jede dieser sieben erfüllenden Belegungen zwei Kombinationen an Nebenbedingungen ausprobiert werden. Insgesamt ergeben sich $2^7 = 128$ mögliche LP-Probleme, für die versucht werden muss, eine Lösung zu finden. Durch das Einfügen einer weiteren Hilfsvariable entstehen für jede erfüllende Belegung vier verschiedene Nebenbedingungskombinationen, also insgesamt $4^7 = 16.384$ mögliche LP-Probleme. $\diamond$

Es ist leicht zu erkennen, dass für größere Probleme dieses Verfahren schnell an seine Grenzen stößt. Aus diesem Grund wurde in [Pak21] eine Heuristik eingeführt, die das Finden eines Qubo-Ausdrucks beschleunigen kann. Um diese Heuristik soll es im folgenden Abschnitt gehen.

LP-Heuristik. Auch schon bei aussagenlogischen Ausdrücken mit wenigen Variablen kann es vorkommen, dass sich diese nicht direkt als Qubo-Ausdruck darstellen lassen. Ein konkretes Beispiel ist der Ausdruck $\neg(x \wedge y \wedge z)$, welcher schon im Beispiel 4.3.2 verwendet wurde. Hierbei sind alle Belegungen erfüllend bis auf $(1, 1, 1)$. Eine genauere Betrachtung des LP-Problems zeigt schnell, warum keine Lösung ohne Hilfsvariablen existieren kann.

Die Belegung $(0, 0, 0)$ ist erfüllend. Also wird dem LP-Problem folgende Bedingung hinzugefügt:

$$0a_1 + 0a_2 + 0a_3 + 0a_{12} + 0a_{13} + 0a_{23} - 1t + 0g = 0 \quad (4.52)$$

Um diese Gleichung zu erfüllen, muss t zwangsläufig mit 0 belegt werden. Das eindeutige Minimum steht somit fest auf 0 und kann nicht geändert werden. Für die nächste erfüllende Belegung $(0, 0, 1)$ wird die folgende Bedingung eingeführt:

$$0a_1 + 0a_2 + 1a_3 + 0a_{12} + 0a_{13} + 0a_{23} - 1 \cdot 0 + 0g = 0 \quad (4.53)$$

Auch hier wird schnell deutlich, dass a_3 mit 0 belegt werden muss, um diese Gleichung zu erfüllen. Für die erfüllenden Belegungen $(0, 1, 0)$ und $(1, 0, 0)$ gilt analog, dass a_2 und a_1 mit 0 belegt werden müssen. Weiterhin folgt aus den erfüllenden Belegungen $(0, 1, 1)$, $(1, 1, 0)$ und $(1, 0, 1)$, dass auch die letzten Variablen a_{23} , a_{12} und a_{13} mit 0 belegt werden müssen.

Jetzt ergibt sich allerdings ein Problem für die letzte noch nicht behandelte Belegung $(1, 1, 1)$. Da diese eine nicht erfüllende Belegung ist, wird folgende Bedingung dem LP-Problem hinzugefügt:

$$-1 \cdot 0 - 1 \cdot 0 - 0 \cdot 0 - 0 \cdot 0 - 0 \cdot 0 - 0 \cdot 0 + 1 \cdot 0 + 1g \leq -\epsilon \quad (4.54)$$

Da ϵ eine Zahl, die echt größer als Null, und g eine Variable, die echt größer als Null sein muss, lässt sich die Bedingung $g \leq -\epsilon$ nicht erfüllen. Somit ist das gesamte LP-Problem nicht erfüllbar.

Dieses Beispiel verdeutlicht, dass gerade erfüllende Belegungen mit vielen Nullen starke Bedingungen an die Variablen des LP-Problems stellen können. Aus diesem Grund versucht die in [Pak21] eingeführte Heuristik, genau diese intuitiv vermeintlich schweren Fälle mit vielen Nullen zuerst zu betrachten. Ist also das initiale LP-Problem nicht erfüllbar, wird eine Hilfsvariable hinzugefügt. Anstatt nun alle möglichen Kombinationen zu probieren, wird systematisch mit den vermeintlich schwersten Bedingungen begonnen.

Die Wahrheitswerttabelle wird nach der Anzahl der auftretenden Nullen in der Belegung sortiert, und zwar so, dass die Belegungen mit den meisten Nullen zuerst abgearbeitet werden. Weiterhin werden für die Belegungen der Hilfsvariablen zuerst die Belegungen getestet, die die meisten Einsen besitzen. Konkret würde das für das obige Beispiel bedeuten, dass zuerst die Nebenbedingung der Belegung $(0, 0, 0)$ eingeführt wird und die Hilfsvariablenbelegung 1 getestet wird.

Existiert für dieses kleine LP-Problem eine Lösung wird die nächste Belegung untersucht. Wieder wird zuerst die Hilfsvariablenbelegung mit den meisten Einsen getestet. Falls keine Lösung gefunden wird, wird die nächste Belegung der Hilfsvariablen ausprobiert. Falls für keine der möglichen Belegungen der Hilfsvariablen eine Lösung des LP-Problems gefunden werden kann, wird abgebrochen und kein Qubo-Ausdruck ausgegeben.

Da beim ersten Scheitern sofort abgebrochen wird, ist nicht garantiert, dass sich tatsächlich kein Qubo-Ausdruck für die gegebene Wahrheitswerttabelle finden lässt. Die Evaluation des Verfahrens in [Pak21] zeigt allerdings, dass für verschiedenste Probleme mithilfe dieser Heuristik schnell ein Qubo-Ausdruck gefunden werden kann. Das Verfahren ist dabei schneller als das Penalty-Modell [pen] von D-Wave. Eine dritte

Möglichkeit, Qubo-Ausdrücke zu finden, kann mithilfe von Constraintprogrammierung realisiert werden. Um diesen Ansatz soll es im folgenden Abschnitt gehen.

4.3.3 Qubo-Ausdrücke erstellen mithilfe von CSPs

Die Idee, Qubo-Ausdrücke mithilfe von Constraintprogrammierung zu finden, stammt aus dem Paper [PVVL22]. Hier wurden lineare Gleichungen gesucht, die genau dann erfüllt sind, wenn eine erfüllende Belegung des aussagenlogischen Ausdrucks eingesetzt wird. Wie in dem Paper erwähnt, können solche Terme durch das simple Quadrieren des Ausdrucks in einen Qubo-Ausdruck überführt werden, der genau dann sein eindeutiges Minimum erreicht, wenn eine erfüllende Belegung eingesetzt wird.

Allerdings können mit diesem Ansatz nicht alle möglichen Qubo-Ausdrücke erzeugt werden. Ein Beispiel aus dem Paper ist der Ausdruck $x \wedge y = z$. Für diesen Ausdruck wird folgende lineare Gleichung gefunden:

$$x + y - 2z - a = 0 \quad (4.55)$$

Die Variable a ist eine Hilfsvariable, die benötigt wird, um eine gültige lineare Gleichung zu finden, die genau dann erfüllt ist, wenn eine erfüllende Belegung für den aussagenlogischen Ausdruck eingesetzt wird. Wollen wir daraus einen Qubo-Ausdruck erzeugen, der für erfüllende Belegungen minimal ist, muss die Gleichung quadriert werden.

$$(x + y - 2z - a)^2 = 0^2 \quad (4.56)$$

$$x^2 + y^2 + 4z^2 + a^2 + 2xy - 4xz - 2xa - 4yz - 2ya + 4za = 0 \quad (4.57)$$

Allerdings existiert auch ein kompakterer Qubo-Ausdruck für $x \wedge y = z$, der sogar ohne Hilfsvariable auskommt.

$$3z + xy - 2xz - 2yz \quad (4.58)$$

Dieser Ausdruck erreicht für erfüllende Belegungen sein eindeutiges Minimum. Weiterhin werden nur drei statt sechs Coupler benötigt und es ist keine Hilfsvariable nötig. Mithilfe von linearen Gleichungen kann dieser Ausdruck aber nicht gefunden werden, da er nicht als Quadrat einer linearen Gleichung dargestellt werden kann.

Das obige Beispiel zeigt deutlich, dass eine Erweiterung zum direkten Suchen von Qubo-Ausdrücken bessere Ergebnisse liefert. Aus diesem Grund wurde als Teil dieser Arbeit ein Programm entwickelt, welches mithilfe von Constraint-Programmierung aus einer Wahrheitswerttabelle Qubo- und auch Ising-Ausdrücke finden kann. Dies wird im Folgenden vorgestellt.

Qubo-Finder. Die Idee, Constraintprogrammierung zu nutzen, um Qubo-Ausdrücke zu finden, funktioniert sehr ähnlich zu dem Ansatz über LP-Probleme. Als Eingabe erhält das Programm eine Wahrheitswerttabelle der erfüllenden und nicht-erfüllenden Belegungen. Daraus werden Constraints aufgestellt, die sich am LP-Problem aus Absatz 4.3 orientieren. Dabei ist zu beachten, dass der CSP-Ansatz vor allem für Probleme, die Hilfsvariablen benötigen, sinnvoll sein kann. Werden keine Hilfsvariablen benötigt, muss lediglich ein einziges LP-Problem gelöst werden. Dafür können direkte LP-Solver verwendet werden.

Das CSP wird wie folgt aufgebaut. Für jeden Koeffizienten des zu suchenden Qubo-Ausdrucks wird eine Variable zu dem CSP hinzugefügt. Somit wird für jede der n vielen normalen Variablen und für jede der m vielen Hilfsvariablen eine Variable für die linearen Terme hinzugefügt. Zusätzlich werden noch $\binom{n+m}{2}$ viele Variablen für Koeffizienten der quadratischen Terme hinzugefügt.

Das formale CSP für n Variablen und m Hilfsvariablen wird im folgenden Abschnitt angegeben. Dabei entspricht q der Anzahl an quadratischen Koeffizienten ($q = \binom{n+m}{2}$) und M_{ij} der Wahrheitswerttabelle für das zu bestimmende Constraint. Eine Zeile der Matrix M entspricht dabei der Variablenbelegung der n Variablen und der letzte Eintrag der Zeile bestimmt, ob es sich um eine erfüllende Belegung 1 oder eine nicht erfüllende Belegung 0 handelt. Analog ist M' die Matrix, die zusätzlich alle Belegungen inklusive der Hilfsvariablen besitzt. Die Variablen d_u und d_l geben jeweils die oberen und unteren Domänengrenzen der Koeffizienten an. Das vollständige CSP $P = (V, D, C)$ ist dann wie folgt gegeben:

$$V = \{a_1, \dots, a_{n+m}\} \cup \bigcup_{i < j}^{n+m} a_{i,j} \cup \{h_{1,1}, \dots, h_{m,2^n}\} \cup \{t\} \quad (4.59)$$

Die a_i sind die Koeffizienten der linearen Terme und die $a_{i,j}$ sind die Koeffizienten der quadratischen Terme. Die $h_{i,j}$ entsprechen den m vielen Hilfsvariablen, wobei für jede erfüllende Belegung eine eigene Menge an Hilfsvariablen eingeführt werden muss. Das t ist die Variable, die dem eindeutigen Minimum des Qubo-Ausdrucks entsprechen soll.

$$D = \{D_1, \dots, D_{n+m+q} \mid D_1 = \dots = D_{n+m+q} = \{d_l, \dots, d_u\}\} \quad (4.60)$$

$$\cup \{D_{n+m+1}, \dots, D_{n+m+q} \mid D_{n+m+1} = \dots = D_{n+m+q} = \{d_l, \dots, d_u\}\} \quad (4.61)$$

$$\cup \{D_{n+m+q+1}, \dots, D_{n+m+q+m} \mid D_{n+m+q+1} = \dots = D_{n+m+q+m} = \{0, 1\}\} \quad (4.62)$$

$$\cup \{D_{n+m+q+m \cdot 2^n + 1} \mid D_{n+m+q+m \cdot 2^n + 1} = \{-1000, \dots, 1000\}\} \quad (4.63)$$

In den Mengen 4.60 und 4.61 liegen die Domänen der Koeffizienten des Qubo-Ausdrucks. Die Menge 4.62 enthält die Domänen der Hilfsvariablen, welche in diesem Fall binäre

Variablen sind, die Werte aus der Menge $\{0, 1\}$ annehmen. Die letzte Domäne 4.63 gehört zur Variable t . Hier wird eine möglichst große Domäne gewählt, sodass keine Einschränkung in der Wahl des eindeutigen Minimums vorliegt.

$$\begin{aligned}
C = \{ & M_{1,j}a_1 + \dots + M_{n,j}a_n + h_{1,j}a_{n+1} + \dots + h_{m,j}a_{n+m} + \\
& M_{1,j}M_{2,j}a_{1,2} + \dots + M_{1,j}M_{n,j}a_{1,n} + \\
& M_{1,j}h_{1,j}a_{1,n+1} + \dots + M_{1,j}h_{m,j}a_{1,n+m} + \\
& M_{2,j}M_{3,j}a_{2,3} + \dots + h_{m-1,j}h_{m,j}a_{n+m-1,n+m} = t \\
& | \forall j \in \{1, \dots, 2^n\} : M_{n+1,j} = 1 \}
\end{aligned} \tag{4.64}$$

∪

$$\begin{aligned}
& \{ M'_{1,j}a_1 + \dots + M'_{n+m,j}a_{n+m} \\
& M'_{1,j}M'_{2,j}a_{1,2} + \dots + M'_{n+m-1,j}M'_{n+m,j}a_{n+m-1,n+m} \geq t \\
& | \forall j \in \{1, \dots, 2^{n+m}\} : M'_{n+m+1,j} = 1 \}
\end{aligned} \tag{4.65}$$

∪

$$\begin{aligned}
& \{ M'_{1,j}a_1 + \dots + M'_{n+m,j}a_{n+m} \\
& M'_{1,j}M'_{2,j}a_{1,2} + \dots + M'_{n+m-1,j}M'_{n+m,j}a_{n+m-1,n+m} > t \\
& | \forall j \in \{1, \dots, 2^{n+m}\} : M'_{n+m+1,j} = 0 \}
\end{aligned} \tag{4.66}$$

Die Constraints in der Menge 4.64 sorgen dafür, dass mindestens eine Belegung der Hilfsvariablen existiert, die das eindeutige Minimum des Qubo-Ausdrucks für erfüllende Belegungen erreichen. Um sicherzustellen, dass keine Belegung der Hilfsvariablen existiert, die kleinere Werte als das eindeutige Minimum für erfüllende Belegungen liefert, werden in der Menge 4.65 zusätzliche Constraints aufgestellt. Abschließend werden in der Menge 4.66 Constraints eingeführt, die garantieren, dass für nicht erfüllende Belegungen der Qubo-Ausdruck einen größeren Wert als das eindeutige Minimum annimmt.

Die Domänen der Variablen sind hierbei auf ganze Zahlen beschränkt. Da zu kleine Zahlen aufgrund von technischen Limitationen innerhalb des Annealers nicht umgesetzt werden können, könnten die Domänen so groß gewählt werden, um mehrere Nachkommastellen zu simulieren. Beispielsweise würde, um Zahlen mit drei Nachkommastellen zu finden, eine Domäne von -10000 bis 10000 gewählt werden.

Die Lösungszeit kann allerdings mit steigender Domänengröße wachsen, weshalb meistens kleine Domänen wie -20 bis 20 gewählt wurden. Weiterhin wird auch eine Variable für das eindeutige Minimum angelegt. Für die Domäne werden hier größeren Werten wie -100 bis 100 als Schranken gewählt. Dabei sollte beachtet werden, dass betragsmäßig größere Werte bei den Koeffizienten auch zu betragsmäßig größeren

Werten des eindeutigen Minimums führen können. Als Letztes müssen für jede erfüllende Belegung und jede Hilfsvariable zusätzliche Entscheidungsvariablen eingeführt werden, die das Ausprobieren aller möglichen Belegungen der Hilfsvariablen simulieren sollen. Die Constraints, die danach eingefügt werden, sind analog zu den Nebenbedingungen des LP-Problems aufgebaut. Diese sollen anhand des Beispiels $\neg(x_1 \wedge x_2 \wedge x_3)$ verdeutlicht werden.

Beispiel 4.3.3 (Qubo CSP). Im vorherigen Abschnitt wurde bereits gezeigt, dass für den aussagenlogischen Ausdruck $\neg(x_1 \wedge x_2 \wedge x_3)$ kein direkter Qubo-Ausdruck ohne Hilfsvariablen gefunden wurde. Das CSP wird somit von Beginn an mit einer Hilfsvariable aufgebaut. Zunächst werden die benötigten Variablen des CSPs eingeführt.

Variablen. Für jeden linearen und quadratischen Term des Qubo-Ausdrucks wird eine Entscheidungsvariable zum CSP hinzugefügt. Insgesamt entstehen dadurch die folgenden Variablen:

$$V = \{a_1, a_2, a_3, a_4, a_{12}, a_{13}, a_{14}, a_{23}, a_{24}, a_{34}\} \quad (4.67)$$

$$\cup \{h_1, h_2, h_3, h_4, h_5, h_6, h_7\} \quad (4.68)$$

Die Variablen a_i gehören zu den linearen Termen und die Variablen a_{ij} zu den quadratischen. Die Domänen der Variablen sind von -20 bis $+20$ festgelegt. Weiterhin werden die Variablen h_i benötigt, um für jede erfüllende Belegung anstelle der Hilfsvariable eingesetzt zu werden. Somit dürfen die Domänen von h_i nur 0 und 1 enthalten.

Im ersten Schritt werden alle erfüllenden Belegungen der ursprünglichen Variablen durchlaufen und für jede Belegung wird ein Gleichheits-Constraint aufgestellt. Die Constraints sehen dann wie folgt aus:

$$0a_1 + 0a_2 + 0a_3 + h_1a_4 + 0a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + 0a_{34} = t \quad (4.69)$$

$$0a_1 + 0a_2 + 1a_3 + h_2a_4 + 0a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + h_2a_{34} = t \quad (4.70)$$

$$\vdots$$

$$1a_1 + 1a_2 + 0a_3 + h_7a_4 + 1a_{12} + 0a_{13} + h_7a_{14} + 0a_{23} + h_7a_{24} + 0a_{34} = t \quad (4.71)$$

Die Hilfsvariablen h_i sorgen dafür, dass für die i -te erfüllende Belegung auch mindestens eine Belegung der neu eingeführten Hilfsvariablen existiert, für die der Qubo-Ausdruck sein eindeutiges Minimum t erreicht.

Im zweiten Schritt werden alle möglichen Belegungen der ursprünglichen und Hilfsvariablen durchlaufen, und abhängig davon, ob sie erfüllend oder nicht erfüllend sind,

werden unterschiedliche Constraints eingefügt. Die eingeführten Constraints für erfüllende Belegungen sehen wie folgt aus:

$$0a_1 + 0a_2 + 0a_3 + 0a_4 + 0a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + 0a_{34} \geq t \quad (4.72)$$

$$0a_1 + 0a_2 + 0a_3 + 1a_4 + 0a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + 0a_{34} \geq t \quad (4.73)$$

$$0a_1 + 0a_2 + 1a_3 + 0a_4 + 0a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + 0a_{34} \geq t \quad (4.74)$$

$$0a_1 + 0a_2 + 1a_3 + 1a_4 + 0a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + 1a_{34} \geq t \quad (4.75)$$

$\vdots$

$$1a_1 + 1a_2 + 0a_3 + 0a_4 + 1a_{12} + 0a_{13} + 0a_{14} + 0a_{23} + 0a_{24} + 0a_{34} \geq t \quad (4.76)$$

$$1a_1 + 1a_2 + 0a_3 + 1a_4 + 1a_{12} + 0a_{13} + 1a_{14} + 0a_{23} + 1a_{24} + 0a_{34} \geq t \quad (4.77)$$

Diese Constraints sorgen dafür, dass keine Belegung an Hilfsvariablen existieren kann, sodass ein kleinerer Wert als das eindeutige Minimum erreicht wird. Größere Werte sind aber zulässig. Für jede nicht erfüllende Belegung wird analog ein echt größer-Constraint eingeführt, um sicherzustellen, dass nicht erfüllende Belegungen immer einen größeren Wert als das eindeutige Minimum erreichen.

$$1a_1 + 1a_2 + 1a_3 + 0a_4 + 1a_{12} + 1a_{13} + 0a_{14} + 1a_{23} + 0a_{24} + 0a_{34} > t \quad (4.78)$$

$$1a_1 + 1a_2 + 1a_3 + 1a_4 + 1a_{12} + 1a_{13} + 1a_{14} + 1a_{23} + 1a_{24} + 1a_{34} > t \quad (4.79)$$

Diese Constraints reichen aus, um erste Qubo-Ausdrücke aus Wahrheitstabelle zu erzeugen. Für dieses konkrete Problem lässt sich mithilfe des Solvers von OR-Tools [PD] folgender Qubo-Ausdruck finden:

$$h + x_1x_2 - x_1h - x_2h + x_3h \quad (4.80)$$

Die Variable h ist dabei die neu eingeführte Hilfsvariable. Das eindeutige Minimum dieses Ausdrucks ist null. Für die nicht erfüllende Belegung $(1, 1, 1)$ kann keine Belegung der Hilfsvariablen gefunden werden, sodass das eindeutige Minimum erreicht oder unterschritten wird. Für sowohl $h = 0$ als auch $h = 1$ ergibt der Qubo-Ausdruck den Wert 1. Für die erfüllende Belegung $(1, 1, 0)$ ergibt sich zwar für $h = 0$ der Wert 1, allerdings kann h mit 1 belegt werden, um wieder das eindeutige Minimum von 0 zu erreichen. $\diamond$

Natürlich kann das Programm noch erweitert werden, um auch optimierte Qubo-Ausdrücke zu bestimmen. Also solche Ausdrücke, die besonders gute Ergebnisse auf dem Quanten-Annealer liefern. Dies wird genauer in Abschnitt 4.5 besprochen.

4.4 Überführung zum Ising-Modell

Bisher wurden ausschließlich Qubo-Ausdrücke bestimmt. Natürlich können Qubo und Ising-Ausdrücke äquivalent ineinander überführt werden. Dabei bleiben sowohl das eindeutige Minimum als auch alle anderen Werte bis auf eine Konstante erhalten. Allerdings können sich die Koeffizienten der beiden Modelle stark unterscheiden. Konkret verändern sich die Koeffizienten beim Umwandeln der Ausdrücke wie folgt:

$$a \cdot x_i = a \cdot \frac{s_i + 1}{2} = \frac{a}{2} \cdot s_i + \frac{a}{2} \quad (4.81)$$

$$a \cdot x_i \cdot x_j = a \cdot \frac{s_i + 1}{2} \cdot \frac{s_j + 1}{2} = \frac{a}{4} \cdot s_i + \frac{a}{4} \cdot s_j + \frac{a}{4} \cdot s_i \cdot s_j + \frac{a}{4} \quad (4.82)$$

Das a entspricht hier wieder dem Koeffizienten der linearen und quadratischen Terme. Die x_i und x_j sind boolesche Variablen des Qubo-Ausdrucks und die s_i und s_j sind die dazu äquivalenten Ising-Variablen des Ising-Modells. Lineare Koeffizienten werden mit einem Faktor von $\frac{1}{2}$ übersetzt 4.81, während quadratische Ausdrücke mit einem Faktor von $\frac{1}{4}$ übersetzt werden 4.82. Allerdings entstehen zusätzlich noch weitere lineare Koeffizienten mit einem Vorfaktor von $\frac{1}{4}$. Somit werden nicht immer alle Koeffizienten gleichermaßen skaliert. Damit ein Problem auf dem Quanten-Annealer ausgeführt werden kann, muss es immer zuerst in ein Ising-Modell überführt werden. Somit muss auch jede Qubo immer zuerst in ein Ising-Modell transformiert werden. Damit während der Modellierung die Koeffizienten genau so bleiben, wie sie initial angegeben werden, wird ab diesem Punkt in der Arbeit ausschließlich mit Ising-Modellen gearbeitet.

4.4.1 Ising-Ausdrücke erzeugen

Die bisher vorgestellten Techniken (Penalty-Modell, LP-Heuristik und CSP-Qubo-Finder) wurden alle genutzt, um Qubo-Ausdrücke aus Wahrheitswertetabellen zu erzeugen. Jedes dieser Verfahren lässt sich durch leichte Modifikation so anpassen, dass damit auch Ising-Ausdrücke gefunden werden können.

Penalty-Model. Beim Penalty-Model wird ein LP-Problem aufgestellt, welches als Lösung die Koeffizienten des Qubo-Ausdrucks liefert. Die einzige Änderung, die hier vorgenommen werden muss, ist das Austauschen der Nullen innerhalb der Belegungen durch minus Eins. Somit verändern sich die Nebenbedingungen aus Beispiel 4.3.1 wie

folgt:

$$+1a_1 - 1a_2 + 1a_3 - 1a_{12} + 1a_{13} - 1a_{23} - 1t + 0g = 0 \quad (4.83)$$

$$-1a_1 + 1a_2 + 1a_3 - 1a_{12} - 1a_{13} + 1a_{23} - 1t + 0g = 0 \quad (4.84)$$

$$+1a_1 + 1a_2 + 1a_3 + 1a_{12} + 1a_{13} + 1a_{23} - 1t + 1g = 0 \quad (4.85)$$

$$+1a_1 + 1a_2 + 1a_3 - 1a_{12} - 1a_{13} - 1a_{23} + 1t + 1g \leq -\epsilon \quad (4.86)$$

$$+1a_1 + 1a_2 - 1a_3 - 1a_{12} + 1a_{13} + 1a_{23} + 1t + 1g \leq -\epsilon \quad (4.87)$$

$$+1a_1 - 1a_2 + 1a_3 + 1a_{12} - 1a_{13} + 1a_{23} + 1t + 1g \leq -\epsilon \quad (4.88)$$

$$-1a_1 + 1a_2 + 1a_3 + 1a_{12} + 1a_{13} - 1a_{23} + 1t + 1g \leq -\epsilon \quad (4.89)$$

$$-1a_1 - 1a_2 + 1a_3 - 1a_{12} + 1a_{13} + 1a_{23} + 1t + 1g \leq -\epsilon \quad (4.90)$$

Hierbei handelt es sich weiterhin um ein Standard LP-Problem. Ein LP-Solver liefert für dieses Problem die Lösung $a_1 = a_2 = a_3 = -1$, $a_{12} = 1$ und $a_{13} = a_{23} = 0$, was folgendem Ising-Ausdruck entspricht: $-s_1 - s_2 - s_3 + s_{12}$.

Auch beim Verwenden von Hilfsvariablen muss keine Änderung des Verfahrens vorgenommen werden. Es werden wieder nur Nullen durch -1 in der Belegung ersetzt. Somit werden anstatt aller möglichen booleschen Hilfsvariablenbelegungen alle möglichen Ising-Hilfsvariablenbelegungen ausprobiert, bis eine Lösung des LP-Problems gefunden werden kann oder mehr Hilfsvariablen hinzugefügt werden müssen.

LP-Heuristik. Da die LP-Heuristik auch auf dem Lösen von LP-Problemen basiert, kann hier dieselbe Modifikation vorgenommen werden, die beim Penalty-Modell angewendet wurde. Allerdings basierte die Heuristik darauf, die Belegungen mit den meisten Nullen zuerst abzuarbeiten. Natürlich könnte die Heuristik modifiziert werden und zuerst die Belegungen abarbeiten, in denen der Wert -1 am häufigsten vorkommt. Ob diese Heuristik dann immer noch gut funktioniert, müsste getestet werden. In dieser Arbeit geht es aber vor allem um den CSP-Ansatz weshalb keine weiteren Tests dieser Heuristik für Ising-Ausdrücke durchgeführt werden.

Qubo-CSP Der CSP-Ansatz lässt sich auch durch einfaches Austauschen von 0 mit -1 innerhalb der Variablenbelegungen so modifizieren, dass Ising-Ausdrücke gefunden werden können. Zusätzlich muss die Domäne der Hilfsvariablen h_i von $\{0, 1\}$ zu $\{-1, 1\}$ geändert werden.

4.5 Optimierung von Ising-Ausdrücken

Zu einer gegebenen Wahrheitswerttabelle lassen sich viele verschiedene Qubo- und Ising-Ausdrücke finden, die alle die benötigte Qubo-Bedingung erfüllen (vgl. Def.

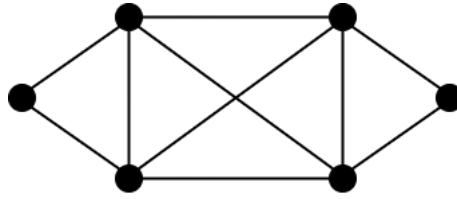


Abbildung 4.5: Vier färbbarer Beispielgraph.

3.2.1). Allerdings besitzen nicht alle verschiedenen Ausdrücke die gleiche Qualität. Für den Anneal-Prozess kann es einen großen Unterschied machen, welche Ausdrücke für ein konkretes Problem übergeben werden. Welche Faktoren dabei eine Rolle spielen und wie sich die konkreten Auswirkungen dabei verhalten, wird in den folgenden Abschnitten gezeigt. Dabei wird die Optimierung auf den in dieser Arbeit entwickelten CSP-Ansatz beschränkt.

Das durchgängige Beispiel dieses Abschnitts ist ein kleines Graphfärbungsproblem auf sechs Knoten. Der zu färbende Graph ist in Abbildung 4.5 zu sehen. Dieser kann mithilfe von vier verschiedenen Farben gefärbt werden. Da vier eine Zweierpotenz ist, können die einzelnen Farben mithilfe von Binärkodierung durch zwei boolesche Variablen dargestellt werden. Konkrete Ising-Ausdrücke werden dann in den einzelnen Teilabschnitten bestimmt. Zu beachten ist, dass eine Hilfsvariable benötigt wird, um einen Ising-Ausdruck zu finden, der auf zwei mit jeweils zwei Bit binär kodierten Zahlen Ungleichheit garantiert.

4.5.1 Energielücke

Das Quanten Adiabatic Theorem (vgl. Abschnitt 2.3) besagt, dass für eine größere Energielücke der Eigenwerte des Hamiltonoperators weniger Zeit benötigt wird, um ein erfolgreiches Annealing (bei dem der Annealer im energetisch niedrigsten Zustand bleibt) durchzuführen. Dadurch wird auch die Wahrscheinlichkeit erhöht, im energetisch niedrigsten Zustand zu bleiben, was die Robustheit des Verfahrens erhöht. Daraus kann direkt gefolgert werden, dass der finale Hamiltonoperator der erzeugt wird, möglichst große Abstände zwischen dem minimalen und nächsthöheren Eigenwerten besitzen soll. Auch wenn die Eigenwerte des finalen Hamiltonoperators (Summe aus Start- und Ziel-Hamiltonien) nur aufwendig konkret bestimmt werden können, sind sie doch abhängig vom Zielhamiltonien, welcher von der Modellierung abhängt.

Der Zielhamiltonien setzt sich wiederum aus den einzelnen Ising-Termen zusammen, die für jedes Constraint eingeführt werden. Der Abstand zwischen dem minimalen Eigenwert und dem nächsthöheren hängt somit von den Abständen der einzelnen

eingeführten Terme ab. Deswegen sollte versucht werden, den Abstand zwischen dem eindeutigen Minimum und dem nächst höheren Wert eines Ising-Ausdrucks zu maximieren.

Konkret realisiert wird diese Optimierung durch das Einführen von neuen Variablen in das bisherige CSP. Für jede nicht erfüllende Belegung wird ein neues Constraint eingeführt, welches einer neuen Variable g_i die Differenz zwischen dem eindeutigen Minimum und dem tatsächlich erreichten Wert zuweist. Das Minimum dieser g_i wird einer neuen Variable $g_{\min}$ zugewiesen. Die Variable $g_{\min}$ soll dann maximiert werden. Daraus resultiert ein neues COP. Dies wird nachfolgend an einer Belegung für das Graphfärbungsproblem demonstriert.

Beispiel 4.5.1 (Energielücke maximieren). Eine nicht erfüllende Belegung zweier benachbarter Knoten ist $(-1, -1, -1, -1)$, wobei die ersten beiden Variablen dem ersten Knoten entsprechen und die anderen beiden Werte dem zweiten Knoten. Im CSP wurden bisher die folgenden Constraints für diese Belegung eingeführt:

$$\begin{aligned} & -1a_1 - 1a_2 - 1a_3 - 1a_4 - 1a_5 \\ & +1a_{12} + 1a_{13} + 1a_{14} + 1a_{15} + 1a_{23} + 1a_{24} + 1a_{25} + 1a_{34} + 1a_{35} + 1a_{45} > t \end{aligned} \quad (4.91)$$

$$\begin{aligned} & -1a_1 - 1a_2 - 1a_3 - 1a_4 + 1a_5 \\ & +1a_{12} + 1a_{13} + 1a_{14} - 1a_{15} + 1a_{23} + 1a_{24} - 1a_{25} + 1a_{34} - 1a_{35} - 1a_{45} > t \end{aligned} \quad (4.92)$$

Der Koeffizient a_5 gehört zur eingefügten Hilfsvariable. Für diese beiden Constraints wird jeweils ein zusätzliches Constraint eingefügt, welches den Variablen g_1 und g_2 jeweils die Differenz der linken Seite der Ungleichung und dem eindeutigen Minimum t zuweist. Nachdem dies für alle nicht erfüllenden Belegungen durchgeführt wurde, kann eine weitere Variable $g_{\min}$ eingeführt werden. Dieser wird über ein weiteres Constraint das Minimum aus allen g_i zugewiesen. Über eine Optimierungsfunktion kann diese Variable dann maximiert werden. $\diamond$

Wie die konkrete Auswirkung von zwei verschiedenen Ising-Ausdrücken mit unterschiedlichen minimalen Energieabständen aussieht, wird an folgenden zwei Beispielen gezeigt. Dabei wurde einmal absichtlich ein Ising-Ausdruck mit einer besonders kleinen Energielücke und einmal ein Ising-Ausdruck mit einer besonders großen Energielücke garantiert.

Beispiel 4.5.2 (Verschiedene Energielücken). Das Problem ist weiterhin das Graphfärbungsproblem. Für das Constraint, das ausdrückt, dass zwei benachbarte Knoten nicht die selbe Farbe erhalten dürfen, wurden folgende zwei Ising-Ausdrücke generiert:

$$7x_0x_1 - 8x_0x_3 + 15x_0x_4 - 8x_1x_2 + 15x_1x_4 + 7x_2x_3 - 15x_2x_4 - 15x_3x_4 \quad (4.93)$$

$$3x_0x_1 - x_0x_2 - 6x_0x_3 - 10x_0x_4 - 8x_1x_2 - 11x_1x_4 + x_2x_3 + 10x_2x_4 + 7x_3x_4 \quad (4.94)$$

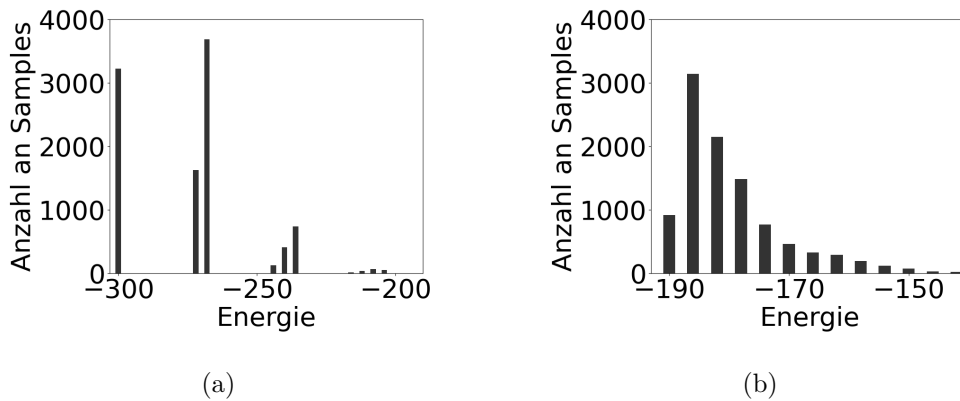


Abbildung 4.6: (a) Anneal-Ergebnisse für eine große minimale Energielücke. (b) Anneal-Ergebnisse für eine kleine minimale Energielücke.

Der Ising-Ausdruck 4.93 besitzt mit 28 eine besonders große Energielücke, während der Ising-Ausdruck 4.94 mit vier im Verhältnis eine sehr kleine Energielücke aufweist. Diese beiden Ausdrücke wurden jeweils genutzt, um das Graphfärbungsproblem auf dem Graphen 4.5 zu realisieren. Dabei wurden jeweils fünf Testdurchläufe mit jeweils 2000 Anneal-Durchläufen und einer Anneal-Zeit von $5\mu s$ durchgeführt. Bei den fünf Testdurchläufen wurde immer ein neues Embedding generiert, um den Einfluss von verschiedenen guten Embeddings zu minimieren. Die Ergebnisse sind in der Abbildung 4.6 zu sehen.

Das Optimum für die große Energielücke beträgt -300 (a), während das Optimum für die kleine Energielücke -190 beträgt (b). In der Abbildung 4.6 ist deutlich zu erkennen, dass von den insgesamt 10000 Anneal-Durchläufen der Ising-Term mit der größeren Energielücke besser abgeschnitten hat. Es wurde 3226 mal ein Optimum erreicht, wohingegen bei der kleinen Energielücke nur 915 mal ein Optimum erreicht wurde. Es wird also deutlich, wie wichtig es ist, Ising-Ausdrücke zu finden, die eine möglichst große minimale Energielücke besitzen. $\diamond$

4.5.2 Koeffizienten-Domäne

Die Koeffizienten-Domäne, also der Wertebereich der Koeffizienten des Ising-Ausdrucks, spielt auch eine essentielle Rolle für die Qualität des Ising-Ausdrucks. Der Grund dafür liegt in den technischen Limitationen des Quanten-Annealers. Bevor der Ising-Ausdruck auf dem Quanten-Annealer ausgeführt werden kann, muss nicht nur ein passendes Embedding gefunden werden. Zusätzlich müssen die Koeffizienten der linearen und quadratischen Terme so skaliert werden, dass diese im möglichen Wertebereich

der physischen Coupler und Biases liegen, was die Qualität des Annealing-Ergebnisses beeinflussen kann.

Domänen Advantage System 7.1[adv]. Als konkretes Beispiel kann dafür das Advantage System 7.1 betrachtet werden. Der Wertebereich der linearen Terme liegt bei $[-4, 4]$ und der Wertebereich der quadratischen Terme bei $[-2, 1]$. Außerdem ist die maximale Stärke an Couplern, die auf ein Qubit gleichzeitig einwirken können, begrenzt. Für das Advantage System 7.1 liegt diese Grenze bei -18 und 15 . Dies bedeutet, auch wenn innerhalb der Pegasus-Architektur bis zu 16 Qubits untereinander verbunden sind, können diese nicht alle gleichzeitig ihre maximale negative Coupler-Wirkung anwenden. Die Ising-Koeffizienten werden also automatisch so skaliert, dass die Wertebereiche eingehalten werden.

Die Skalierung erfolgt durch das Multiplizieren mit einem Skalierungsfaktor $\frac{1}{f}$, wobei sich $\frac{1}{f}$ wie folgt berechnen lässt:

$$\max \left\{ \frac{\max(h_i)}{h_{\max}}, \frac{\min(h_i)}{h_{\min}}, \frac{\max(J_{ij})}{J_{\max}}, \frac{\min(J_{ij})}{J_{\min}}, \frac{\max(J_{\text{total}_i})}{J_{\text{total}_{\max}}}, \frac{\min(J_{\text{total}_i})}{J_{\text{total}_{\min}}} \right\} \quad (4.95)$$

Die Variablen h_i sind die linearen Terme und die J_{ij} die quadratischen Koeffizienten des Ising-Ausdrucks. Die J_{total_i} entsprechen der Summe aller Coupler, die auf die Variable i einwirken. Die technischen Grenzen des Annealers sind jeweils mit $h_{\max}$, $h_{\min}$, $J_{\max}$, $J_{\min}$, $J_{\text{total}_{\max}}$ und $J_{\text{total}_{\min}}$ gegeben. Alle Koeffizienten (h_i und J_{ij}) werden dann mit dem berechneten f multipliziert. Dadurch werden auch die minimalen Energielücken um den Faktor f skaliert, was zu einer Verschlechterung der Lösungsqualität führen kann. Aus diesem Grund sollten die Koeffizienten so gewählt werden, dass f so groß wie möglich ist.

Die konkrete Umsetzung in der COP-Modellierung ist dabei sehr einfach. Es werden zwei neue Variablen eingeführt: *minKoe* und *maxKoe*. Diesen werden über jeweils ein Constraint der minimale und maximale Koeffizient aller Koeffizienten im COP zugewiesen:

$$\begin{aligned} \text{minKoe} &= \min(h_i, J_{ij}) \\ \text{maxKoe} &= \max(h_i, J_{ij}) \end{aligned}$$

Die Differenz dieser beiden Koeffizienten wird dann in der Optimierungsfunktion minimiert. Das hat den Effekt, dass der benötigte Skalierungsfaktor groß gehalten werden kann. Dadurch bleibt die minimale Energielücke größer als bei einem sehr kleinen Skalierungsfaktor. Allerdings existiert noch ein weiterer Faktor, der Einfluss auf den Skalierungsfaktor hat. Die Verkettung von einzelnen physischen Qubits zu einem logischen Qubit wird über die gleichen Coupler realisiert, die für die Umsetzung des

Ising-Ausdrucks genutzt werden. Dafür wird zum einen eine geeignete Kettenstärke berechnet und diese muss auch auf den Wertebereich des Annealers skaliert werden. Dazu wird die Kettenstärke zu den J_{ij} mit aufgenommen.

Kettenstärke. Dem Annealer kann entweder eine Kettenstärke übergeben werden oder automatisch, durch ein von Dwave vorgegebenes Verfahren [cha], eine geeignete Stärke berechnet werden. Die Formel dafür sieht wie folgt aus:

$$c = k \cdot \sqrt{\frac{1}{n} \sum_{i=1}^n J_i^2} \cdot \sqrt{\frac{1}{m} \sum_{i=1}^m d_i} \quad (4.96)$$

Dabei ist k eine Konstante, die standardmäßig mit 1,414 belegt ist. Der zweite Term ist das quadratische Mittel aller Coupler-Koeffizienten und der dritte Term ist die Wurzel des Durchschnitts aller Grade, die die Variablen v_i annehmen. Die Idee ist es, eine Kettenstärke zu finden, die groß genug ist, sodass keine Ketten während des Annealings gebrochen werden, also alle Variablen innerhalb einer Kette den gleichen Wert annehmen. Allerdings soll sie auch nicht so groß sein, dass ein zu kleiner Skalierungsfaktor gewählt werden muss, der die Energielücke zu klein skaliert. Dabei wird die Kettenstärke automatisch durch die oben gegebene Formel 4.96 berechnet, kann aber auch manuell übergeben und angepasst werden.

Auch die Kettenstärke muss, genau wie die anderen Werte der Coupler, auf den physischen Wertebereich skaliert werden. Die Auswirkungen von verschiedenen Koeffizienten-Domänen und der daraus resultierenden Skalierungsfaktoren, werden wieder anhand des Graphfärbungsbeispiels verdeutlicht.

Beispiel 4.5.3 (Verschiedene Koeffizienten-Domänen). Wieder wird ein Ising-Ausdruck gesucht, der dafür sorgt, dass benachbarte Knoten nicht die gleiche Farbe erhalten. Dabei wird einmal ein Ausdruck mit einer absichtlich großen Koeffizienten-Domäne gewählt und einmal ein Ausdruck mit einer kleinen Koeffizienten-Domäne:

$$1x_0x_1 - 1x_0x_3 - 2x_0x_4 - 1x_1x_2 - 2x_1x_4 + 1x_2x_3 + 2x_2x_4 + 2x_3x_4 \quad (4.97)$$

$$1x_0x_1 - 1x_0x_3 + 2x_0x_4 - 19x_1x_2 + 20x_1x_4 + 1x_2x_3 - 20x_2x_4 - 2x_3x_4 \quad (4.98)$$

Der Term 4.97 besitzt mit $[-2, 2]$ eine besonders kleine Koeffizienten-Domäne, wohingegen der Term 4.98 mit $[-20, 20]$ eine besonders große Koeffizienten-Domäne besitzt. Die minimale Energielücke ist allerdings bei beiden Ausdrücken mit dem Wert 4 identisch. Für den Term 4.97 mit einer kleinen Domäne muss ein Skalierungsfaktor von $f = \frac{1}{4} = 0,25$ gewählt werden. Die automatisch berechnete Kettenstärke für den Term entspricht $-6,768$ und liefert, da Coupler negative Werte bis -2 erlauben, einen Skalierungsfaktor von $\frac{-2}{-6,768} = 0,296$. Von diesen beiden Faktoren muss der jeweils

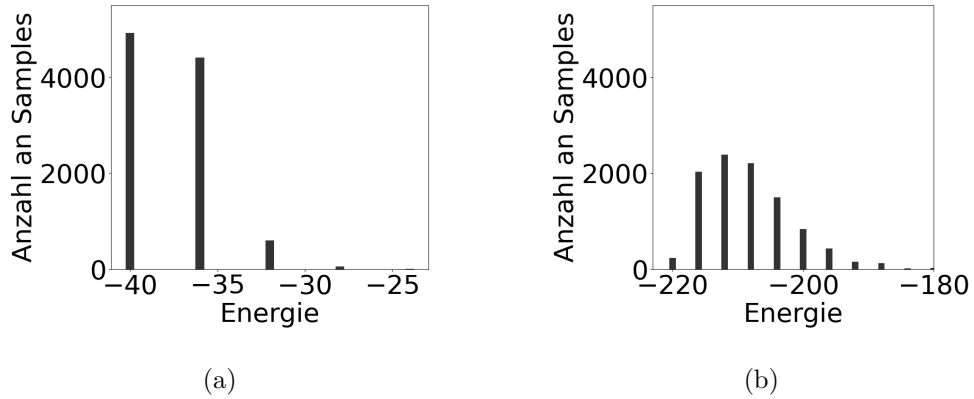


Abbildung 4.7: (a) Anneal-Ergebnis für eine kleine Koeffizienten-Domäne. (b) Anneal-Ergebnis für eine große Koeffizienten-Domäne.

kleine verwendet werden, in diesem Fall also 0,25. Für den Term 4.98 muss aufgrund der hohen Coupler-Stärke eine große Kettenstärke gewählt werden. In diesem Fall ergibt sich eine Kettenstärke von $-46,257$. Daraus resultiert auch der sehr kleine Skalierungsfaktor mit $f = \frac{-2}{-46,257} = 0,0432$.

Für beide Terme wurden wieder jeweils fünf Anneal-Durchläufe mit jeweils 2000 Samples und einer Anneal-Zeit von $5\mu s$ durchgeführt. Die Ergebnisse sind in der Abbildung 4.7 dargestellt. Es ist deutlich zu erkennen, dass für eine kleine Koeffizienten-Domäne wesentlich öfter das Minimum erreicht wurde als für eine große Domäne. Das Minimum für (a) liegt bei -40 und das Minimum für (b) bei -220 . Beim Annealer mit der kleinen Domäne erreichten 49,23% aller Samples das Minimum wohingegen bei der großen Domäne nur 2,38% das Minimum erreicht haben. $\diamond$

4.5.3 Anzahl der Variablen und Kanten

Gerade für größere Probleme kann es relevant sein, dass so wenig Variablen und Kanten wie möglich verwendet werden, um das CSP als Ising-Ausdruck darzustellen. Je mehr Variablen und Kanten benötigt werden, desto schwerer wird es, ein geeignetes Embedding auf dem Quanten-Annealer zu finden. Teilweise kann es auch passieren, dass erst gar kein Embedding gefunden werden kann. Die Begrenzung an die Variablenanzahl bezieht sich hier vor allem auf die Anzahl an Hilfsvariablen, da die konkrete Anzahl an benötigten Variablen stark von der Modellierung abhängt.

Außerdem erhöht die Anzahl an verwendeten Qubits auch den Einfluss von ungewollten Störfaktoren. Zum einen ist die Wahrscheinlichkeit für zufällige thermische

Sprünge zwischen Energieniveaus erhöht. Es kann zu ungewollten Wechselwirkungen zwischen physisch nahe beieinanderliegenden Qubits kommen, die nicht miteinander gekoppelt sein sollen. Deshalb sollten immer Ising-Ausdrücke mit so wenig Variablen und Kanten wie möglich gefunden werden.

Umgesetzt wird diese Optimierung innerhalb des Ising-COPs wie folgt: Für jeden Koeffizienten wird eine boolesche Variable eingeführt, die genau dann 1 wird, wenn der entsprechende Koeffizient ungleich 0 ist. In der Optimierungsfunktion wird dann die Summe dieser booleschen Variablen minimiert. Der Einfluss von Variablen und Kantenanzahl wird wieder am Graphfärbungsproblem verdeutlicht.

Beispiel 4.5.4 (Anzahl Variablen und Kanten). Das Graphfärbungs-Constraint (zwei benachbarte Knoten dürfen nicht die selbe Farbe erhalten) kann mit nur einer Hilfsvariable h_1 realisiert werden 4.99. Allerdings werden für diesen Test absichtlich drei Hilfsvariablen h_1 , h_2 , h_3 und eine maximale Anzahl an möglichen Kanten verwendet 4.100, um das Constraint als Ising-Term umzusetzen. Die zwei zu vergleichenden Ising-Terme sehen wie folgt aus:

$$1x_0x_1 - 1x_0x_3 - 2x_0h_1 - 1x_1x_2 - 2x_1h_1 + 1x_2x_3 + 2x_2h_1 + 2x_3h_1 \quad (4.99)$$

$$\begin{aligned} &1x_0x_1 + 2x_0x_2 + 1x_0x_3 + 1x_0h_1 + 1x_0h_2 + 2x_0h_3 + 1x_1x_2 + 2x_1x_3 \\ &+ 1x_1h_1 + 1x_1h_2 + 2x_1h_3 + 1x_2x_3 + 1x_2h_1 + 1x_2h_2 + 2x_2h_3 + 1x_3h_1 \\ &+ 1x_3h_2 + 2x_3h_3 + 2h_1h_2 + 2h_1h_3 + 2h_2h_3 \end{aligned} \quad (4.100)$$

Wieder wurden für beide Ising-Terme jeweils fünf Anneal-Durchläufe mit jeweils 2000 Samples und einer Zeit von $5\mu s$ durchgeführt. Die Ergebnisse sind in der Abbildung 4.8 zu sehen. Auch hier wird deutlich, dass weniger Kanten und Hilfsvariablen zu besseren Ergebnissen führen. Beim Ising-Ausdruck mit wenig Kanten und Variablen wurde das Minimum für 49,23% der Samples erreicht und beim Ising-Ausdruck mit vielen Kanten und Variablen wurde das Minimum in 27,17% der Samples erreicht.

◇

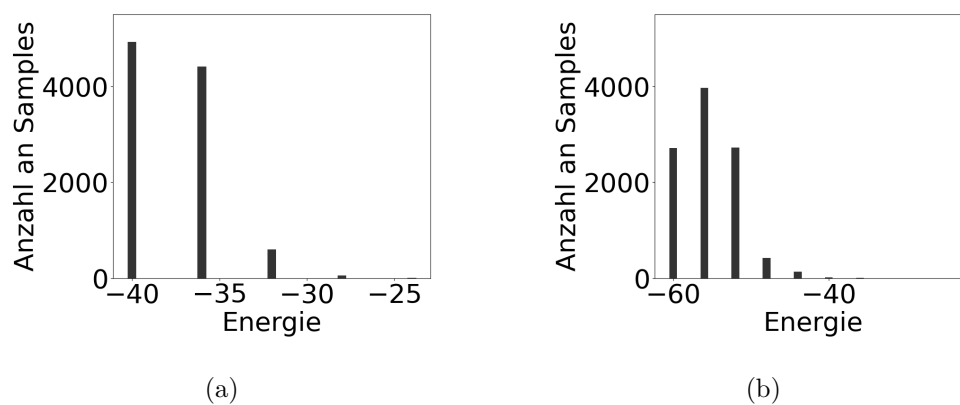


Abbildung 4.8: (a) Anneal-Ergebnis für eine minimale Anzahl von Kanten und Hilfsvariablen. (b) Ergebnis für drei Hilfsvariablen und maximale Kantenanzahl.

Kapitel 5

Evaluation

In diesem Kapitel werden die in der Arbeit behandelten Constraint-Probleme genauer untersucht und für verschiedene Anneal-Konfigurationen getestet. In Abschnitt 5.1 wird zunächst das in dieser Arbeit entwickelte Werkzeug zum Finden von Qubo/Ising-Ausdrücken mit den anderen in der Arbeit erwähnten Verfahren verglichen. Danach wird in Abschnitt 5.2 das Problem der Graphfärbung für verschiedene Problemgrößen und Anneal-Konfigurationen genauer untersucht und ausgewertet. Das Rotating Rostering-Problem wird in Abschnitt 5.3 auch einer ausführlichen Testreihe unterzogen und eine alternative Kodierung des Problems wird untersucht. Abschließend wird in Abschnitt 5.4 das Sonet-Problem [ALL23] als COP ausgewertet.

5.1 Vergleich Qubo/Ising-Finder

Um ein CSP in ein Qubo- oder Ising-Modell umzuwandeln, wurden innerhalb dieser Arbeit verschiedene Techniken vorgestellt. Der Fokus in der Auswertung liegt dabei vor allem auf den Verfahren, die einzelne Constraints, gegeben als Wahrheitstabelle, in Qubo/Ising-Ausdrücke umwandeln können. Die drei zu vergleichenden Techniken sind dabei das D-Wave Penalty Model [pen], die LP-Heuristik [Pak21] und das in dieser Arbeit entwickelte COP-Programm 4.3.3.

5.1.1 Beispielproblem

Das untersuchte Problem wurde dem Paper [Pak21], welches die LP-Heuristik eingeführt hat, entnommen. Bei dem Problem handelt es sich um Fizz Buzz. Dies wurde ursprünglich als Gruppen-Wortspiel für Kinder konzipiert. Dabei sitzen die spielenden Personen im Kreis und müssen der Reihe nach die natürlichen Zahlen aufsagen.

Zahl	x_0	x_1	x_2	x_3	Fizz	Buzz
0	0	0	0	0	1	1
1	0	0	0	1	0	0
2	0	0	1	0	0	0
3	0	0	1	1	1	0
4	0	1	0	0	0	0
5	0	1	0	1	0	1

Tabelle 5.1: Ausschnitt der Fizz Buzz-Wahrheitstabelle.

Immer wenn eine Zahl durch drei teilbar ist, muss die Zahl durch das Wort Fizz ersetzt werden. Wenn die Zahl durch fünf teilbar ist, muss die Zahl durch Buzz ersetzt werden. Falls eine Zahl durch drei und fünf teilbar ist, müssen beide Wörter, also Fizz Buzz, gesagt werden.

Das Problem wird jetzt mit sechs booleschen Variablen wie folgt dargestellt. Die ersten vier Variablen kodieren binär eine Zahl. Die beiden darauf folgenden Variablen geben an, ob die Zahl durch drei oder fünf teilbar ist. Ein Ausschnitt der Wahrheitstabelle ist in Tabelle 5.1 zu sehen. In dem Paper [Pak21] wurde zu diesem Problem ein Qubo-Ausdruck gesucht. Aus diesem Grund wird auch das COP-Programm verwendet, um einen Qubo-Ausdruck zu finden. Die D-Wave Penalty Model-Implementierung erlaubt allerdings nur, Ising-Ausdrücke zu finden, was die Vergleichbarkeit reduzieren kann. Bei durchgeführten Stichprobentests wurden allerdings meist keine relevanten Unterschiede festgestellt.

Der Qubo-Term, der mithilfe der LP-Heuristik gefunden wurde, ist direkt aus dem Paper entnommen und im Anhang 7.2 zu finden. Der durch das Penalty Model gefundene Ising-Term wurde nachträglich in einen Qubo-Term umgewandelt und ist auch im Anhang 7.2 aufgeführt.

Für den COP-Ansatz wurden zwei verschiedene Qubo-Ausdrücke untersucht. Der Grund dafür liegt in der benötigten Zeit zum Finden dieser Ausdrücke. Sowohl das D-Wave Penalty Model als auch die LP-Heuristik liefern ihr Ergebnis in weniger als einer Sekunde. Das COP-Programm benötigt zum Finden einer ersten Lösung $\text{COP}_{\text{first}}$ zehn Minuten und um das Optimum COP_{opt} zu finden mehrere Stunden. Die beiden konkreten Ausdrücke sind auch im Anhang 7.2 zu finden.

5.1.2 Auswertung

Für die Tests wurden jeweils fünf Anneal-Tests mit einer Anneal-Zeit von $5 \mu\text{s}$ und 2000 Samples durchgeführt. Die Ergebnisse des Ausdrucks, der vom Penalty Model

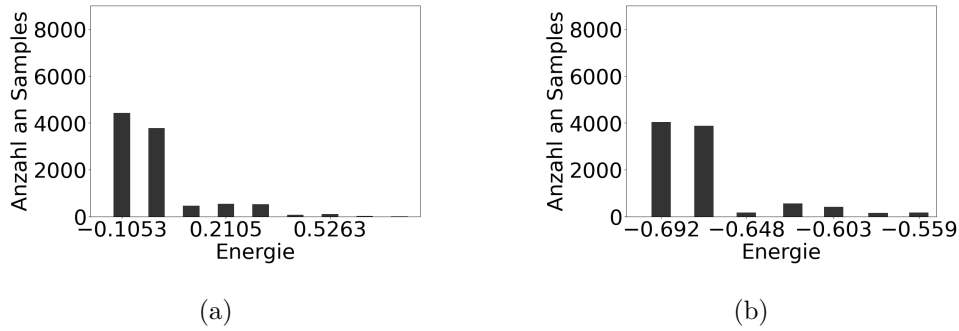


Abbildung 5.1: Anneal-Ergebnisse für das Penalty Model (a) und die LP-Heuristik (b).

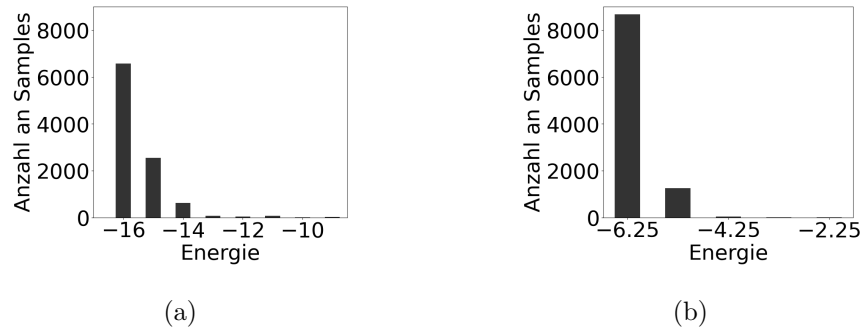


Abbildung 5.2: (a) Erstes COP-Ergebnis nach ungefähr zehn Minuten. (b) COP-Optimum nach mehreren Stunden.

und der LP-Heuristik berechnet wurden, sind in Abbildung 5.1 zu sehen. Das Minimum für das Penalty Model liegt bei $-0,1053$ und für die LP-Heuristik bei $-0,692$. Für die beiden Qubo-Ausdrücke des COP-Programms wurde der gleiche Test durchgeführt. Hierbei liegt das Minimum bei -16 und $-6,25$. Die Ergebnisse der beiden Tests sind in Abbildung 5.2 dargestellt.

Es ist deutlich zu sehen, dass die Qubo-Ausdrücke des COP-Programms bessere Ergebnisse erzielen als die beiden LP-Ansätze. Konkret erreichten im Fall des Penalty Models nur 44,23% der Samples das Minimum. Bei der LP-Heuristik sind es sogar nur 40,38%. Die Qubo-Ausdrücke des COP-Programms erzielen jeweils in 65,81% und 86,79% der Samples das Optimum. Diese starke Verbesserung im Vergleich zu den anderen Methoden lässt sich mithilfe des Quantum Adiabatic Theorems erklären.

Betrachten wir die minimalen Energielücken der einzelnen Terme, wird deutlich,

	LP-Heuristik	Penalty Modell	COP _{first}	COP _{opt.}
Energielücke	0,019	0,105	1	1
Skalierungsfaktor	4	1,310	0,2	0,571
skalierte Energielücke	0,076	0,138	0,2	0,571
Kanten	28	28	25	20
Prozentsatz Optimum	42,78%	44,23%	65,81%	86,79%

Tabelle 5.2: Vergleich der minimalen Energielücken und Anneal-Ergebnisse der verschiedenen Qubo-Ausdrücke.

wie die unterschiedlichen Ergebnisse zustande kommen. Für den Ausdruck der LP-Heuristik ergibt sich nach dem Skalieren der Koeffizienten eine minimale Energielücke von 0,076. Für den Ausdruck des Penalty Modells eine Energielücke von 0,138 und die Terme des COP-Programms liefern jeweils die Energielücken 0,2 und 0,571. Die Werte sind auch noch einmal in der Tabelle 5.2 aufgelistet. Der Skalierungsfaktor wurde dabei, wie in Abschnitt 4.5.2 beschrieben, berechnet. Durch Multiplikation der Energielücke mit dem Skalierungsfaktor entsteht dann die skalierte Energielücke. Hier wird deutlich, dass größere Energielücken zu deutlich besseren Anneal-Ergebnissen führen. Außerdem benötigt der optimale COP-Ausdruck acht Kanten weniger als die LP-Ausdrücke, was auch noch einmal zu besseren Ergebnissen führen kann.

Natürlich ist das große Problem des COP-Ansatzes die benötigte Rechenzeit. Allerdings wurden noch keine Optimierungsversuche in Bezug auf die Laufzeit unternommen. Bessere Variablen und Werteheuristiken während der Suche, das Einfügen von weiteren Constraints oder auch das Entfernen von vielleicht nicht benötigten Constraints kann zu einer schnelleren Laufzeit führen. Außerdem werden die Qubo- bzw. Ising-Ausdrücke auf klassischen Computern berechnet, für die die Laufzeit nicht so limitiert ist wie für einen Quantencomputer. Bessere Ising-Ausdrücke können somit zu einer Minimierung der benötigten Quantencomputerzeit führen. Die so gefundenen Ising-Ausdrücke sind außerdem wiederverwendbar. So muss für jedes Constraint dieser Ausdruck nur einmalig berechnet werden und kann dann auch bei anderen Problemen mit demselben Constraint wiederverwendet werden.

5.2 Graphfärbung

Das Graphfärbungsproblem, welches schon in Abschnitt 3.2.1 behandelt wurde, wird in diesem Abschnitt einmal ausführlicher für verschieden große Probleminstanzen untersucht. Die Graphen wurden dabei so generiert, dass die entstandenen Ising-Ausdrücke sich besonders gut auf dem Quanten-Annealer embedden lassen. Dafür

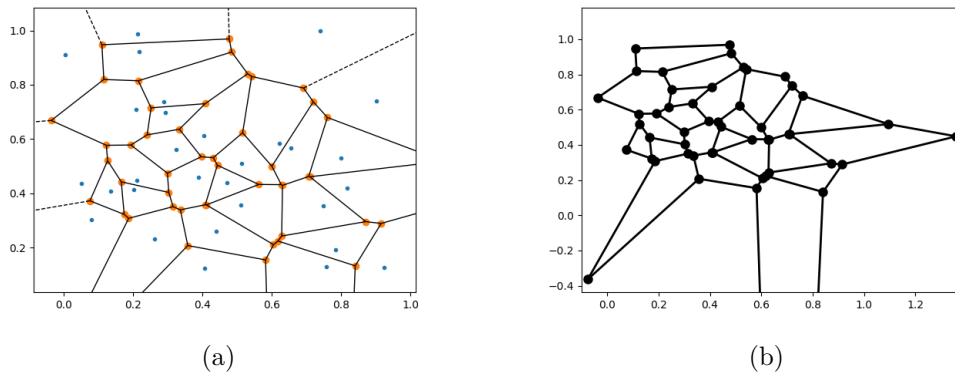


Abbildung 5.3: (a) Voronoi Diagramm zur Grapherzeugung. (b) Planarer Graph erstellt aus Voronoi Diagramm (a).

wurde der Grad jedes einzelnen Knotens limitiert und es wurden ausschließlich planare Graphen erzeugt. Diese Graphenstruktur passt ungefähr zu der des Quantenannealers. Innerhalb des Annealers besitzt jedes Qubit nur eine beschränkte Anzahl an Couplern und ist nur mit seinen lokal benachbarten Qubits gekoppelt.

5.2.1 Grapherzeugung

Um planare Graphen mit beschränktem Knotengrad zu erzeugen, wurden Voronoi-Diagramme verwendet. Zuerst werden zufällige Punkte im Raum platziert und daraus ein Voronoi-Diagramm erzeugt. Die Punkte, in denen sich die Trennlinien des Voronoi-Diagramms schneiden, werden als Knoten für den zu erzeugenden Graphen verwendet. Die Kanten sind dann die Trennlinien zwischen zwei so gewählten Knoten. Ein Beispiel für einen so entstandenen Graphen ist in Abbildung 5.3 zu sehen.

Diese Methode zum Erstellen von planaren Graphen sorgt dafür, dass der Grad jedes Knotens für alle hier erstellten Graphen auf maximal drei beschränkt bleibt. Dadurch wächst die Anzahl der Kanten des Graphen mit steigender Knotenzahl nur linear, was zu einem leichteren Embedding auf dem Quanten-Annealer führt. Insgesamt wurden Graphen von 10 bis 380 Knoten mit Abständen von 10 Knoten erstellt und getestet.

5.2.2 Auswertung Kodierung

Da es sich hier um das Färben von planaren Graphen handelt und diese immer mit vier Farben färbbar sind [RSST97], wurde die Domäne der Variablen auf $\{1, 2, 3, 4\}$ beschränkt. Weiterhin wurden drei verschiedene Variablenkodierungen (One Hot, Bi-

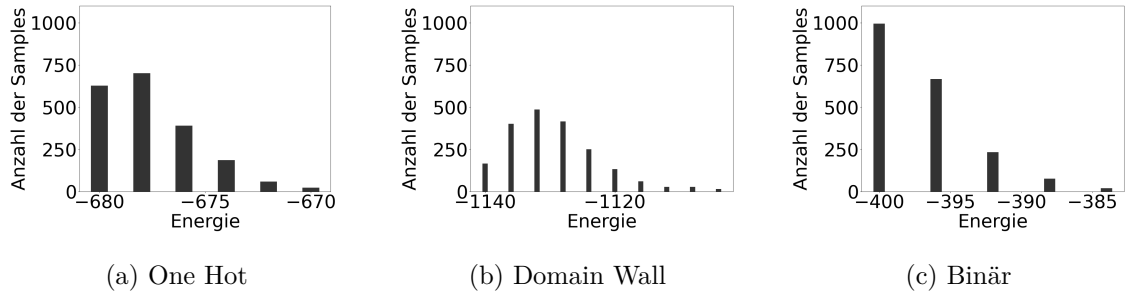


Abbildung 5.4: Anneal-Ergebnisse für die One Hot- (a), Domain Wall- (b) und Binärkodierung (c).

när und Domain Wall) getestet. Die genaue Transformation ist im Anhang 7.3 aufgeführt. Zum Vergleich der einzelnen Kodierungen wurde ein Graph mit 70 Knoten gewählt. Für diesen Graphen wurden die drei verschiedenen Ising-Modelle mit jeweils 2000 Anneal-Durchläufen und einer Anneal-Zeit von $20\mu s$ getestet. Die Ergebnisse der Samples sind in Abbildung 5.4 zu sehen.

Es ist deutlich zu erkennen, dass die Binärkodierung die besten Ergebnisse liefert. Für diese Kodierung erreichten 49,75% der Samples das Optimum von -400 . Die One Hot-Kodierung erreichte nur in 31,4% der Samples das Optimum von -680 und die Domain Wall-Kodierung sogar in nur 8,3% der Samples. Dieser Unterschied in den Anneal-Ergebnissen lässt sich auch theoretisch durch das Betrachten der Energielücken und benötigten Qubits erklären.

Bei der Binärkodierung werden für das Embedding des Ising-Modells nur 395 Qubits benötigt, wohingegen die One Hot-Kodierung 421 und die Domain Wall-Kodierung 488 Qubits benötigen. Die hier genannten Zahlen und noch ein paar weitere Vergleichswerte sind auch noch einmal in der Tabelle 5.3 aufgeführt. Außerdem entsteht beim skalierten Ising-Modell der Binärkodierung die größte Energielücke mit 1,272. Die One Hot-Kodierung liegt dicht dahinter mit 1,168, benötigt aber 23,54% mehr Qubits. Die Domain Wall-Kodierung liefert mit Abstand die schlechteste Energielücke mit 0,648, was sich auch in dem schlechtesten Anneal-Ergebnis widerspiegelt.

5.2.3 Auswertung Anneal-Zeit

Damit das Quanten-Annealing erfolgreich durchgeführt werden kann, ist nicht nur die Energielücke zwischen zwei Eigenzuständen relevant, sondern auch die zeitliche Ableitung des Hamiltonoperators. Dabei sollte diese so klein wie möglich sein. Eine Verlängerung der Anneal-Zeit führt dabei zu einer langsameren Veränderung des Ha-

	One Hot	Domain Wall	Binär
Koeffizientendomäne	1 bis 5	-10 bis 10	-2 bis 3
Ising-Modell Variablen	280	210	240
Max. Knotengrad	6	11	7
Anzahl Kanten	820	847	670
Anzahl Qubits	488	421	395
Max. Kettenlänge	4	4	4
Min. Energielücke	2	4	4
Skalierungsfaktor	0,584	0,162	0,318
Skalierte Energielücke	1,168	0,648	1,272

Tabelle 5.3: Eigenschaften des Ising-Modells und des dazu gehörigen Embeddings für die One Hot-, Domain Wall- und Binärcodierung.

miltonian was zu einer kleineren zeitlichen Ableitung führt. Dieses Phänomen wird an dem Graphfärbungsbeispiel für 150 Knoten und der Binärcodierung genauer untersucht. Dafür wurden 10000 Anneal-Durchläufe mit den Anneal-Zeiten $0,5\mu s$, $2\mu s$, $5\mu s$, $10\mu s$, $20\mu s$, $50\mu s$, $100\mu s$, $150\mu s$ und $200\mu s$ durchgeführt. Abbildungen der Samples für die neun Anneal-Zeiten sind im Anhang 7.4 zu finden.

Interessant wird hier die Betrachtung der TTT für die einzelnen Anneal-Zeiten (vgl. Abschnitt 3.2.4). In diesem Kapitel wird TTT immer als $TTT_{90\%}$ benutzt. Außerdem werden nur die Anneal-, Readout- und Delay-Zeiten für die TTT -Berechnung genutzt. Programming-, Overhead- und Postprocessing-Zeiten werden vernachlässigt. Die Readout-Zeit ist Probleminstanz abhängig und beträgt hier konstant $186,9\mu s$. Die Delay-Zeit ist im allgemeinen auch konstant mit $20,6\mu s$. Für sehr kleine Anneal-Zeiten wird das Optimum von nur sehr wenigen Samples erreicht. Dadurch ergibt sich trotz der sehr geringen Anneal-Zeit eine doch relativ hohe TTT . Die Anzahl an optimalen Samples und der jeweiligen TTT lässt sich im Anhang in Tabelle 7.1 nachlesen. Vor allem die im Vergleich hohen Readout- und Delay-Zeiten mit $186,9\mu s$ und $20,6\mu s$ haben großen Einfluss auf die TTT . Dadurch beträgt die TTT bei einer Anneal-Zeit von $0,5\mu s$ insgesamt $368333\mu s$, aber nur $4075\mu s$ bei einer Anneal-Zeit von $200\mu s$. Die bestmögliche TTT wird auch bei $200\mu s$ erreicht. Die Kurve der TTT wird für größere Anneal-Zeiten immer flacher. Es erreichen zwar immer mehr Samples das Optimum, allerdings ist die Verbesserung der TTT dadurch nur minimal. Für noch längere Anneal-Zeiten ist sogar eher mit einer Verschlechterung der TTT zu rechnen, da die Qubits natürlich auch länger äußeren und thermischen Störeinflüssen ausgesetzt sind. Der Verlauf der TTT für die verschiedenen Anneal-Zeiten ist in Abbildung 5.5 grafisch dargestellt.

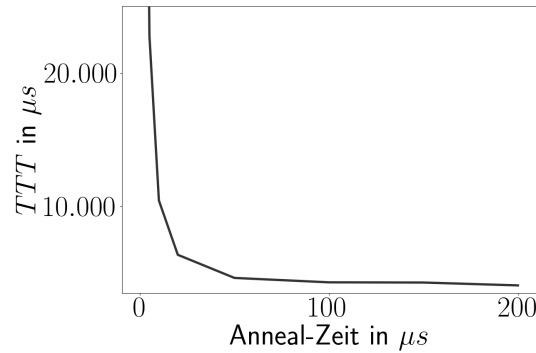
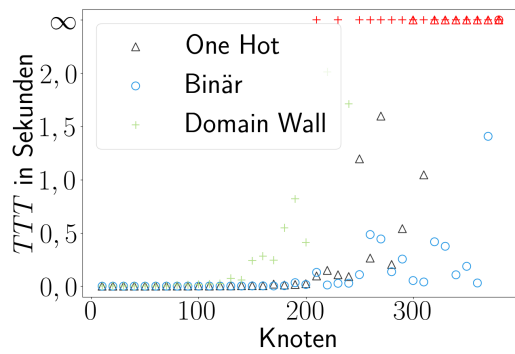
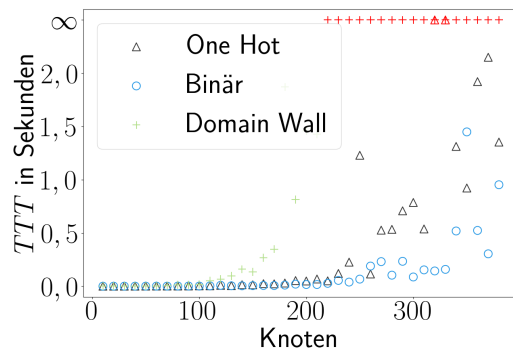


Abbildung 5.5: TTT in μs für Anneal-Zeiten von 0, $5\mu s$, $2\mu s$, $5\mu s$, $10\mu s$, $20\mu s$, $50\mu s$, $100\mu s$, $150\mu s$ und $200\mu s$.

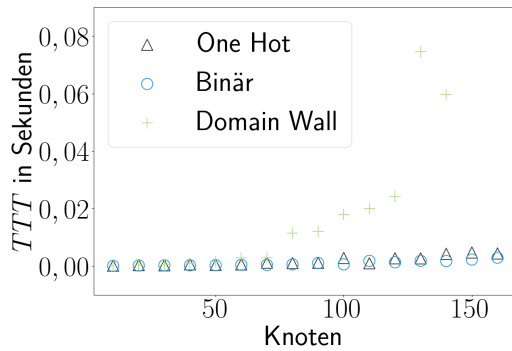
Skalieren der Problemgröße. Auch die Entwicklung der TTT mit steigender Problemgröße liefert interessante Ergebnisse. Zu diesem Zweck wurden jeweils zwei Testläufe durchgeführt, in denen die Graphfärbungsprobleme für 10 bis 380 Knoten in jeweils Zehner-Schritten auf dem Quanten-Annealer gelöst wurden. Dabei wurden für jede Kodierungstechnik jeweils Durchläufe mit Anneal-Zeiten von 0, $5\mu s$, $2\mu s$, $5\mu s$, $10\mu s$, $20\mu s$, $50\mu s$, $100\mu s$, $150\mu s$ und $200\mu s$ durchgeführt. Für die zwei Tests wurden jeweils verschiedene Embeddings berechnet, um die Abhängigkeit von diesen zu demonstrieren. Die Ergebnisse der Tests für die beiden Embeddings sind in Abbildung 5.6 zu sehen. Dabei wurde für eine Problemgröße immer die beste TTT für eine Kodierung abgebildet.



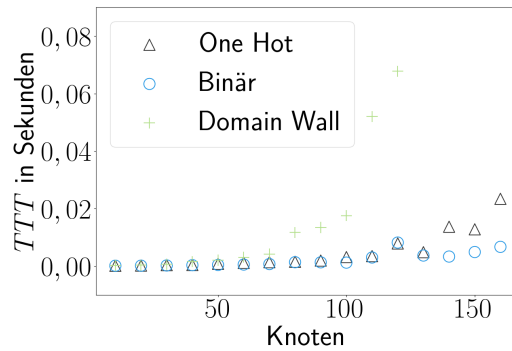
(a)



(b)



(c)



(d)

Abbildung 5.6: TTT s in Sekunden für das Graphfärbungsproblem von 10 bis 380 Knoten. In den Abbildungen (a) und (b) wurden jeweils verschiedene Embeddings verwendet. Rot markierte TTT s bedeuten, dass kein Sample das Optimum erreichen konnte. Die Abbildungen (c) und (d) sind jeweils Vergrößerungen der darüber liegenden Abbildungen für die Knoten 10 bis 160.

Die Testergebnisse decken sich hierbei mit der Vorbetrachtung der verschiedenen Kodierungen. Die Domain Wall-Kodierung schneidet am schlechtesten ab. Nicht nur ist mehr Zeit nötig, um das Problem zu lösen, sondern es werden auch früher keine Lösungen mehr gefunden. Direkt darauf folgt die One Hot-Kodierung, welche besonders für kleine Probleme eine ähnliche Zeit wie die Binärkodierung benötigt. Allerdings werden für größere Probleme keine Lösungen gefunden oder sehr viel Zeit benötigt. Die Binärkodierung findet im Vergleich zu den anderen Kodierungen bis auf eine Ausnahme immer eine Lösung und bleibt bei der Lösungszeit bis 130 Knoten in einem

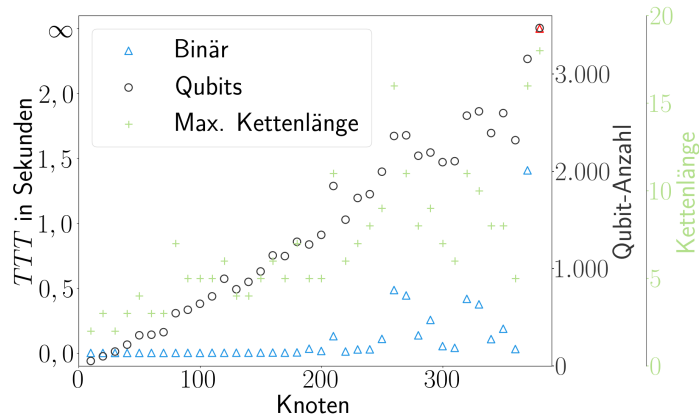


Abbildung 5.7: TTT in Sekunden für die Binärcodierung inklusive der maximalen Kettenlänge und Anzahl benötigter Qubits des ersten berechneten Embeddings.

Bereich von $78\mu s$ bis $1822\mu s$. Die extremen Schwankungen in der TTT für gerade größere Probleme liegen vor allem in der Unzuverlässigkeit der Ergebnisse. Bei den Anneal-Durchläufen wurde das Minimum zum Teil nur zwei oder drei Mal von 2000 Samples erreicht. Daraus folgt, dass die Wahrscheinlichkeit, das Minimum zu erreichen sehr gering ist, was eine sehr hohe Varianz in den Ergebnissen zur Folge hat.

Ein weiterer Erklärungsversuch für die starken TTT -Schwankungen kann im Embedding gefunden werden. Hierbei wird die Untersuchung auf die Binärcodierung beschränkt, da diese noch die verlässlichsten Ergebnisse von allen Kodierungen liefert. Zusätzlich zur TTT können auch die Anzahl an benötigten Qubits und die maximale Kettenlänge geplottet werden. Die dazugehörige Darstellung ist in Abbildung 5.7 zu sehen. Vor allem ab Problemen mit 200 Knoten können Zusammenhänge zwischen Embedding-Qualität und TTT gefunden werden. Für 210, 260 und 370 Knoten ist deutlich zu sehen, dass eine erhöhte Anzahl an Qubits und eine längere maximale Kettenlänge für einen Anstieg in der TTT sorgen. Da in diesen Fällen mehr Qubits und längere Ketten verwendet werden mussten, ist die Anfälligkeit für ungewollte Störfaktoren wie thermische Zustandsprünge, ungewollter magnetischer Einfluss oder Widersprüche in Qubit-Ketten erhöht, was zu schlechteren Ergebnissen führt.

Vergleich klassische Hardware. Auch der Vergleich mit klassischen Computern soll einmal näher beleuchtet werden. Dafür wurde das Graphfärbungsproblem als CSP mithilfe von OR-Tools [PD] modelliert und gelöst. Das Lösen erfolgte auf einem Laptop mit einem Intel Core i5-7200 2,5GHz. Da für die Binärcodierung die besten

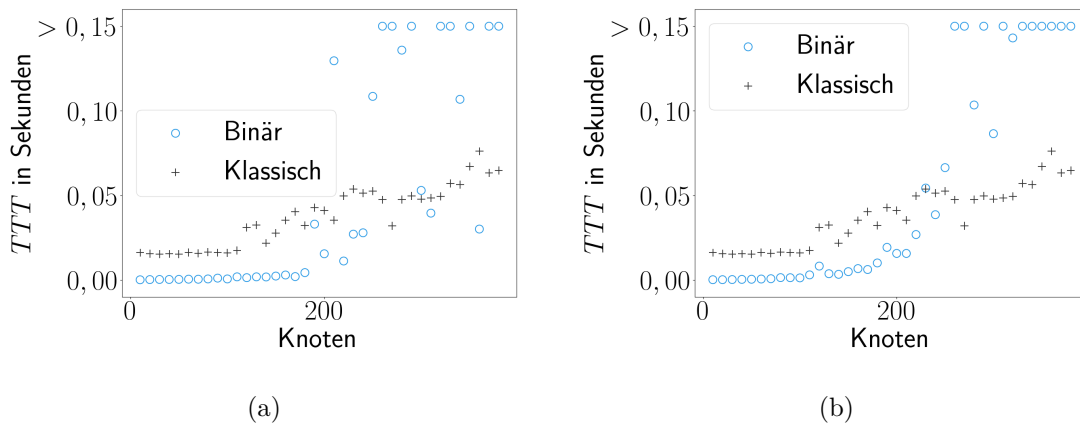


Abbildung 5.8: Vergleich der TTT mit der benötigten Lösungszeit eines klassischen Computers in Sekunden.

Ergebnisse geliefert werden, beschränkt sich die grafische Darstellung nur auf den Vergleich mit dieser Kodierung. Die Abbildung 5.8 zeigt die TTT der Binärkodierung im Vergleich mit der klassischen Lösungszeit. Für beide Embeddings wird deutlich, dass für kleine Probleme der Annealer deutlich im Vorteil ist. Für z.B. 50 Knoten benötigt der klassische Computer 0,015 Sekunden und der Quanten-Annealer nur 0,000322 Sekunden. Mit steigender Problemgröße verkleinert sich dieser Abstand allerdings. Für einen Graphen mit 200 Knoten benötigte der Quanten-Annealer 0,015 Sekunden und der klassische Computer 0,041 Sekunden. Für noch größere Probleme braucht der Quanten-Annealer teilweise sogar länger als der klassische Computer.

5.2.4 Auswertung Anneal Schedule

Zusätzlich zu der Anneal-Zeit kann auch noch das Anneal Schedule modifiziert werden. Damit kann die Geschwindigkeit, mit der das Annealing durchgeführt wird, feiner kontrolliert werden. So kann z.B. in der ersten Hälfte des Annealings das Standard-Schedule schneller durchgeschaltet werden und in der anderen Hälfte langsamer. Eine grafische Repräsentation des Standard-Schedules ist in Abbildung 5.9 zu sehen. Experimentell lassen sich auch hier Verbesserungen der TTT beobachten. Wieder wird das Graphfärbungsproblem mit 150 Knoten untersucht. Die sechs verschiedenen Schedules, die zum Durchführen der Tests verwendet wurden, sind in Abbildung 5.10 dargestellt.

Die Abbildungen sind wie folgt zu interpretieren: Für z.B. Schedule (a) aus Abbildung 5.10 wird im ersten Drittel des Anneal-Durchlaufs das Schedule schneller

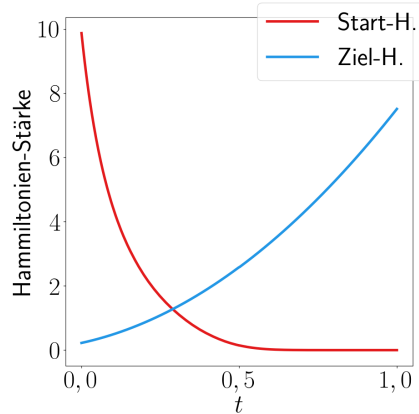
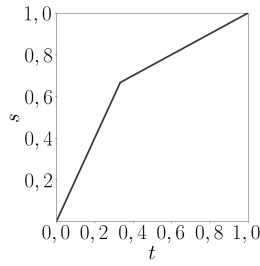


Abbildung 5.9: Standard-Anneal Schedule des Advantage System 7.1. Die x-Achse gibt den Fortschritt des Annealings von 0 bis 1 an und die y-Achse die Stärke des Hamiltonien.

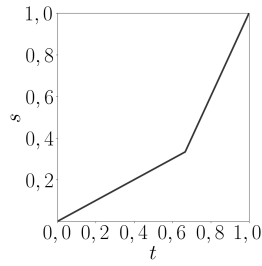
durchgeschaltet. In den letzten zwei Dritteln wird das Schedule dann wieder langsamer durchgeschaltet. Mit dem Durchschalten ist hierbei das Zuschalten des Ziel-Hamiltonians und das Abschalten des Start-Hamiltonians gemeint. Für den Graphen mit 150 Knoten wurden jeweils die sechs Anneal Schedules mit jeweils den Anneal-Zeiten $5\mu s$, $10\mu s$, $20\mu s$, $50\mu s$, $100\mu s$, $150\mu s$ und $200\mu s$ getestet. Die Anneal-Zeiten $0,5\mu s$ und $2\mu s$ konnten für dieses Experiment nicht gewählt werden. Der Anneal ist physisch limitiert, wie schnell die Coupler und Biase verändert werden können. Aus diesem Grund können für kleine Anneal-Zeiten keine steileren Kurven im Anneal Schedule gewählt werden.

Die Testergebnisse sind einmal als Tabelle im Anhang 7.5 und grafisch in Abbildung 5.11 zu finden. Bei diesem Test ist vor allem der Vergleich zum normalen Anneal Schedule interessant. Dabei ist zu beobachten, dass das Schedule S3, welches zu Beginn und am Ende beschleunigt und in der Mitte eine langsamere Anneal-Kurve aufweist, fast immer besser als das normale Schedule abschneidet. Die beste erreichte TTT mithilfe des normalen Schedules betrug $0,0041s$ mit einer Anneal-Zeit von $200\mu s$. Für das Schedule S3 wurde eine TTT von $0,0037s$ mit einer Anneal-Zeit von $100\mu s$ erreicht. Dies entspricht einer Verbesserung von $9,76\%$. Auch die Schedules S5 und S6, welche eine ähnliche Kurve wie S3 aufweisen, schneiden in den meisten Tests besser ab als das normale Schedule. Die anderen Schedules, die in der mittleren Phase des Anneal-Durchlaufs beschleunigen, schneiden gerade bei kurzen Anneal-Zeiten wesentlich schlechter ab als das normale Schedule.

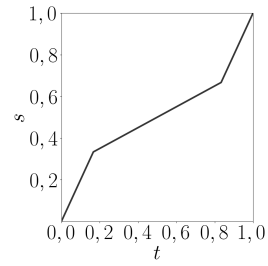
Eine exakte theoretische Erklärung dieser Anneal-Ergebnisse ist nur schwer mög-



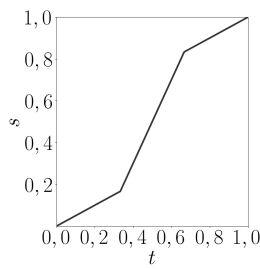
(a) S1



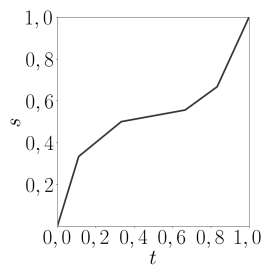
(b) S2



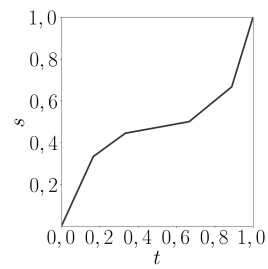
(c) S3



(d) S4



(e) S5



(f) S6

Abbildung 5.10: Anneal Schedules S1 bis S6. Die x-Achse gibt den zeitlichen Fortschritt an, wobei 1 das Ende des Annealing-Durchlaufs symbolisiert. Die y-Achse gibt den Fortschritt des Zuschaltens des Annealing Schedules an, wobei 1 für das Ende des Schedules steht.

lich. Natürlich sind wieder die minimale Energielücke und die zeitliche Änderung des Hamiltonoperators relevant. Allerdings ist das Annealing ein kontinuierlicher Prozess. Das bedeutet, der Hamiltonian, der als Summe des Start- und Ziel-Hamiltonian gebildet wird, verändert sich kontinuierlich. Um genau zu bestimmen, warum ein Schedule besser als das andere funktioniert, könnten die minimalen Energielücken und die zeitlichen Ableitungen des Hamiltonoperators zu verschiedenen Zeitpunkten bestimmt werden.

Dazu müssten unter anderem die Eigenwerte des Hamiltonoperators berechnet werden. Die Größe der Matrix skaliert allerdings exponentiell mit der Anzahl der Variablen. Für das Graphfärbungsproblem mit 150 Knoten und der Binärcodierung werden 251 Variablen benötigt. Die dazugehörige Matrix des Hamiltonoperators besitzt eine Größe von $2^{251} \times 2^{251}$. Zusätzlich hängt der tatsächliche Problemhamiltonien auch vom embeddeten Problem ab, welches noch mehr Qubits als Variablen benötigt. Für solch große Matrizen wird es unrealistisch, die Eigenwerte vor allem zu verschiedenen

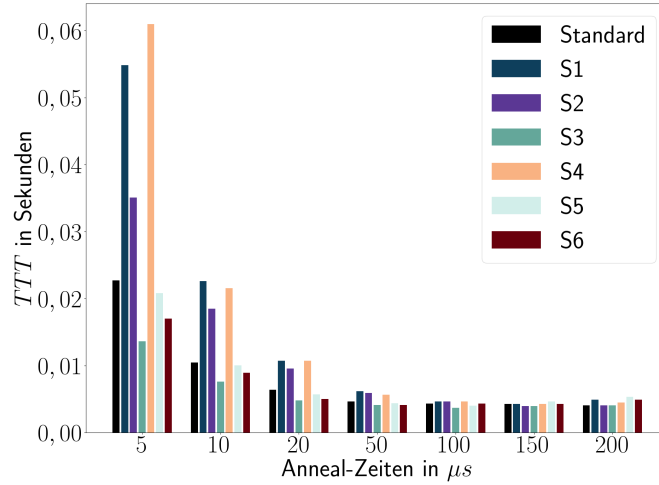


Abbildung 5.11: TTT s in Sekunden für verschiedene Anneal Schedules. Die Werte sind dabei auf der x-Achse in Blöcken aufgetragen. Im ersten Block wurde jedes Schedule mit einer Anneal-Zeit von $5\mu s$ ausgeführt, im zweiten Block mit $10\mu s$ usw. bis $200\mu s$.

Zeitpunkten zu berechnen. Eine Vermutung lässt sich trotzdem abgeben.

Die Eigenwerte des Hamiltonians H , der sich als Summe aus Starthamiltonian H_s und Zielhamiltonian H_t zusammensetzt, hängen von den Eigenwerten der einzelnen Summanden ab [Ful00]. Werden Matrizen mit einem Vorfaktor multipliziert, werden deren Eigenwerte um den gleichen Faktor skaliert. Betrachtet man jetzt die Summe der beiden Faktoren, mit denen H_s und H_t während des Annealings multipliziert werden, entsteht die Kurve in Abbildung 5.12. Hierbei ist deutlich zu erkennen, dass die Summe der beiden Koeffizienten und damit die Eigenwerte der beiden Matrizen in der Mitte des Anneal-Vorgangs am kleinsten sind. Daraus kann vermutet werden, dass auch die Eigenwerte des Hamiltonians H in diesem Bereich klein sind. Somit würde eine kleinere zeitliche Änderung des Hamiltonoperators in diesem Bereich zu besseren Ergebnissen führen.

5.2.5 Fazit

Aus den hier durchgeführten Experimenten kann eindeutig gefolgert werden, dass die Binärcodierung für das Graphfärbungsproblem mit vier Farben die besten Ergebnisse liefert. Sowohl die One Hot- als auch die Domain Wall-Kodierung schneiden schlechter ab. Weiterhin führen erhöhte Anneal-Zeiten zu besseren Ergebnissen. Eine zu starke Erhöhung der Zeit führt jedoch zu rückläufigen Erträgen. Teilweise sogar

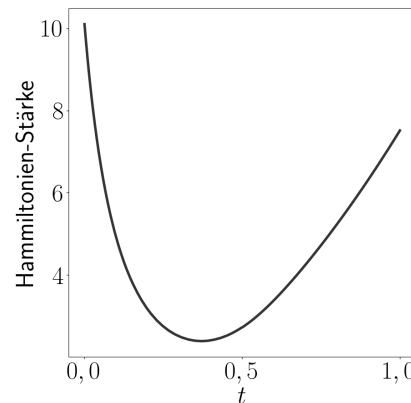


Abbildung 5.12: Summe der Koeffizienten des Ziel- und Start-Hamiltonians im Laufe des Anneal-Vorgangs.

zu schlechteren Ergebnissen. Die optimale Dauer der zu nutzenden Anneal-Zeit ist vom konkreten Problem abhängig. Dabei gilt als Faustregel: je größer das Problem, desto längere Anneal-Zeiten sollten verwendet werden. Wobei für zu lange Zeiten eine Sättigung auftritt.

Die Modifikation des Anneal-Schedules kann zu einer Verbesserung der Anneal-Ergebnisse führen. Das optimale Schedule ist auch wieder problemspezifisch und kann variieren. Gerade da nur eine relativ kleine Verbesserung von weniger als 10% in der TTT erreicht wurde, sollte die Modifikation des Schedules einer der letzten Optimierungsschritte sein. Viel relevanter ist das Finden einer guten Modellierung, die eine große Energielücke liefert und sich gut embedden und skalieren lässt.

5.3 Rotating Rostering

Das Rotating Rostering ist ein sehr praxisorientiertes Constraint-Problem, welches dadurch natürlich von besonders großem Interesse ist. In Abschnitt 3.2.4 wurde bereits ein erster Blick auf die doch relativ schlechten Anneal-Ergebnisse für dieses Problem geworfen. In diesem Abschnitt wird eine größere Testreihe durchgeführt und evaluiert.

5.3.1 Problem

Die Testdurchführung orientiert sich dabei am Abschnitt 5.2. Jede Problemistanz wird mit Anneal-Zeiten von $5\mu s$ bis $200\mu s$ und den Anneal-Schedules S1 bis S6 (vgl. Abbildung 5.10) getestet. Zum einen wurde die modifizierte Version des Rotating

	One Hot	Domain Wall
Anzahl Variablen	448	504
Variablengrad	(4, 20, 21)	(3, 9, 15, 18, 20)
Anzahl Variablengrad	(224, 112, 112)	(224, 112, 56, 56, 56, 56)
Anzahl gekoppelter Var.	2744	2324
Bias Domäne	$[-0, 5 \dots 6, 25]$	$[-8 \dots 11]$
Coupler Domäne	$[-0, 5 \dots 1, 5]$	$[-7 \dots 3]$
Kettenstärke	2,963	9,422
Skalierungsfaktor	0,667	0,212
Minimale Energielücke	1	2
Skalierte Energielücke	0,667	0,424
Maximale Kettenlänge	35	30
Anzahl Qubits	4125	3666

Tabelle 5.4: Modellierungs- und Embedding-Statistiken des Rotating Rostering-Problems mit der One Hot- und Domain Wall-Kodierung

Rosterings wie in Abschnitt 3.2.4 getestet. Zusätzlich wurde noch eine kleinere Instanz mit nur vier Angestellten und den gleichen Modifikationen, also einem Maximum von drei identischen Schichten in Folge und ohne Constraint für genug freie Tage, evaluiert. Weiterhin wurde für beide Instanzen nicht nur die Modellierung mithilfe der One Hot-Kodierung, sondern auch die Domain Wall-Kodierung umgesetzt. Eine genaue Beschreibung der Domain Wall-Kodierung für das Rotating Rostering-Problem ist im Anhang 7.6 zu finden.

5.3.2 Auswertung Kodierung

Zunächst werden die beiden Kodierungen des Rotating Rostering-Problems für acht Angestellte miteinander verglichen. Die in diesem Abschnitt erwähnten und verglichenen Zahlen sind auch nochmal in Tabelle 5.4 dargestellt. Zunächst wird die rein theoretische Modellierung betrachtet und anschließend das praktische Embedding des Problems auf dem Quanten-Annealer.

Modellierung. Die One Hot-Modellierung des Rotating Rostering-Problems verwendet zwar insgesamt 448 Variablen und somit 56 weniger Variablen als die Domain Wall-Modellierung, allerdings ist der durchschnittliche Grad der Variablen wesentlich höher. Bei der One Hot-Modellierung besitzen die Hälfte aller Variablen einen Grad von 20 oder 21, was zu einem schlechteren Embedding führen wird. Im Vergleich dazu besitzen nur ein Drittel aller Variablen der Domain Wall-Modellierung einen hohen

Grad von 15, 18 und 20. Zusätzlich verwendet die One Hot-Modellierung auch eine größere Anzahl an Kanten zwischen den Variablen. Insgesamt werden 2744 Kanten im Vergleich zu den 2324 Kanten benötigt.

Bei den Koeffizientendomänen hat die One Hot-Modellierung wieder einen Vorteil. Hier werden nur Koeffizienten im Bereich $-0,5$ bis $6,25$ angenommen. Im Vergleich dazu werden bei der Domain Wall-Modellierung Koeffizientenwerte im Bereich von -8 bis 11 angenommen, was vor allem für das Embedding und die Skalierung relevant wird. Für die minimale Energielücke sehen die Ergebnisse wieder anders aus. Die One Hot-Modellierung weist eine minimale Energielücke von 1 auf, während die Domain Wall-Modellierung eine minimale Energielücke von 2 aufweist. Zusätzlich besitzen die meisten Ising-Ausdrücke der One Hot-Modellierung nur eine Energielücke von 1, was dazu führen kann, dass jedes Constraint mit der gleichen Wahrscheinlichkeit verletzt werden kann. Die Domain Wall-Modellierung besitzt für einige Constraints eine minimale Energielücke von 4 oder 6, was insgesamt zu besseren Anneal-Ergebnissen führen kann.

Embedding. Die reine Modellierung des Problems ist aber nicht der einzige entscheidende Punkt für die Qualität des Anneal-Ergebnisses. Das Problem muss sowohl embedded als auch skaliert werden. Wie schon im Modellierungsabschnitt erwähnt, schneidet hier die One Hot-Modellierung schlechter ab. Obwohl weniger Variablen als bei der Domain Wall-Modellierung verwendet werden, müssen mehr Qubits genutzt werden, um das Problem zu embedden. Auch wird eine längere maximale Qubit-Kette bzw. im Allgemeinen längere Qubit-Ketten benötigt, um das Problem zu embedden. Beim Skalieren liegt die One-Hot-Modellierung aber wieder vorne. Da nur ein Skalierungsfaktor von $0,667$ benötigt wird, ist die skalierte Energielücke der One Hot-Modellierung mit $0,667$ größer als die skalierte Energielücke der Domain Wall-Modellierung mit $0,424$.

Insgesamt ergibt sich also ein gemischtes Bild zwischen den beiden Modellierungen. Die One Hot-Modellierung verliert beim Embedding, aber sie besitzt nach der Skalierung eine größere Energielücke als die Domain Wall-Modellierung. Allgemein sollte aber festgehalten werden, dass sich beide Probleme nicht gut auf dem Quanten-Annealer embedden lassen. Bei der One Hot-Modellierung werden fast zehnmal so viele Qubits wie Variablen benötigt und auch Qubit-Ketten der Länge 35 sind verhängend für das Annealing-Ergebnis. Die Domain Wall-Modellierung schneidet zwar etwas besser ab, allerdings werden trotzdem mehr als siebenmal so viele Qubits wie Variablen benötigt und auch eine Kettenlänge von 30 ist schlecht für das Annealing.

	$5\mu s$	$10\mu s$	$20\mu s$	$50\mu s$	$100\mu s$	$150\mu s$	$200\mu s$
Std.	-642	-645	-650	-655	-660	-667	-669
S1	-641	-640	-651	-656	-656	-661	-666
S2	-640	-653	-652	-666	-662	-667	-678
S3	-634	-650	-646	-650	-660	-662	-663
S4	-639	-639	-641	-656	-660	-662	-666
S5	-626	-641	-647	-654	-660	-660	-664
S6	-645	-647	-645	-651	-659	-661	-662

Tabelle 5.5: Ergebnisse der Testreihe für die One Hot-Modellierung. Das zu erreichende Optimum liegt bei -770.

	$5\mu s$	$10\mu s$	$20\mu s$	$50\mu s$	$100\mu s$	$150\mu s$	$200\mu s$
Std.	-1926	-1944	-1950	-1966	-1982	-1978	-1982
S1	-1914	-1926	-1922	-1956	-1947	-1966	-1974
S2	-1940	-1946	-1962	-1970	-1992	-1992	-1988
S3	-1910	-1928	-1930	-1966	-1988	-1974	-1974
S4	-1908	-1948	-1936	-1960	-1968	-1964	-1972
S5	-1896	-1936	-1940	-1942	-1956	-1966	-1960
S6	-1908	-1936	-1926	-1944	-1968	-1982	-1970

Tabelle 5.6: Ergebnisse der Testreihe für die Domain Wall-Modellierung. Das zu erreichende Optimum liegt bei -2088.

5.3.3 Auswertung Anneal-Zeit und Schedule

Im vorherigen Abschnitt 5.3.2 wurden schon erste theoretische Betrachtungen der beiden Modellierungen unternommen. Dieser Abschnitt zeigt die konkreten Anneal-Ergebnisse der beiden Modellierungen. Da bei keinem der Anneal-Durchläufe das Optimum erreicht wurde, konnte keine TTT berechnet werden. Aus diesem Grund wurden nur die erreichten minimalen Energiewerte betrachtet. Für das Rotating Rostering-Problem mit acht Angestellten sind die Ergebnisse in den Tabellen 5.5 und 5.6 zu sehen. Im Anhang 7.7 sind die Ergebnisse für dieselbe Testreihe mit einer Problemistanz für vier Angestellte zu sehen.

Es lässt sich ein ähnliches Muster wie in Abschnitt 5.2.3 beobachten. Längere Anneal-Zeiten führen meist zu besseren Ergebnissen. Dabei sollte beachtet werden, dass die Ergebnisse relativ unzuverlässig sind, da fast immer nur ein einziges Sample den in den Tabellen 5.5 und 5.6 dargestellten Energiewert erreicht. Dadurch sind die Ergebnisse mit hoher Varianz versehen. Generell lässt sich aber für die One Hot-Modellierung von $5\mu s$ bis $200\mu s$ eine Verbesserung der Anneal-Ergebnisse beobachten. Die einzelnen

Schedules haben nur einen minimalen Einfluss auf das Anneal-Ergebnis. Das beste Ergebnis von -678 wurde mit einer Anneal-Zeit von $200\mu s$ und dem Schedule S2 erreicht. Die Differenz zum Optimum liegt bei 92.

Die Domain Wall-Modellierung weist ein ähnliches Verhalten zu der One Hot-Modellierung auf. Mit steigender Anneal-Zeit werden bessere Ergebnisse erzielt. Für einige Schedules wird das beste Ergebnis aber schon für Anneal-Zeiten von $100\mu s$ bis $150\mu s$ erreicht. Wieder wird das beste Ergebnis von -1992 von dem Schedule S2 erreicht. Der Einfluss verschiedener Schedules ist allerdings wieder relativ gering. Die Differenz zum Minimum beträgt in diesem Test 96, was minimal schlechter ist als bei der One Hot-Modellierung. Allerdings können diese beiden Werte nicht direkt verglichen werden. Die beiden Modellierungen realisieren unterschiedlich große Energielücken innerhalb der Constraint-Ausdrücke. Das bedeutet, dass eine Abweichung von z.B. 4 vom Optimum innerhalb der One Hot-Kodierung dem Verletzten von vier verschiedenen Constraint entsprechen kann. Da die minimale Energielücke der Domain Wall-Kodierung 2 beträgt, kann also eine Abweichung von 4 maximal zwei Constraints verletzen. Natürlich ist es auch möglich, dass ein einzelner Ising-Ausdruck eine Abweichung von 4 oder mehr aufweisen kann. Nur das Minimum ist eindeutig bestimmt, verschiedene nicht erfüllende Belegungen können aber auch verschieden große Abweichungen vom Minimum besitzen.

Um einen besseren Eindruck von den verschiedenen Anneal-Ergebnissen zu bekommen, werden die jeweils besten Schichtpläne der beiden Modellierungen in den Abbildungen 5.7 und 5.8 dargestellt. Durch genaues Betrachten der beiden Schichtpläne wird deutlich, dass bei der One Hot-Modellierung das Kodierungs-Constraint viermal verletzt wurde, während bei der Domain Wall-Modellierung dieses Constraint nur ein einziges mal verletzt wurde. Das liegt vor allem daran, dass die minimale Energielücke bei der One Hot-Modellierung einen Wert von 2 annimmt, während die Energielücke bei der Domain Wall-Modellierung 4 beträgt. Die Domain Wall-Kodierung ist somit die stabilere Kodierung.

Ein Punkt, bei dem die One Hot-Modellierung vorne liegt, ist das Constraint der maximalen Schichtwiederholung. Dieses wurde kein einziges Mal verletzt. Im Vergleich dazu wurde dieses Constraint in der Domain Wall-Modellierung sechsmal verletzt. Ein besonderes Problem sind dabei die siebenmal hintereinander auftretenden Spätschichten. Der Grund dafür kann leider nicht einfach über die Energielücken gefunden werden. Der One Hot-Ausdruck für dieses Constraint besitzt eine Energielücke von 1, während der Ausdruck der Domain Wall-Kodierung eine Lücke von 4 aufweist. Allerdings wurden bei der Domain Wall-Kodierung minimale und maximale Schichtwiederholung kombiniert, was der Grund für das schlechtere Ergebnis sein kann.

Ansonsten verhalten sich die beiden Modellierungen sehr ähnlich. Bei beiden Model-

	Mo	Di	Mi	Do	Fr	Sa	So
A1	Frei	Frei	Spät	E_1	Nacht	Frei	Frei
A2	Leer	Nacht	Nacht	Früh	Früh	Frei	Frei
A3	Nacht	Nacht	E_2	Frei	E_3	Nacht	Nacht
A4	Früh	Früh	Spät	Frei	Frei	Früh	Früh
A5	Nacht	Spät	Früh	Nacht	Nacht	Frei	Frei
A6	Spät	Nacht	Nacht	Früh	Früh	Frei	Frei
A7	Spät	Spät	Frei	Nacht	Nacht	Früh	Früh
A8	Nacht	Frei	Frei	Spät	Spät	Nacht	Nacht

Tabelle 5.7: Schichtplan des Anneal-Ergebnisses der One Hot-Modellierung mit einem Energiewert von -678 . Die Bezeichner E_i werden dann genutzt, wenn mehrere One Hot-Variablen den Wert $+1$ annehmen, also mehrere Schichten gleichzeitig zugewiesen werden. Die zugewiesenen Schichten für diese Fälle lauten wie folgt: $E_1 = (\text{Spät}, \text{Nacht})$, $E_2 = (\text{Früh}, \text{Nacht})$, $E_3 = (\text{Frei}, \text{Spät})$. Die Schicht "Leer" bedeutet, dass keine One Hot-Variable den Wert $+1$ angenommen hat.

lierungen wird an fast jedem Tag das Schichtbedarfs-Constraint verletzt. Da dies das Constraint mit den meisten beteiligten Variablen ist, ist dies auch zu erwarten. Dagegen wurde das Constraint für gleiche Schichten am Wochenende bei keiner der beiden Modellierungen verletzt. Da bei diesem Constraint die wenigsten Variablen involviert sind, war dieses Ergebnis auch zu erwarten.

Im Anhang 7.7 sind auch nochmal zwei Schichtpläne des verkleinerten Rotating Rostering-Problems für beide Modellierungstechniken abgebildet. Auch hier lässt sich ein ähnliches Verhalten wie für die größere Instanz beobachten. Eine nennenswerte Besonderheit ist aber, dass bei der Domain Wall-Kodierung das Constraint des Schichtbedarfs kein einziges Mal verletzt wurde. Gerade für kleinere Probleminstanzen wird die minimale Energielücke wieder relevant und erzielt somit trotz des nicht optimalen Embeddings ein besseres Ergebnis.

5.3.4 Fazit

Aus den Testergebnissen wird deutlich, dass Probleme mit Constraints in denen viele Variablen beteiligt sind, zu schlechten Anneal-Ergebnissen führen. Vor allem wenn für diese Variablen eine starke Verknüpfung wie über ein Summen-Constraint hergestellt werden soll. Auch die Modifikation der Anneal-Zeiten und des Schedules konnte nur minimale Verbesserungen hervorrufen. Probleme in denen sehr viele Variablen stark untereinander Verknüpft sind, lassen sich also schwer oder gar nicht direkt auf dem Annealer lösen. Die beiden Modellierungen zeigen in der Lösungsqualität auch keinen

	Mo	Di	Mi	Do	Fr	Sa	So
A1	Nacht	Frei	Frei	Früh	Früh	Früh	Früh
A2	Frei	Frei	Frei	Früh	Früh	Spät	Spät
A3	Spät	Spät	Spät	Spät	Spät	Frei	Frei
A4	Früh	Früh	Früh	Spät	Spät	Frei	Frei
A5	Frei	Nacht	Nacht	Frei	Frei	Früh	Früh
A6	Spät	Nacht	Nacht	Nacht	Nacht	Frei	Frei
A7	Früh	Früh	Frei	Frei	Spät	Früh	Früh
A8	Spät	Spät	Spät	Error	Frei	Nacht	Nacht

Tabelle 5.8: Schichtplan des Anenal-Ergebnisses der Domain Wall-Modellierung für eine Anneal-Zeit von $100\mu s$ und dem Schedule S2. Die Schicht Error bedeutet, dass das Domain Wall-Constraint verletzt wurde und keine Schicht in den Variablen kodiert ist. Die Belegung in diesem Fall sieht wie folgt aus: $(-1, +1, -1)$.

deutlichen Unterschied.

5.4 COPS

In der bisherigen Arbeit wurden ausschließlich CSPs untersucht. Oftmals soll aber noch eine Zielfunktion optimiert werden, während verschiedene Constraints erfüllt werden müssen. Um einen Überblick über die COP-Modellierung zu geben, wird in diesem Abschnitt das Sonet-Problem [ALL23] modelliert und untersucht.

5.4.1 Sonet-Problem

Beim Sonet-Problem müssen Sensorknoten auf sogenannten Sonet-Ringen installiert werden, damit diese untereinander kommunizieren können. Dabei ist vorgegeben, welche Knoten untereinander kommunizieren müssen, wie viele Knoten maximal auf einem Ring installiert werden dürfen und wie viele Ringe insgesamt existieren. Ein Knoten kann dabei auch auf mehreren Ringen installiert werden. Ziel ist es, so wenig Knoten wie möglich auf Ringen zu installieren und dabei trotzdem die geforderte Kommunikation aller Sensorknoten zu ermöglichen.

Eine mehr mathematische Beschreibung des Problems mithilfe von Graphen kann wie folgt gegeben werden. Gegeben ist ein Graph G mit Knoten K und Kanten E . Gesucht sind neue Graphen G_i , so dass jeder Graph G_i eine maximale Anzahl an Knoten nicht überschreitet und jede Kante aus E auch in den Graphen G_i vorkommt. Dabei soll die Anzahl der Knoten in den Graphen G_i minimiert werden.

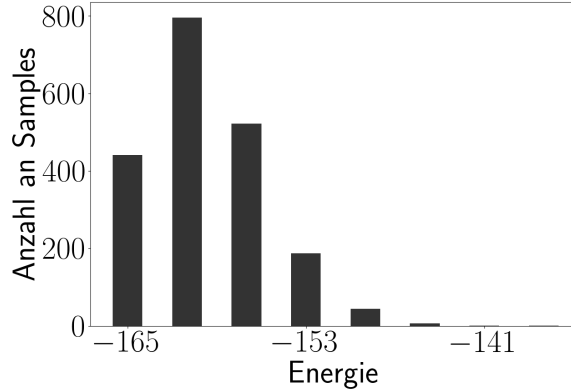


Abbildung 5.13: Anneal-Ergebnisse des Sonet-Constraint-Problems ohne Zielfunktion.

Die hier untersuchte Probleminstance ist wie folgt gegeben: Es sind sechs Sensorknoten gegeben, die über maximal fünf Ringe verteilt werden dürfen. Auf jedem Ring können maximal drei Sensorknoten installiert werden. Die Liste der benötigten Kanten, also welche Sensorknoten untereinander kommunizieren müssen, ist wie folgt gegeben: $[(K_0, K_1), (K_0, K_2), (K_0, K_3), (K_2, K_3), (K_2, K_5), (K_4, K_5)]$. Die Modellierung für diese konkrete Probleminstance ist im Anhang 7.8 zu finden.

5.4.2 Auswertung

Zunächst wird nur das Sonet-CSP ohne Optimierungsfunktion ausgewertet. Falls schon für dieses Constraint-Problem keine Lösungen gefunden werden können, ist die Wahrscheinlichkeit hoch, dass auch keine Lösungen für das Problem mit Optimierungsfunktion gefunden werden. Die Ergebnisse des Sonet-CSP-Tests sind in Abbildung 5.13 zu sehen. Für diesen Test wurde eine Anneal-Zeit von $50\mu s$ und 2000 Samples gewählt. Insgesamt haben 441 Samples das Optimum erreicht und eine Lösung des CSPs gefunden. Somit kann im folgenden Teil zusätzlich die Optimierungsfunktion zum Problem hinzugefügt werden.

Zielfunktion. Zusätzlich zu den zwei Constraints des Sonet-Problems sollen so wenig Knoten wie möglich auf den Ringen installiert werden. Um die Anzahl der Knoten zu minimieren, wird jede Spin-Variable $s_{i,j}$ mit einem positiven Faktor versehen. Wird ein Knoten nicht auf einem Ring installiert, also nimmt die Variable den Wert -1 an, wird dadurch der Ising-Ausdruck minimiert. Dabei muss aber sichergestellt werden, dass es für den minimalen Wert des Ising-Modells besser ist, alle Constraints zu erfüllen und nicht einfach nur die Anzahl aller installierten Knoten zu minimieren. Aus

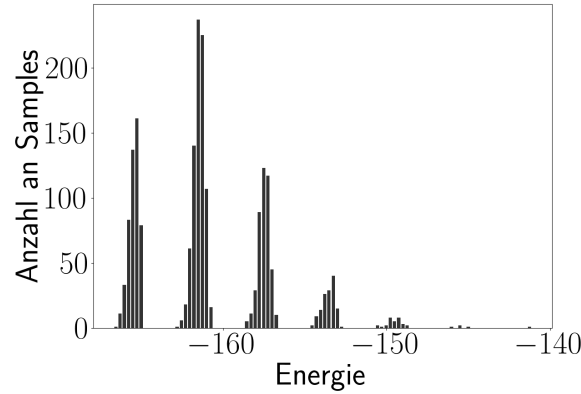


Abbildung 5.14: Anneal-Ergebnisse des Sonet-Problems mit einfacher Optimierungsfunktion.

diesem Grund wird jede Spin-Variable mit dem folgenden Faktor versehen:

$$f \cdot \frac{g}{6 \cdot 5 + 1} = 1 \cdot \frac{4}{6 \cdot 5 + 1} = 0,129 \quad (5.1)$$

Das f ist ein frei wählbarer Skalierungsfaktor, der später relevant wird und hier erstmal auf 1 gesetzt wird. Der Bruch setzt sich im Zähler aus der minimalen Energielücke aller verwendeten Ising-Ausdrücke (in diesem Fall 4) und der Nenner aus der Anzahl aller Knoten plus eins zusammen. Dadurch wird garantiert, dass das Verletzten eines einzigen Constraints stärker gewichtet ist als das Installieren von keinem einzigen Knoten. Das Minimum des Ising-Modells wird also genau nur dann angenommen, wenn alle Constraints erfüllt sind und die minimale Anzahl an Knoten verwendet wird. Für dieses Ising-Modell wurden dieselben Anneal-Parameter gewählt wie für den vorherigen Test ohne Zielfunktion. Das Ergebnis ist in Abbildung 5.14 zu sehen.

Das zu erreichende Minimum für diese Problemistanz liegt bei $-166,806$. Von den 2000 Samples erreichte nur ein einziges Sample den Wert $-166,548$ und liegt somit knapp über dem Minimum. Die gefundene Lösung ist in Abbildung 5.15 dargestellt. Die Abbildung ist dabei wie folgt zu interpretieren: Jeder Graph ist ein einzelner Ring, auf dem Sensorknoten installiert werden. Der erste Graph ist dabei als Referenz für die benötigten Kanten des Problems gegeben. In dieser konkreten Lösung wird ein Knoten zu viel verwendet. Der Grund für dieses schlechte Ergebnis liegt bei den Energielücken. Durch das Addieren des sehr kleinen Wertes $0,129$ zu jeder Spin-Variable entstehen viele Zustände mit sehr kleinen Energielücken untereinander. Die energetisch nah beieinanderliegenden Zustände sind auch nochmal gut in Abbildung 5.14 zu erkennen. Um ein besseres Anneal-Ergebnis zu erhalten, müssen die Energielücken zwischen



Abbildung 5.15: Bestes Sample des einfachen Optimierungsansatzes.

den verschiedenen Zuständen vergrößert werden. Dazu kann der Skalierungsfaktor f verwendet werden.

Bei der Anpassung des Skalierungsfaktors müssen zwei Probleme beachtet werden. Zum einen muss der Faktor so groß gewählt werden, dass die entstehenden Energie-lücken zu guten Ergebnissen führen. Allerdings darf der Faktor auch nicht so groß gewählt werden, dass es besser ist, weniger Knoten auf den Ringen zu installieren, als alle Constraints zu erfüllen. Hier muss also eine gute Balance aus diesen beiden Bedingungen gefunden werden. Der dazu optimale Wert liegt bei ungefähr 7,7. Dieser Wert kann wie folgt bestimmt werden.

Das Verletzen eines Constraints erhöht den Energiewert einer gefundenen Lösung um mindestens 4. Das wichtigste Constraint, welches in Konkurrenz mit der Knotenminimierung steht, ist das Constraint der Kommunikation der Sensorknoten untereinander. Es soll also immer besser sein, eine Kante (und somit zwei Knoten) zu wählen, als dieses Constraint zu verletzen. Daraus folgt, dass der Unterschied zwischen dem Installieren von zwei Knoten kleiner als 4 sein muss. Es ergibt sich somit folgende Ungleichung:

$$f \cdot \frac{4}{31} \cdot 4 < 4 \quad (5.2)$$

$$f < \frac{31}{4} = 7,75 \quad (5.3)$$

Der Faktor 4 auf der linken Seite der Ungleichung 5.2 entsteht durch den Wertebereich der Ising-Variablen. Diese nehmen die Werte -1 und $+1$ an. Wird also eine Variable mit einem Faktor c versehen, liegt der Unterschied zwischen den beiden Belegungen bei $2 \cdot c$. Da hier zwei Variablen betrachtet werden, liegt der Unterschied bei $4 \cdot c$. Somit wird 7,7 als Skalierungsfaktor f gewählt. Das Annealing-Ergebnis für diesen Skalierungsfaktor ist in Abbildung 5.16 zu sehen. Zusätzlich wurde noch das Annealing-Ergebnis für einen Skalierungsfaktor von 6 dargestellt.

In Abbildung 5.16 (a) ist deutlich zu erkennen, dass größere Abstände zwischen den Zuständen der einzelnen Ergebnisse entstehen. Die einzige problematische Energie-lücke existiert zwischen dem angestrebtem Minimum von $-178,91$ und dem nächst höheren Energiezustand mit $-178,88$. Ein Sample, das den minimalen Energiewert

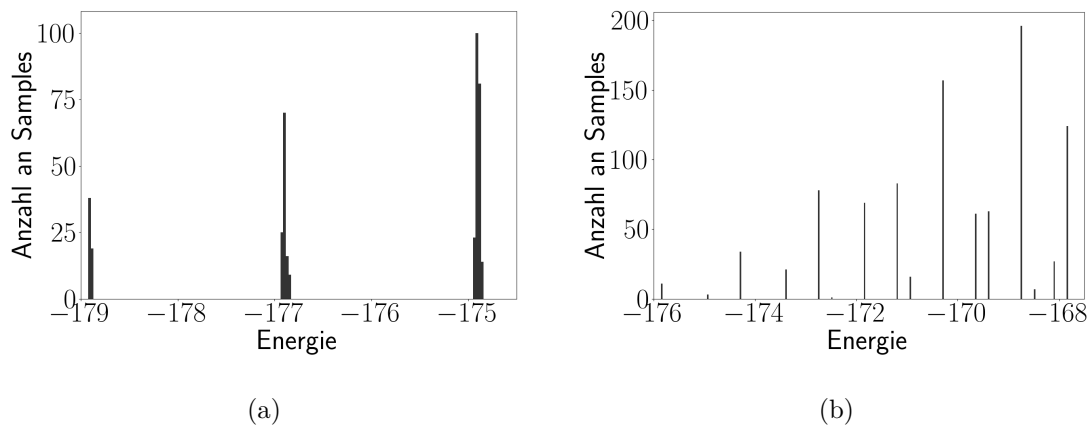


Abbildung 5.16: Anneal-Ergebnisse für einen Skalierungsfaktor von 7,7 (a) und einen Skalierungsfaktor von 6 (b).



Abbildung 5.17: Optimales Sample des Sonet-Problems mit einem Skalierungsfaktor von 7,7.

erreicht hat, ist in Abbildung 5.17 zu sehen. Das dargestellte Ergebnis entspricht auch der Optimallösung. Ein Ergebnis mit der Energie $-178,88$ nutzt zwar zwei Knoten weniger, verletzt aber auch ein Constraint einer benötigten Kante. Ein beispielhaftes Sample mit einer Energie von $-178,88$ ist in Abbildung 5.18 dargestellt.

Wird ein kleinerer Skalierungsfaktor gewählt, wie in Abbildung 5.16 (b), entstehen regelmäßiger Energieklücken zwischen den einzelnen Zuständen. Die kritische Energieklücke zwischen dem minimalen und dem nächst höheren energetischen Zustand weitet sich aus. Allerdings schrumpfen dadurch insgesamt die Abstände zwischen verschiedenen Zuständen und dem Minimum. Dies spiegelt sich in einem schlechteren



Abbildung 5.18: Sample des Sonet-Problems mit einem Skalierungsfaktor von 7,7 und einer erreichten Energie von $-178,88$.

Anneal-Ergebnis wider. Das Minimum von $-175,84$ wurde insgesamt 11 Mal erreicht. Dadurch lässt sich das Problem immer noch lösen, es schneidet aber schlechter ab als für einen Skalierungsfaktor von $7,7$.

5.4.3 Fazit

Auch Optimierungsprobleme können auf dem Quanten-Annealer gelöst werden. Dabei sei zu beachten, dass sich die Zielfunktion für das Sonet-Problem sehr direkt modellieren lässt. Generell können Probleme, in denen über die Anzahl aller Variablen (hier Sensorknoten) optimiert werden muss, leicht in ein Ising-Modell überführen. Für komplexere Zielfunktionen muss dies nicht unbedingt möglich sein. Trotz der einfachen Modellierung ist aber eine korrekte Skalierung der Zielfunktion essentiell für die Qualität der Anneal-Ergebnisse. Der korrekte Skalierungsfaktor hängt dabei von der bisherigen Constraint-Modellierung, also der bisherigen minimalen Energielücke der verwendeten Ising-Ausdrücke, und dem Optimierungsproblem selber ab. Somit muss für jedes Optimierungsproblem eine individuelle Zielfunktion gefunden werden, die optimale Ergebnisse liefert.

Kapitel 6

Zusammenfassung, Fazit und Ausblick

Im folgenden Kapitel werden die Ergebnisse dieser Arbeit zusammengefasst und ein allgemeines Fazit bezüglich dem Lösen von Constraint-Problemen auf Quanten-Annealern abgegeben. Abschließend wird ein Ausblick auf noch offene und zukünftige Fragen und mögliche Arbeiten gegeben.

6.1 Zusammenfassung

Zunächst wurden in Kapitel 2 die grundlegenden Begriffe rund um die Constraint-Programmierung eingeführt und erläutert. Anschließend wurde das Konzept von CSP-Solvern eingeführt und Vertreter klassischer Solver wurden aufgelistet und vorgestellt. Darauf folgend wurden die zum besseren Verständnis des Quanten-Annealers nötigen Grundlagen der Quantenmechanik eingeführt. Dabei wurde der Fokus auf die verwendete Dirac-Notation, Operatoren und die dazugehörigen Eigenwerte und Eigenzustände gelegt. Anschließend wurde das Quantum Adiabatic Theorem, welches die physikalische Grundlage des Quanten-Annealers bildet, eingeführt und aus der Schrödinger-Gleichung hergeleitet. Abschließend wurden noch verwandte Arbeiten betrachtet.

Die praktische Umsetzung des Quantum Adiabatic Theorems wurde in Kapitel 3 thematisiert. Dabei wurden der Aufbau und die Funktionsweise des Quanten-Annealers erklärt. Die einzelnen Bauteile wie Qubits und Coupler wurden eingeführt und schematisch erklärt. Anschließend wurde der Zusammenhang zwischen dem physischen Aufbau des Annealers und dem Ising-Modell hergestellt. Dabei wurde auch die Äqui-

valenz zum Qubo-Modell gezeigt. Der theoretische Ablauf des Annealings wurde erklärt und anhand des XOR-Beispiels präsentiert. Abschließend wurde der komplette Prozess zum Lösen eines CSPs auf dem Quanten-Annealer anhand des Graphfärbungs- und Rotating Rostering-Problems präsentiert. Dazu gehören auch die Auswertung und die Interpretation der Anneal-Ergebnisse und der dabei anfallenden benötigten Zeiten. Für die Auswertung und den Vergleich mit klassischen Verfahren wurde die *TTT*-Metrik eingeführt.

Eine ausführliche Betrachtung der Problemmodellierung für Quanten-Annealer erfolgte in Kapitel 4 der Arbeit. Dabei wurden zunächst die Variablenkodierungen One Hot, Binär und Domain Wall eingeführt und an einem kleinen Beispiel verglichen. Anschließend wurde mit der Konflikt-Graph-Technik eine direkte Methode zum Umwandeln von Constraint-Problemen in Qubo-Modelle präsentiert. Eine alternative Technik, die direkte Umwandlung von einzelnen Constraints in Qubo- oder Ising-Ausdrücke, wurde in den nachfolgenden Abschnitten präsentiert. Sowohl die Nutzung von allgemeinen Umformungen bestimmter Constraint-Typen als auch das Erzeugen von Qubo-Ausdrücken aus Wahrheitswertetabellen wurde eingeführt. Dabei wurde auch die Funktionsweise des in dieser Arbeit entwickelten Werkzeugs zum Finden von Qubo- und Ising-Ausdrücken mithilfe von Constraint-Programmierung eingeführt und erklärt. Abschließend wurden noch verschiedene Möglichkeiten zum Optimieren von Qubo- und Ising-Ausdrücken entwickelt, was die Energielücke, Koeffizientendomäne, Anzahl an Couplern und Variablen beinhaltet.

Eine ausführliche Auswertung und ein Vergleich des in dieser Arbeit entwickelten Tools sowie verschiedener CSPs und COPs erfolgte in Kapitel 5. Dabei wurde der Qubo/Ising-Finder mit der von D-Wave implementierten Methode und der darauf basierenden eingeführten Heuristik verglichen. Anschließend wurde das Graphfärbungsproblem für Graphen von 10 bis 380 Knoten mit verschiedenen Variablenkodierungen modelliert, ausgewertet und mit einem klassischen Computer verglichen. Für das Rotating Rostering-Problem wurden sowohl eine Modellierung mithilfe der One Hot- als auch der Domain Wall-Kodierung getestet und evaluiert. Bei beiden Constraint-Problemen wurden jeweils verschiedene Anneal-Parameter-Konfigurationen verwendet und analysiert. Dies beinhaltet verschiedene Anneal-Zeiten und Anneal-Schedules. Zuletzt wurde noch ein Constraint Optimization-Problem, das Sonet-Problem, modelliert und evaluiert. Dabei lag der Fokus vor allem auf dem Finden einer Zielfunktion, die gute Anneal-Ergebnisse garantiert.

6.2 Fazit

Das Ziel, verschiedene Constraint Satisfaction- und Optimization-Probleme so zu modellieren, dass sie auf dem Quanten-Annealer lösbar sind, kann als erfüllt angesehen werden. Dabei wurde deutlich, dass vor allem Probleme mit Constraints, in denen nur wenig Variablen beteiligt sind, besonders gut auf dem Annealer lösbar sind. Weiterhin ist es auch vorteilhaft, wenn nur lokale Constraints zwischen Variablen existieren. Das Graphfärbungsproblem für planare Graphen ist dabei ein Beispiel für so ein konkretes Problem. Hier konnte vor allem für kleine Probleminstanzen sogar ein Vorteil gegenüber klassischer Hardware festgestellt werden.

Allerdings wird für größere Probleminstanzen die Limitation des Quanten-Annealers deutlich. Zum einen sind die Quantencomputer stark durch ihre verfügbare Anzahl an Qubits und deren Verknüpfungen untereinander limitiert. So lassen sich z.B. Probleme wie das Rotating Rostering, in denen viele Variablen untereinander stark verknüpft sind und Constraints sehr viele Variablen beinhalten können, nur schlecht auf dem Annealer ausführen. Zusätzlich zu den reinen Limitationen der Qubits kommen auch noch die nicht idealen Umgebungsbedingungen hinzu. Unter realen Bedingungen kann weder sichergestellt werden, dass der Anneal-Prozess adiabatisch erfolgt, noch dass keine anderen äußeren Störfaktoren wie Wärme, elektrische oder magnetische Felder Einfluss auf die Hardware nehmen. Dadurch verschlechtert sich das Anneal-Ergebnis für große Probleme.

All diesen negativen Faktoren kann aber durch eine genügend gute Modellierung wenigstens bis zu einem bestimmten Grad entgegengewirkt werden. Dabei wurde auch deutlich, dass die Werkzeuge und Techniken zum Modellieren von Constraint-Problemen noch bei weitem nicht ihre Grenze erreicht haben. Durch das Einfügen von zwei neuen Optimierungsparametern wie der Minimierung der Koeffizientendomäne sowie der Minimierung der benötigten Kanten eines Ising-Ausdrucks können bessere Anneal-Ergebnisse und somit größere Probleminstanzen gelöst werden. Ein wesentliches Problem des in dieser Arbeit entwickelten Werkzeugs ist die Laufzeit. Diese kann aber durch Optimierung des COPs und Optimierung der Solver-Parameter vermutlich minimiert werden. Auch lassen sich die beiden zusätzlichen Optimierungsfaktoren in den LP-Ansatz integrieren, was auch zu einer schnelleren Lösungszeit führen kann.

Allgemein existieren unzählige Möglichkeiten ein einzelnes Constraint-Problem in ein Ising-Modell zu transformieren. Gerade bei dem Rotating-Rostering-Problem konnte durch Optimierung des Ising-Modells eine extreme Verkleinerung der benötigten Variablen und Coupler herbeigeführt werden. Trotzdem war der Anneal nicht in der Lage, eine optimale Lösung zu finden. Dies zeigt auch aktuell die größte Schwäche des Quanten-Annealers auf. Durch die aktuellen Hardwarelimitationen ist es immer

möglich, so große Probleminstanzen zu generieren, dass diese nicht mehr auf dem Quanten-Annealer lösbar sind. Weil sie entweder zu groß sind um ein Embedding zu finden oder aber zu fehleranfällig, so dass kein Minimum gefunden wird. Die dabei entstehenden Probleminstanzen stellen aber für klassische Computer kein Problem dar.

Zusammengefasst kann also festgehalten werden, dass das direkte Lösen von CSPs und COPs, jedenfalls für die hier untersuchten Probleme, noch keinen praktischen Nutzen liefert. Die Ergebnisse deuten auch stark darauf hin, dass die meisten relevanten praktischen Constraint-Probleme auf der aktuellen Quanten-Hardware nur schwer oder gar nicht direkt lösbar sind. Sowohl Hardware als auch Modellierungstechniken werden jedoch kontinuierlich verbessert, und daher kann sich dieses Fazit in Zukunft rasch ändern..

6.3 Ausblick

Der Ausblick unterteilt sich in zwei thematische Abschnitte. Im ersten Abschnitt wird auf weitere Techniken und Möglichkeiten der Modellierung eingegangen. Der zweite Abschnitt wirft weitere Optimierungsmöglichkeiten bezüglich des Annealings auf.

Modellierung. Die Transformation von Constraint-Problemen in Ising-Modelle bietet viel Spielraum für Verbesserungen und Neuerungen. In dieser Arbeit wurden nur drei verschiedene Variablenkodierungen untersucht. Es ist aber durchaus denkbar, dass noch andere neue Kodierungen existieren, die bessere Anneal-Ergebnisse liefern können und sich besser embedden lassen. Denkbar wäre es auch, Variablenkodierungen mehr zu kombinieren und die jeweiligen Vorteile innerhalb der einzelnen Problemvariablen auszunutzen.

Auch die Umwandlung von Constraints in Ising-Ausdrücke lässt noch viele Fragen offen. Zum einen kann der in dieser Arbeit entworfene Qubo/Ising-Finder weiterentwickelt werden. Dabei sollte vor allem die benötigte Laufzeit minimiert werden, was zum Beispiel durch das Optimieren des zugrunde liegenden COPs erfolgen kann. Außerdem können die verwendeten Solver-Einstellungen angepasst und Werte- und Variablenheuristiken eingebaut werden. Dazu könnte die in dieser Arbeit vorgestellte LP-Heuristik direkt in das COP-Programm übernommen werden. Zusätzlich könnten weitere Modifikationen an der Zielfunktion vorgenommen werden, um Ising-Ausdrücke zu erzeugen, die noch bessere Anneal-Ergebnisse liefern.

Denkbar wäre auch eine Modifikation, die es erlaubt, verschiedene Ising-Ausdrücke aufeinander abzustimmen. Aktuell werden einzelne Constraints nur für sich betrachtet

übersetzt. Die einzelnen Ising-Ausdrücke werden aber im finalen Modell alle aufsummiert. Hier könnte eine Verknüpfung der verschiedenen Constraints zu einem in Summe besseren Ergebnis führen. Aber nicht nur der COP-Ansatz muss weiterverfolgt werden. Auch der LP-Ansatz kann so modifiziert werden, dass die zusätzlichen Optimierungsansätze wie Koeffizientendomäne und Kantenanzahl mitbetrachtet werden. Es könnte sich auch herausstellen, dass der LP-Ansatz mit diesen Modifikationen und der zusätzlichen LP-Heuristik die überlegenere Technik zum Umwandeln von Constraints in Ising-Ausdrücke bietet.

In dieser Arbeit wurde immer versucht, die CSPs und COPs als ein großes Ising-Modell darzustellen und vollständig in einem Durchlauf auf dem Quanten-Annealer zu lösen. Denkbar wäre aber auch ein hybrider Ansatz, wie er auch schon von D-Wave untersucht wird. Hier könnten die CSPs oder COPs von einem klassischen Computer in kleinere Teilprobleme aufgeteilt werden und diese dann auf dem Quanten-Annealer gelöst werden. Die Ergebnisse müssten dann wieder von einem klassischen Computer zusammengeführt werden. Wie praktikabel und zielführend dieser Modellierungsansatz ist, müsste weiter untersucht werden.

Annealing. Auch das Annealing selbst bietet noch viel Raum für Verbesserungen. In dieser Arbeit wurde vor allem der Fokus auf die Anneal-Zeit und das Anneal-Schedule gesetzt. Hierbei wurde zwar festgestellt, dass verschiedene Anneal-Schedules bessere Ergebnisse liefern können, aber kein konkretes Verfahren zum Finden eines solchen besseren Schedules entwickelt. Außerdem existieren noch eine Vielzahl von anderen Anneal-Parametern, für die es sich lohnen kann, Optimierungen vorzunehmen.

Hierbei wäre es denkbar, dass auch das Anpassen der Kettenstärke und das Setzen eines Anneal-Versatzes für einzelne Qubits bessere Ergebnisse produzieren könnte. Auch alternative Anneal-Techniken wie das Reverse Annealing oder das Fast Annealing-Protokoll welches in dieser Arbeit nicht verwendet wurde, könnten im Kontext des Lösen von Constraint-Problemen untersucht und deren Nutzen evaluiert werden.

Zuletzt muss immer wieder erwähnt werden, dass das Gebiet der Quantencomputer noch in den Kinderschuhen steckt und noch viele grundlegende Probleme, gerade im Bereich der Hardware, nicht gelöst sind. Aber auch im Bereich der Software stehen wir gerade erst am Anfang einer langen Entwicklung mit noch vielen offenen Forschungsfragen.

Kapitel 7

Anhang

7.1 One Hot-Kodierung

Umwandlung des Constraints (exakt eine Variable muss den Wert 1 annehmen) für die One Hot-Kodierung einer Variable mit einer Domänenengröße von vier in einen Qubo-Ausdruck:

$$\sum_{j=0}^3 x_{i,j} = 1 \quad (7.1)$$

$$x_{i,0} + x_{i,1} + x_{i,2} + x_{i,3} = 1 \quad (7.2)$$

$$x_{i,0} + x_{i,1} + x_{i,2} + x_{i,3} - 1 = 0 \quad (7.3)$$

$$(x_{i,0} + x_{i,1} + x_{i,2} + x_{i,3} - 1)^2 = 0^2 \quad (7.4)$$

$$x_{i,0}^2 + x_{i,1}^2 + x_{i,2}^2 + x_{i,3}^2 + 1 \quad (7.5)$$

$$+2x_{i,0}x_{i,1} + 2x_{i,0}x_{i,2} + 2x_{i,0}x_{i,3} \quad (7.6)$$

$$+2x_{i,1}x_{i,2} + 2x_{i,1}x_{i,3} + 2x_{i,2}x_{i,3} \quad (7.7)$$

$$-2x_{i,0} - 2x_{i,1} - 2x_{i,2} - 2x_{i,3} = 0 \text{ mit } x_{i,j} = x_{i,j}^2 \text{ für } x_{i,j} \in \{0, 1\} \quad (7.8)$$

$$2x_{i,0}x_{i,1} + 2x_{i,0}x_{i,2} + 2x_{i,0}x_{i,3} \quad (7.9)$$

$$+2x_{i,1}x_{i,2} + 2x_{i,1}x_{i,3} + 2x_{i,2}x_{i,3} \quad (7.10)$$

$$-1x_{i,0} - 1x_{i,1} - 1x_{i,2} - 1x_{i,3} + 1 = 0 \quad (7.11)$$

$$\sum_{j < k}^3 2x_{i,j}x_{i,k} - \sum_{j=0}^3 x_{i,j} = -1 \quad (7.12)$$

7.2 Qubo-Ausdrücke für den Vergleich der Transformationstechniken

Die Variablen x_0 bis x_3 kodieren jeweils die Binärzahl. Die Variable x_4 steht für Fizz und die Variable x_5 für Buzz. Die Variablen x_6 und x_7 sind die jeweils benötigten Hilfsvariablen, um das Problem als Qubo-Ausdruck darzustellen. Der Qubo-Ausdruck der LP-Heuristik sieht wie folgt aus:

$$\begin{aligned} & -0,267x_0 + 0,244x_1 - 0,267x_2 + 0,689x_3 + 0,911x_4 + 1,000x_5 - 0,600x_6 \\ & - 0,578x_7 - 0,089x_0x_1 + 0,111x_0x_2 - 0,200x_0x_3 - 0,267x_0x_4 - 0,089x_0x_5 \\ & + 0,178x_0x_6 + 0,533x_0x_7 - 0,089x_1x_2 + 0,133x_1x_3 + 0,178x_1x_4 + 0,044x_1x_5 \\ & - 0,089x_1x_6 - 0,356x_1x_7 - 0,200x_2x_3 - 0,267x_2x_4 - 0,089x_2x_5 + 0,178x_2x_6 \\ & + 0,533x_2x_7 + 0,400x_3x_4 + 0,156x_3x_5 - 0,311x_3x_6 - 0,800x_3x_7 + 0,178x_4x_5 \\ & - 0,378x_4x_6 - 1,000x_4x_7 - 0,844x_5x_6 - 0,378x_5x_7 + 1,000x_6x_7 \end{aligned}$$

Der Qubo-Ausdruck, der mithilfe des Penalty Models von D-Wave ermittelt wurde, lautet wie folgt:

$$\begin{aligned} & -0.105263x_0 - 0.105263x_1 - 0.105263x_2 - 0.105263x_3 + 0.842105x_4 + 2.315789x_5 \\ & + 1.684211x_6 + 2.315789x_7 + 0.421053x_0x_1 + 0.842105x_0x_2 + 0.421053x_0x_3 \\ & - 0.736842x_0x_4 - 1.789474x_0x_5 + 3.052632x_0x_6 - 1.473684x_0x_7 + 0.421053x_1x_2 \\ & + 0.210526x_1x_3 - 0.421053x_1x_4 - 0.631579x_1x_5 + 1.263158x_1x_6 - 0.842105x_1x_7 \\ & + 0.421053x_2x_3 - 0.736842x_2x_4 - 1.263158x_2x_5 + 2.526316x_2x_6 - 1.473684x_2x_7 \\ & - 0.421053x_3x_4 - 0.631579x_3x_5 + 1.263158x_3x_6 - 0.842105x_3x_7 + 1.052632x_4x_5 \\ & - 2.000000x_4x_6 + 1.473684x_4x_7 - 4.000000x_5x_6 + 2.000000x_5x_7 - 4.000000x_6x_7 \end{aligned}$$

Der erste Qubo-Ausdruck, der durch das COP-Programm gefunden wurde, sieht wie folgt aus:

$$\begin{aligned} & -5x_0 - 11x_1 - 5x_2 - 13x_3 + 20x_5 + 6x_6 - 20x_7 + 2x_0x_1 + 3x_0x_2 + 2x_0x_3 + 9x_0x_4 \\ & - 4x_0x_5 - 8x_0x_6 + 12x_0x_7 + 2x_1x_2 + 4x_1x_3 - 4x_1x_5 + 6x_1x_6 + 10x_1x_7 + 2x_2x_3 \\ & + 12x_2x_4 - 4x_2x_5 - 11x_2x_6 + 15x_2x_7 - 5x_3x_5 + 8x_3x_6 + 12x_3x_7 - 7x_4x_5 - 20x_4x_6 \\ & + 20x_4x_7 - 14x_5x_7 - 6x_6x_7 \end{aligned}$$

Der Qubo-Ausdruck, der als Optimum durch das COP-Programm bestimmt wurde, ist wie folgt gegeben:

$$\begin{aligned} & -1x_0 + 3x_1 - 1x_2 + 3x_3 + 7x_4 + 3x_5 + 8x_7 + 2x_0x_2 - 2x_0x_5 + 4x_0x_6 - 4x_0x_7 \\ & + 2x_1x_3 + 4x_1x_4 - 2x_1x_5 - 4x_1x_6 - 4x_1x_7 - 2x_2x_5 + 4x_2x_6 - 4x_2x_7 + 4x_3x_4 \\ & - 2x_3x_5 - 4x_3x_6 - 4x_3x_7 - 4x_4x_5 - 7x_4x_6 - 7x_4x_7 + 7x_5x_7 \end{aligned}$$

7.3 Kodierungen des Graphfärbungsproblems

One Hot-Kodierung. Hierbei wird analog zu Abschnitt 3.2.1 vorgegangen. Für jeden Knoten i werden vier neue Ising-Variablen $x_{i,1}$, $x_{i,2}$, $x_{i,3}$ und $x_{i,4}$ eingeführt. Eine Variable $x_{i,j}$ nimmt genau dann den Wert 1 an, wenn der Knoten i mit der Farbe j gefärbt wird. Der Ising-Ausdruck um zur Sicherstellung, dass immer nur exakt eine der vier Variablen den Wert 1 annimmt, sieht wie folgt aus:

$$\begin{aligned} \forall i \in \{1, \dots, n\} : & 2x_{i,1} + 2x_{i,2} + 2x_{i,3} + 2x_{i,4} \\ & + 1x_{i,1}x_{i,2} + 1x_{i,1}x_{i,3} + 1x_{i,1}x_{i,4} + 1x_{i,2}x_{i,3} + 1x_{i,2}x_{i,4} + 1x_{i,3}x_{i,4} \end{aligned} \quad (7.13)$$

Hierbei ist n die Anzahl der Knoten des Graphen. Das Constraint, dass ausdrückt, dass zwei adjazente Knoten nicht dieselbe Farbe erhalten dürfen, wird über den folgenden Ausdruck realisiert:

$$\forall (i, j) \in E \forall k \in \{1, 2, 3, 4\} : 1x_{i,k} + 1x_{j,k} + 1x_{i,k}x_{j,k} \quad (7.14)$$

Die Menge E ist die Menge aller Kanten des Graphen.

Binärkodierung. Da nur vier verschiedene Farben benötigt werden, können die Knoten mithilfe von zwei Ising-Variablen $x_{i,1}$ und $x_{i,2}$ binär kodiert werden. Da die Domäne exakt einer Zweierpotenz entspricht, muss kein zusätzliches Constraint für die Kodierung eingeführt werden. Allerdings existiert kein Ising-Ausdruck ohne Hilfsvariable, der die Ungleichheit zwischen zwei gefärbten Knoten garantiert. Aus diesem Grund wird zusätzlich für jede Kante (i, j) eine Hilfsvariable $h_{i,j}$ eingeführt. Der Ising-Ausdruck sieht dann wie folgt aus:

$$\begin{aligned} \forall (i, j) \in E : & 1x_{i,1}x_{i,2} - 1x_{i,1}x_{j,1} - 2x_{i,1}h_{i,j} \\ & - 1x_{i,2}x_{j,1} - 2x_{i,2}h_{i,j} + 1x_{j,1}x_{j,2} + 2x_{j,1}h_{i,j} + 2x_{j,2}h_{i,j} \end{aligned} \quad (7.15)$$

Domain Wall-Kodierung. Um vier Farben zu kodieren, werden insgesamt drei Ising-Variablen $x_{i,1}$, $x_{i,2}$ und $x_{i,3}$ pro Knoten benötigt. Diese Variablen werden über eine Kette verbunden, der dazu verwendete Ising-Ausdruck lautet wie folgt:

$$\forall i \in \{1, \dots, n\} : x_{i,1} - x_{i,1}x_{i,2} - x_{i,2}x_{i,3} - x_{i,3} \quad (7.16)$$

Das Constraint für zwei benachbarte Knoten wird über vier einzelne Teilausdrücke realisiert. Da bei der Domain Wall-Kodierung der Wert einer Variable durch die Position der Domain Wall festgelegt ist, existieren vier mögliche Belegungen. Zu den jeweils einzelnen Fällen gehören die Belegungen $(-1, -1, -1)$, $(-1, -1, 1)$, $(-1, 1, 1)$ und $(1, 1, 1)$. Jeder dieser Fälle wird im Folgenden separat betrachtet.

- **Fall $(-1, -1, -1)$:**

Unter der Annahme, dass das Domain Wall-Constraint eingehalten wird, also nur exakt eine Domain Wall innerhalb der Kette existiert, reicht es, sich die letzte Variable in der Kette anzuschauen. Besitzt diese den Wert -1 , ist klar, dass die vollständige Belegung $(-1, -1, -1)$ sein muss. Somit reicht es für zwei benachbarte Knoten (i, j) , sicherzustellen, dass die Variablen $x_{i,3}$ und $x_{j,3}$ nicht gleichzeitig -1 sein dürfen. Der Ising-Ausdruck, der diese Bedingung realisiert, sieht wie folgt aus:

$$\forall (i, j) \in E : -1x_{i,3} - 1x_{j,3} + 1x_{i,3}x_{j,3} \quad (7.17)$$

- **Fall $(+1, +1, +1)$:**

Dieser Fall funktioniert analog zu dem Fall $(-1, -1, -1)$. Hierbei ist es ausreichend, wenn für zwei Knoten i und j jeweils die erste Variable der Kette betrachtet wird. Nimmt die Variable $x_{i,1}$ den Wert 1 an, muss es sich um die Belegung $(1, 1, 1)$ handeln. Somit ist es wieder ausreichend, wenn ein Constraint eingeführt wird, welches dafür sorgt, dass die Variablen $x_{i,1}$ und $x_{j,1}$ nicht gleichzeitig den Wert 1 annehmen können. Der dazu nötige Ising-Ausdruck lautet wie folgt:

$$\forall (i, j) \in E : 1x_{i,1} + 1x_{j,1} + 1x_{i,1}x_{j,1} \quad (7.18)$$

- **Fall $(-1, -1, +1)$:**

Für diese Belegung ist es nicht ausreichend, nur eine einzige Variable zu betrachten. Jede der drei Variablen kommt mit demselben Wert auch in einer der drei anderen Belegungen vor. Hier wird es nötig, jeweils die beiden Variablen an der Domain Wall selbst zu betrachten. In diesem Fall werden die Variablen $x_{i,2}$ und $x_{i,3}$ genutzt. Für jeweils zwei adjazente Knoten muss sichergestellt werden, dass für beide Knoten diese zwei Variablen nicht gleichzeitig die Belegung $(-1, +1)$ besitzen. Der dazu nötige Ising-Ausdruck ist wie folgt gegeben:

$$\begin{aligned} \forall (i, j) \in E : & 2x_{i,2} - 2x_{i,3} + 2x_{j,2} - 2x_{j,3} \\ & - 2x_{i,2}x_{i,3} + 1x_{i,2}x_{j,2} - 1x_{i,2}x_{j,3} - 1x_{i,3}x_{j,2} + 1x_{i,3}x_{j,3} - 2x_{j,2}x_{j,3} \end{aligned} \quad (7.19)$$

- **Fall $(-1, +1, +1)$:**

Dieser Fall kann analog zum vorherigen Fall $(-1, -1, +1)$ behandelt werden. Hier werden die Variablen $x_{i,1}$ und $x_{i,2}$ in das Constraint aufgenommen. Der Ising-Ausdruck ist ansonsten identisch zu dem vorherigen Ising-Ausdruck 7.19:

$$\begin{aligned} \forall (i, j) \in E : & 2x_{i,1} - 2x_{i,2} + 2x_{j,1} - 2x_{j,2} \\ & - 2x_{i,1}x_{i,2} + 1x_{i,1}x_{j,1} - 1x_{i,1}x_{j,2} - 1x_{i,2}x_{j,1} + 1x_{i,2}x_{j,2} - 2x_{j,1}x_{j,2} \end{aligned} \quad (7.20)$$

7.4 Anneal-Ergebnisse des Anneal-Zeit Tests

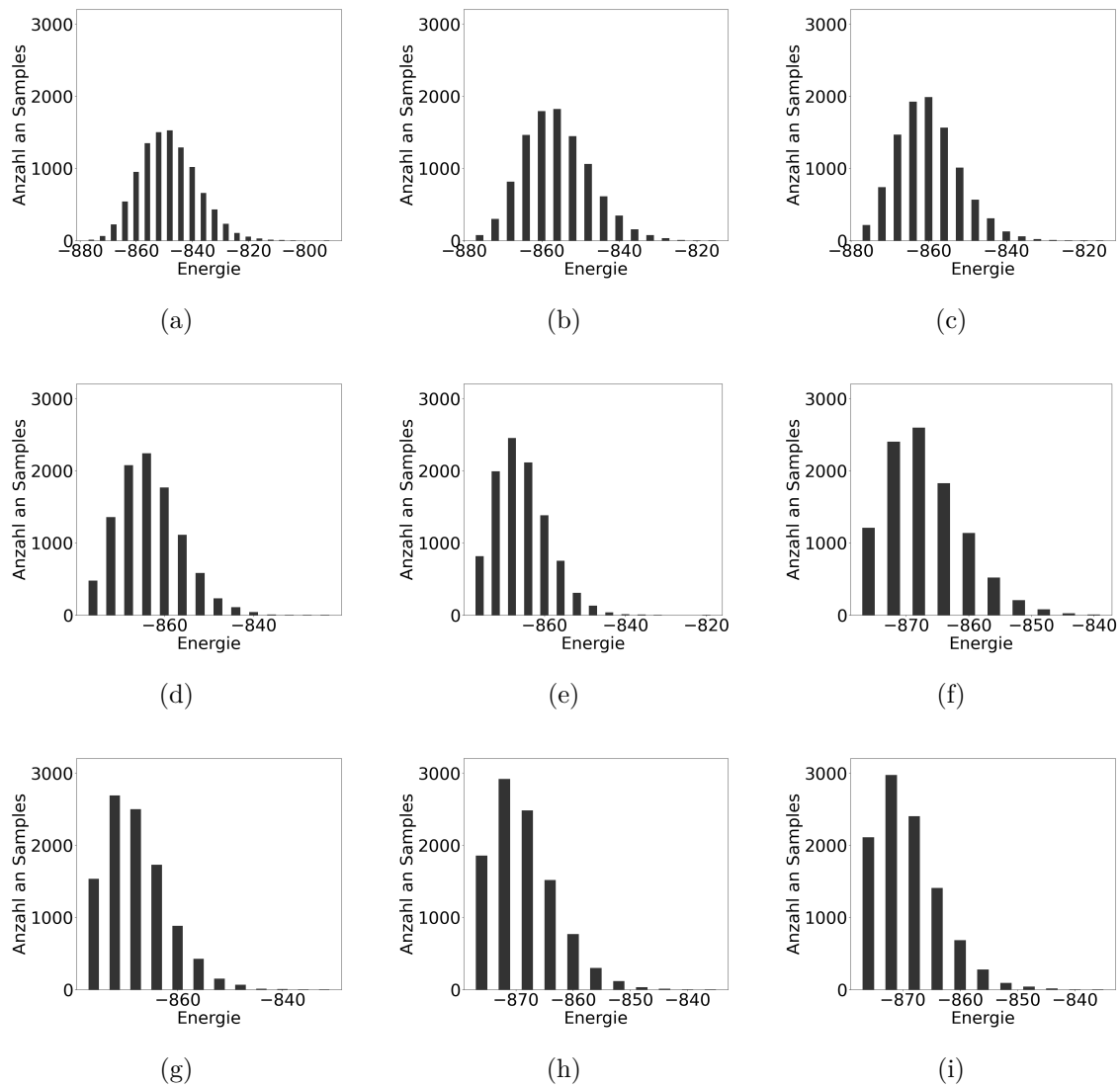


Abbildung 7.1: Anneal Samples für die Anneal-Zeiten 0, 5 μs , 2 μs , 5 μs , 10 μs , 20 μs , 50 μs , 100 μs , 150 μs und 200 μs .

	$0,5\mu s$	$2\mu s$	$5\mu s$	$10\mu s$	$20\mu s$	$50\mu s$	$100\mu s$	$150\mu s$	$200\mu s$
Samples	13	75	214	478	815	1209	1535	1856	2109
<i>TTT</i>	368333	64101	22735	10439	6369	4635	4305	4290	4075

Tabelle 7.1: Anzahl der Samples die das Optimum erreicht haben und die konkrete *TTT* in μs für Anneal-Zeiten von $0,5\mu s$ bis $200\mu s$.

7.5 Anneal-Ergebnisse des Anneal Schedule-Tests

	$5\mu s$	$10\mu s$	$20\mu s$	$50\mu s$	$100\mu s$	$150\mu s$	$200\mu s$
Std.	22, 74	10, 44	6, 37	4, 64	4, 31	4, 29	4, 08
S1	54, 83	22, 62	10, 69	6, 18	4, 61	4, 29	4, 89
S2	35, 06	18, 49	9, 56	5, 92	4, 61	3, 93	4, 08
S3	13, 6	7, 61	4, 78	4, 12	3, 69	3, 93	4, 08
S4	60, 99	21, 53	10, 69	5, 67	4, 61	4, 29	4, 48
S5	20, 83	10, 01	5, 69	4, 379	4	4, 64	5, 3
S6	17, 01	8, 92	5, 01	4, 12	4, 31	4, 29	4, 89

Tabelle 7.2: Anneal-Ergebnisse für das Graphfärbungsproblem mit 150 Knoten. Die Spalten entsprechen den verschiedenen Anneal-Zeiten und die Zeilen den jeweiligen genutzten Schedules. Alle Ergebniszeiten sind in Millisekunden angegeben.

7.6 Domain Wall-Kodierung des Rotating Rostering

Variablenkodierung. Im Falle der Domain Wall-Kodierung wird jede Schicht, mit drei in einer Kette Verknüpften Variablen, kodiert. Die Zuweisung der einzelnen Schichten zu den jeweiligen gültigen Variablenbelegungen ist in Tabelle 7.3 dargestellt. Das Problem wird direkt als Ising-Modell modelliert, wodurch alle Variablen als Ising-Variablen mit der Domäne $\{-1, +1\}$ angegeben werden. Für jede Woche und jeden Tag werden jeweils die Variablen $s_{i,j,0}$, $s_{i,j,1}$ und $s_{i,j,2}$ erstellt. Wobei i für die Woche und j für den Tag steht. Die Variablen innerhalb der Kette sind von links nach rechts nummeriert. Die Belegung $s_{i,j,0} = s_{i,j,1} = -1$, $s_{i,j,2} = +1$ steht somit für eine Frühschicht. Untereinander sind die Variablen mit dem Term 4.9 aus Abschnitt 4.1.3 verknüpft. Konkret werden folgende Terme eingeführt:

$$\forall i \in \{1, \dots, w\} \forall j \in \{1, \dots, 7\} : s_{i,j,0} - s_{i,j,0}s_{i,j,1} - s_{i,j,1}s_{i,j,2} - s_{i,j,2} \quad (7.21)$$

Jetzt müssen alle fünf Constraints des Rotating Rostering-Problems (vgl. **C1**, **C2**, **C3**, **C4**, **C5** in Abschnitt 5.3) für die Domain Wall-Kodierung modelliert werden. Das konkrete Vorgehen wird in dem folgenden Abschnitten beschrieben.

Schichtbedarf. C1 Im Fall der One Hot-Kodierung, ist diese Modellierung relativ intuitiv durch Summen über die einzelnen Schichtvariablen möglich. Der Gleiche

Variablenbelegung	Schicht
$-1, -1, -1$	Frei
$-1, -1, +1$	Früh
$-1, +1, +1$	Spät
$+1, +1, +1$	Nacht

Tabelle 7.3: Zuweisung der einzelnen Schichten zu den jeweiligen Variablenbelegungen der Domain Wall-Kodierung.

Ansatz kann auch hier verwendet werden. Warum dieser Ansatz aber wieder funktioniert muss genauer erklärt werden. In dieser Variablenkodierung korreliert nicht jede Variable eindeutig zu einer bestimmten Schicht. Dies gilt in diesem konkreten Problem für die Schichten Früh und Spät. Nur durch das betrachten einer einzelnen Variable lässt sich nicht feststellen, welche der beiden Schichten kodiert wird. Sein z.B. die Variable $s_{i,j,1} = -1$. Dann kann nicht eindeutig festgestellt werden, ob es sich um eine Frei oder Frühschicht handelt.

Allerdings können die beiden Schichten Frei und Nacht eindeutig zu einer Variable zugeordnet werden. Wir die Variable $s_{i,j,0}$ mit einer $+1$ belegt, muss es sich um eine Nachtschicht handeln. Analog, wenn die Variable $s_{i,j,2}$ mit einer -1 belegt wird, muss es sich um eine Freischicht handeln. Natürlich immer unter der Annahme, dass das Domain Wall-Constraint nicht verletzt ist. Wenn also für einen Wochentag j jede Schicht zweimal besetzt werden muss, dann müssen innerhalb der Variablen $s_{0,j,0}, \dots, s_{8,j,0}$ exakt zwei Einsen auftauchen. Dieses Muster kann jetzt fortgesetzt werden. Wenn zwei Spätschichten besetzt sein müssen, dann müssen innerhalb der Variablen $s_{0,j,1}, \dots, s_{8,j,1}$ exakt vier Einsen auftauchen. Zwei dieser Einsen kommen von den Vorherigen zwei Nachtschichten und die anderen beiden Einsen durch die benötigten zwei Spätschichten. Analog müssen innerhalb der Variablen $s_{0,j,2}, \dots, s_{8,j,2}$ insgesamt sechs Einsen auftreten. Vier dieser Einsen kommen aus den zwei Nacht- und Spätschichten und die anderen beiden aus den zwei benötigten Frühschichten. Da auch zwei Frei Schichten gefordert sind, dürfen auch keine weiteren Variablen mit eins belegt werden. Für den leicht modifizierten Schichtbedarf am Wochenende lässt sich ein analoger Zusammenhang herstellen. Die eben erwähnten Belegungen und anzahl an benötigten einsen sind auch nochmal in Abbildung 7.4 dargestellt.

Für jede der sechs Summen Constraints wird ein äquivalenter Ising-Ausdruck mithilfe des in dieser Arbeit entwickelten COP-Programms gefunden. Für den Schichtbedarf

$s_{i,j,0}$	$s_{i,j,1}$	$s_{i,j,2}$	$s_{i,j,0}$	$s_{i,j,1}$	$s_{i,j,2}$
-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1
-1	-1	+1	-1	-1	-1
-1	-1	+1	-1	-1	-1
-1	+1	+1	-1	-1	+1
-1	+1	+1	-1	-1	+1
+1	+1	+1	-1	+1	+1
+1	+1	+1	+1	+1	+1
2	4	6	1	2	4

Tabelle 7.4: Darstellung der benötigten Schichten unter der Woche und am Wochenende. Die letzte Zeile ist jeweils die Summe der Einsen einer einzelnen Spalte.

unter der Woche ergeben sich folgende drei Ising-Ausdrücke:

$$\forall j \in \{1, \dots, 5\} : \sum_{i=1}^8 4s_{i,j,0} + \sum_{i < k}^8 s_{i,j,0}s_{k,j,0} \quad (7.22)$$

$$\forall j \in \{1, \dots, 5\} : \sum_{i < k}^8 3s_{i,j,1}s_{k,j,1} \quad (7.23)$$

$$\forall j \in \{1, \dots, 5\} : \sum_{i=1}^8 -4s_{i,j,2} + \sum_{i < k}^8 s_{i,j,2}s_{k,j,2} \quad (7.24)$$

Für den Schichtbedarf am Wochenende werden die folgenden drei Ising-Ausdrücke verwendet:

$$\forall j \in \{6, 7\} : \sum_{i=1}^8 6s_{i,j,0} + \sum_{i < k}^8 s_{i,j,0}s_{k,j,0} \quad (7.25)$$

$$\forall j \in \{6, 7\} : \sum_{i=1}^8 4s_{i,j,1} + \sum_{i < k}^8 s_{i,j,1}s_{k,j,1} \quad (7.26)$$

$$\forall j \in \{6, 7\} : \sum_{i < k}^8 3s_{i,j,2}s_{k,j,2} \quad (7.27)$$

Wochenendschichten. C2 Um am Wochenende die gleichen Schichten zu garantieren, können die Domain Wall-Kodierten Variablen über einen Koeffizienten von -1 verknüpft werden. Dadurch wird das Minimum genau dann erreicht, wenn beide Variablen die selbe Schicht kodieren. Der Ising-Ausdruck dazu sieht wie folgt aus:

$$\forall i \in \{1, \dots, 8\} : -s_{i,6,0}s_{i,7,0} - s_{i,6,1}s_{i,7,1} - s_{i,6,2}s_{i,7,2} \quad (7.28)$$

Minimale und Maximale Schichtwiederholung. C3 C5 Die Umwandlung dieser Constraints gestaltet sich wieder komplizierter. Der Grund liegt in der wie vorher schon erwähnten nicht Eindeutigkeit für die Schichten Früh und Spät. Aus diesem Grund werden zur einfacheren Modellierung zwei Hilfsvariablen $s_{i,j,früh}$, $s_{i,j,spät}$ zur Kodierung hinzu gefügt. Die Variablen sollen genau dann den Wert +1 annehmen, wenn innerhalb der Domain Wall-Kodierung eine Früh- bzw. Spätschicht kodiert ist. Dazu muss die Variable $s_{i,j,früh}$ mit den beiden Variablen $s_{i,j,1}$ und $s_{i,j,2}$ verknüpft werden, um eine eindeutige Beziehung herzustellen. Es ist genau dann eine Früh-schicht kodiert, wenn die Variable $s_{i,j,1}$ den Wert -1 und die Variable $s_{i,j,2}$ den Wert +1 annimmt. Analog wird die Variable $s_{i,j,spät}$ mit den Variablen $s_{i,j,0}$ und $s_{i,j,1}$ verknüpft. Es ist genau dann eine Spätschicht kodiert, wenn die Variable $s_{i,j,0}$ den Wert -1 und die Variable $s_{i,j,1}$ den Wert +1 annimmt. Die beiden Ising-Terme sehen dann wie folgt aus:

$$\begin{aligned} \forall i \in \{1, \dots, 8\} \forall j \in \{1, \dots, 7\} : \\ s_{i,j,1} - s_{i,j,2} + s_{i,j,früh} - s_{i,j,1}s_{i,j,2} + s_{i,j,1}s_{i,j,früh} - s_{i,j,2}s_{i,j,früh} \end{aligned} \quad (7.29)$$

$$\begin{aligned} \forall i \in \{1, \dots, 8\} \forall j \in \{1, \dots, 7\} : \\ s_{i,j,0} - s_{i,j,1} + s_{i,j,spät} - s_{i,j,1}s_{i,j,2} + s_{i,j,1}s_{i,j,früh} - s_{i,j,2}s_{i,j,früh} \end{aligned} \quad (7.30)$$

Um zu garantieren, dass immer mindestens zweimal dieselbe Schicht hintereinander belegt wird, müssen drei aufeinander folgende Schichtvariablen verknüpft werden. Analog müssen um zu garantieren, dass nur maximal drei mal dieselbe Schicht zugewiesen wird, vier Schichtvariablen untereinander verknüpft werden. Da für die maximale Schichtwiederholung schon vier Variablen verknüpft werden müssen, wird das minimale Schichtwiederhoungs-Constraint direkt in den selben Ising-Ausdruck mit Integriet. Für die Schichten Früh, Spät und Nacht wird also garantiert, dass mindestens zwei +1 hintereinander folgend und nur maximal drei +1 hintereinander belegt werden. Der Ising-Ausdruck, welcher zusätzlich noch eine Hilfsvariable h benötigt, lautet wie folgt:

$$\begin{aligned} \forall i \in \{1, \dots, 8 \cdot 7\} : \\ -s_{i+3} - h_i + s_i s_{i+2} - s_i h_i - 2s_{i+1} s_{i+2} - 2s_{i+1} h_i - s_{i+2} s_{i+3} + 2s_{i+3} h_i \end{aligned} \quad (7.31)$$

Die Variablen s_i sind dabei jeweils vier aufeinander folgende Schichtvariablen. Für eine Nachtschicht an Tag sechs sind folgende vier Variablen verknüpft:

$$s_{1,6,0}, s_{1,7,0}, s_{2,1,0}, s_{2,2,0} \quad (7.32)$$

Analog werden für Früh- und Spätschichten die jeweils dazu korrespondierenden neu eingeführten Variablen $s_{i,j,früh}$ und $s_{i,j,spät}$ verknüpft. Um das Constraint für Freischichten umzusetzen, muss ein etwas anderer Ising-Ausdruck angewendet werden.

Die Variable $s_{i,j,2}$ kodiert zwar eindeutig, wenn eine Freischicht gewählt wird, allerdings nur wenn die Variable mit einer -1 belegt ist. Es wird also ein Ising-Ausdruck verwendet, der mindestens zwei -1 und maximal drei -1 Werte hintereinander garantiert. Der Ising-Ausdruck sieht dann wie folgt aus:

$$\begin{aligned} \forall i \in \{1, \dots, 8 \cdot 7\} : \\ -s_{i+3} - h_i + s_i s_{i+2} - s_i h_i - 2s_{i+1} s_{i+2} - 2s_{i+1} h_i - s_{i+2} s_{i+3} + 2s_{i+3} h_i \end{aligned} \quad (7.33)$$

Die Variablen sind wieder Analog wie in Ausdruck 7.31 zu interpretieren und es wird auch eine Hilfsvariable h benötigt.

Vorwärtsrotation. C4 Das letzte noch zu transformierende Constraint ist die Vorwärtsrotation, in der immer nur dieselbe, spätere oder eine Freischicht folgen darf. Dabei werden wieder nur Nacht- und Spätschichten betrachtet, da nur für diese Einschränkungen bezüglich der Vorwärtsrotation gelten. Das auf eine Nachtschicht nur eine weitere Nacht- oder Freischicht folgen darf, lässt sich durch einfaches verknüpfen von drei Variablen realisieren. Zur einfacheren Darstellung des Constraints, werden die Variablen wieder als Liste von 1 bis $8 \cdot 7$ kodiert.

$$\forall i \in \{1, \dots, 8 \cdot 7\} : s_{i+1,0} - s_{i+1,2} - s_{i,0} s_{i+1,0} + s_{i,0} s_{i+1,2} - 2s_{i+1,0} s_{i+2,2} \quad (7.34)$$

Dieser Ising-Ausdruck garantiert, dass auf eine Nachtschicht, kodiert durch die Variable $s_{i,0}$ nur eine weitere Nachtschicht ($s_{i+1,0}$) oder eine Freischicht ($s_{i+1,2}$) folgen kann. Weiterhin darf auf eine Spätschicht keine Frühschicht folgen, alle anderen Schichten sind aber erlaubt. Da sowohl Spät- und Frühschicht nicht eindeutig durch eine Variable kodiert werden, müssen in diesem Fall vier Variablen miteinander verknüpft werden.

$$\begin{aligned} \forall i \in \{1, \dots, 8 \cdot 7\} : s_{i+1,0} - s_{i+1,1} \\ -2s_{i,0} s_{i,1} - 2s_{i,0} s_{i+1,0} + 2s_{i,0} s_{i+1,1} + s_{i,1} s_{i+1,0} - s_{i,1} s_{i+1,1} - 2s_{i+1,0} s_{i+1,1} \end{aligned} \quad (7.35)$$

Die Variablen $s_{i,0}$ und $s_{i,1}$ kodieren die Spätschicht des aktuell betrachteten Tages und die Variablen $s_{i+1,1}$, $s_{i+1,2}$ die Frühschicht des folgenden Tages. Ergebnis Domain Wall:

	Montag	Dienstag	Mittwoch	Donnerstag	Freitag	Samstag	Sonntag
A1	Spät	Spät	Nacht	Nacht	Frei	Frei	Frei
A2	Spät	Früh	Nacht	Nacht	Nacht	Frei	Frei
A3	Spät	Nacht	Nacht	Früh	Früh	Frei	Frei
A4	Früh	Früh	Frei	Frei	Früh	Früh	Früh
A5	Früh	Früh	Früh	Früh	Früh	Frei	Frei
A6	Frei	Spät	Spät	Spät	Spät	Früh	Früh
A7	Nacht	Nacht	Frei	Frei	Frei	Früh	Früh
A8	Früh	Früh	Früh	Spät	Spät	Spät	Spät

Ergebnis One Hot:

	Montag	Dienstag	Mittwoch	Donnerstag	Freitag	Samstag	Sonntag
A1	Frei	Frei	Error	Nacht	Nacht	Frei	Frei
A2	Früh	Error	Nacht	Spät	Spät	Error	Frei
A3	Nacht	Nacht	Früh	Frei	Frei	Nacht	Nacht
A4	Spät	Spät	Nacht	Nacht	Frei	Frei	Frei
A5	Nacht	Früh	Spät	Spät	Nacht	Früh	Früh
A6	Spät	Error	Nacht	Früh	Früh	Error	Error
A7	Frei	Frei	Frei	Error	Error	Frei	Frei
A8	Error	Früh	Error	Frei	Früh	Nacht	Nacht

7.7 Anneal-Ergebnisse Rotating Rostering mit vier Angestellten

Für die kleinere Modellierung mit vier Angestellten wurde das Constraint des Schichtbedarfs **C1** wie folgt modifiziert. Unter der Woche muss jede Schicht genau einmal belegt werden und am Wochenende sind zwei Frei-, eine Früh- und eine Spätschicht gefordert. Das am Wochenende keine Nachtschicht benötigt wird ist nicht realistisch allerdings sollte die Idee der Ursprünglichen Modellierung, an den Wochenenden einen anderen Schichtbedarf zu fordern nicht verloren gehen.

	$5\mu s$	$10\mu s$	$20\mu s$	$50\mu s$	$100\mu s$	$150\mu s$	$200\mu s$
Std.	-308	-313	-313	-313	-315	-316	-315
S1	-309	-309	-306	-313	-314	-314	-314
S2	-312	-310	-313	-313	-314	-314	-314
S3	-310	-310	-314	-313	-314	-314	-315
S4	-310	-308	-312	-314	-313	-313	-314
S5	-309	-308	-311	-311	-314	-313	-314
S6	-309	-309	-312	-313	-314	-314	-314

Tabelle 7.5: Ergebnisse der Testreihe für die One Hot-Modellierung mit vier Angestellten. Das zu erreichende Optimum liegt bei -355.

	$5\mu s$	$10\mu s$	$20\mu s$	$50\mu s$	$100\mu s$	$150\mu s$	$200\mu s$
Std.	-968	-972	-968	-976	-980	-984	-982
S1	-962	-968	-970	-974	-980	-978	-982
S2	-968	-976	-976	-976	-978	-984	-986
S3	-972	-970	-976	-974	-976	-984	-986
S4	-968	-968	-968	-970	-982	-980	-982
S5	-958	-966	-980	-970	-974	-976	-976
S6	-960	-972	-968	-972	-972	-980	-982

Tabelle 7.6: Ergebnisse der Testreihe für die Domain Wall-Modellierung mit vier Angestellten. Das zu erreichende Optimum liegt bei -1004.

	Montag	Dienstag	Mittwoch	Donnerstag	Freitag	Samstag	Sonntag
A1	Früh	Früh	Früh	Frei	Frei	Spät	Spät
A2	Frei	Frei	Nacht	Nacht	Nacht	Früh	Früh
A3	Nacht	Nacht	Frei	Spät	Spät	Frei	Frei
A4	Nacht	Spät	Spät	Nacht	Nacht	Frei	Frei

Tabelle 7.7: Bester Schichtplan der One Hot-Modellierung mit einem Energiewert von -316.

	Montag	Dienstag	Mittwoch	Donnerstag	Freitag	Samstag	Sonntag
A1	Spät	Spät	Früh	Nacht	Nacht	Frei	Frei
A2	Frei	Früh	Spät	Spät	Spät	Spät	Spät
A3	Früh	Frei	Frei	Früh	Früh	Frei	Frei
A4	Nacht	Nacht	Nacht	Frei	Frei	Früh	Früh

Tabelle 7.8: Bester Schichtplan der Domain Wall-Kodierung mit einem Energiewert von -986 . Dabei wurde das Ergebnis des Schedules S3 verwendet.

7.8 Sonet-Problem Modellierung

Variablenkodierung Jeder Sensorknoten eines Ringes wird als Spin-Variable modelliert. Dabei nimmt eine Variable $s_{i,j}$ genau dann den Wert $+1$ an, wenn der Sensorknoten K_i auf dem Ring R_j installiert wird. Analog nimmt die Variable $s_{i,j}$ den Wert -1 an, wenn sie nicht auf dem Ring installiert ist.

Ringkapazität Für jeden Ring muss sichergestellt werden, dass maximal drei Knoten installiert werden. Dazu wird ein Ising-Ausdruck eingeführt, der genau dann minimal ist, wenn maximal drei der Sensorknotenvariablen eines Rings den Wert $+1$ annehmen. Um so einen Ising-Ausdruck zu finden, wird eine Hilfsvariable h benötigt. Der dazugehörige Ising-Ausdruck sieht wie folgt aus:

$$\forall j \in \{1, \dots, 5\} : \sum_{i=0}^6 3s_{i,j} + 6h + \sum_{i < k} s_{i,j}s_{i,k} + \sum_{i=1}^6 2s_{i,j}h \quad (7.36)$$

Sensorknotenkommunikation Die Sensorknoten müssen so auf den Ringen installiert werden, dass alle Kanten, die in der Probleminstance gegeben sind, auch auf den einzelnen Ringen wiederzufinden sind. Dazu wird für jede geforderte Kante auf jedem Ring eine Hilfsvariable $h_{r,i,j}$ eingeführt. Dabei steht der Index r für den jeweiligen Ring und i und j für die beiden zu verbindenden Knoten. Diese Variable $h_{r,i,j}$ nimmt genau dann den Wert $+1$ an, wenn sowohl die Variable $s_{i,r}$ als auch $s_{j,r}$ den Wert $+1$ annehmen. Die Hilfsvariable symbolisiert also, ob zwei Knoten in einem Ring kommunizieren können oder nicht. Der dazu benötigte Ising-Ausdruck sieht wie folgt aus:

$$\forall (i, j) \in E \forall r \in \{1, \dots, 5\} : -s_{i,r} - s_{j,r} + 2h_{r,i,j} + s_{i,r}s_{j,r} - 2s_{i,r}h_{r,i,j} - 2s_{j,r}h_{r,i,j} \quad (7.37)$$

Mithilfe dieser Hilfsvariablen kann jetzt ein Ising-Ausdruck gefunden werden, der genau dann minimal ist, wenn für eine geforderte Kante mindestens eine der neuen Hilfsvariablen eines Ringes den Wert $+1$ annimmt. Um einen solchen Ising-Ausdruck zu finden, müssen zwei Hilfsvariablen $h_{e1,i,j}$ und $h_{e2,i,j}$ eingeführt werden. Der Ausdruck lautet dann wie folgt:

$$\begin{aligned} \forall (i, j) \in E : & -h_{2,i,j} - h_{3,i,j} - h_{5,i,j} + h_{e1,i,j} - 2h_{e2,i,j} + h_{1,i,j}h_{4,i,j} + h_{1,i,j}h_{e1,i,j} \\ & + h_{2,i,j}h_{3,i,j} + h_{2,i,j}h_{5,i,j} - h_{2,i,j}h_{e1,i,j} + 2h_{2,i,j}h_{e2,i,j} + h_{3,i,j}h_{5,i,j} - h_{3,i,j}h_{e1,i,j} \\ & + 2h_{3,i,j}h_{e2,i,j} + h_{4,i,j}h_{e1,i,j} - h_{5,i,j}h_{e1,i,j} + 2h_{5,i,j}h_{e2,i,j} - 2h_{e1,i,j}h_{e2,i,j} \end{aligned} \quad (7.38)$$

Zielfunktion. Zusätzlich zu den beiden oberen Constraints soll die Anzahl der installierten Knoten auf den Ringen minimiert werden. Dazu werden die folgenden

Ising-Ausdrücke eingeführt:

$$\forall i \in \{1, \dots, 6\} \forall j \in \{1, \dots, 5\} : f \cdot \frac{g}{6 \cdot 5 + 1} \cdot s_{i,j} \quad (7.39)$$

Die Variable g ist dabei die kleinste minimale Energielücke aller verwendeten Ising-Ausdrücke. Für diese Modellierung liegt ihr Wert bei 4. Die Variable f ist ein Skalierungsfaktor, der frei gewählt werden kann. Die besten Ergebnisse werden mit einem Faktor von 7,7 erzielt. Eine genaue Erklärung dazu lässt sich in Abschnitt 5.4.2 finden.

Literaturverzeichnis

- [Abu25] Muhammad AbuGhanem. Ibm quantum computers: evolution, performance, and future directions. *The Journal of Supercomputing*, 81(5), April 2025.
- [adv] Advantage system 7.1. https://docs.dwavequantum.com/en/latest/quantum_research/solver_properties_specific.html#advantage-system7-1. zuletzt besucht 2025-05-11.
- [ALL23] Gilles Audemard, Christophe Lecoutre, and Emmanuel Lonca. Proceedings of the 2023 xcsp3 competition, 2023. arXiv:2312.05877.
- [BCS57] J. Bardeen, L. N. Cooper, and J. R. Schrieffer. Theory of superconductivity. *Phys. Rev.*, 108:1175–1204, Dec 1957.
- [Bea12] Mathieu Beau. Feynman path integral approach to electron diffraction for one and two slits: analytical results. *European Journal of Physics*, 33(5):1023–1039, June 2012.
- [Bel12] Dr Tonomura and Belsazar. Français : Diffraction d’électrons. https://commons.wikimedia.org/wiki/File:Double-slit_experiment_results_Tanamura_four.jpg, August 2012. zuletzt besucht 2025-05-10.
- [BF28] M. Born and V. Fock. Beweis des Adiabatenatzes. *Zeitschrift für Physik*, 51:165–180, mar 1928.
- [BG00] D.A. Bromley and W. Greiner. *Quantum Mechanics: An Introduction*. Physics and Astronomy. Springer Berlin Heidelberg, 2000.
- [cha] Chain strength utility. https://github.com/dwavesystems/dwave-system/blob/master/dwave/embedding/chain_strength.py. zuletzt besucht 2025-05-11.

- [Cha19] Nicholas Chancellor. Domain wall encoding of discrete variables for quantum annealing and qaoa. *Quantum Science and Technology*, 4(4):045004, August 2019.
- [cho] Choco. <https://choco-solver.org/>. zuletzt besucht 2025-05-11.
- [CMR14] Jun Cai, William G. Macready, and Aidan Roy. A practical heuristic for finding graph minors. *CoRR*, abs/1406.2741, 2014. arXiv:1406.2741.
- [Cod21] Philippe Codognet. Constraint solving by quantum annealing. In *50th International Conference on Parallel Processing Workshop, ICPP Workshops '21*, New York, NY, USA, 2021. Association for Computing Machinery.
- [cpl] Ibm ilog cplex optimizer. <https://www.ibm.com/de-de/products/ilog-cplex-optimization-studio/cplex-optimizer>. zuletzt besucht 2025-05-11.
- [csp99] CSPLib: A problem library for constraints. <http://www.csplib.org>, 1999. zuletzt besucht 2025-05-11.
- [dig] Digital annealer fujitsu. <https://www.fujitsu.com/de/services/hybrid-it/digital-annealer/>. zuletzt besucht 2025-05-11.
- [Ful00] William Fulton. Eigenvalues, invariant factors, highest weights, and schubert calculus. *BULLETIN (New Series) OF THE AMERICAN MATHEMATICAL SOCIETY*, 37(3):209–249, 2000. arXiv:math/9908012.
- [HJB⁺10] R. Harris, J. Johansson, A. J. Berkley, M. W. Johnson, T. Lanting, Siyuan Han, P. Bunyk, E. Ladizinsky, T. Oh, I. Perminov, E. Tolkacheva, S. Uchaikin, E. M. Chapple, C. Enderud, C. Rich, M. Thom, J. Wang, B. Wilson, and G. Rose. Experimental demonstration of a robust and scalable flux qubit. *Phys. Rev. B*, 81:134510, Apr 2010.
- [HLB⁺09] R. Harris, T. Lanting, A. J. Berkley, J. Johansson, M. W. Johnson, P. Bunyk, E. Ladizinsky, N. Ladizinsky, T. Oh, and S. Han. Compound josephson-junction coupler for flux qubits with minimal crosstalk. *Phys. Rev. B*, 80:052506, Aug 2009.
- [Hur09] G. Hurlbert. *Linear Optimization: The Simplex Workbook*. Undergraduate Texts in Mathematics. Springer New York, 2009.
- [HW07] Petra Hofstedt and Armin Wolf. *Einführung in die Constraint-Programmierung - Grundlagen, Methoden, Sprachen, Anwendungen*. eXamen.press. Springer, 2007.

- [JBTS19] Farzan Jazaeri, Arnout Beckers, Armin Tajalli, and Jean-Michel Sallese. A review on quantum computing: From qubits to front-end electronics and cryogenic MOSFET physics. In Andrzej Napieralksi, editor, *26th International Conference on Mixed Design of Integrated Circuits and Systems, MIXDES 2019, Rzeszów, Poland, June 27-29, 2019*, pages 15–25. IEEE, 2019.
- [KYN⁺15] James King, Sheir Yarkoni, Mayssam M. Nevisi, Jeremy P. Hilton, and Catherine C. McGeoch. Benchmarking a quantum annealing processor with the time-to-target metric, 2015. arXiv:1508.05087.
- [Lan13] S. Lang. *Linear Algebra*. Undergraduate Texts in Mathematics. Springer New York, 2013.
- [LBH] Sven Löffler, Ilja Becker, and Petra Hofstedt. CSPLib problem 087: Rotating rostering problem. <http://www.csplib.org/Problems/prob087>. zuletzt besucht 2025-05-11.
- [LL24] Elisabeth Lobe and Annette Lutz. Minor embedding in broken chimera and derived graphs is np-complete. *Theoretical Computer Science*, 989:114369, 2024.
- [min17] minorminer. <https://github.com/dwavesystems/minorminer>, 2017. zuletzt besucht 2025-05-11.
- [oce] Dwave ocean sdk. <https://github.com/dwavesystems/dwave-ocean-sdk>. zuletzt besucht 2025-05-11.
- [Pak21] Scott Pakin. A simple heuristic for expressing a truth table as a quadratic pseudo-boolean function. In *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 218–224, 2021.
- [PD] Laurent Perron and Frédéric Didier. Cp-sat. https://developers.google.com/optimization/cp/cp_solver/. zuletzt besucht 2025-05-26.
- [pen] Dwave penalty model. <https://docs.dwavequantum.com/en/latest/concepts/penalty.html>. zuletzt besucht 2025-05-11.
- [Pet15] Justyna Petke. *Bridging Constraint Satisfaction and Boolean Satisfiability*. Springer Publishing Company, Incorporated, 1st edition, 2015.
- [PVVL22] Aleksey I. Pakhomchik, Vladimir V. Voloshinov, Valerii M. Vinokur, and Gordey B. Lesovik. Converting of boolean expression to linear equations, inequalities and qubo penalties for cryptanalysis. *Algorithms*, 15(2), 2022.

- [Raz03] M. Razavy. *Quantum Theory of Tunneling*. G - Reference, Information and Interdisciplinary Subjects Series. World Scientific, 2003.
- [RN20] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020.
- [RSA78] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120–126, February 1978.
- [RSST97] Neil Robertson, Daniel Sanders, Paul Seymour, and Robin Thomas. The four-colour theorem. *Journal of Combinatorial Theory, Series B*, 70(1):2–44, 1997.
- [Sho97] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997.
- [SJ21] Ashkan Shekaari and Mahmoud Jafari. Theory and simulation of the ising model, 2021. arXiv:2105.00841.
- [SN20] J. J. Sakurai and Jim Napolitano. *Modern Quantum Mechanics*. Cambridge University Press, 3 edition, 2020.
- [SS05] N. Samaras and K. Stergiou. Binary encodings of non-binary constraint satisfaction problems: Algorithms and experimental results. *Journal of Artificial Intelligence Research*, 24:641–684, November 2005.
- [TTMT19] Kotaro Tanahashi, Shinichi Takayanagi, Tomomitsu Motohashi, and Shu Tanaka. Application of ising machines and a software development for ising machines. *Journal of the Physical Society of Japan*, 88(6):061010, 2019.
- [Van20] Robert J. Vanderbei. *Linear Programming: Foundations and Extensions*, volume 285 of *International Series in Operations Research & Management Science*. Springer International Publishing, Cham, 2020.
- [Von32] John VonNeumann. *Mathematische Grundlagen der Quantenmechanik*. Springer, 1932.
- [WG23] Armin Wolf and Cristian Grozea. Automatic conversion of minizinc programs to qubo, 2023. arXiv:2307.10032.
- [Wol24] Armin Wolf. Conversion of boolean and integer flatzinc builtins to quadratic or linear integer problems, 2024. arXiv:2404.12797.

- [Zim05] James Zimmerman. Physical chemistry for the biosciences: Chang, raymond. *Biochemistry and Molecular Biology Education*, 33(5):382–382, 2005.
- [ZTT21] Mashiyat Zaman, Kotaro Tanahashi, and Shu Tanaka. Pyqubo: Python library for qubo creation. *IEEE Transactions on Computers*, 2021.

Eigenständigkeitserklärung

Der Verfasser erklärt, dass er die vorliegende Arbeit selbständig, ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt hat. Die aus fremden Quellen (einschließlich elektronischer Quellen) direkt oder indirekt übernommenen Gedanken sind ausnahmslos als solche kenntlich gemacht. Wörtlich und inhaltlich verwendete Quellen wurden entsprechend den anerkannten Regeln wissenschaftlichen Arbeitens zitiert. Die Arbeit ist nicht in gleicher oder vergleichbarer Form, auch nicht auszugsweise im Rahmen einer anderen Prüfung bei einer anderen Hochschule vorgelegt worden.

Sie wurde bisher auch nicht veröffentlicht.

Ich erkläre mich damit einverstanden, dass die Arbeit mit Hilfe eines Plagiatserkennungsdienstes auf enthaltene Plagiate überprüft wird.

Ort, Datum

Unterschrift des Verfassers